

H2 Database Engine

Version 2.5.250 (2026-08-29)

Table of Contents

Quickstart.....	30
Embedding H2 in an Application.....	30
The H2 Console Application.....	30
Step-by-Step.....	30
Installation.....	30
Start the Console.....	31
Login.....	31
Sample.....	32
Execute.....	33
Disconnect.....	34
End.....	34
Installation.....	35
Requirements.....	35
Database Engine.....	35
H2 Console.....	35
Supported Platforms.....	35
Installing the Software.....	35
Directory Structure.....	35
Tutorial.....	37
Starting and Using the H2 Console.....	37
Firewall.....	39
Testing Java.....	39
Error Message 'Port may be in use'.....	39
Using another Port.....	39
Connecting to the Server using a Browser.....	40
Multiple Concurrent Sessions.....	40
Login.....	40
Error Messages.....	40
Adding Database Drivers.....	40
Using the H2 Console.....	41
Inserting Table Names or Column Names.....	41
Disconnecting and Stopping the Application.....	41
Special H2 Console Syntax.....	41
Settings of the H2 Console.....	43
Connecting to a Database using JDBC.....	44

Creating New Databases.....	45
Using the Server.....	46
Starting the Server Tool from Command Line.....	46
Connecting to the TCP Server.....	47
Starting the TCP Server within an Application.....	47
Stopping a TCP Server from Another Process.....	47
Using Hibernate.....	48
Using TopLink and Glassfish.....	48
Using EclipseLink.....	49
Using Apache ActiveMQ.....	49
Using H2 within NetBeans.....	49
Using H2 with jOOQ.....	50
Using Databases in Web Applications.....	51
Embedded Mode.....	51
Server Mode.....	51
Using a Servlet Listener to Start and Stop a Database.....	52
Using the H2 Console Servlet.....	53
CSV (Comma Separated Values) Support.....	54
Reading a CSV File from Within a Database.....	54
Importing Data from a CSV File.....	55
Writing a CSV File from Within a Database.....	55
Writing a CSV File from a Java Application.....	55
Reading a CSV File from a Java Application.....	56
Upgrade, Backup, and Restore.....	56
Database Upgrade.....	56
Backup using the Script Tool.....	56
Restore from a Script.....	57
Online Backup.....	57
Command Line Tools.....	58
The Shell Tool.....	59
Using OpenOffice Base.....	59
Java Web Start / JNLP.....	60
Using a Connection Pool.....	61
Fulltext Search.....	62
Using the Native Fulltext Search.....	62
Using the Apache Lucene Fulltext Search.....	63
User-Defined Variables.....	64

Date and Time.....	65
Using Spring.....	66
Using the TCP Server.....	66
OSGi.....	66
Java Management Extension (JMX).....	67
Features.....	69
Feature List.....	70
Main Features.....	70
Additional Features.....	70
SQL Support.....	70
Security Features.....	71
Other Features and Tools.....	71
H2 in Use.....	72
Connection Modes.....	72
Embedded Mode.....	72
Server Mode.....	73
Mixed Mode.....	74
Database URL Overview.....	74
Connecting to an Embedded (Local) Database.....	76
In-Memory Databases.....	76
Database Files Encryption.....	77
Creating a New Database with File Encryption.....	78
Connecting to an Encrypted Database.....	78
Encrypting or Decrypting a Database.....	78
Database File Locking.....	78
Opening a Database Only if it Already Exists.....	79
Closing a Database.....	80
Delayed Database Closing.....	80
Don't Close a Database when the VM Exits.....	80
Execute SQL on Connection.....	81
Ignore Unknown Settings.....	81
Changing Other Settings when Opening a Connection.....	82
Custom File Access Mode.....	82
Multiple Connections.....	82
Opening Multiple Databases at the Same Time.....	82
Multiple Connections to the Same Database: Client/Server.....	82
Multithreading Support.....	83

Locking, Lock-Timeout, Deadlocks.....	83
Database File Layout.....	84
Moving and Renaming Database Files.....	85
Backup.....	85
Logging and Recovery.....	85
Compatibility.....	85
Compatibility Modes.....	86
REGULAR Compatibility mode.....	86
STRICT Compatibility Mode.....	86
LEGACY Compatibility Mode.....	87
DB2 Compatibility Mode.....	88
Derby Compatibility Mode.....	89
HSQLDB Compatibility Mode.....	89
MS SQL Server Compatibility Mode.....	89
MariaDB Compatibility Mode.....	90
MySQL Compatibility Mode.....	92
Oracle Compatibility Mode.....	93
PostgreSQL Compatibility Mode.....	94
Auto-Reconnect.....	95
Automatic Mixed Mode.....	95
Page Size.....	97
Using the Trace Options.....	97
Trace Options.....	97
Setting the Maximum Size of the Trace File.....	98
Java Code Generation.....	98
Using Other Logging APIs.....	99
Read Only Databases.....	99
Read Only Databases in Zip or Jar File.....	100
Opening a Corrupted Database.....	100
Generated Columns (Computed Columns) / Function Based Index.....	101
Multi-Dimensional Indexes.....	102
User-Defined Functions and Stored Procedures.....	102
Referencing a Compiled Method.....	102
Declaring Functions as Source Code.....	103
Method Overloading.....	104
Function Data Type Mapping.....	104
Functions That Require a Connection.....	105

Functions Throwing an Exception.....	105
Functions Returning a Result Set.....	105
Using SimpleResultSet.....	105
Using a Function as a Table.....	106
Pluggable or User-Defined Tables.....	107
Triggers.....	108
Compacting a Database.....	109
Cache Settings.....	110
External authentication (Experimental).....	111
Securing your H2.....	114
Introduction.....	114
Network exposed.....	114
Alias / Stored procedures.....	114
Grants / Roles / Permissions.....	114
Encrypted storage.....	115
Performance.....	116
Performance Comparison.....	116
Embedded.....	116
Client-Server.....	117
Benchmark Results and Comments.....	118
H2.....	118
HSQLDB.....	118
Derby.....	118
PostgreSQL.....	119
MySQL.....	119
SQLite.....	120
Firebird.....	120
Why Oracle / MS SQL Server / DB2 are Not Listed.....	120
About this Benchmark.....	120
How to Run.....	120
Separate Process per Database.....	120
Number of Connections.....	121
Real-World Tests.....	121
Comparing Embedded with Server Databases.....	121
Test Platform.....	121
Multiple Runs.....	121
Memory Usage.....	121

Delayed Operations.....	122
Transaction Commit / Durability.....	122
Using Prepared Statements.....	122
Currently Not Tested: Startup Time.....	122
PolePosition Benchmark.....	123
Database Performance Tuning.....	124
Keep Connections Open or Use a Connection Pool.....	124
Use a Modern JVM.....	124
Virus Scanners.....	124
Using the Trace Options.....	125
Index Usage.....	125
Index Hints.....	125
How Data is Stored Internally.....	126
Optimizer.....	126
Expression Optimization.....	127
COUNT(*) Optimization.....	127
Updating Optimizer Statistics / Column Selectivity.....	127
In-Memory (Hash) Indexes.....	128
Use Prepared Statements.....	128
Prepared Statements and IN(...)......	128
Optimization Examples.....	129
Cache Size and Type.....	129
Data Types.....	129
Sorted Insert Optimization.....	129
Using the Built-In Profiler.....	129
Application Profiling.....	130
Analyze First.....	130
Database Profiling.....	131
Statement Execution Plans.....	132
Displaying the Scan Count.....	133
Special Optimizations.....	134
How Data is Stored and How Indexes Work.....	135
Indexes.....	136
Using Multiple Indexes.....	137
Fast Database Import.....	138
Advanced.....	139
Result Sets.....	140

Statements that Return a Result Set.....	140
Limiting the Number of Rows.....	140
Large Result Sets and External Sorting.....	140
Large Objects.....	140
Storing and Reading Large Objects.....	140
When to use CLOB/BLOB.....	141
Linked Tables.....	141
Updatable Views.....	142
Transaction Isolation.....	142
Multi-Version Concurrency Control (MVCC).....	144
Lock Timeout.....	144
Clustering / High Availability.....	144
Using the CreateCluster Tool.....	145
Detect Which Cluster Instances are Running.....	146
Clustering Algorithm and Limitations.....	147
Two Phase Commit.....	148
Compatibility.....	148
Transaction Commit when Autocommit is On.....	148
Keywords / Reserved Words.....	149
Standards Compliance.....	154
Supported Character Sets, Character Encoding, and Unicode.....	154
Run as Windows Service.....	154
Install the Service.....	155
Start the Service.....	155
Connect to the H2 Console.....	155
Stop the Service.....	155
Uninstall the Service.....	155
Additional JDBC drivers.....	155
ODBC Driver.....	156
ODBC Installation.....	156
Starting the Server.....	156
ODBC Configuration.....	157
PG Protocol Support Limitations.....	158
Security Considerations.....	158
Using Microsoft Access.....	158
ACID.....	158
Atomicity.....	159

Consistency.....	159
Isolation.....	159
Durability.....	159
Durability Problems.....	159
Ways to (Not) Achieve Durability.....	159
Running the Durability Test.....	161
Using the Recover Tool.....	162
Traditional 2-phase Recover.....	162
Direct Recover.....	162
File Locking Protocols.....	164
File Locking Method 'File'.....	164
File Locking Method 'Socket'.....	166
File Locking Method 'FS'.....	166
Using Passwords.....	167
Using Secure Passwords.....	167
Passwords: Using Char Arrays instead of Strings.....	167
Passing the User Name and/or Password in the URL.....	168
Password Hash.....	168
Protection against SQL Injection.....	169
What is SQL Injection.....	169
Disabling Literals.....	169
Using Constants.....	170
Using the ZERO() Function.....	171
Protection against Remote Access.....	171
Restricting Class Loading and Usage.....	171
Security Protocols.....	172
User Password Encryption.....	172
File Encryption.....	173
Wrong Password / User Name Delay.....	174
HTTPS Connections.....	175
TLS Connections.....	175
Universally Unique Identifiers (UUID).....	175
Spatial Features.....	177
Recursive Queries.....	178
Settings Read from System Properties.....	179
Setting the Server Bind Address.....	179
Pluggable File System.....	180

Split File System.....	181
Java Objects Serialization.....	181
Limits and Limitations.....	182
Glossary and Links.....	183
Commands.....	185
Index.....	185
Commands (Data Manipulation).....	185
Commands (Data Definition).....	185
Commands (Other).....	187
Details.....	189
Commands (Data Manipulation).....	189
SELECT.....	189
INSERT.....	192
UPDATE.....	192
DELETE.....	193
BACKUP.....	193
CALL.....	193
EXECUTE IMMEDIATE.....	194
EXPLAIN.....	194
MERGE INTO.....	194
MERGE USING.....	195
RUNSCRIPT.....	196
SCRIPT.....	197
SHOW.....	198
Explicit table.....	198
Table value.....	199
WITH.....	199
Commands (Data Definition).....	200
ALTER DOMAIN.....	200
ALTER DOMAIN ADD CONSTRAINT.....	201
ALTER DOMAIN DROP CONSTRAINT.....	201
ALTER DOMAIN RENAME.....	201
ALTER DOMAIN RENAME CONSTRAINT.....	202
ALTER INDEX RENAME.....	202
ALTER SCHEMA RENAME.....	202
ALTER SEQUENCE.....	202
ALTER TABLE ADD.....	203

ALTER TABLE ADD CONSTRAINT.....	203
ALTER TABLE RENAME CONSTRAINT.....	204
ALTER TABLE ALTER COLUMN.....	204
ALTER TABLE DROP COLUMN.....	206
ALTER TABLE DROP CONSTRAINT.....	206
ALTER TABLE SET.....	207
ALTER TABLE RENAME.....	207
ALTER USER ADMIN.....	208
ALTER USER RENAME.....	208
ALTER USER SET PASSWORD.....	208
ALTER VIEW RECOMPILE.....	209
ALTER VIEW RENAME.....	209
ANALYZE.....	209
COMMENT ON.....	210
CREATE AGGREGATE.....	210
CREATE ALIAS.....	210
CREATE CONSTANT.....	212
CREATE DOMAIN.....	212
CREATE INDEX.....	213
CREATE LINKED TABLE.....	213
CREATE ROLE.....	215
CREATE SCHEMA.....	215
CREATE SEQUENCE.....	216
CREATE TABLE.....	216
CREATE TRIGGER.....	218
CREATE USER.....	220
CREATE VIEW.....	220
CREATE MATERIALIZED VIEW.....	221
DROP AGGREGATE.....	221
DROP ALIAS.....	221
DROP ALL OBJECTS.....	222
DROP CONSTANT.....	222
DROP DOMAIN.....	222
DROP INDEX.....	223
DROP ROLE.....	223
DROP SCHEMA.....	223
DROP SEQUENCE.....	224

DROP TABLE.....	224
DROP TRIGGER.....	224
DROP USER.....	224
DROP VIEW.....	225
DROP MATERIALIZED VIEW.....	225
REFRESH MATERIALIZED VIEW.....	225
TRUNCATE TABLE.....	226
ALTER TYPE.....	226
Commands (Other).....	226
CHECKPOINT.....	226
CHECKPOINT SYNC.....	227
COMMIT.....	227
COMMIT TRANSACTION.....	227
GRANT RIGHT.....	228
GRANT ALTER ANY SCHEMA.....	228
GRANT ROLE.....	228
HELP.....	229
PREPARE COMMIT.....	229
REVOKE RIGHT.....	229
REVOKE ALTER ANY SCHEMA.....	229
REVOKE ROLE.....	230
ROLLBACK.....	230
ROLLBACK TRANSACTION.....	230
SAVEPOINT.....	231
SET @.....	231
SET ALLOW_LITERALS.....	231
SET AUTOCOMMIT.....	232
SET CACHE_SIZE.....	232
SET CLUSTER.....	233
SET BUILTIN_ALIAS_OVERRIDE.....	233
SET CATALOG.....	233
SET COLLATION.....	234
SET DATABASE_EVENT_LISTENER.....	235
SET DB_CLOSE_DELAY.....	235
SET DEFAULT_LOCK_TIMEOUT.....	236
SET DEFAULT_NULL_ORDERING.....	236
SET DEFAULT_TABLE_TYPE.....	237

SET EXCLUSIVE.....	237
SET IGNORECASE.....	238
SET IGNORE_CATALOGS.....	238
SET JAVA_OBJECT_SERIALIZER.....	239
SET LAZY_QUERY_EXECUTION.....	239
SET LOCK_MODE.....	240
SET LOCK_TIMEOUT.....	240
SET MAX_LENGTH_INPLACE_LOB.....	241
SET MAX_LOG_SIZE.....	241
SET MAX_MEMORY_ROWS.....	241
SET MAX_MEMORY_UNDO.....	242
SET MAX_OPERATION_MEMORY.....	242
SET MODE.....	243
SET NON_KEYWORDS.....	243
SET OPTIMIZE_REUSE_RESULTS.....	243
SET PASSWORD.....	244
SET QUERY_STATISTICS.....	244
SET QUERY_STATISTICS_MAX_ENTRIES.....	244
SET QUERY_TIMEOUT.....	245
SET REFERENTIAL_INTEGRITY.....	245
SET RETENTION_TIME.....	246
SET SALT_HASH.....	246
SET SCHEMA.....	246
SET SCHEMA_SEARCH_PATH.....	247
SET SESSION CHARACTERISTICS.....	247
SET THROTTLE.....	248
SET TIME_ZONE.....	248
SET TRACE_LEVEL.....	249
SET TRACE_MAX_FILE_SIZE.....	249
SET TRUNCATE_LARGE_LENGTH.....	249
SET VARIABLE_BINARY.....	250
SET WRITE_DELAY.....	250
SHUTDOWN.....	251
Functions.....	252
Index.....	252
Numeric Functions.....	252
String Functions.....	253

Time and Date Functions.....	255
System Functions.....	256
JSON Functions.....	257
Table Functions.....	257
Details.....	258
Numeric Functions.....	258
ABS.....	258
ACOS.....	258
ASIN.....	259
ATAN.....	259
COS.....	259
COSH.....	259
COT.....	260
SIN.....	260
SINH.....	260
TAN.....	260
TANH.....	261
ATAN2.....	261
BITAND.....	261
BITOR.....	262
BITXOR.....	262
BITNOT.....	262
BITNAND.....	262
BITNOR.....	263
BITXNOR.....	263
BITGET.....	264
BITCOUNT.....	264
LSHIFT.....	264
RSHIFT.....	265
ULSHIFT.....	265
URSHIFT.....	266
ROTATELEFT.....	266
ROTATERIGHT.....	266
MOD.....	267
CEIL.....	267
DEGREES.....	267
EXP.....	268

FLOOR.....	268
LN.....	268
LOG.....	268
LOG10.....	269
ORA_HASH.....	269
RADIANS.....	269
SQRT.....	270
PI.....	270
POWER.....	270
RAND.....	270
RANDOM_UUID.....	271
ROUND.....	271
SECURE_RAND.....	271
SIGN.....	272
ENCRYPT.....	272
DECRYPT.....	272
HASH.....	272
TRUNC.....	273
COMPRESS.....	274
EXPAND.....	274
ZERO.....	274
GCD.....	274
LCM.....	275
String Functions.....	275
ASCII.....	275
BIT_LENGTH.....	275
CHAR_LENGTH.....	276
OCTET_LENGTH.....	276
CHAR.....	276
CONCAT.....	276
CONCAT_WS.....	277
DIFFERENCE.....	277
HEXTORAW.....	277
RAWTOHEX.....	278
INSERT Function.....	278
LOWER.....	278
UPPER.....	278

LEFT.....	279
RIGHT.....	279
LOCATE.....	279
LPAD.....	279
RPAD.....	280
LTRIM.....	280
RTRIM.....	280
BTRIM.....	280
TRIM.....	281
REGEXP_REPLACE.....	281
REGEXP_LIKE.....	282
REGEXP_SUBSTR.....	282
REPEAT.....	283
REPLACE.....	283
SOUNDEX.....	284
SPACE.....	284
STRINGDECODE.....	284
STRINGENCODE.....	284
STRINGTOUTF8.....	285
SUBSTRING.....	285
UTF8TOSTRING.....	285
QUOTE_IDENT.....	286
XMLATTR.....	286
XMLNODE.....	286
XMLCOMMENT.....	286
XMLCDATA.....	287
XMLSTARTDOC.....	287
XMLTEXT.....	287
TO_CHAR.....	287
TRANSLATE.....	288
Time and Date Functions.....	288
CURRENT_DATE.....	288
CURRENT_TIME.....	288
CURRENT_TIMESTAMP.....	289
LOCALTIME.....	289
LOCALTIMESTAMP.....	290
DATEADD.....	290

DATEDIFF.....	291
DATE_TRUNC.....	291
LAST_DAY.....	292
DAYNAME.....	292
DAY_OF_MONTH.....	292
DAY_OF_WEEK.....	292
ISO_DAY_OF_WEEK.....	293
DAY_OF_YEAR.....	293
EXTRACT.....	293
FORMATDATETIME.....	293
HOUR.....	294
MINUTE.....	294
MONTH.....	295
MONTHNAME.....	295
PARSEDATETIME.....	295
QUARTER.....	296
SECOND.....	296
WEEK.....	296
ISO_WEEK.....	296
YEAR.....	297
ISO_YEAR.....	297
System Functions.....	297
ABORT_SESSION.....	297
ARRAY_GET.....	298
CARDINALITY.....	298
ARRAY_CONTAINS.....	298
ARRAY_CAT.....	299
ARRAY_APPEND.....	299
ARRAY_MAX_CARDINALITY.....	299
TRIM_ARRAY.....	299
ARRAY_SLICE.....	300
AUTOCOMMIT.....	300
CANCEL_SESSION.....	300
CASEWHEN Function.....	301
COALESCE.....	301
CONVERT.....	301
CURRVAL.....	301

CSVWRITE.....	302
CURRENT_SCHEMA.....	303
CURRENT_CATALOG.....	303
DATABASE_PATH.....	303
DATA_TYPE_SQL.....	303
DB_OBJECT_ID.....	304
DB_OBJECT_SQL.....	305
DB_OBJECT_SIZE.....	305
DB_OBJECT_TOTAL_SIZE.....	306
DB_OBJECT_APPROXIMATE_SIZE.....	306
DB_OBJECT_APPROXIMATE_TOTAL_SIZE.....	306
DECODE.....	307
DISK_SPACE_USED.....	307
SIGNAL.....	307
ESTIMATED_ENVELOPE.....	307
FILE_READ.....	308
FILE_WRITE.....	308
GREATEST.....	309
LEAST.....	309
LOCK_MODE.....	309
LOCK_TIMEOUT.....	309
MEMORY_FREE.....	310
MEMORY_USED.....	310
NEXTVAL.....	310
NULLIF.....	311
NVL2.....	311
READONLY.....	311
ROWNUM.....	311
SESSION_ID.....	312
SET.....	312
TRANSACTION_ID.....	313
TRUNCATE_VALUE.....	313
CURRENT_PATH.....	313
CURRENT_ROLE.....	313
CURRENT_USER.....	314
H2VERSION.....	314
JSON Functions.....	314

JSON_OBJECT.....	314
JSON_ARRAY.....	315
Table Functions.....	315
CSVREAD.....	315
LINK_SCHEMA.....	316
TABLE.....	317
UNNEST.....	317
Aggregate Functions.....	318
Index.....	318
General Aggregate Functions.....	318
Binary Set Functions.....	318
Ordered Aggregate Functions.....	319
Hypothetical Set Functions.....	319
Inverse Distribution Functions.....	319
JSON Aggregate Functions.....	319
Details.....	319
General Aggregate Functions.....	320
AVG.....	320
MAX.....	320
MIN.....	320
SUM.....	321
EVERY.....	321
ANY.....	322
COUNT.....	322
STDDEV_POP.....	322
STDDEV_SAMP.....	323
VAR_POP.....	323
VAR_SAMP.....	323
ANY_VALUE.....	324
BIT_AND_AGG.....	324
BIT_OR_AGG.....	324
BIT_XOR_AGG.....	325
BIT_NAND_AGG.....	325
BIT_NOR_AGG.....	325
BIT_XNOR_AGG.....	326
ENVELOPE.....	326
GCD_AGG.....	326

LCM_AGG.....	327
Binary Set Functions.....	327
COVAR_POP.....	327
COVAR_SAMP.....	327
CORR.....	328
REGR_SLOPE.....	328
REGR_INTERCEPT.....	328
REGR_COUNT.....	329
REGR_R2.....	329
REGR_AVGX.....	329
REGR_AVGY.....	330
REGR_SXX.....	330
REGR_SYY.....	330
REGR_SXY.....	331
Ordered Aggregate Functions.....	331
LISTAGG.....	331
ARRAY_AGG.....	332
Hypothetical Set Functions.....	332
RANK aggregate.....	332
DENSE_RANK aggregate.....	333
PERCENT_RANK aggregate.....	333
CUME_DIST aggregate.....	334
Inverse Distribution Functions.....	334
PERCENTILE_CONT.....	334
PERCENTILE_DISC.....	335
MEDIAN.....	335
MODE.....	335
JSON Aggregate Functions.....	336
JSON_OBJECTAGG.....	336
JSON_ARRAYAGG.....	336
Window Functions.....	338
Index.....	338
Row Number Function.....	338
Rank Functions.....	338
Lead or Lag Functions.....	338
Nth Value Functions.....	338
Other Window Functions.....	338

Details.....	338
Row Number Function.....	339
ROW_NUMBER.....	339
Rank Functions.....	339
RANK.....	339
DENSE_RANK.....	340
PERCENT_RANK.....	340
CUME_DIST.....	341
Lead or Lag Functions.....	341
LEAD.....	341
LAG.....	342
Nth Value Functions.....	342
FIRST_VALUE.....	342
LAST_VALUE.....	343
NTH_VALUE.....	343
Other Window Functions.....	344
NTILE.....	344
RATIO_TO_REPORT.....	344
Data Types.....	346
Index.....	346
Details.....	347
CHARACTER.....	347
CHARACTER VARYING.....	347
CHARACTER LARGE OBJECT.....	348
VARCHAR_IGNORECASE.....	349
BINARY.....	349
BINARY VARYING.....	350
BINARY LARGE OBJECT.....	350
BOOLEAN.....	351
TINYINT.....	351
SMALLINT.....	351
INTEGER.....	352
BIGINT.....	352
NUMERIC.....	352
REAL.....	353
DOUBLE PRECISION.....	353
DECFLOAT.....	353

DATE.....	354
TIME.....	355
TIME WITH TIME ZONE.....	355
TIMESTAMP.....	356
TIMESTAMP WITH TIME ZONE.....	357
INTERVAL.....	357
JAVA_OBJECT.....	358
ENUM.....	359
GEOMETRY.....	359
JSON.....	360
UUID.....	361
ARRAY.....	361
ROW.....	362
Interval Data Types.....	362
INTERVAL YEAR.....	362
INTERVAL MONTH.....	363
INTERVAL DAY.....	363
INTERVAL HOUR.....	363
INTERVAL MINUTE.....	363
INTERVAL SECOND.....	364
INTERVAL YEAR TO MONTH.....	364
INTERVAL DAY TO HOUR.....	364
INTERVAL DAY TO MINUTE.....	365
INTERVAL DAY TO SECOND.....	365
INTERVAL HOUR TO MINUTE.....	365
INTERVAL HOUR TO SECOND.....	366
INTERVAL MINUTE TO SECOND.....	366
SQL Grammar.....	367
Index.....	367
Literals.....	367
Datetime fields.....	368
Other Grammar.....	369
Details.....	371
Literals.....	372
Value.....	372
Approximate numeric.....	372
Array.....	372

Boolean.....	373
Bytes.....	373
Date.....	373
Date and time.....	373
Dollar Quoted String.....	374
Exact numeric.....	374
Hex Number.....	374
Octal Number.....	375
Binary Number.....	375
Int.....	375
GEOMETRY.....	375
JSON.....	376
Long.....	376
Null.....	376
Number.....	377
Numeric.....	377
String.....	377
UUID.....	378
Time.....	378
Time with time zone.....	379
Timestamp.....	379
Timestamp with time zone.....	379
Interval.....	380
INTERVAL YEAR.....	380
INTERVAL MONTH.....	380
INTERVAL DAY.....	380
INTERVAL HOUR.....	381
INTERVAL MINUTE.....	381
INTERVAL SECOND.....	381
INTERVAL YEAR TO MONTH.....	381
INTERVAL DAY TO HOUR.....	382
INTERVAL DAY TO MINUTE.....	382
INTERVAL DAY TO SECOND.....	382
INTERVAL HOUR TO MINUTE.....	382
INTERVAL HOUR TO SECOND.....	383
INTERVAL MINUTE TO SECOND.....	383
Datetime fields.....	383

Datetime field.....	383
Year field.....	384
Month field.....	384
Day of month field.....	384
Hour field.....	384
Minute field.....	385
Second field.....	385
Timezone hour field.....	385
Timezone minute field.....	385
Timezone second field.....	386
Millennium field.....	386
Century field.....	386
Decade field.....	386
Quarter field.....	386
Millisecond field.....	387
Microsecond field.....	387
Nanosecond field.....	387
Day of year field.....	387
ISO day of week field.....	388
ISO week field.....	388
ISO week year field.....	388
Day of week field.....	388
Week field.....	388
Week year field.....	389
Epoch field.....	389
Other Grammar.....	389
Alias.....	389
And Condition.....	390
Array element reference.....	390
Field reference.....	390
Array value constructor by query.....	390
Case expression.....	391
Simple case.....	391
Searched case.....	392
Cast specification.....	392
Cipher.....	393
Column Definition.....	394

Column Constraint Definition.....	395
Comment.....	396
Bracketed comment.....	396
Compare.....	396
Condition.....	397
Condition Right Hand Side.....	397
Comparison Right Hand Side.....	398
Quantified Comparison Right Hand Side.....	398
Null Predicate Right Hand Side.....	399
Distinct Predicate Right Hand Side.....	400
Quantified Distinct Predicate Right Hand Side.....	400
Boolean Test Right Hand Side.....	401
Type Predicate Right Hand Side.....	401
JSON Predicate Right Hand Side.....	401
Between Predicate Right Hand Side.....	402
In Predicate Right Hand Side.....	402
Like Predicate Right Hand Side.....	403
Regex Predicate Right Hand Side.....	403
Nulls Distinct.....	403
Table Constraint Definition.....	404
Constraint Name Definition.....	405
Csv Options.....	405
Data Change Delta Table.....	406
Data Type or Domain.....	406
Data Type.....	407
Predefined Type.....	407
Digit.....	407
Expression.....	407
Factor.....	408
Grouping element.....	408
Hex.....	408
Index Column.....	409
Insert values.....	409
Interval qualifier.....	409
Join specification.....	409
Merge when clause.....	410
Merge when matched clause.....	410

Merge when not matched clause.....	410
Name.....	411
Operand.....	411
Override clause.....	412
Query.....	412
Quoted Name.....	412
Referential Constraint.....	413
References Specification.....	413
Referential Triggered Action.....	414
Referential Action.....	414
Script Compression Encryption.....	415
Select order.....	415
Row value expression.....	415
Select Expression.....	416
Sequence value expression.....	416
Sequence option.....	416
Alter sequence option.....	417
Alter identity column option.....	417
Basic sequence option.....	418
Set clause list.....	419
Sort specification.....	419
Sort specification list.....	419
Summand.....	420
Table Expression.....	420
Update target.....	420
Within group specification.....	421
Wildcard expression.....	421
Window name or specification.....	421
Window specification.....	422
Window frame.....	422
Window frame preceding.....	423
Window frame bound.....	424
Term.....	424
Time zone.....	425
Column.....	425
System Tables.....	426
Index.....	426

Information Schema.....	427
CHECK_CONSTRAINTS.....	427
COLLATIONS.....	427
COLUMNS.....	428
COLUMN_PRIVILEGES.....	432
CONSTANTS.....	433
CONSTRAINT_COLUMN_USAGE.....	435
DOMAINS.....	435
DOMAIN_CONSTRAINTS.....	438
ELEMENT_TYPES.....	439
ENUM_VALUES.....	441
FIELDS.....	442
INDEXES.....	444
INDEX_COLUMNS.....	445
INFORMATION_SCHEMA_CATALOG_NAME.....	446
IN_DOUBT.....	446
KEY_COLUMN_USAGE.....	447
LOCKS.....	447
PARAMETERS.....	448
QUERY_STATISTICS.....	450
REFERENTIAL_CONSTRAINTS.....	451
RIGHTS.....	452
ROLES.....	453
ROUTINES.....	453
SCHEMATA.....	456
SEQUENCES.....	457
SESSIONS.....	458
SESSION_STATE.....	459
SETTINGS.....	460
SYNONYMS.....	460
TABLES.....	461
TABLE_CONSTRAINTS.....	462
TABLE_PRIVILEGES.....	463
TRIGGERS.....	464
USERS.....	465
VIEWS.....	465
Range Table.....	466

Build.....	468
Portability.....	468
Environment.....	468
Building the Software.....	468
Using Apache Lucene.....	469
Using Maven 2.....	469
Using a Central Repository.....	469
Maven Plugin to Start and Stop the TCP Server.....	469
Using Snapshot Version.....	470
Native Image.....	470
Using Eclipse.....	471
Translating.....	471
Submitting Source Code Changes.....	471
Reporting Problems or Requests.....	472
Automated Build.....	474
Generating Railroad Diagrams.....	474
History.....	475
Change Log.....	475
History of this Database Engine.....	475
Why Java.....	475
Supporters.....	476
Frequently Asked Questions.....	479
I Have a Problem or Feature Request.....	479
Are there Known Bugs? When is the Next Release?.....	479
Is this Database Engine Open Source?.....	480
Is Commercial Support Available?.....	480
How to Create a New Database?.....	480
How to Connect to a Database?.....	480
Where are the Database Files Stored?.....	481
What is the Size Limit (Maximum Size) of a Database?.....	481
Is it Reliable?.....	481
Why is Opening my Database Slow?.....	482
My Query is Slow.....	483
H2 is Very Slow.....	483
Column Names are Incorrect?.....	483
Float is Double?.....	484
How to Translate this Project?.....	484

How to Contribute to this Project?.....	484
---	-----

Quickstart

Embedding H2 in an Application

The H2 Console Application

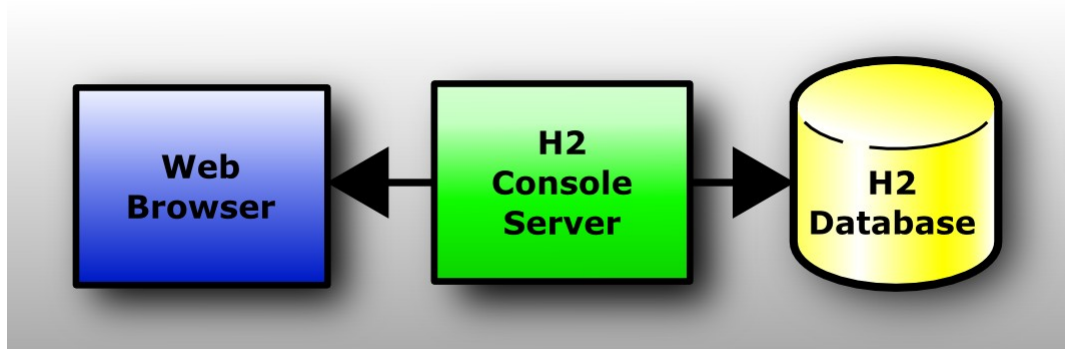
Embedding H2 in an Application

This database can be used in embedded mode, or in server mode. To use it in embedded mode, you need to:

- Add the h2*.jar to the classpath (H2 does not have any dependencies)
- Use the JDBC driver class: org.h2.Driver
- The database URL jdbc:h2:~/test opens the database test in your user home directory
- A new database is automatically created

The H2 Console Application

The Console lets you access a SQL database using a browser interface.



If you don't have Windows XP, or if something does not work as expected, please see the detailed description in the [Tutorial](#).

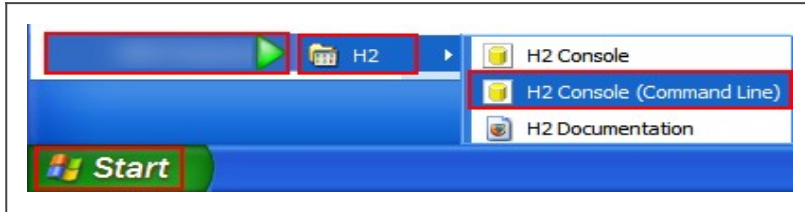
Step-by-Step

Installation

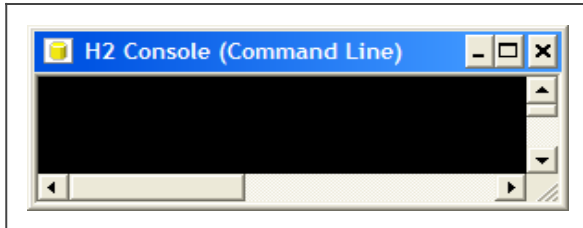
Install the software using the Windows Installer (if you did not yet do that).

Start the Console

Click [Start], [All Programs], [H2], and [H2 Console (Command Line)]:



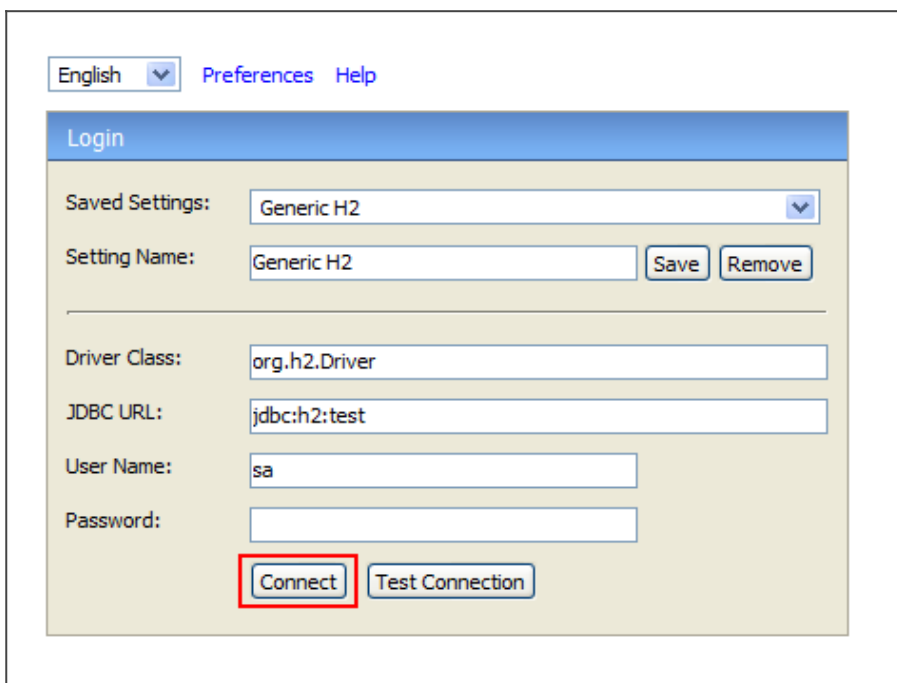
A new console window appears:



Also, a new browser page should open with the URL <http://localhost:8082>. You may get a security warning from the firewall. If you don't want other computers in the network to access the database on your machine, you can let the firewall block these connections. Only local connections are required at this time.

Login

Select [Generic H2] and click [Connect]:



You are now logged in.

Sample

Click on the [Sample SQL Script]:

The screenshot shows a database management tool interface. At the top, there is a toolbar with icons for connecting, executing, and other functions. Below the toolbar, the connection is set to 'jdbc:h2:test'. The left sidebar shows a tree view with 'INFORMATION_SCHEMA' and 'Users'. The main area contains a 'Run' button, a 'Clear' button, and a text area for the 'SQL statement:'. Below this, there is a section titled 'Important Commands' with a table of icons and actions. Further down, there is a section titled 'Sample SQL Script' with a table of operations and their corresponding SQL statements. The first row of the 'Sample SQL Script' table is highlighted with a red border.

jdbc:h2:test

Run Clear SQL statement:

INFORMATION_SCHEMA

Users

Important Commands

Icon	Action
?	Displays this Help Page
≡	Shows the Command History
▶	Executes the current SQL statement
🔌	Disconnects from the database

Sample SQL Script

Operations	SQL statements
Delete the table if it exists Create a new table with ID and NAME columns Add a new row Add another row Query the table Change data in a row Remove a row	DROP TABLE IF EXISTS TEST; CREATE TABLE TEST(ID INT PRIMARY KEY, NAME VARCHAR(255)); INSERT INTO TEST VALUES(1, 'Hello'); INSERT INTO TEST VALUES(2, 'World'); SELECT * FROM TEST ORDER BY ID; UPDATE TEST SET NAME='Hi' WHERE ID=1; DELETE FROM TEST WHERE ID=2;

The SQL commands appear in the command area.

Execute

Click [Run]

The screenshot shows a SQL IDE interface. At the top, there is a toolbar with icons for undo, redo, Autocommit (checked), Max Rows (set to 1000), and other controls. Below the toolbar, on the left, is a tree view showing the database structure: 'jdbc:h2:test', 'INFORMATION_SCHEMA', and 'Users'. The 'Run' button is highlighted with a red rectangle. To the right of the 'Run' button is a 'Clear' button and a text area for the 'SQL statement:'. The SQL statement area contains the following code:

```
DROP TABLE IF EXISTS TEST;  
CREATE TABLE TEST(ID INT PRIMARY KEY, NAME VARCHAR(255));  
INSERT INTO TEST VALUES(1, 'Hello');  
INSERT INTO TEST VALUES(2, 'World');  
SELECT * FROM TEST ORDER BY ID;  
UPDATE TEST SET NAME='Hi' WHERE ID=1;  
DELETE FROM TEST WHERE ID=2;
```

Below the SQL statement area, there is a section titled 'Important Commands' with a table of icons and their actions:

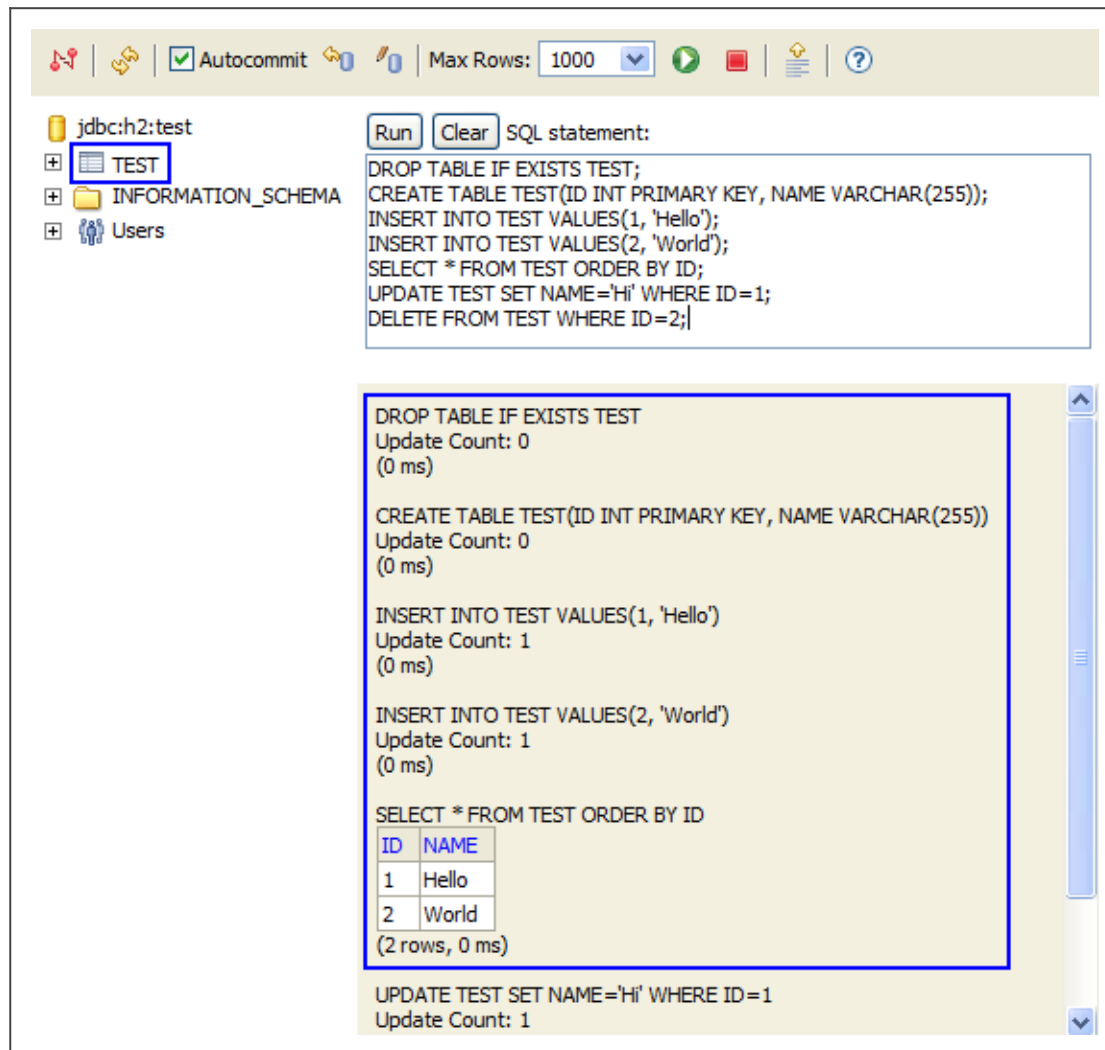
Icon	Action
?	Displays this Help Page
📜	Shows the Command History
▶	Executes the current SQL statement
🔌	Disconnects from the database

Below the 'Important Commands' section, there is a section titled 'Sample SQL Script' with a table of operations and their corresponding SQL statements:

Operations	SQL statements
Delete the table if it exists:	DROP TABLE IF EXISTS TEST;
Create a new table: with ID and NAME columns:	CREATE TABLE TEST(ID INT PRIMARY KEY, NAME VARCHAR(255));
Add a new row:	INSERT INTO TEST VALUES(1, 'Hello');
Add another row:	INSERT INTO TEST VALUES(2, 'World');
Query the table:	SELECT * FROM TEST ORDER BY ID;
Change data in a row:	UPDATE TEST SET NAME='Hi' WHERE ID=1;
Remove a row:	DELETE FROM TEST WHERE ID=2;

On the left side, a new entry TEST is added below the database icon. The

operations and results of the statements are shown below the script.



The screenshot shows a SQL IDE interface with a script execution window. The script contains the following SQL statements:

```
DROP TABLE IF EXISTS TEST;  
CREATE TABLE TEST(ID INT PRIMARY KEY, NAME VARCHAR(255));  
INSERT INTO TEST VALUES(1, 'Hello');  
INSERT INTO TEST VALUES(2, 'World');  
SELECT * FROM TEST ORDER BY ID;  
UPDATE TEST SET NAME='Hi' WHERE ID=1;  
DELETE FROM TEST WHERE ID=2;
```

The results of the execution are displayed below the script, showing update counts and a table of data:

```
DROP TABLE IF EXISTS TEST  
Update Count: 0  
(0 ms)  
  
CREATE TABLE TEST(ID INT PRIMARY KEY, NAME VARCHAR(255))  
Update Count: 0  
(0 ms)  
  
INSERT INTO TEST VALUES(1, 'Hello')  
Update Count: 1  
(0 ms)  
  
INSERT INTO TEST VALUES(2, 'World')  
Update Count: 1  
(0 ms)  
  
SELECT * FROM TEST ORDER BY ID  


| ID | NAME  |
|----|-------|
| 1  | Hello |
| 2  | World |

  
(2 rows, 0 ms)  
  
UPDATE TEST SET NAME='Hi' WHERE ID=1  
Update Count: 1
```

Disconnect

Click on [Disconnect]:



to close the connection.

End

Close the console window. For more information, see the [Tutorial](#).

Installation

[Requirements](#)

[Supported Platforms](#)

[Installing the Software](#)

[Directory Structure](#)

Requirements

To run this database, the following software stack is known to work. Other software most likely also works, but is not tested as much.

Database Engine

- Windows, Mac OS X, or Linux
- Java 11 or newer

H2 Console

- Mozilla Firefox

Supported Platforms

As this database is written in Java, it can run on many different platforms. It is tested with Java 11 and 17. All major operating systems (Windows, Mac OS X, Linux, ...) are supported.

Installing the Software

To install the software, run the installer or unzip it to a directory of your choice.

Directory Structure

After installing, you should get the following directory structure:

Directory	Contents
bin	JAR and batch files
docs	Documentation

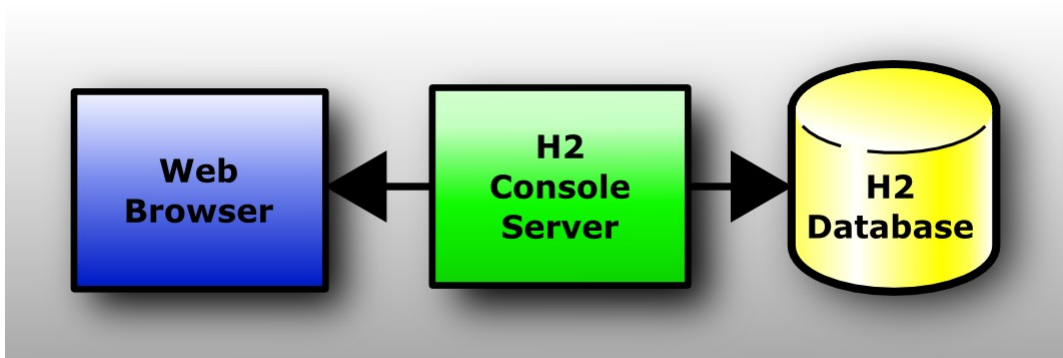
docs/html	HTML pages
docs/javadoc	Javadoc files
ext	External dependencies (downloaded when building)
service	Tools to run the database as a Windows Service
src	Source files
src/docsrc	Documentation sources
src/installer	Installer, shell, and release build script
src/main	Database engine source code
src/test	Test source code
src/tools	Tools and database adapters source code

Tutorial

- Starting and Using the H2 Console
- Special H2 Console Syntax
- Settings of the H2 Console
- Connecting to a Database using JDBC
- Creating New Databases
- Using the Server
- Using Hibernate
- Using TopLink and Glassfish
- Using EclipseLink
- Using Apache ActiveMQ
- Using H2 within NetBeans
- Using H2 with jOOQ
- Using Databases in Web Applications
- CSV (Comma Separated Values) Support
- Upgrade, Backup, and Restore
- Command Line Tools
- The Shell Tool
- Using OpenOffice Base
- Java Web Start / JNLP
- Using a Connection Pool
- Fulltext Search
- User-Defined Variables
- Date and Time
- Using Spring
- OSGi
- Java Management Extension (JMX)

Starting and Using the H2 Console

The H2 Console application lets you access a database using a browser. This can be a H2 database, or another database that supports the JDBC API.



This is a client/server application, so both a server and a client (a browser) are required to run it.

Depending on your platform and environment, there are multiple ways to start the H2 Console:

OS	Start
Windows	Click [Start], [All Programs], [H2], and [H2 Console (Command Line)] An icon will be added to the system tray:  If you don't get the window and the system tray icon, then maybe Java is not installed correctly (in this case, try another way to start the application). A browser window should open and point to the login page at http://localhost:8082.
Windows	Open a file browser, navigate to h2/bin, and double click on h2.bat. A console window appears. If there is a problem, you will see an error message in this window. A browser window will open and point to the login page (URL: http://localhost:8082).
Any	Double click on the h2*.jar file. This only works if the .jar suffix is associated with Java.
Any	Open a console window, navigate to the directory h2/bin, and type: <pre>java -jar h2*.jar</pre>

If the console startup procedure is unable to locate the default system web browser, an error message may be displayed. It is possible to explicitly tell H2 which program/script to use when opening a system web browser by setting either the BROWSER environment variable, or the h2.browser java property.

Firewall

If you start the server, you may get a security warning from the firewall (if you have installed one). If you don't want other computers in the network to access the application on your machine, you can let the firewall block those connections. The connection from the local machine will still work. Only if you want other computers to access the database on this computer, you need allow remote connections in the firewall.

It has been reported that when using Kaspersky 7.0 with firewall, the H2 Console is very slow when connecting over the IP address. A workaround is to connect using 'localhost'.

A small firewall is already built into the server: other computers may not connect to the server by default. To change this, go to 'Preferences' and select 'Allow connections from other computers'.

Testing Java

To find out which version of Java is installed, open a command prompt and type:

```
java -version
```

If you get an error message, you may need to add the Java binary directory to the path environment variable.

Error Message 'Port may be in use'

You can only start one instance of the H2 Console, otherwise you will get the following error message: "The Web server could not be started. Possible cause: another server is already running...". It is possible to start multiple console applications on the same computer (using different ports), but this is usually not required as the console supports multiple concurrent connections.

Using another Port

If the default port of the H2 Console is already in use by another application, then a different port needs to be configured. The settings are stored in a

properties file. For details, see [Settings of the H2 Console](#). The relevant entry is webPort.

If no port is specified for the TCP and PG servers, each service will try to listen on its default port. If the default port is already in use, a random port is used.

Connecting to the Server using a Browser

If the server started successfully, you can connect to it using a web browser. Javascript needs to be enabled. If you started the server on the same computer as the browser, open the URL `http://localhost:8082`. If you want to connect to the application from another computer, you need to provide the IP address of the server, for example: `http://192.168.0.2:8082`. If you enabled TLS on the server side, the URL needs to start with `https://`.

Multiple Concurrent Sessions

Multiple concurrent browser sessions are supported. As that the database objects reside on the server, the amount of concurrent work is limited by the memory available to the server application.

Login

At the login page, you need to provide connection information to connect to a database. Set the JDBC driver class of your database, the JDBC URL, user name, and password. If you are done, click [Connect].

You can save and reuse previously saved settings. The settings are stored in a properties file (see [Settings of the H2 Console](#)).

Error Messages

Error messages in are shown in red. You can show/hide the stack trace of the exception by clicking on the message.

Adding Database Drivers

To register additional JDBC drivers (MySQL, PostgreSQL, HSQLDB,...), add the jar file names to the environment variables H2DRIVERS or CLASSPATH. Example

(Windows): to add the HSQLDB JDBC driver C:\Programs\hsqldb\lib\hsqldb.jar, set the environment variable H2DRIVERS to C:\Programs\hsqldb\lib\hsqldb.jar.

Multiple drivers can be set; entries need to be separated by ; (Windows) or : (other operating systems). Spaces in the path names are supported. The settings must not be quoted.

Using the H2 Console

The H2 Console application has three main panels: the toolbar on top, the tree on the left, and the query/result panel on the right. The database objects (for example, tables) are listed on the left. Type a SQL command in the query panel and click [Run]. The result appears just below the command.

Inserting Table Names or Column Names

To insert table and column names into the script, click on the item in the tree. If you click on a table while the query is empty, then `SELECT * FROM ...` is added. While typing a query, the table that was used is expanded in the tree. For example if you type `SELECT * FROM TEST T WHERE T.` then the table TEST is expanded.

Disconnecting and Stopping the Application

To log out of the database, click [Disconnect] in the toolbar panel. However, the server is still running and ready to accept new sessions.

To stop the server, right click on the system tray icon and select [Exit]. If you don't have the system tray icon, navigate to [Preferences] and click [Shutdown], press [Ctrl]+[C] in the console where the server was started (Windows), or close the console window.

Special H2 Console Syntax

The H2 Console supports a few built-in commands. Those are interpreted within the H2 Console, so they work with any database. Built-in commands need to be at the beginning of a statement (before any remarks), otherwise they are not parsed correctly. If in doubt, add ; before the command.

Command(s)	Description
@autocommit_true; @autocommit_false;	Enable or disable autocommit.
@cancel;	Cancel the currently running statement.
@columns null null TEST; @index_info null null TEST; @tables; @tables null null TEST;	Call the corresponding DatabaseMetaData.get method. Patterns are case sensitive (usually identifiers are uppercase). For information about the parameters, see the Javadoc documentation. Missing parameters at the end of the line are set to null. The complete list of metadata commands is: @attributes, @best_row_identifier, @catalogs, @columns, @column_privileges, @cross_references, @exported_keys, @imported_keys, @index_info, @primary_keys, @procedures, @procedure_columns, @pseudo_columns, @schemas, @super_tables, @super_types, @tables, @table_privileges, @table_types, @type_info, @udts, @version_columns
@edit select * from test;	Use an updatable result set.
@generated insert into test() values(); @generated(1) insert into test() values(); @generated(ID, "TIMESTAMP") insert into test() values();	Show the result of Statement.getGeneratedKeys(). Names or one-based indexes of required columns can be optionally specified.
@history;	List the command history.
@info;	Display the result of various Connection and DatabaseMetaData methods.
@list select * from test;	Show the result set in list format (each column on its own line, with row numbers).
@loop 1000 select ?,	Run the statement this many times. Parameters

<code>?/*rnd*/; @loop 1000 @statement select ?;</code>	(?) are set using a loop from 0 up to x - 1. Random values are used for each ?/*rnd*/. A Statement object is used instead of a PreparedStatement if @statement is used. Result sets are read until ResultSet.next() returns false. Timing information is printed.
<code>@maxrows 20;</code>	Set the maximum number of rows to display.
<code>@memory;</code>	Show the used and free memory. This will call System.gc().
<code>@meta select 1;</code>	List the ResultSetMetaData after running the query.
<code>@parameter_meta select ?;</code>	Show the result of the PreparedStatement.getParameterMetaData() calls. The statement is not executed.
<code>@prof_start; call hash('SHA256', '', 100000 0); @prof_stop;</code>	Start/stop the built-in profiling tool. The top 3 stack traces of the statement(s) between start and stop are listed (if there are 3).
<code>@prof_start; @sleep 10; @prof_stop;</code>	Sleep for a number of seconds. Used to profile a long running query or operation that is running in another session (but in the same process).
<code>@transaction_isolation; @transaction_isolation 2;</code>	Display (without parameters) or change (with parameters 1, 2, 4, 8) the transaction isolation level.

Settings of the H2 Console

The settings of the H2 Console are stored in a configuration file called `.h2.server.properties` in your user home directory. For Windows installations, the user home directory is usually `C:\Documents and Settings\[username]` or `C:\Users\[username]`. The configuration file contains the settings of the application and is automatically created when the H2 Console is first started. Supported settings are:

- `webAllowOthers`: allow other computers to connect.

- webPort: the port of the H2 Console
- webSSL: use encrypted TLS (HTTPS) connections.
- webAdminPassword: hash of password to access preferences and tools of H2 Console, use
`org.h2.server.web.WebServer.encodeAdminPassword(String)` to generate a hash for your password. Always use long complex passwords, especially when access from other hosts is enabled.

In addition to those settings, the properties of the last recently used connection are listed in the form `<number>=<name>|<driver>|<url>|<user>` using the escape character `\`. Example: `1=Generic H2 (Embedded)|org.h2.Driver|jdbc\:h2\:~/test|sa`

Connecting to a Database using JDBC

To connect to a database, a Java application first needs to load the database driver, and then get a connection. A simple way to do that is using the following code:

```
import java.sql.*;
public class Test {
    public static void main(String[] a)
        throws Exception {
        Connection conn = DriverManager.
            getConnection("jdbc:h2:~/test", "sa", "");
        // add application code here
        conn.close();
    }
}
```

This code opens a connection (using `DriverManager.getConnection()`). The driver name is `"org.h2.Driver"`. The database URL always needs to start with `jdbc:h2:` to be recognized by this database. The second parameter in the `getConnection()` call is the user name (`sa` for System Administrator in this example). The third parameter is the password. In this database, user names are not case sensitive, but passwords are.

Creating New Databases

By default, if the database specified in the [embedded](#) URL does not yet exist, a new (empty) database is created automatically. The user that created the database automatically becomes the administrator of this database.

Auto-creation of databases can be disabled, see [Opening a Database Only if it Already Exists](#).

H2 Console does not allow creation of databases unless a browser window is opened by Console during its startup or from its icon in the system tray and remote access is not enabled. A context menu of the tray icon can also be used to create a new database.

You can also create a new local database from a command line with a Shell tool:

```
> java -cp h2-*.jar org.h2.tools.Shell

Welcome to H2 Shell
Exit with Ctrl+C
[Enter] jdbc:h2:mem:2
URL    jdbc:h2:./path/to/database
[Enter] org.h2.Driver
Driver
[Enter] sa
User   your_username
Password (hidden)
Type the same password again to confirm database creation.
Password (hidden)
Connected

sql> quit
Connection closed
```

By default remote creation of databases from a TCP connection or a web interface is not allowed. It's not recommended to enable remote creation of databases due to security reasons. User who creates a new database becomes its administrator and therefore gets the same access to your JVM as H2 has and

the same access to your operating system as Java and your system account allows. It's recommended to create all databases locally using an embedded URL, local H2 Console, or the Shell tool.

If you really need to allow remote database creation, you can pass `-ifNotExists` parameter to TCP, PG, or Web servers (but not to the Console tool). Its combination with `-tcpAllowOthers`, `-pgAllowOthers`, or `-webAllowOthers` effectively creates a remote security hole in your system, if you use it, always guard your ports with a firewall or some other solution and use such combination of settings only in trusted networks.

H2 Servlet also supports such option. When you use it always protect the servlet with security constraints, see [Using the H2 Console Servlet](#) for example; don't forget to uncomment and adjust security configuration for your needs.

Using the Server

H2 currently supports three server: a web server (for the H2 Console), a TCP server (for client/server connections) and an PG server (for PostgreSQL clients). Please note that only the web server supports browser connections. The servers can be started in different ways, one is using the Server tool. Starting the server doesn't open a database - databases are opened as soon as a client connects.

Starting the Server Tool from Command Line

To start the Server tool from the command line with the default settings, run:

```
java -cp h2*.jar org.h2.tools.Server
```

This will start the tool with the default options. To get the list of options and default values, run:

```
java -cp h2*.jar org.h2.tools.Server -?
```

There are options available to use other ports, and start or not start parts.

Connecting to the TCP Server

To remotely connect to a database using the TCP server, use the following driver and database URL:

- JDBC driver class: org.h2.Driver
- Database URL: jdbc:h2:tcp://localhost/~ /test

For details about the database URL, see also in Features. Please note that you can't connection with a web browser to this URL. You can only connect using a H2 client (over JDBC).

Starting the TCP Server within an Application

Servers can also be started and stopped from within an application. Sample code:

```
import org.h2.tools.Server;
...
// start the TCP Server
Server server = Server.createTcpServer(args).start();
...
// stop the TCP Server
server.stop();
```

Stopping a TCP Server from Another Process

The TCP server can be stopped from another process. To stop the server from the command line, run:

```
java org.h2.tools.Server -tcpShutdown tcp://localhost:9092 -tcpPassword
password
```

To stop the server from a user application, use the following code:

```
org.h2.tools.Server.shutdownTcpServer("tcp://localhost:9092", "password",
false, false);
```

This function will only stop the TCP server. If other server were started in the same process, they will continue to run. To avoid recovery when the databases

are opened the next time, all connections to the databases should be closed before calling this method. To stop a remote server, remote connections must be enabled on the server. Shutting down a TCP server is protected using the option `-tcpPassword` (the same password must be used to start and stop the TCP server).

Using Hibernate

This database supports Hibernate version 3.1 and newer. You can use the HSQLDB Dialect, or the native H2 Dialect.

When using Hibernate, try to use the H2Dialect if possible. When using the H2Dialect, compatibility modes such as `MODE=MySQL` are not supported. When using such a compatibility mode, use the Hibernate dialect for the corresponding database instead of the H2Dialect; but please note H2 does not support all features of all databases.

Using TopLink and Glassfish

To use H2 with Glassfish (or Sun AS), set the Datasource Classname to `org.h2.jdbcx.JdbcDataSource`. You can set this in the GUI at Application Server - Resources - JDBC - Connection Pools, or by editing the file `sun-resources.xml`: at element `jdbc-connection-pool`, set the attribute `datasource-classname` to `org.h2.jdbcx.JdbcDataSource`.

The H2 database is compatible with HSQLDB and PostgreSQL. To take advantage of H2 specific features, use the H2Platform. The source code of this platform is included in H2 at `src/tools/oracle/toplink/essentials/platform/database/DatabasePlatform.java.txt`. You will need to copy this file to your application, and rename it to `.java`. To enable it, change the following setting in `persistence.xml`:

```
<property
  name="toplink.target-database"
  value="oracle.toplink.essentials.platform.database.H2Platform"/>
```

In old versions of Glassfish, the property name is `toplink.platform.class.name`.

To use H2 within Glassfish, copy the h2*.jar to the directory glassfish/glassfish/lib.

Using EclipseLink

To use H2 in EclipseLink, use the platform class `org.eclipse.persistence.platform.database.H2Platform`. If this platform is not available in your version of EclipseLink, you can use the `OraclePlatform` instead in many case. See also [H2Platform](#).

Using Apache ActiveMQ

When using H2 as the backend database for Apache ActiveMQ, please use the `TransactDatabaseLocker` instead of the default locking mechanism. Otherwise the database file will grow without bounds. The problem is that the default locking mechanism uses an uncommitted UPDATE transaction, which keeps the transaction log from shrinking (causes the database file to grow). Instead of using an UPDATE statement, the `TransactDatabaseLocker` uses `SELECT ... FOR UPDATE` which is not problematic. To use it, change the ApacheMQ configuration element `<jdbcPersistenceAdapter>` element, property `databaseLocker="org.apache.activemq.store.jdbc.adapter.TransactDatabaseLocker"`. However, using the MVCC mode will again result in the same problem. Therefore, please do not use the MVCC mode in this case. Another (more dangerous) solution is to set `useDatabaseLock` to false.

Using H2 within NetBeans

There is a known issue when using the Netbeans SQL Execution Window: before executing a query, another query in the form `SELECT COUNT(*) FROM <query>` is run. This is a problem for queries that modify state, such as `SELECT NEXT VALUE FOR SEQ`. In this case, two sequence values are allocated instead of just one.

Using H2 with jOOQ

jOOQ adds a thin layer on top of JDBC, allowing for type-safe SQL construction, including advanced SQL, stored procedures and advanced data types. jOOQ takes your database schema as a base for code generation. If this is your example schema:

```
CREATE TABLE USER (ID INT, NAME VARCHAR(50));
```

then run the jOOQ code generator on the command line using this command:

```
java -cp jooq.jar;jooq-meta.jar;jooq-codegen.jar;h2-1.4.199.jar;.  
org.jooq.util.GenerationTool /codegen.xml
```

...where codegen.xml is on the classpath and contains this information

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>  
<configuration xmlns="http://www.jooq.org/xsd/jooq-codegen-3.11.0.xsd">  
  <jdbc>  
    <driver>org.h2.Driver</driver>  
    <url>jdbc:h2:~/test</url>  
    <user>sa</user>  
    <password></password>  
  </jdbc>  
  <generator>  
    <database>  
      <includes>.*</includes>  
      <excludes></excludes>  
      <inputSchema>PUBLIC</inputSchema>  
    </database>  
    <target>  
      <packageName>org.jooq.h2.generated</packageName>  
      <directory>./src</directory>  
    </target>  
  </generator>  
</configuration>
```

Using the generated source, you can query the database as follows:

```
DSLContext dsl = DSL.using(connection);
Result<UserRecord> result =
dsl.selectFrom(USER)
    .where(NAME.like("Johnny%"))
    .orderBy(ID)
    .fetch();
```

See more details on [jOOQ Homepage](#) and in the [jOOQ Tutorial](#)

Using Databases in Web Applications

There are multiple ways to access a database from within web applications. Here are some examples if you use Tomcat or JBoss.

Embedded Mode

The (currently) simplest solution is to use the database in the embedded mode, that means open a connection in your application when it starts (a good solution is using a Servlet Listener, see below), or when a session starts. A database can be accessed from multiple sessions and applications at the same time, as long as they run in the same process. Most Servlet Containers (for example Tomcat) are just using one process, so this is not a problem (unless you run Tomcat in clustered mode). Tomcat uses multiple threads and multiple classloaders. If multiple applications access the same database at the same time, you need to put the database jar in the shared/lib or server/lib directory. It is a good idea to open the database when the web application starts, and close it when the web application stops. If using multiple applications, only one (any) of them needs to do that. In the application, an idea is to use one connection per Session, or even one connection per request (action). Those connections should be closed after use if possible (but it's not that bad if they don't get closed).

Server Mode

The server mode is similar, but it allows you to run the server in another process.

Using a Servlet Listener to Start and Stop a Database

Add the h2*.jar file to your web application, and add the following snippet to your web.xml file (between the context-param and the filter section):

```
<listener>
  <listener-class>org.h2.server.web.DbStarter</listener-class>
</listener>
```

If your servlet container is already Servlet 5-compatible, use the following snippet instead:

```
<listener>
  <listener-class>org.h2.server.web.JakartaDbStarter</listener-class>
</listener>
```

For details on how to access the database, see the file DbStarter.java. By default this tool opens an embedded connection using the database URL jdbc:h2:~/test, user name sa, and password sa. If you want to use this connection within your servlet, you can access as follows:

```
Connection conn = getServletContext().getAttribute("connection");
```

DbStarter can also start the TCP server, however this is disabled by default. To enable it, use the parameter db.tcpServer in the file web.xml. Here is the complete list of options. These options need to be placed between the description tag and the listener / filter tags:

```
<context-param>
  <param-name>db.url</param-name>
  <param-value>jdbc:h2:~/test</param-value>
</context-param>
<context-param>
  <param-name>db.user</param-name>
  <param-value>sa</param-value>
</context-param>
<context-param>
  <param-name>db.password</param-name>
```

```
<param-value>sa</param-value>
</context-param>
<context-param>
  <param-name>db.tcpServer</param-name>
  <param-value>-tcpAllowOthers</param-value>
</context-param>
```

When the web application is stopped, the database connection will be closed automatically. If the TCP server is started within the DbStarter, it will also be stopped automatically.

Using the H2 Console Servlet

The H2 Console is a standalone application and includes its own web server, but it can be used as a servlet as well. To do that, include the h2*.jar file in your application, and add the following configuration to your web.xml:

```
<servlet>
  <servlet-name>H2Console</servlet-name>
  <servlet-class>org.h2.server.web.WebServlet</servlet-class>
  <!--
  <init-param>
    <param-name>webAllowOthers</param-name>
    <param-value></param-value>
  </init-param>
  <init-param>
    <param-name>trace</param-name>
    <param-value></param-value>
  </init-param>
  -->
  <load-on-startup>1</load-on-startup>
</servlet>
<servlet-mapping>
  <servlet-name>H2Console</servlet-name>
  <url-pattern>/console/*</url-pattern>
</servlet-mapping>
<!--
```

```
<security-role>
  <role-name>admin</role-name>
</security-role>
<security-constraint>
  <web-resource-collection>
    <web-resource-name>H2 Console</web-resource-name>
    <url-pattern>/console/*</url-pattern>
  </web-resource-collection>
  <auth-constraint>
    <role-name>admin</role-name>
  </auth-constraint>
</security-constraint>
-->
```

For details, see also `src/tools/WEB-INF/web.xml`.

If your application is already Servlet 5-compatible, use the servlet class `org.h2.server.web.JakartaWebServlet` instead.

To create a web application with just the H2 Console, run the following command:

```
build warConsole
```

CSV (Comma Separated Values) Support

The CSV file support can be used inside the database using the functions `CSVREAD` and `CSVWRITE`, or it can be used outside the database as a standalone tool.

Reading a CSV File from Within a Database

A CSV file can be read using the function `CSVREAD`. Example:

```
SELECT * FROM CSVREAD('test.csv');
```

Please note for performance reason, `CSVREAD` should not be used inside a join. Instead, import the data first (possibly into a temporary table), create the required indexes if necessary, and then query this table.

Importing Data from a CSV File

A fast way to load or import data (sometimes called 'bulk load') from a CSV file is to combine table creation with import. Optionally, the column names and data types can be set when creating the table. Another option is to use INSERT INTO ... SELECT.

```
CREATE TABLE TEST AS SELECT * FROM CSVREAD('test.csv');
CREATE TABLE TEST(ID INT PRIMARY KEY, NAME VARCHAR(255))
AS SELECT * FROM CSVREAD('test.csv');
```

Writing a CSV File from Within a Database

The built-in function CSVWRITE can be used to create a CSV file from a query. Example:

```
CREATE TABLE TEST(ID INT, NAME VARCHAR);
INSERT INTO TEST VALUES(1, 'Hello'), (2, 'World');
CALL CSVWRITE('test.csv', 'SELECT * FROM TEST');
```

Writing a CSV File from a Java Application

The Csv tool can be used in a Java application even when not using a database at all. Example:

```
import java.sql.*;
import org.h2.tools.Csv;
import org.h2.tools.SimpleResultSet;
public class TestCsv {
    public static void main(String[] args) throws Exception {
        SimpleResultSet rs = new SimpleResultSet();
        rs.addColumn("NAME", Types.VARCHAR, 255, 0);
        rs.addColumn("EMAIL", Types.VARCHAR, 255, 0);
        rs.addRow("Bob Meier", "bob.meier@abcde.abc");
        rs.addRow("John Jones", "john.jones@abcde.abc");
        new Csv().write("data/test.csv", rs, null);
    }
}
```

Reading a CSV File from a Java Application

It is possible to read a CSV file without opening a database. Example:

```
import java.sql.*;
import org.h2.tools.Csv;
public class TestCsv {
    public static void main(String[] args) throws Exception {
        ResultSet rs = new Csv().read("data/test.csv", null, null);
        ResultSetMetaData meta = rs.getMetaData();
        while (rs.next()) {
            for (int i = 0; i < meta.getColumnCount(); i++) {
                System.out.println(
                    meta.getColumnLabel(i + 1) + ": " +
                    rs.getString(i + 1));
            }
            System.out.println();
        }
        rs.close();
    }
}
```

Upgrade, Backup, and Restore

Database Upgrade

The recommended way to upgrade from one version of the database engine to the next version is to create a backup of the database (in the form of a SQL script) using the old engine, and then execute the SQL script using the new engine.

Backup using the Script Tool

The recommended way to backup a database is to create a compressed SQL script file. This will result in a small, human readable, and database version independent backup. Creating the script will also verify the checksums of the database file. The Script tool is ran as follows:

```
java org.h2.tools.Script -url jdbc:h2:~/test -user sa -script test.zip -options
```



```
compression zip
```

It is also possible to use the SQL command SCRIPT to create the backup of the database. For more information about the options, see the SQL command SCRIPT. The backup can be done remotely, however the file will be created on the server side. The built in FTP server could be used to retrieve the file from the server.

Restore from a Script

To restore a database from a SQL script file, you can use the RunScript tool:

```
java org.h2.tools.RunScript -url jdbc:h2:~/test -user sa -script test.zip -options  
compression zip
```

For more information about the options, see the SQL command RUNSCRIPT. The restore can be done remotely, however the file needs to be on the server side. The built in FTP server could be used to copy the file to the server. It is also possible to use the SQL command RUNSCRIPT to execute a SQL script. SQL script files may contain references to other script files, in the form of RUNSCRIPT commands. However, when using the server mode, the references script files need to be available on the server side.

If the script was generated by H2 1.4.200 or an older version, add VARIABLE_BINARY option to import it into more recent version.

```
java org.h2.tools.RunScript -url jdbc:h2:~/test -user sa -script test.zip -options  
compression zip variable_binary
```

Online Backup

The BACKUP SQL statement and the Backup tool both create a zip file with the database file. However, the contents of this file are not human readable.

The resulting backup is transactionally consistent, meaning the consistency and atomicity rules apply.

```
BACKUP TO 'backup.zip'
```

The Backup tool (org.h2.tools.Backup) can not be used to create a online backup; the database must not be in use while running this program.

Creating a backup by copying the database files while the database is running is not supported, except if the file systems support creating snapshots. With other file systems, it can't be guaranteed that the data is copied in the right order.

Command Line Tools

This database comes with a number of command line tools. To get more information about a tool, start it with the parameter '-?', for example:

```
java -cp h2*.jar org.h2.tools.Backup -?
```

The command line tools are:

- Backup creates a backup of a database.
- ChangeFileEncryption allows changing the file encryption password or algorithm of a database.
- Console starts the browser based H2 Console.
- ConvertTraceFile converts a .trace.db file to a Java application and SQL script.
- CreateCluster creates a cluster from a standalone database.
- DeleteDbFiles deletes all files belonging to a database.
- Recover helps recovering a corrupted database.
- Restore restores a backup of a database.
- RunScript runs a SQL script against a database.
- Script allows converting a database to a SQL script for backup or migration.
- Server is used in the server mode to start a H2 server.
- Shell is a command line database tool.

The tools can also be called from an application by calling the main or another public method. For details, see the Javadoc documentation.

The Shell Tool

The Shell tool is a simple interactive command line tool. To start it, type:

```
java -cp h2*.jar org.h2.tools.Shell
```

You will be asked for a database URL, JDBC driver, user name, and password. The connection setting can also be set as command line parameters. After connecting, you will get the list of options. The built-in commands don't need to end with a semicolon, but SQL statements are only executed if the line ends with a semicolon ;. This allows to enter multi-line statements:

```
sql> select * from test  
...> where id = 0;
```

By default, results are printed as a table. For results with many column, consider using the list mode:

```
sql> list  
Result list mode is now on  
sql> select * from test;  
ID : 1  
NAME: Hello  
  
ID : 2  
NAME: World  
(2 rows, 0 ms)
```

Using OpenOffice Base

OpenOffice.org Base supports database access over the JDBC API. To connect to a H2 database using OpenOffice Base, you first need to add the JDBC driver to OpenOffice. The steps to connect to a H2 database are:

- Start OpenOffice Writer, go to [Tools], [Options]
- Make sure you have selected a Java runtime environment in OpenOffice.org / Java
- Click [Class Path...], [Add Archive...]

- Select your h2 jar file (location is up to you, could be wherever you choose)
- Click [OK] (as much as needed), stop OpenOffice (including the Quickstarter)
- Start OpenOffice Base
- Connect to an existing database; select [JDBC]; [Next]
- Example datasource URL: jdbc:h2:~/test
- JDBC driver class: org.h2.Driver

Now you can access the database stored in the current users home directory.

To use H2 in NeoOffice (OpenOffice without X11):

- In NeoOffice, go to [NeoOffice], [Preferences]
- Look for the page under [NeoOffice], [Java]
- Click [Class Path], [Add Archive...]
- Select your h2 jar file (location is up to you, could be wherever you choose)
- Click [OK] (as much as needed), restart NeoOffice.

Now, when creating a new database using the "Database Wizard" :

- Click [File], [New], [Database].
- Select [Connect to existing database] and the select [JDBC]. Click next.
- Example datasource URL: jdbc:h2:~/test
- JDBC driver class: org.h2.Driver

Another solution to use H2 in NeoOffice is:

- Package the h2 jar within an extension package
- Install it as a Java extension in NeoOffice

This can be done by create it using the NetBeans OpenOffice plugin. See also [Extensions Development](#).

Java Web Start / JNLP

When using Java Web Start / JNLP (Java Network Launch Protocol), permissions tags must be set in the .jnlp file, and the application .jar file must be signed.

Otherwise, when trying to write to the file system, the following exception will

occur: java.security.AccessControlException: access denied
(java.io.FilePermission ... read). Example permission tags:

```
<security>
  <all-permissions/>
</security>
```

Using a Connection Pool

For H2, opening a connection is fast if the database is already open. Still, using a connection pool improves performance if you open and close connections a lot. A simple connection pool is included in H2. It is based on the [Mini Connection Pool Manager](#) from Christian d'Heureuse. There are other, more complex, open source connection pools available, for example the [Apache Commons DBCP](#). For H2, it is about twice as faster to get a connection from the built-in connection pool than to get one using `DriverManager.getConnection()`. The build-in connection pool is used as follows:

```
import java.sql.*;
import org.h2.jdbcx.JdbcConnectionPool;
public class Test {
    public static void main(String[] args) throws Exception {
        JdbcConnectionPool cp = JdbcConnectionPool.create(
            "jdbc:h2:~/test", "sa", "sa");
        for (int i = 0; i < args.length; i++) {
            Connection conn = cp.getConnection();
            conn.createStatement().execute(args[i]);
            conn.close();
        }
        cp.dispose();
    }
}
```

Fulltext Search

H2 includes two fulltext search implementations. One is using Apache Lucene, and the other (the native implementation) stores the index data in special tables in the database.

Using the Native Fulltext Search

To initialize, call:

```
CREATE ALIAS IF NOT EXISTS FT_INIT FOR "org.h2.fulltext.FullText.init";  
CALL FT_INIT();
```

You need to initialize it in each database where you want to use it. Afterwards, you can create a fulltext index for a table using:

```
CREATE TABLE TEST(ID INT PRIMARY KEY, NAME VARCHAR);  
INSERT INTO TEST VALUES(1, 'Hello World');  
CALL FT_CREATE_INDEX('PUBLIC', 'TEST', NULL);
```

PUBLIC is the schema name, TEST is the table name. The list of column names (comma separated) is optional, in this case all columns are indexed. The index is updated in realtime. To search the index, use the following query:

```
SELECT * FROM FT_SEARCH('Hello', 0, 0);
```

This will produce a result set that contains the query needed to retrieve the data:

```
QUERY: "PUBLIC"."TEST" WHERE "ID"=1
```

To drop an index on a table:

```
CALL FT_DROP_INDEX('PUBLIC', 'TEST');
```

To get the raw data, use FT_SEARCH_DATA('Hello', 0, 0);. The result contains the columns SCHEMA (the schema name), TABLE (the table name), COLUMNS (an array of column names), and KEYS (an array of objects). To join a table, use a

join as in: `SELECT T.* FROM FT_SEARCH_DATA('Hello', 0, 0) FT, TEST T WHERE FT."TABLE"='TEST' AND T.ID=FT."KEYS"[1];`

You can also call the index from within a Java application:

```
org.h2.fulltext.FullText.search(conn, text, limit, offset);
org.h2.fulltext.FullText.searchData(conn, text, limit, offset);
```

Using the Apache Lucene Fulltext Search

To use the Apache Lucene full text search, you need the Lucene library in the classpath. Apache Lucene 9.7.0 or binary compatible version is required. How to do that depends on the application; if you use the H2 Console, you can add the Lucene jar file to the environment variables H2DRIVERS or CLASSPATH. To initialize the Lucene fulltext search in a database, call:

```
CREATE ALIAS IF NOT EXISTS FTL_INIT FOR "org.h2.fulltext.FullTextLucene.init";
CALL FTL_INIT();
```

You need to initialize it in each database where you want to use it. Afterwards, you can create a full text index for a table using:

```
CREATE TABLE TEST(ID INT PRIMARY KEY, NAME VARCHAR);
INSERT INTO TEST VALUES(1, 'Hello World');
CALL FTL_CREATE_INDEX('PUBLIC', 'TEST', NULL);
```

PUBLIC is the schema name, TEST is the table name. The list of column names (comma separated) is optional, in this case all columns are indexed. The index is updated in realtime. To search the index, use the following query:

```
SELECT * FROM FTL_SEARCH('Hello', 0, 0);
```

This will produce a result set that contains the query needed to retrieve the data:

```
QUERY: "PUBLIC"."TEST" WHERE "ID"=1
```

To drop an index on a table (be warned that this will re-index all of the full-text indices for the entire database):

```
CALL FTL_DROP_INDEX('PUBLIC', 'TEST');
```

To get the raw data, use `FTL_SEARCH_DATA('Hello', 0, 0);`. The result contains the columns `SCHEMA` (the schema name), `TABLE` (the table name), `COLUMNS` (an array of column names), and `KEYS` (an array of objects). To join a table, use a join as in: `SELECT T.* FROM FTL_SEARCH_DATA('Hello', 0, 0) FT, TEST T WHERE FT."TABLE"='TEST' AND T.ID=FT."KEYS"[1];`

You can also call the index from within a Java application:

```
org.h2.fulltext.FullTextLucene.search(conn, text, limit, offset);  
org.h2.fulltext.FullTextLucene.searchData(conn, text, limit, offset);
```

The Lucene fulltext search supports searching in specific column only. Column names must be uppercase (except if the original columns are double quoted). For column names starting with an underscore (`_`), another underscore needs to be added. Example:

```
CREATE ALIAS IF NOT EXISTS FTL_INIT FOR "org.h2.fulltext.FullTextLucene.init";  
CALL FTL_INIT();  
DROP TABLE IF EXISTS TEST;  
CREATE TABLE TEST(ID INT PRIMARY KEY, FIRST_NAME VARCHAR, LAST_NAME  
VARCHAR);  
CALL FTL_CREATE_INDEX('PUBLIC', 'TEST', NULL);  
INSERT INTO TEST VALUES(1, 'John', 'Wayne');  
INSERT INTO TEST VALUES(2, 'Elton', 'John');  
SELECT * FROM FTL_SEARCH_DATA('John', 0, 0);  
SELECT * FROM FTL_SEARCH_DATA('LAST_NAME:John', 0, 0);  
CALL FTL_DROP_ALL();
```

User-Defined Variables

This database supports user-defined variables. Variables start with `@` and can be used wherever expressions or parameters are allowed. Variables are not persisted and session scoped, that means only visible from within the session in which they are defined. A value is usually assigned using the `SET` command:


```
SET @USER = 'Joe';
```

The value can also be changed using the SET() method. This is useful in queries:

```
SET @TOTAL = NULL;  
SELECT X, SET(@TOTAL, COALESCE(@TOTAL, 1.) * X) F FROM SYSTEM_RANGE(1,  
50);
```

Variables that are not set evaluate to NULL. The data type of a user-defined variable is the data type of the value assigned to it, that means it is not necessary (or possible) to declare variable names before using them. There are no restrictions on the assigned values; large objects (LOBs) are supported as well. Rolling back a transaction does not affect the value of a user-defined variable.

Date and Time

Date, time and timestamp values support standard literals:

```
VALUES (  
    DATE '2008-01-01',  
    TIME '12:00:00',  
    TIME WITH TIME ZONE '12:00:00+01:00',  
    TIMESTAMP '2008-01-01 12:00:00',  
    TIMESTAMP WITH TIME ZONE '2008-01-01 12:00:00+01:00'  
);
```

ISO 8601-style datetime formats with T instead of space between date and time parts are also supported.

TIME and TIMESTAMP values are preserved without time zone information as local time. That means if you store the value '2000-01-01 12:00:00' in one time zone, then change time zone of the session you will also get '2000-01-01 12:00:00', the value will not be adjusted to the new time zone, therefore its absolute value in UTC may be different.

TIME WITH TIME ZONE and TIMESTAMP WITH TIME ZONE values preserve the specified time zone offset and if you store the value '2008-01-01 12:00:00+01:00'

it also remains the same even if you change time zone of the session, and because it has a time zone offset its absolute value in UTC will be the same. `TIMESTAMP WITH TIME ZONE` values may be also specified with time zone name like '2008-01-01 12:00:00 Europe/Berlin'. In that case this name will be converted into time zone offset. Names of time zones are not stored.

Using Spring

Using the TCP Server

Use the following configuration to start and stop the H2 TCP server using the Spring Framework:

```
<bean id = "org.h2.tools.Server"
    class="org.h2.tools.Server"
    factory-method="createTcpServer"
    init-method="start"
    destroy-method="stop">
    <constructor-arg value="-tcp,-tcpAllowOthers,-tcpPort,8043" />
</bean>
```

The `destroy-method` will help prevent exceptions on hot-redployment or when restarting the server.

OSGi

The standard H2 jar can be dropped in as a bundle in an OSGi container. H2 implements the JDBC Service defined in OSGi Service Platform Release 4 Version 4.2 Enterprise Specification. The H2 Data Source Factory service is registered with the following properties: `OSGI_JDBC_DRIVER_CLASS=org.h2.Driver` and `OSGI_JDBC_DRIVER_NAME=H2 JDBC Driver`. The `OSGI_JDBC_DRIVER_VERSION` property reflects the version of the driver as is.

The following standard configuration properties are supported: `JDBC_USER`, `JDBC_PASSWORD`, `JDBC_DESCRIPTION`, `JDBC_DATASOURCE_NAME`, `JDBC_NETWORK_PROTOCOL`, `JDBC_URL`, `JDBC_SERVER_NAME`,

JDBC_PORT_NUMBER. Any other standard property will be rejected. Non-standard properties will be passed on to H2 in the connection URL.

Java Management Extension (JMX)

Management over JMX is supported, but not enabled by default. To enable JMX, append ;JMX=TRUE to the database URL when opening the database. Various tools support JMX, one such tool is the jconsole. When opening the jconsole, connect to the process where the database is open (when using the server mode, you need to connect to the server process). Then go to the MBeans section. Under org.h2 you will find one entry per database. The object name of the entry is the database short name, plus the path (each colon is replaced with an underscore character).

The following attributes and operations are supported:

- CacheSize: the cache size currently in use in KB.
- CacheSizeMax (read/write): the maximum cache size in KB.
- Exclusive: whether this database is open in exclusive mode or not.
- FileReadCount: the number of file read operations since the database was opened.
- FileSize: the file size in KB.
- FileWriteCount: the number of file write operations since the database was opened.
- FileWriteCountTotal: the number of file write operations since the database was created.
- LogMode (read/write): the current transaction log mode. See SET LOG for details.
- Mode: the compatibility mode (REGULAR if no compatibility mode is used).
- MultiThreaded: true if multi-threaded is enabled.
- Mvcc: true if MVCC is enabled.
- ReadOnly: true if the database is read-only.
- TraceLevel (read/write): the file trace level.
- Version: the database version in use.
- listSettings: list the database settings.

- `listSessions`: list the open sessions, including currently executing statement (if any) and locked tables (if any).

To enable JMX, you may need to set the system properties `com.sun.management.jmxremote` and `com.sun.management.jmxremote.port` as required by the JVM.

Features

[Feature List](#)

[H2 in Use](#)

[Connection Modes](#)

[Database URL Overview](#)

[Connecting to an Embedded \(Local\) Database](#)

[In-Memory Databases](#)

[Database Files Encryption](#)

[Database File Locking](#)

[Opening a Database Only if it Already Exists](#)

[Closing a Database](#)

[Ignore Unknown Settings](#)

[Changing Other Settings when Opening a Connection](#)

[Custom File Access Mode](#)

[Multiple Connections](#)

[Database File Layout](#)

[Logging and Recovery](#)

[Compatibility](#)

[Auto-Reconnect](#)

[Automatic Mixed Mode](#)

[Page Size](#)

[Using the Trace Options](#)

[Using Other Logging APIs](#)

[Read Only Databases](#)

[Read Only Databases in Zip or Jar File](#)

[Generated Columns \(Computed Columns\) / Function Based Index](#)

[Multi-Dimensional Indexes](#)

[User-Defined Functions and Stored Procedures](#)

[Pluggable or User-Defined Tables](#)

[Triggers](#)

[Compacting a Database](#)

Feature List

Main Features

- Very fast database engine
- Open source
- Written in Java
- Supports standard SQL, JDBC API
- Embedded and Server mode, Clustering support
- Strong security features
- The PostgreSQL ODBC driver can be used
- Multi version concurrency

Additional Features

- Disk based or in-memory databases and tables, read-only database support, temporary tables
- Transaction support (read uncommitted, read committed, repeatable read, snapshot), 2-phase-commit
- Multiple connections, row-level locking
- Cost based optimizer, using a genetic algorithm for complex queries, zero-administration
- Scrollable and updatable result set support, large result set, external result sorting, functions can return a result set
- Encrypted database (AES), SHA-256 password encryption, encryption functions, SSL

SQL Support

- Support for multiple schemas, information schema
- Referential integrity / foreign key constraints with cascade, check constraints
- Inner and outer joins, subqueries, read only views and inline views

- Triggers and Java functions / stored procedures
- Many built-in functions, including XML and lossless data compression
- Wide range of data types including large objects (BLOB/CLOB) and arrays
- Sequences and identity columns, generated columns (can be used for function based indexes)
- ORDER BY, GROUP BY, HAVING, UNION, OFFSET / FETCH (including PERCENT and WITH TIES), LIMIT, TOP, DISTINCT / DISTINCT ON (...)
- Window functions
- Collation support, including support for the ICU4J library
- Support for users and roles
- Compatibility modes for IBM DB2, Apache Derby, HSQLDB, MS SQL Server, MySQL, Oracle, and PostgreSQL.

Security Features

- Includes a solution for the SQL injection problem
- User password authentication uses SHA-256 and salt
- For server mode connections, user passwords are never transmitted in plain text over the network (even when using insecure connections; this only applies to the TCP server and not to the H2 Console however; it also doesn't apply if you set the password in the database URL)
- All database files (including script files that can be used to backup data) can be encrypted using the AES-128 encryption algorithm
- The remote JDBC driver supports TCP/IP connections over TLS
- The built-in web server supports connections over TLS
- Passwords can be sent to the database using char arrays instead of Strings

Other Features and Tools

- Small footprint (around 2.5 MB), low memory requirements
- Multiple index types (b-tree, tree, hash)
- Support for multi-dimensional indexes
- CSV (comma separated values) file support
- Support for linked tables, and a built-in virtual 'range' table

- Supports the EXPLAIN PLAN statement; sophisticated trace options
- Database closing can be delayed or disabled to improve the performance
- Web-based Console application (translated to many languages) with autocomplete
- The database can generate SQL script files
- Contains a recovery tool that can dump the contents of the database
- Support for variables (for example to calculate running totals)
- Automatic re-compilation of prepared statements
- Uses a small number of database files
- Uses a checksum for each record and log entry for data integrity
- Well tested (high code coverage, randomized stress tests)

H2 in Use

For a list of applications that work with or use H2, see: [Links](#).

Connection Modes

The following connection modes are supported:

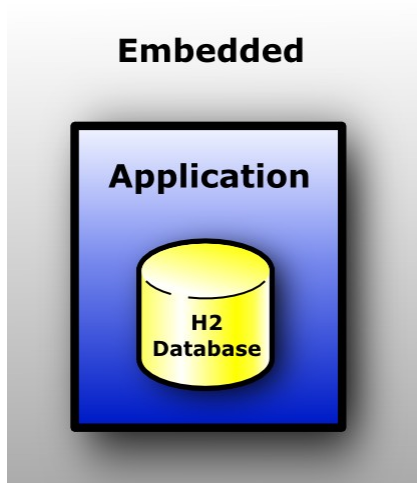
- Embedded mode (local connections using JDBC)
- Server mode (remote connections using JDBC or ODBC over TCP/IP)
- Mixed mode (local and remote connections at the same time)

Embedded Mode

In embedded mode, an application opens a database from within the same JVM using JDBC. This is the fastest and easiest connection mode. The disadvantage is that a database may only be open in one virtual machine (and class loader) at any time. As in all modes, both persistent and in-memory databases are supported. There is no limit on the number of database open concurrently, or on the number of open connections.

In embedded mode I/O operations can be performed by application's threads that execute a SQL command. The application may not interrupt these threads, it can lead to database corruption, because JVM closes I/O handle during thread interruption. Consider other ways to control execution of your application.

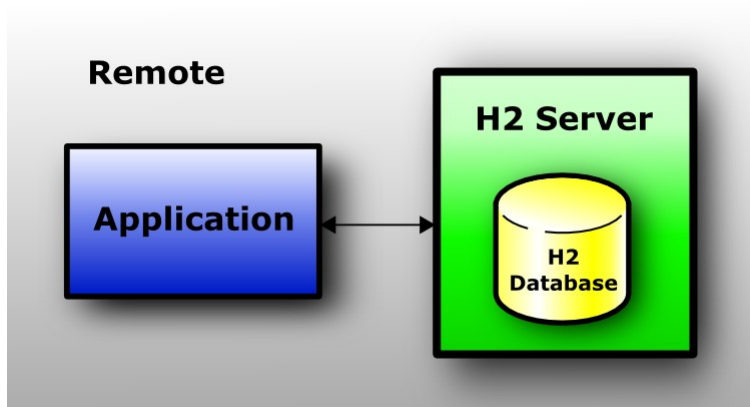
When interrupts are possible the [async](#) file system can be used as a workaround, but full safety is not guaranteed. It's recommended to use the client-server model instead, the client side may interrupt own threads.



Server Mode

When using the server mode (sometimes called remote mode or client/server mode), an application opens a database remotely using the JDBC or ODBC API. A server needs to be started within the same or another virtual machine, or on another computer. Many applications can connect to the same database at the same time, by connecting to this server. Internally, the server process opens the database(s) in embedded mode.

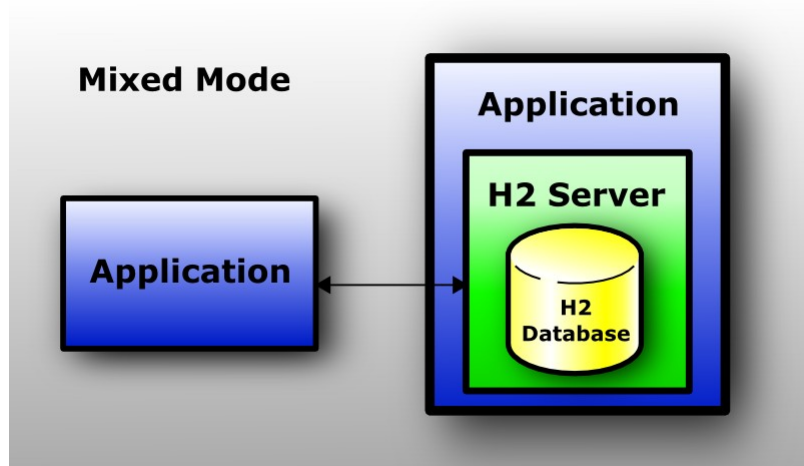
The server mode is slower than the embedded mode, because all data is transferred over TCP/IP. As in all modes, both persistent and in-memory databases are supported. There is no limit on the number of database open concurrently per server, or on the number of open connections.



Mixed Mode

The mixed mode is a combination of the embedded and the server mode. The first application that connects to a database does that in embedded mode, but also starts a server so that other applications (running in different processes or virtual machines) can concurrently access the same data. The local connections are as fast as if the database is used in just the embedded mode, while the remote connections are a bit slower.

The server can be started and stopped from within the application (using the server API), or automatically (automatic mixed mode). When using the [automatic mixed mode](#), all clients that want to connect to the database (no matter if it's an local or remote connection) can do so using the exact same database URL.



Database URL Overview

This database supports multiple connection modes and connection settings. This is achieved using different database URLs. Settings in the URLs are not case sensitive.

Topic	URL Format and Examples
Embedded (local) connection	<code>jdbc:h2:[file:][<path>]<databaseName></code> <code>jdbc:h2:~/test</code> <code>jdbc:h2:file:/data/sample</code> <code>jdbc:h2:file:C:/data/sample (Windows only)</code>

In-memory (private)	jdbc:h2:mem:
In-memory (named)	jdbc:h2:mem:<databaseName> jdbc:h2:mem:test_mem
Server mode (remote connections) using TCP/IP	jdbc:h2:tcp://<server>[:<port>]/ [<path>]<databaseName> jdbc:h2:tcp://localhost/~/test jdbc:h2:tcp://dbserve:8084/~/sample jdbc:h2:tcp://localhost/mem:test
Server mode (remote connections) using TLS	jdbc:h2:ssl://<server>[:<port>]/ [<path>]<databaseName> jdbc:h2:ssl://localhost:8085/~/sample;
Using encrypted files	jdbc:h2:<url>;CIPHER=AES jdbc:h2:ssl://localhost/~/test;CIPHER=AES jdbc:h2:file:~/secure;CIPHER=AES
File locking methods	jdbc:h2:<url>;FILE_LOCK={FILE SOCKET FS NO} jdbc:h2:file:~/private;CIPHER=AES;FILE_LOCK=SOCKET
Only open if it already exists	jdbc:h2:<url>;IFEXISTS=TRUE jdbc:h2:file:~/sample;IFEXISTS=TRUE
Don't close the database when the VM exits	jdbc:h2:<url>;DB_CLOSE_ON_EXIT=FALSE
Execute SQL on connection	jdbc:h2:<url>;INIT=RUNSCRIPT FROM '~/create.sql' jdbc:h2:file:~/sample;INIT=RUNSCRIPT FROM '~/create.sql';RUNSCRIPT FROM '~/populate.sql'
User name and/or password	jdbc:h2:<url>;[USER=<username>] [;PASSWORD=<value>] jdbc:h2:file:~/sample;USER=sa;PASSWORD=123
Debug trace settings	jdbc:h2:<url>;TRACE_LEVEL_FILE=<level 0..3> jdbc:h2:file:~/sample;TRACE_LEVEL_FILE=3
Ignore unknown settings	jdbc:h2:<url>;IGNORE_UNKNOWN_SETTINGS=TRUE
Custom file access mode	jdbc:h2:<url>;ACCESS_MODE_DATA=rws

Database in a zip file	jdbc:h2:zip:<zipFileName>!/<databaseName> jdbc:h2:zip:~/db.zip!/test
Compatibility mode	jdbc:h2:<url>;MODE=<databaseType> jdbc:h2:~/test;MODE=MYSQL;DATABASE_TO_LOWER=TRUE
Auto-reconnect	jdbc:h2:<url>;AUTO_RECONNECT=TRUE jdbc:h2:tcp://localhost/~/test;AUTO_RECONNECT=TRUE
Automatic mixed mode	jdbc:h2:<url>;AUTO_SERVER=TRUE jdbc:h2:~/test;AUTO_SERVER=TRUE
Page size	jdbc:h2:<url>;PAGE_SIZE=512
Changing other settings	jdbc:h2:<url>;<setting>=<value>[;<setting>=<value>.. .] jdbc:h2:file:~/sample;TRACE_LEVEL_SYSTEM_OUT=3

Connecting to an Embedded (Local) Database

The database URL for connecting to a local database is jdbc:h2:[file:] [<path>]<databaseName>. The prefix file: is optional. If no or only a relative path is used, then the current working directory is used as a starting point. The case sensitivity of the path and database name depend on the operating system, however it is recommended to use lowercase letters only. The database name must be at least three characters long (a limitation of File.createTempFile). The database name must not contain a semicolon. To point to the user home directory, use ~/, as in: jdbc:h2:~/test.

In-Memory Databases

For certain use cases (for example: rapid prototyping, testing, high performance operations, read-only databases), it may not be required to persist data, or persist changes to the data. This database supports the in-memory mode, where the data is not persisted.

In some cases, only one connection to a in-memory database is required. This means the database to be opened is private. In this case, the database URL is

`jdbc:h2:mem:` Opening two connections within the same virtual machine means opening two different (private) databases.

Sometimes multiple connections to the same in-memory database are required. In this case, the database URL must include a name. Example:

`jdbc:h2:mem:db1`. Accessing the same database using this URL only works within the same virtual machine and class loader environment.

To access an in-memory database from another process or from another computer, you need to start a TCP server in the same process as the in-memory database was created. The other processes then need to access the database over TCP/IP or TLS, using a database URL such as:

`jdbc:h2:tcp://localhost/mem:db1`.

By default, closing the last connection to a database closes the database. For an in-memory database, this means the content is lost. To keep the database open, add `;DB_CLOSE_DELAY=-1` to the database URL. To keep the content of an in-memory database as long as the virtual machine is alive, use `jdbc:h2:mem:test;DB_CLOSE_DELAY=-1`. This may create a memory leak, when you need to remove the database, use the [SHUTDOWN](#) command.

Database Files Encryption

The database files can be encrypted. Three encryption algorithms are supported:

- "AES" - also known as Rijndael, only AES-128 is implemented.
- "XTEA" - the 32 round version.
- "FOG" - pseudo-encryption only useful for hiding data from a text editor.

To use file encryption, you need to specify the encryption algorithm (the 'cipher') and the file password (in addition to the user password) when connecting to the database.

Creating a New Database with File Encryption

By default, a new database is automatically created if it does not exist yet when the [embedded](#) url is used. To create an encrypted database, connect to it as it would already exist locally using the embedded URL.

Connecting to an Encrypted Database

The encryption algorithm is set in the database URL, and the file password is specified in the password field, before the user password. A single space separates the file password and the user password; the file password itself may not contain spaces. File passwords and user passwords are case sensitive. Here is an example to connect to a password-encrypted database:

```
String url = "jdbc:h2:~/test;CIPHER=AES";  
String user = "sa";  
String pwds = "filepwd userpwd";  
conn = DriverManager.  
    getConnection(url, user, pwds);
```

Encrypting or Decrypting a Database

To encrypt an existing database, use the ChangeFileEncryption tool. This tool can also decrypt an encrypted database, or change the file encryption key. The tool is available from within the H2 Console in the tools section, or you can run it from the command line. The following command line will encrypt the database test in the user home directory with the file password filepwd and the encryption algorithm AES:

```
java -cp h2*.jar org.h2.tools.ChangeFileEncryption -dir ~ -db test -cipher AES  
-encrypt filepwd
```

Database File Locking

Whenever a database is opened, a lock file is created to signal other processes that the database is in use. If database is closed, or if the process that opened the database terminates, this lock file is deleted.

The following file locking methods are implemented:

- The default method is FILE and uses a watchdog thread to protect the database file. The watchdog reads the lock file each second.
- The second method is SOCKET and opens a server socket. The socket method does not require reading the lock file every second. The socket method should only be used if the database files are only accessed by one (and always the same) computer.
- The third method is FS. This will use native file locking using FileChannel.lock.
- It is also possible to open the database without file locking; in this case it is up to the application to protect the database files. Failing to do so will result in a corrupted database. Using the method NO forces the database to not create a lock file at all. Please note that this is unsafe as another process is able to open the same database, possibly leading to data corruption.

To open the database with a different file locking method, use the parameter FILE_LOCK. The following code opens the database with the 'socket' locking method:

```
String url = "jdbc:h2:~/test;FILE_LOCK=SOCKET";
```

For more information about the algorithms, see [Advanced / File Locking Protocols](#).

Opening a Database Only if it Already Exists

By default, when an application calls DriverManager.getConnection(url, ...) with [embedded](#) URL and the database specified in the URL does not yet exist, a new (empty) database is created. In some situations, it is better to restrict creating new databases, and only allow to open existing databases. To do this, add ;IFEXISTS=TRUE to the database URL. In this case, if the database does not already exist, an exception is thrown when trying to connect. The connection only succeeds when the database already exists. The complete URL may look like this:

```
String url = "jdbc:h2:/data/sample;IFEXISTS=TRUE";
```

Closing a Database

Delayed Database Closing

Usually, a database is closed when the last connection to it is closed. In some situations this slows down the application, for example when it is not possible to keep at least one connection open. The automatic closing of a database can be delayed or disabled with the SQL statement `SET DB_CLOSE_DELAY <seconds>`. The parameter `<seconds>` specifies the number of seconds to keep a database open after the last connection to it was closed. The following statement will keep a database open for 10 seconds after the last connection was closed:

```
SET DB_CLOSE_DELAY 10
```

The value `-1` means the database is not closed automatically. The value `0` is the default and means the database is closed when the last connection is closed. This setting is persistent and can be set by an administrator only. It is possible to set the value in the database URL: `jdbc:h2:~/test;DB_CLOSE_DELAY=10`.

Don't Close a Database when the VM Exits

By default, a database is closed when the last connection is closed. However, if it is never closed, a persistent database is closed when the virtual machine exits normally, using a shutdown hook. In some situations, the database should not be closed in this case, for example because the database is still used at virtual machine shutdown (to store the shutdown process in the database for example). For those cases, the automatic closing of the database can be disabled in the database URL. The first connection (the one that is opening the database) needs to set the option in the database URL (it is not possible to change the setting afterwards). The database URL to disable database closing on exit is:

```
String url = "jdbc:h2:~/test;DB_CLOSE_ON_EXIT=FALSE";
```


Warning: when database closing on exit is disabled, an application **must** execute the **SHUTDOWN** command by itself in its own shutdown hook after completion of all operations with database to avoid data loss and should not try to establish new connections to database after that.

Execute SQL on Connection

Sometimes, particularly for in-memory databases, it is useful to be able to execute DDL or DML commands automatically when a client connects to a database. This functionality is enabled via the INIT property. Note that multiple commands may be passed to INIT, but the semicolon delimiter must be escaped, as in the example below.

```
String url = "jdbc:h2:mem:test;INIT=runscript from '~/create.sql'\\;runscript from '~/init.sql'";
```

Please note the double backslash is only required in a Java or properties file. In a GUI, or in an XML file, only one backslash is required:

```
<property name="url" value="
jdbc:h2:mem:test;INIT=create schema if not exists test\;runscript from
'~/sql/init.sql'
"/>
```

Backslashes within the init script (for example within a runscript statement, to specify the folder names in Windows) need to be escaped as well (using a second backslash). It might be simpler to avoid backslashes in folder names for this reason; use forward slashes instead.

Ignore Unknown Settings

Some applications (for example OpenOffice.org Base) pass some additional parameters when connecting to the database. Why those parameters are passed is unknown. The parameters **PREFERDOSLIKELINEENDS** and **IGNOREDRIVERPRIVILEGES** are such examples; they are simply ignored to improve the compatibility with OpenOffice.org. If an application passes other parameters when connecting to the database, usually the database throws an

exception saying the parameter is not supported. It is possible to ignore such parameters by adding `;IGNORE_UNKNOWN_SETTINGS=TRUE` to the database URL.

Changing Other Settings when Opening a Connection

In addition to the settings already described, other database settings can be passed in the database URL. Adding `;setting=value` at the end of a database URL is the same as executing the statement `SET setting value` just after connecting. For a list of supported settings, see [SQL Grammar](#) or the [DbSettings](#) javadoc.

Custom File Access Mode

Usually, the database opens the database file with the access mode `rw`, meaning read-write (except for read only databases, where the mode `r` is used). To open a database in read-only mode if the database file is not read-only, use `ACCESS_MODE_DATA=r`. Also supported are `rws` and `rwd`. This setting must be specified in the database URL:

```
String url = "jdbc:h2:~/test;ACCESS_MODE_DATA=rws";
```

For more information see [Durability Problems](#). On many operating systems the access mode `rws` does not guarantee that the data is written to the disk.

Multiple Connections

Opening Multiple Databases at the Same Time

An application can open multiple databases at the same time, including multiple connections to the same database. The number of open database is only limited by the memory available.

Multiple Connections to the Same Database: Client/Server

If you want to access the same database at the same time from different processes or computers, you need to use the client / server mode. In this case, one process acts as the server, and the other processes (that could reside on

other computers as well) connect to the server via TCP/IP (or TLS over TCP/IP for improved security).

Multithreading Support

This database is multithreading-safe. If an application is multi-threaded, it does not need to worry about synchronizing access to the database. An application should normally use one connection per thread. This database synchronizes access to the same connection, but other databases may not do this. To get higher concurrency, you need to use multiple connections.

An application can use multiple threads that access the same database at the same time. Threads that use different connections can use the database concurrently.

Locking, Lock-Timeout, Deadlocks

Usually, SELECT statements will generate read locks. This includes subqueries. Statements that modify data use write locks on the modified rows. It is also possible to issue write locks without modifying data, using the statement SELECT ... FOR UPDATE. Data definition statements may issue exclusive locks on tables. The statements COMMIT and ROLLBACK releases all open locks. The commands SAVEPOINT and ROLLBACK TO SAVEPOINT don't affect locks. The locks are also released when the autocommit mode changes, and for connections with autocommit set to true (this is the default), locks are released after each statement. The following statements generate locks:

Type of Lock	SQL Statement
Read	SELECT * FROM TEST; CALL SELECT MAX(ID) FROM TEST; SCRIPT;
Write (row-level)	SELECT * FROM TEST WHERE 1=0 FOR UPDATE;
Write (row-level)	INSERT INTO TEST VALUES(1, 'Hello'); INSERT INTO TEST SELECT * FROM TEST; UPDATE TEST SET NAME='Hi'; DELETE FROM TEST;

Exclusive	ALTER TABLE TEST ...; CREATE INDEX ... ON TEST ...; DROP INDEX ...;
-----------	---

The number of seconds until a lock timeout exception is thrown can be set separately for each connection using the SQL command SET LOCK_TIMEOUT <milliseconds>. The initial lock timeout (that is the timeout used for new connections) can be set using the SQL command SET DEFAULT_LOCK_TIMEOUT <milliseconds>. The default lock timeout is persistent.

Database File Layout

The following files are created for persistent databases:

File Name	Description	Number of Files
test.mv.db	Database file. Contains the transaction log, indexes, and data for all tables. Format: <database>.mv.db	1 per database
test.newFile	Temporary file for database compaction. Contains the new MVStore file. Format: <database>.newFile	0 or 1 per database
test.tempFile	Temporary file for database compaction. Contains the temporary MVStore file. Format: <database>.tempFile	0 or 1 per database
test.lock.db	Database lock file. Automatically (re-)created while the database is in use. Format: <database>.lock.db	1 per database (only if in use)
test.trace.db	Trace file (if the trace option is enabled). Contains trace information. Format: <database>.trace.db Renamed to <database>.trace.db.old if	0 or 1 per database

	too big.	
test.123.temp.db	Temporary file. Contains a temporary blob or a large result set. Format: <database>.<id>.temp.db	1 per object

Moving and Renaming Database Files

Database name and location are not stored inside the database files.

While a database is closed, the files can be moved to another directory, and they can be renamed as well (as long as all files of the same database start with the same name and the respective extensions are unchanged).

As there is no platform specific data in the files, they can be moved to other operating systems without problems.

Backup

When the database is closed, it is possible to backup the database files.

To backup data while the database is running, the SQL commands SCRIPT and BACKUP can be used.

Logging and Recovery

Whenever data is modified in the database and those changes are committed, the changes are written to the transaction log (except for in-memory objects). The changes to the main data area itself are usually written later on, to optimize disk access. If there is a power failure, the main data area is not up-to-date, but because the changes are in the transaction log, the next time the database is opened, the changes are re-applied automatically.

Compatibility

All database engines behave a little bit different. Where possible, H2 supports the ANSI SQL standard, and tries to be compatible to other databases. There are still a few differences however:

In MySQL text columns are case insensitive by default, while in H2 they are case sensitive. However H2 supports case insensitive columns as well. To create the tables with case insensitive texts, append IGNORECASE=TRUE to the database URL (example: jdbc:h2:~/test;IGNORECASE=TRUE).

Compatibility Modes

For certain features, this database can emulate the behavior of specific databases. However, only a small subset of the differences between databases are implemented in this way. Here is the list of currently supported modes and the differences to the regular mode:

REGULAR Compatibility mode

This mode is used by default.

- Empty IN predicate is allowed.
- TOP clause in SELECT is allowed.
- OFFSET/LIMIT clauses are allowed.
- MINUS can be used instead of EXCEPT.
- IDENTITY can be used as a data type.
- Legacy SERIAL and BIGSERIAL data types are supported.
- AUTO_INCREMENT clause can be used instead of GENERATED { ALWAYS | BY DEFAULT } AS IDENTITY.

STRICT Compatibility Mode

To use the STRICT mode, use the database URL jdbc:h2:~/test;MODE=STRICT or the SQL statement SET MODE STRICT. In this mode some deprecated features are disabled.

If your application or library uses only the H2 or it generates different SQL for different database systems it is recommended to use this compatibility mode in unit tests to reduce possibility of accidental misuse of such features. This mode cannot be used as SQL validator, however.

It is not recommended to enable this mode in production builds of libraries, because this mode may become more restrictive in future releases of H2 that may break your library if it will be used together with newer version of H2.

- Empty IN predicate is disallowed.
- TOP and OFFSET/LIMIT clauses are disallowed, only OFFSET/FETCH can be used.
- MINUS cannot be used instead of EXCEPT.
- IDENTITY cannot be used as a data type and AUTO_INCREMENT clause cannot be specified. Use GENERATED BY DEFAULT AS IDENTITY clause instead.
- SERIAL and BIGSERIAL data types are disallowed. Use INTEGER GENERATED BY DEFAULT AS IDENTITY or BIGINT GENERATED BY DEFAULT AS IDENTITY instead.

LEGACY Compatibility Mode

To use the LEGACY mode, use the database URL `jdbc:h2:~/test;MODE=LEGACY` or the SQL statement `SET MODE LEGACY`. In this mode some compatibility features for applications written for H2 1.X are enabled. This mode doesn't provide full compatibility with H2 1.X.

- Empty IN predicate is allowed.
- TOP clause in SELECT is allowed.
- OFFSET/LIMIT clauses are allowed.
- MINUS can be used instead of EXCEPT.
- IDENTITY can be used as a data type.
- MS SQL Server-style IDENTITY clause is supported.
- Legacy SERIAL and BIGSERIAL data types are supported.
- SMALLDATETIME, DATETIME, and DATETIME2 data types are treated like TIMESTAMP data type.
- AUTO_INCREMENT clause can be used instead of GENERATED { ALWAYS | BY DEFAULT } AS IDENTITY.
- If a value for identity column was specified in an INSERT command the base value of sequence generator of this column is updated if current

value of generator was smaller (larger for generators with negative increment) than the inserted value.

- Identity columns have implicit DEFAULT ON NULL clause. It means a NULL value may be specified for this column in INSERT command and it will be treated as DEFAULT.
- Oracle-style CURRVAL and NEXTVAL can be used on sequences.
- TOP clause can be used in DELETE and UPDATE.
- Non-standard Oracle-style WHERE clause can be used in standard MERGE command.
- Attempt to reference a non-unique set of columns from a referential constraint will create an UNIQUE constraint on them automatically.
- Unsafe comparison operators between numeric and boolean values are allowed.
- GREATEST and LEAST ignore NULL values by default.
- IDENTITY() and SCOPE_IDENTITY() are supported, but both are implemented like SCOPE_IDENTITY()
- SYSDATE, SYSTIMESTAMP, and TODAY are supported.

DB2 Compatibility Mode

To use the IBM DB2 mode, use the database URL

`jdbc:h2:~/test;MODE=DB2;DEFAULT_NULL_ORDERING=HIGH` or the SQL statement `SET MODE DB2`.

- For aliased columns, `ResultSetMetaData.getColumnNames()` returns the alias name and `getTableName()` returns null.
- Support the pseudo-table `SYSIBM.SYSDUMMY1`.
- Timestamps with dash between date and time are supported.
- Datetime value functions return the same value within a command.
- Second and third arguments of `TRANSLATE()` function are swapped.
- `SEQUENCE.NEXTVAL` and `SEQUENCE.CURRVAL` are supported
- `LIMIT / OFFSET` clauses are supported.
- `MINUS` can be used instead of `EXCEPT`.

- Unsafe comparison operators between numeric and boolean values are allowed.

Derby Compatibility Mode

To use the Apache Derby mode, use the database URL

`jdbc:h2:~/test;MODE=Derby;DEFAULT_NULL_ORDERING=HIGH` or the SQL statement `SET MODE Derby`.

- For aliased columns, `ResultSetMetaData.getColumnNames()` returns the alias name and `getTableNames()` returns null.
- For unique indexes, NULL is distinct. That means only one row with NULL in one of the columns is allowed.
- Support the pseudo-table `SYSIBM.SYSDUMMY1`.
- Datetime value functions return the same value within a command.

HSQLDB Compatibility Mode

To use the HSQLDB mode, use the database URL

`jdbc:h2:~/test;MODE=HSQLDB;DEFAULT_NULL_ORDERING=FIRST` or the SQL statement `SET MODE HSQLDB`.

- Text can be concatenated using `'+'`.
- NULL value works like DEFAULT value is assignments to identity columns.
- Datetime value functions return the same value within a command.
- TOP clause in SELECT is supported.
- LIMIT / OFFSET clauses are supported.
- MINUS can be used instead of EXCEPT.
- Unsafe comparison operators between numeric and boolean values are allowed.
- SYSDATE and TODAY are supported.

MS SQL Server Compatibility Mode

To use the MS SQL Server mode, use the database URL

`jdbc:h2:~/test;MODE=MSSQLServer;DATABASE_TO_UPPER=FALSE;CASE_INSENSITIVE`

TIVE_IDENTIFIERS=TRUE. Do not change value of DATABASE_TO_LOWER and CASE_INSENSITIVE_IDENTIFIERS after creation of database.

- For aliased columns, ResultSetMetaData.getColumnNames() returns the alias name and getTableName() returns null.
- Identifiers may be quoted using square brackets as in [Test].
- For unique indexes, NULL is distinct. That means only one row with NULL in one of the columns is allowed.
- GREATEST and LEAST ignore NULL values by default.
- Text can be concatenated using '+'.
- Arguments of LOG() function are swapped.
- SMALLDATETIME, DATETIME, and DATETIME2 data types are treated like TIMESTAMP data type. DATETIMEOFFSET data type is treated like TIMESTAMP WITH TIME ZONE data type.
- MONEY data type is treated like NUMERIC(19, 4) data type. SMALLMONEY data type is treated like NUMERIC(10, 4) data type.
- IDENTITY can be used for automatic id generation on column level.
- Table hints are discarded. Example: SELECT * FROM table WITH (NOLOCK).
- Datetime value functions return the same value within a command.
- 0x literals are parsed as binary string literals.
- TRUNCATE TABLE restarts next values of generated columns.
- TOP clause in SELECT, UPDATE, and DELETE is supported.
- Unsafe comparison operators between numeric and boolean values are allowed.

MariaDB Compatibility Mode

To use the MariaDB mode, use the database URL

jdbc:h2:~/test;MODE=MariaDB;DATABASE_TO_LOWER=TRUE. When case-insensitive identifiers are needed append

;CASE_INSENSITIVE_IDENTIFIERS=TRUE to URL. Do not change value of DATABASE_TO_LOWER after creation of database.

- Creating indexes in the CREATE TABLE statement is allowed using INDEX(..) or KEY(..). Example: create table test(id int primary key, name varchar(255), key idx_name(name));
- When converting a floating point number to an integer, the fractional digits are not truncated, but the value is rounded.
- ON DUPLICATE KEY UPDATE is supported in INSERT statements, due to this feature VALUES has special non-standard meaning in some contexts.
- INSERT IGNORE is partially supported and may be used to skip rows with duplicate keys if ON DUPLICATE KEY UPDATE is not specified.
- REPLACE INTO is partially supported.
- Spaces are trimmed from the right side of CHAR values.
- REGEXP_REPLACE() uses \ for back-references.
- Datetime value functions return the same value within a command.
- 0x literals are parsed as binary string literals.
- Unrelated expressions in ORDER BY clause of DISTINCT queries are allowed.
- Some MariaDB-specific ALTER TABLE commands are partially supported.
- TRUNCATE TABLE restarts next values of generated columns.
- NEXT VALUE FOR returns different values when invoked multiple times within the same row.
- If value of an identity column was manually specified, its sequence is updated to generate values after inserted.
- NULL value works like DEFAULT value in assignments to identity columns.
- LIMIT / OFFSET clauses are supported.
- AUTO_INCREMENT clause can be used.
- DATETIME data type is treated like TIMESTAMP data type.
- YEAR data type is treated like SMALLINT data type.
- GROUP BY clause can contain 1-based positions of expressions from the SELECT list.
- Unsafe comparison operators between numeric and boolean values are allowed.

- Accepts non-standard JSON_OBJECT and JSON_OBJECTAGG syntax using comma as key/value separator.

Text comparison in MariaDB is case insensitive by default, while in H2 it is case sensitive (as in most other databases). H2 does support case insensitive text comparison, but it needs to be set separately, using SET IGNORECASE TRUE. This affects comparison using =, LIKE, REGEXP.

MySQL Compatibility Mode

To use the MySQL mode, use the database URL `jdbc:h2:~/test;MODE=MySQL;DATABASE_TO_LOWER=TRUE`. When case-insensitive identifiers are needed append `;CASE_INSENSITIVE_IDENTIFIERS=TRUE` to URL. Do not change value of `DATABASE_TO_LOWER` after creation of database.

- Creating indexes in the CREATE TABLE statement is allowed using INDEX(..) or KEY(..). Example: `create table test(id int primary key, name varchar(255), key idx_name(name));`
- When converting a floating point number to an integer, the fractional digits are not truncated, but the value is rounded.
- ON DUPLICATE KEY UPDATE is supported in INSERT statements, due to this feature VALUES has special non-standard meaning in some contexts.
- INSERT IGNORE is partially supported and may be used to skip rows with duplicate keys if ON DUPLICATE KEY UPDATE is not specified.
- REPLACE INTO is partially supported.
- Spaces are trimmed from the right side of CHAR values.
- REGEXP_REPLACE() uses \ for back-references.
- Datetime value functions return the same value within a command.
- 0x literals are parsed as binary string literals.
- Unrelated expressions in ORDER BY clause of DISTINCT queries are allowed.
- Some MySQL-specific ALTER TABLE commands are partially supported.
- TRUNCATE TABLE restarts next values of generated columns.

- If value of an identity column was manually specified, its sequence is updated to generate values after inserted.
- NULL value works like DEFAULT value is assignments to identity columns.
- Referential constraints don't require an existing primary key or unique constraint on referenced columns and create a unique constraint automatically if such constraint doesn't exist.
- LIMIT / OFFSET clauses are supported.
- AUTO_INCREMENT clause can be used.
- DATETIME data type is treated like TIMESTAMP data type.
- YEAR data type is treated like SMALLINT data type.
- GROUP BY clause can contain 1-based positions of expressions from the SELECT list.
- Unsafe comparison operators between numeric and boolean values are allowed.
- Accepts non-standard JSON_OBJECT and JSON_OBJECTAGG syntax using comma as key/value separator.

Text comparison in MySQL is case insensitive by default, while in H2 it is case sensitive (as in most other databases). H2 does support case insensitive text comparison, but it needs to be set separately, using SET IGNORECASE TRUE. This affects comparison using =, LIKE, REGEXP.

Oracle Compatibility Mode

To use the Oracle mode, use the database URL `jdbc:h2:~/test;MODE=Oracle;DEFAULT_NULL_ORDERING=HIGH` or the SQL statement `SET MODE Oracle`.

- For aliased columns, `ResultSetMetaData.getColumnNames()` returns the alias name and `getTableName()` returns null.
- When using unique indexes, multiple rows with NULL in all columns are allowed, however it is not allowed to have multiple rows with the same values otherwise.
- Empty strings are treated like NULL values, concatenating NULL with another value results in the other value.

- REGEXP_REPLACE() uses \ for back-references.
- RAWTOHEX() converts character strings to hexadecimal representation of their UTF-8 encoding.
- HEXTORAW() decodes a hexadecimal character string to a binary string.
- DATE data type is treated like TIMESTAMP(0) data type.
- Datetime value functions return the same value within a command.
- ALTER TABLE MODIFY COLUMN command is partially supported.
- SEQUENCE.NEXTVAL and SEQUENCE.CURRVAL are supported and return values with DECIMAL/NUMERIC data type.
- Merge when matched clause may have WHERE clause.
- MINUS can be used instead of EXCEPT.
- SYSDATE and SYSTIMESTAMP are supported.

PostgreSQL Compatibility Mode

To use the PostgreSQL mode, use the database URL `jdbc:h2:~/test;MODE=PostgreSQL;DATABASE_TO_LOWER=TRUE;DEFAULT_NULL_ORDERING=HIGH`. Do not change value of DATABASE_TO_LOWER after creation of database.

- For aliased columns, `ResultSetMetaData.getColumnNames()` returns the alias name and `getTableNames()` returns null.
- When converting a floating point number to an integer, the fractional digits are not be truncated, but the value is rounded.
- The system columns `ctid` and `oid` are supported.
- GREATEST and LEAST ignore NULL values by default.
- LOG(x) is base 10 in this mode.
- REGEXP_REPLACE():
 - uses \ for back-references;
 - does not throw an exception when the flagsString parameter contains a 'g';
 - replaces only the first matched substring in the absence of the 'g' flag in the flagsString parameter.
- LIMIT / OFFSET clauses are supported.

- Legacy SERIAL and BIGSERIAL data types are supported.
- ON CONFLICT DO NOTHING is supported in INSERT statements.
- Spaces are trimmed from the right side of CHAR values, but CHAR values in result sets are right-padded with spaces to the declared length.
- NUMERIC and DECIMAL/DEC data types without parameters are treated like DECFLOAT data type.
- MONEY data type is treated like NUMERIC(19, 2) data type.
- Datetime value functions return the same value within a transaction.
- ARRAY_SLICE() out of bounds parameters are silently corrected.
- EXTRACT function with DOW field returns (0-6), Sunday is 0.
- UPDATE with FROM is partially supported.
- GROUP BY clause can contain 1-based positions of expressions from the SELECT list.

Auto-Reconnect

The auto-reconnect feature causes the JDBC driver to reconnect to the database if the connection is lost. The automatic re-connect only occurs when auto-commit is enabled; if auto-commit is disabled, an exception is thrown. To enable this mode, append ;AUTO_RECONNECT=TRUE to the database URL.

Re-connecting will open a new session. After an automatic re-connect, variables and local temporary tables definitions (excluding data) are re-created. The contents of the system table INFORMATION_SCHEMA.SESSION_STATE contains all client side state that is re-created.

If another connection uses the database in exclusive mode (enabled using SET EXCLUSIVE 1 or SET EXCLUSIVE 2), then this connection will try to re-connect until the exclusive mode ends.

Automatic Mixed Mode

Multiple processes can access the same database without having to start the server manually. To do that, append ;AUTO_SERVER=TRUE to the database URL. You can use the same database URL independent of whether the database is

already open or not. This feature doesn't work with in-memory databases.
Example database URL:

```
jdbc:h2:/data/test;AUTO_SERVER=TRUE
```

Use the same URL for all connections to this database. Internally, when using this mode, the first connection to the database is made in embedded mode, and additionally a server is started internally (as a daemon thread). If the database is already open in another process, the server mode is used automatically. The IP address and port of the server are stored in the file `.lock.db`, that's why in-memory databases can't be supported.

The application that opens the first connection to the database uses the embedded mode, which is faster than the server mode. Therefore the main application should open the database first if possible. The first connection automatically starts a server on a random port. This server allows remote connections, however only to this database (to ensure that, the client reads `.lock.db` file and sends the random key that is stored there to the server). When the first connection is closed, the server stops. If other (remote) connections are still open, one of them will then start a server (auto-reconnect is enabled automatically).

All processes need to have access to the database files. If the first connection is closed (the connection that started the server), open transactions of other connections will be rolled back (this may not be a problem if you don't disable autocommit). Explicit client/server connections (using `jdbc:h2:tcp://` or `ssl://`) are not supported. This mode is not supported for in-memory databases.

Here is an example how to use this mode. Application 1 and 2 are not necessarily started on the same computer, but they need to have access to the database files. Application 1 and 2 are typically two different processes (however they could run within the same process).

```
// Application 1:  
DriverManager.getConnection("jdbc:h2:/data/test;AUTO_SERVER=TRUE");
```



```
// Application 2:  
DriverManager.getConnection("jdbc:h2:/data/test;AUTO_SERVER=TRUE");
```

When using this feature, by default the server uses any free TCP port. The port can be set manually using `AUTO_SERVER_PORT=9090`.

Page Size

The page size for new databases is 4 KiB (4096 bytes), unless the page size is set explicitly in the database URL using `PAGE_SIZE=` when the database is created. The page size of existing databases can not be changed, so this property needs to be set when the database is created. The page size of encrypted databases must be a multiple of 4096 (4096, 8192, ...).

Using the Trace Options

To find problems in an application, it is sometimes good to see what database operations were executed. This database offers the following trace features:

- Trace to `System.out` and/or to a file
- Support for trace levels OFF, ERROR, INFO, DEBUG
- The maximum size of the trace file can be set
- It is possible to generate Java source code from the trace file
- Trace can be enabled at runtime by manually creating a file

Trace Options

The simplest way to enable the trace option is setting it in the database URL. There are two settings, one for `System.out` (`TRACE_LEVEL_SYSTEM_OUT`) tracing, and one for file tracing (`TRACE_LEVEL_FILE`). The trace levels are 0 for OFF, 1 for ERROR (the default), 2 for INFO, and 3 for DEBUG. A database URL with both levels set to DEBUG is:

```
jdbc:h2:~/test;TRACE_LEVEL_FILE=3;TRACE_LEVEL_SYSTEM_OUT=3
```

The trace level can be changed at runtime by executing the SQL command SET TRACE_LEVEL_SYSTEM_OUT level (for System.out tracing) or SET TRACE_LEVEL_FILE level (for file tracing). Example:

```
SET TRACE_LEVEL_SYSTEM_OUT 3
```

Setting the Maximum Size of the Trace File

When using a high trace level, the trace file can get very big quickly. The default size limit is 16 MB, if the trace file exceeds this limit, it is renamed to .old and a new file is created. If another such file exists, it is deleted. To limit the size to a certain number of megabytes, use SET TRACE_MAX_FILE_SIZE mb. Example:

```
SET TRACE_MAX_FILE_SIZE 1
```

Java Code Generation

When setting the trace level to INFO or DEBUG, Java source code is generated as well. This simplifies reproducing problems. The trace file looks like this:

```
...
12-20 20:58:09 jdbc[0]:
/**/dbMeta3.getUrl();
12-20 20:58:09 jdbc[0]:
/**/dbMeta3.getTables(null, "", null, new String[]{"BASE TABLE", "VIEW"});
...
```

To filter the Java source code, use the ConvertTraceFile tool as follows:

```
java -cp h2*.jar org.h2.tools.ConvertTraceFile
    -traceFile "~/test.trace.db" -javaClass "Test"
```

The generated file Test.java will contain the Java source code. The generated source code may be too large to compile (the size of a Java method is limited). If this is the case, the source code needs to be split in multiple methods. The password is not listed in the trace file and therefore not included in the source code.

Using Other Logging APIs

By default, this database uses its own native 'trace' facility. This facility is called 'trace' and not 'log' within this database to avoid confusion with the transaction log. Trace messages can be written to both file and System.out. In most cases, this is sufficient, however sometimes it is better to use the same facility as the application, for example Log4j. To do that, this database support SLF4J.

[SLF4J](#) is a simple facade for various logging APIs and allows to plug in the desired implementation at deployment time. SLF4J supports implementations such as Logback, Log4j, Jakarta Commons Logging (JCL), Java logging, x4juli, and Simple Log.

To enable SLF4J, set the file trace level to 4 in the database URL:

```
jdbc:h2:~/test;TRACE_LEVEL_FILE=4
```

Changing the log mechanism is not possible after the database is open, that means executing the SQL statement SET TRACE_LEVEL_FILE 4 when the database is already open will not have the desired effect. To use SLF4J, all required jar files need to be in the classpath. The logger name is h2database. If it does not work, check the file <database>.trace.db for error messages.

Read Only Databases

If the database files are read-only, then the database is read-only as well. It is not possible to create new tables, add or modify data in this database. Only SELECT and CALL statements are allowed. To create a read-only database, close the database. Then, make the database file read-only. When you open the database now, it is read-only. There are two ways an application can find out whether database is read-only: by calling Connection.isReadOnly() or by executing the SQL statement CALL READONLY().

Using the [Custom Access Mode](#) the database can also be opened in read-only mode, even if the database file is not read only.

Read Only Databases in Zip or Jar File

To create a read-only database in a zip file, first create a regular persistent database, and then create a backup. The database must not have pending changes, that means you need to close all connections to the database first. To speed up opening the read-only database and running queries, the database should be closed using SHUTDOWN DEFrag. If you are using a database named test, an easy way to create a zip file is using the Backup tool. You can start the tool from the command line, or from within the H2 Console (Tools - Backup). Please note that the database must be closed when the backup is created. Therefore, the SQL statement BACKUP TO can not be used.

When the zip file is created, you can open the database in the zip file using the following database URL:

```
jdbc:h2:zip:~/data.zip!/test
```

Databases in zip files are read-only. The performance for some queries will be slower than when using a regular database, because random access in zip files is not supported (only streaming). How much this affects the performance depends on the queries and the data. The database is not read in memory; therefore large databases are supported as well. The same indexes are used as when using a regular database.

If the database is larger than a few megabytes, performance is much better if the database file is split into multiple smaller files, because random access in compressed files is not possible. See also the sample application

[ReadOnlyDatabaseInZip](#).

Opening a Corrupted Database

If a database cannot be opened because the boot info (the SQL script that is run at startup) is corrupted, then the database can be opened by specifying a database event listener. The exceptions are logged, but opening the database will continue.

Generated Columns (Computed Columns) / Function Based Index

Each column is either a base column or a generated column. A generated column is a column whose value is calculated before storing and cannot be assigned directly. The formula is evaluated when the row is inserted, and re-evaluated every time the row is updated. One use case is to automatically update the last-modification time:

```
CREATE TABLE TEST(  
  ID INT,  
  NAME VARCHAR,  
  LAST_MOD TIMESTAMP WITH TIME ZONE  
    GENERATED ALWAYS AS CURRENT_TIMESTAMP  
);
```

Function indexes are not directly supported by this database, but they can be emulated by using generated columns. For example, if an index on the upper-case version of a column is required, create a generated column with the upper-case version of the original column, and create an index for this column:

```
CREATE TABLE ADDRESS(  
  ID INT PRIMARY KEY,  
  NAME VARCHAR,  
  UPPER_NAME VARCHAR GENERATED ALWAYS AS UPPER(NAME)  
);  
CREATE INDEX IDX_U_NAME ON ADDRESS(UPPER_NAME);
```

When inserting data, it is not required (and not allowed) to specify a value for the upper-case version of the column, because the value is generated. But you can use the column when querying the table:

```
INSERT INTO ADDRESS(ID, NAME) VALUES(1, 'Miller');  
SELECT * FROM ADDRESS WHERE UPPER_NAME='MILLER';
```

Multi-Dimensional Indexes

A tool is provided to execute efficient multi-dimension (spatial) range queries. This database does not support a specialized spatial index (R-Tree or similar). Instead, the B-Tree index is used. For each record, the multi-dimensional key is converted (mapped) to a single dimensional (scalar) value. This value specifies the location on a space-filling curve.

Currently, Z-order (also called N-order or Morton-order) is used; Hilbert curve could also be used, but the implementation is more complex. The algorithm to convert the multi-dimensional value is called bit-interleaving. The scalar value is indexed using a B-Tree index (usually using a generated column).

The method can result in a drastic performance improvement over just using an index on the first column. Depending on the data and number of dimensions, the improvement is usually higher than factor 5. The tool generates a SQL query from a specified multi-dimensional range. The method used is not database dependent, and the tool can easily be ported to other databases. For an example how to use the tool, please have a look at the sample code provided in `TestMultiDimension.java`.

User-Defined Functions and Stored Procedures

In addition to the built-in functions, this database supports user-defined Java functions. In this database, Java functions can be used as stored procedures as well. A function must be declared (registered) before it can be used. A function can be defined using source code, or as a reference to a compiled class that is available in the classpath. By default, the function aliases are stored in the current schema.

Referencing a Compiled Method

When referencing a method, the class must already be compiled and included in the classpath where the database is running. Only static Java methods are supported; both the class and the method must be public. Example Java class:

```
package acme;
```

```
import java.math.*;
public class Function {
    public static boolean isPrime(int value) {
        return new BigInteger(String.valueOf(value)).isProbablePrime(100);
    }
}
```

The Java function must be registered in the database by calling CREATE ALIAS ... FOR:

```
CREATE ALIAS IS_PRIME FOR "acme.Function.isPrime";
```

For a complete sample application, see `src/test/org/h2/samples/Function.java`.

Declaring Functions as Source Code

When defining a function alias with source code, the database tries to compile the source code using the Java compiler (the class `javax.tools.ToolProvider.getSystemJavaCompiler()`) if it is in the classpath. If not, `javac` is run as a separate process. Only the source code is stored in the database; the class is compiled each time the database is re-opened. Source code can be passed as dollar quoted text (`$$source code$$`) to avoid escaping problems. If you use some third-party script processing tool, use standard single quotes instead and don't forget to repeat each single quotation mark twice within the source code. Example:

```
CREATE ALIAS NEXT_PRIME AS '
String nextPrime(String value) {
    return new BigInteger(value).nextProbablePrime().toString();
}
';
```

By default, the three packages `java.util`, `java.math`, `java.sql` are imported. The method name (`nextPrime` in the example above) is ignored. Method overloading is not supported when declaring functions as source code, that means only one method may be declared for an alias. If different import

statements are required, they must be declared at the beginning and separated with the tag @CODE:

```
CREATE ALIAS IP_ADDRESS AS '  
import java.net.*;  
@CODE  
String ipAddress(String host) throws Exception {  
    return InetAddress.getByName(host).getHostAddress();  
}  
';
```

The following template is used to create a complete Java class:

```
package org.h2.dynamic;  
< import statements before the tag @CODE; if not set:  
import java.util.*;  
import java.math.*;  
import java.sql.*;  
>  
public class <aliasName> {  
    public static <sourceCode>  
}
```

Method Overloading

Multiple methods may be bound to a SQL function if the class is already compiled and included in the classpath. Each Java method must have a different number of arguments. Method overloading is not supported when declaring functions as source code.

Function Data Type Mapping

Functions that accept non-nullable parameters such as int will not be called if one of those parameters is NULL. Instead, the result of the function is NULL. If the function should be called if a parameter is NULL, you need to use java.lang.Integer instead.

SQL types are mapped to Java classes and vice-versa as in the JDBC API. For details, see [Data Types](#). There are a few special cases: `java.lang.Object` is mapped to OTHER (a serialized object). Therefore, `java.lang.Object` can not be used to match all SQL types (matching all SQL types is not supported). The second special case is `Object[]`: arrays of any class are mapped to ARRAY. Objects of type `org.h2.value.Value` (the internal value class) are passed through without conversion.

Functions That Require a Connection

If the first parameter of a Java function is a `java.sql.Connection`, then the connection to database is provided. This connection does not need to be closed before returning. When calling the method from within the SQL statement, this connection parameter does not need to be (can not be) specified.

Functions Throwing an Exception

If a function throws an exception, then the current statement is rolled back and the exception is thrown to the application. `SQLException` are directly re-thrown to the calling application; all other exceptions are first converted to a `SQLException`.

Functions Returning a Result Set

Functions may return a result set. Such a function can be called with the CALL statement:

```
public static ResultSet query(Connection conn, String sql) throws SQLException
{
    return conn.createStatement().executeQuery(sql);
}
```

```
CREATE ALIAS QUERY FOR "org.h2.samples.Function.query";
CALL QUERY('SELECT * FROM TEST');
```

Using SimpleResultSet

A function can create a result set using the `SimpleResultSet` tool:

```

import org.h2.tools.SimpleResultSet;
...
public static ResultSet simpleResultSet() throws SQLException {
    SimpleResultSet rs = new SimpleResultSet();
    rs.addColumn("ID", Types.INTEGER, 10, 0);
    rs.addColumn("NAME", Types.VARCHAR, 255, 0);
    rs.addRow(0, "Hello");
    rs.addRow(1, "World");
    return rs;
}

CREATE ALIAS SIMPLE FOR "org.h2.samples.Function.simpleResultSet";
CALL SIMPLE();

```

Using a Function as a Table

A function that returns a result set can be used like a table. However, in this case the function is called at least twice: first while parsing the statement to collect the column names (with parameters set to null where not known at compile time). And then, while executing the statement to get the data (maybe multiple times if this is a join). If the function is called just to get the column list, the URL of the connection passed to the function is `jdbc:columnlist:connection`. Otherwise, the URL of the connection is `jdbc:default:connection`.

```

public static ResultSet getMatrix(Connection conn, Integer size)
    throws SQLException {
    SimpleResultSet rs = new SimpleResultSet();
    rs.addColumn("X", Types.INTEGER, 10, 0);
    rs.addColumn("Y", Types.INTEGER, 10, 0);
    String url = conn.getMetaData().getURL();
    if (url.equals("jdbc:columnlist:connection")) {
        return rs;
    }
    for (int s = size.intValue(), x = 0; x < s; x++) {
        for (int y = 0; y < s; y++) {
            rs.addRow(x, y);
        }
    }
}

```

```
}  
return rs;  
}
```

```
CREATE ALIAS MATRIX FOR "org.h2.samples.Function.getMatrix";  
SELECT * FROM MATRIX(4) ORDER BY X, Y;
```

Pluggable or User-Defined Tables

For situations where you need to expose other data-sources to the SQL engine as a table, there are "pluggable tables". For some examples, have a look at the code in `org.h2.test.db.TestTableEngines`.

In order to create your own `TableEngine`, you need to implement the `org.h2.api.TableEngine` interface e.g. something like this:

```
package acme;  
public static class MyTableEngine implements org.h2.api.TableEngine {  
  
    private static class MyTable extends org.h2.table.TableBase {  
        .. rather a lot of code here...  
    }  
  
    public EndlessTable createTable(CreateTableData data) {  
        return new EndlessTable(data);  
    }  
}
```

and then create the table from SQL like this:

```
CREATE TABLE TEST(ID INT, NAME VARCHAR)  
ENGINE "acme.MyTableEngine";
```

It is also possible to pass in parameters to the table engine, like so:

```
CREATE TABLE TEST(ID INT, NAME VARCHAR) ENGINE "acme.MyTableEngine"  
WITH "param1", "param2";
```

In which case the parameters are passed down in the `tableEngineParams` field of the `CreateTableData` object.

It is also possible to specify default table engine params on schema creation:

```
CREATE SCHEMA TEST_SCHEMA WITH "param1", "param2";
```

Params from the schema are used when `CREATE TABLE` issued on this schema does not have its own engine params specified.

Triggers

This database supports Java triggers that are called before or after a row is updated, inserted or deleted. Triggers can be used for complex consistency checks, or to update related data in the database. It is also possible to use triggers to simulate materialized views. For a complete sample application, see `src/test/org/h2/samples/TriggerSample.java`. A Java trigger must implement the interface `org.h2.api.Trigger`. The trigger class must be available in the classpath of the database engine (when using the server mode, it must be in the classpath of the server).

```
import org.h2.api.Trigger;
...
public class TriggerSample implements Trigger {

    public void init(Connection conn, String schemaName, String triggerName,
        String tableName, boolean before, int type) {
        // initialize the trigger object is necessary
    }

    public void fire(Connection conn,
        Object[] oldRow, Object[] newRow)
        throws SQLException {
        // the trigger is fired
    }

    public void close() {
```

```

        // the database is closed
    }

    public void remove() {
        // the trigger was dropped
    }

}

```

The connection can be used to query or update data in other tables. The trigger then needs to be defined in the database:

```

CREATE TRIGGER INV_INS AFTER INSERT ON INVOICE
FOR EACH ROW CALL "org.h2.samples.TriggerSample"

```

The trigger can be used to veto a change by throwing a `SQLException`.

As an alternative to implementing the `Trigger` interface, an application can extend the abstract class `org.h2.tools.TriggerAdapter`. This will allow to use the `ResultSet` interface within trigger implementations. In this case, only the `fire` method needs to be implemented:

```

import org.h2.tools.TriggerAdapter;
...
public class TriggerSample extends TriggerAdapter {

    public void fire(Connection conn, ResultSet oldRow, ResultSet newRow)
        throws SQLException {
        // the trigger is fired
    }

}

```

Compacting a Database

Empty space in the database file is re-used automatically. When closing the database, the database is automatically compacted for up to 200 milliseconds by default. To compact more, use the SQL statement `SHUTDOWN COMPACT`.

However re-creating the database may further reduce the database size because this will re-build the indexes. Here is a sample function to do this:

```
public static void compact(String dir, String dbName,
    String user, String password) throws Exception {
    String url = "jdbc:h2:" + dir + "/" + dbName;
    String file = "data/test.sql";
    Script.execute(url, user, password, file);
    DeleteDbFiles.execute(dir, dbName, true);
    RunScript.execute(url, user, password, file, null, false);
}
```

See also the sample application `org.h2.samples.Compact`. The commands `SCRIPT` / `RUNSCRIPT` can be used as well to create a backup of a database and re-build the database from the script.

Cache Settings

The database keeps most frequently used data in the main memory. The amount of memory used for caching can be changed using the setting `CACHE_SIZE`. This setting can be set in the database connection URL (`jdbc:h2:~/test;CACHE_SIZE=131072`), or it can be changed at runtime using `SET CACHE_SIZE` size. The size of the cache, as represented by `CACHE_SIZE` is measured in KB, with each KB being 1024 bytes. This setting has no effect for in-memory databases. For persistent databases, the setting is stored in the database and re-used when the database is opened the next time. However, when opening an existing database, the cache size is set to at most half the amount of memory available for the virtual machine (`Runtime.getRuntime().maxMemory()`), even if the cache size setting stored in the database is larger; however the setting stored in the database is kept. Setting the cache size in the database URL or explicitly using `SET CACHE_SIZE` overrides this value (even if larger than the physical memory). To get the current used maximum cache size, use the query `SELECT * FROM INFORMATION_SCHEMA.SETTINGS WHERE SETTING_NAME = 'info.CACHE_MAX_SIZE'`

An experimental scan-resistant cache algorithm "Two Queue" (2Q) is available. To enable it, append ;CACHE_TYPE=TQ to the database URL. The cache might not actually improve performance. If you plan to use it, please run your own test cases first.

Also included is an experimental second level soft reference cache. Rows in this cache are only garbage collected on low memory. By default the second level cache is disabled. To enable it, use the prefix SOFT_. Example:

jdbc:h2:~/test;CACHE_TYPE=SOFT_LRU. The cache might not actually improve performance. If you plan to use it, please run your own test cases first.

To get information about page reads and writes, and the current caching algorithm in use, call SELECT * FROM INFORMATION_SCHEMA.SETTINGS. The number of pages read / written is listed.

External authentication (Experimental)

External authentication allows to optionally validate user credentials externally (JAAS,LDAP,custom classes). Is also possible to temporary assign roles to externally authenticated users. **This feature is experimental and subject to change**

Master user cannot be externally authenticated

To enable external authentication on a database execute statement SET AUTHENTICATOR TRUE. This setting is persisted on the database.

To connect on a database by using external credentials client must append AUTHREALM=H2 to the database URL. H2 is the identifier of the authentication realm (see later).

External authentication requires to send password to the server. For this reason it works only on local connection or remote over ssl

By default external authentication is performed through JAAS login interface (configuration name is h2). To configure JAAS add argument

-Djava.security.auth.login.config=jaas.conf Here an example of [JAAS login configuration file](#) content:

```
h2 {
    com.sun.security.auth.module.LdapLoginModule REQUIRED \
    userProvider="ldap://127.0.0.1:10389"
    authIdentity="uid={USERNAME},ou=people,dc=example,dc=com" \
    debug=true useSSL=false ;
};
```

Is it possible to specify custom authentication settings by using JVM argument `-Dh2auth.configurationFile={urlOfH2Auth.xml}`. Here an example of `h2auth.xml` file content:

```
<h2Auth allowUserRegistration="false" createMissingRoles="true">

    <!-- realm: DUMMY authenticate users named DUMMY[0-9] with a static
    password -->
    <realm name="DUMMY"

validatorClass="org.h2.security.auth.impl.FixedPasswordCredentialsValidator">
        <property name="userNamePattern" value="DUMMY[0-9]" />
        <property name="password" value="mock" />
    </realm>

    <!-- realm LDAPEXAMPLE:perform credentials validation on LDAP -->
    <realm name="LDAPEXAMPLE"
    validatorClass="org.h2.security.auth.impl.LdapCredentialsValidator">
        <property name="bindDnPattern"
value="uid=%u,ou=people,dc=example,dc=com" />
        <property name="host" value="127.0.0.1" />
        <property name="port" value="10389" />
        <property name="secure" value="false" />
    </realm>

    <!-- realm JAAS: perform credentials validation by using JAAS api -->
    <realm name="JAAS"
    validatorClass="org.h2.security.auth.impl.JaasCredentialsValidator">
        <property name="appName" value="H2" />
    </realm>
```



```
<!--Assign to each user role @{REALM} -->
<userToRolesMapper
class="org.h2.security.auth.impl.AssignRealmNameRole"/>

<!--Assign to each user role REMOTEUSER -->
<userToRolesMapper class="org.h2.security.auth.impl.StaticRolesMapper">
  <property name="roles" value="REMOTEUSER"/>
</userToRolesMapper>
</h2Auth>
```

Custom credentials validators must implement the interface
`org.h2.api.CredentialsValidator`

Custom criteria for role assignments must implement the interface
`org.h2.api.UserToRoleMapper`

Securing your H2

Introduction

Network exposed

Alias / Stored Procedures

Grants / Roles / Permissions

Encrypted storage

Introduction

H2 is __not__ designed to be run in an adversarial environment. You should absolutely not expose your H2 server to untrusted connections.

Running H2 in embedded mode is the best choice - it is not externally exposed.

Network exposed

When running an H2 server in TCP mode, first prize is to run with it only listening to connections on localhost (i.e 127.0.0.1).

Second prize is running listening to restricted ports on a secured network.

If you expose H2 to the broader Internet, you can secure the connection with SSL, but this is a rather tricky thing to get right, between JVM bugs, certificates and choosing a decent cipher.

Alias / Stored procedures

Anything created with CREATE ALIAS can do anything the JVM can do, which includes reading/writing from the filesystem on the machine the JVM is running on.

Grants / Roles / Permissions

GRANT / REVOKE TODO

Encrypted storage

Encrypting your on-disk database will provide a small measure of security to your stored data. You should not assume that this is any kind of real security against a determined opponent however, since there are many repeated data structures that will allow someone with resources and time to extract the secret key.

Also the secret key is visible to anything that can read the memory of the process.

Performance

[Performance Comparison](#)

[PolePosition Benchmark](#)

[Database Performance Tuning](#)

[Using the Built-In Profiler](#)

[Application Profiling](#)

[Database Profiling](#)

[Statement Execution Plans](#)

[How Data is Stored and How Indexes Work](#)

[Fast Database Import](#)

Performance Comparison

In many cases H2 is faster than other (open source and not open source) database engines. Please note this is mostly a single connection benchmark run on one computer, with many very simple operations running against the database. This benchmark does not include very complex queries. The embedded mode of H2 is faster than the client-server mode because the per-statement overhead is greatly reduced.

Embedded

Test Case	Unit	H2	HSQLDB	Derby
Simple: Init	ms	1021	2510	6762
Simple: Query (random)	ms	513	653	2035
Simple: Query (sequential)	ms	1344	2210	7665
Simple: Update (sequential)	ms	1642	3040	7034
Simple: Delete (sequential)	ms	1697	2310	9981
Simple: Memory Usage	MB	18	15	13
BenchA: Init	ms	801	2877	6576
BenchA: Transactions	ms	1369	2629	4987

BenchA: Memory Usage	MB	12	15	9
BenchB: Init	ms	966	2544	7161
BenchB: Transactions	ms	341	2316	815
BenchB: Memory Usage	MB	14	10	10
BenchC: Init	ms	2630	3144	7420
BenchC: Transactions	ms	1732	1742	2735
BenchC: Memory Usage	MB	19	34	11
Executed statements	#	2222032	2222032	2222032
Total time	ms	14056	25975	63171
Statements per second	#/s	158084	85545	35174

Client-Server

Test Case	Unit	H2	HSQldb	Derby	PostgreSQL	MySQL
Simple: Init	ms	27989	48055	47142	32972	109482
Simple: Query (random)	ms	4821	5984	14741	4089	15140
Simple: Query (sequential)	ms	33656	49112	95999	35676	143536
Simple: Update (sequential)	ms	9878	23565	31418	26113	50676
Simple: Delete (sequential)	ms	13056	28584	43955	20985	64647
Simple: Memory Usage	MB	18	15	15	2	4
BenchA: Init	ms	20993	42525	38335	27794	107723
BenchA: Transactions	ms	16549	29255	28995	23113	65036
BenchA: Memory Usage	MB	12	18	11	1	4

BenchB: Init	ms	26785	48772	39756	32369	115398
BenchB: Transactions	ms	898	10046	1916	818	1794
BenchB: Memory Usage	MB	16	11	12	2	5
BenchC: Init	ms	18266	26865	39325	24547	70531
BenchC: Transactions	ms	6569	7783	9412	8916	19150
BenchC: Memory Usage	MB	17	35	13	2	7
Executed statements	#	2222032	2222032	2222032	2222032	2222032
Total time	ms	179460	320546	390994	237392	763113
Statements per second	#/s	12381	6932	5683	9360	2911

Benchmark Results and Comments

H2

Version 2.0.202 (2021-11-25) was used for the test. For most operations, the performance of H2 is about the same as for HSQLDB. One situation where H2 is slow is large result sets, because they are buffered to disk if more than a certain number of records are returned. The advantage of buffering is: there is no limit on the result set size.

HSQLDB

Version 2.5.1 was used for the test. Cached tables are used in this test (hsqldb.default_table_type=cached), and the write delay is 1 second (SET WRITE_DELAY 1).

Derby

Version 10.14.2.0 was used for the test. Derby is clearly the slowest embedded database in this test. This seems to be a structural problem, because all

operations are really slow. It will be hard for the developers of Derby to improve the performance to a reasonable level. A few problems have been identified: leaving autocommit on is a problem for Derby. If it is switched off during the whole test, the results are about 20% better for Derby. Derby calls `FileChannel.force(false)`, but only twice per log file (not on each commit). Disabling this call improves performance for Derby by about 2%. Unlike H2, Derby does not call `FileDescriptor.sync()` on each checkpoint. Derby supports a testing mode (system property `derby.system.durability=test`) where durability is disabled. According to the documentation, this setting should be used for testing only, as the database may not recover after a crash. Enabling this setting improves performance by a factor of 2.6 (embedded mode) or 1.4 (server mode). Even if enabled, Derby is still less than half as fast as H2 in default mode.

PostgreSQL

Version 13.4 was used for the test. The following options were changed in `postgresql.conf`: `fsync = off`, `commit_delay = 100000` (microseconds).

PostgreSQL is run in server mode. The memory usage number is incorrect, because only the memory usage of the JDBC driver is measured.

MySQL

Version 8.0.27 was used for the test. MySQL was run with the InnoDB backend. The setting `innodb_flush_log_at_trx_commit` and `sync_binlog` (found in the `my.ini` / `community-mysql-server.cnf` file) was set to 0. Otherwise (and by default), MySQL is slow (around 140 statements per second in this test) because it tries to flush the data to disk for each commit. For small transactions (when autocommit is on) this is really slow. But many use cases use small or relatively small transactions. Too bad this setting is not listed in the configuration wizard, and it always overwritten when using the wizard. You need to change those settings manually in the file `my.ini` / `community-mysql-server.cnf`, and then restart the service. The memory usage number is incorrect, because only the memory usage of the JDBC driver is measured.

SQLite

SQLite 3.36.0.3, configured to use [WAL](#) and with [synchronous=NORMAL](#) was tested in a separate, less reliable run. A rough estimate is that SQLite performs approximately 2-5x worse in the simple benchmarks, which perform simple work in the database, resulting in a low work-per-transaction ratio. SQLite becomes competitive as the complexity of the database interactions increases. The results seemed to vary drastically across machine, and more reliable results should be obtained. Benchmark on your production hardware.

The benchmarks used include multi-threaded scenarios, and we were not able to get the SQLite JDBC driver we used to work with them. Help with configuring the driver for multi-threaded usage is welcome.

Firebird

Firebird 3.0 (default installation) was tested, but failed on multi-threaded part of the test. It is likely possible to run the performance test with the Firebird database, and any information on how to configure Firebird for this are welcome.

Why Oracle / MS SQL Server / DB2 are Not Listed

The license of these databases does not allow to publish benchmark results. This doesn't mean that they are fast. They are in fact quite slow, and need a lot of memory. But you will need to test this yourself.

About this Benchmark

How to Run

This test was as follows:

```
build benchmark
```

Separate Process per Database

For each database, a new process is started, to ensure the previous test does not impact the current test.

Number of Connections

This is mostly a single-connection benchmark. BenchB uses multiple connections; the other tests use one connection.

Real-World Tests

Good benchmarks emulate real-world use cases. This benchmark includes 4 test cases: BenchSimple uses one table and many small updates / deletes. BenchA is similar to the TPC-A test, but single connection / single threaded (see also: www.tpc.org). BenchB is similar to the TPC-B test, using multiple connections (one thread per connection). BenchC is similar to the TPC-C test, but single connection / single threaded.

Comparing Embedded with Server Databases

This is mainly a benchmark for embedded databases (where the application runs in the same virtual machine as the database engine). However MySQL and PostgreSQL are not Java databases and cannot be embedded into a Java application. For the Java databases, both embedded and server modes are tested.

Test Platform

This test is run on Fedora v.34 with Oracle JVM 1.8 and SSD drive.

Multiple Runs

When a Java benchmark is run first, the code is not fully compiled and therefore runs slower than when running multiple times. A benchmark should always run the same test multiple times and ignore the first run(s). This benchmark runs three times, but only the last run is measured.

Memory Usage

It is not enough to measure the time taken, the memory usage is important as well. Performance can be improved by using a bigger cache, but the amount of memory is limited. HSQLDB tables are kept fully in memory by default; this benchmark uses 'disk based' tables for all databases. Unfortunately, it is not so easy to calculate the memory usage of PostgreSQL and MySQL, because they

run in a different process than the test. This benchmark currently does not print memory usage of those databases.

Delayed Operations

Some databases delay some operations (for example flushing the buffers) until after the benchmark is run. This benchmark waits between each database tested, and each database runs in a different process (sequentially).

Transaction Commit / Durability

Durability means transaction committed to the database will not be lost. Some databases (for example MySQL) try to enforce this by default by calling `fsync()` to flush the buffers, but most hard drives don't actually flush all data. Calling the method slows down transaction commit a lot, but doesn't always make data durable. When comparing the results, it is important to think about the effect. Many database suggest to 'batch' operations when possible. This benchmark switches off autocommit when loading the data, and calls commit after each 1000 inserts. However many applications need 'short' transactions at runtime (a commit after each update). This benchmark commits after each update / delete in the simple benchmark, and after each business transaction in the other benchmarks. For databases that support delayed commits, a delay of one second is used.

Using Prepared Statements

Wherever possible, the test cases use prepared statements.

Currently Not Tested: Startup Time

The startup time of a database engine is important as well for embedded use. This time is not measured currently. Also, not tested is the time used to create a database and open an existing database. Here, one (wrapper) connection is opened at the start, and for each step a new connection is opened and then closed.

PolePosition Benchmark

The PolePosition is an open source benchmark. The algorithms are all quite simple. It was developed / sponsored by db4o. This test was not run for a longer time, so please be aware that the results below are for older database versions (H2 version 1.1, HSQLDB 1.8, Java 1.4).

Test Case	Unit	H2	HSQLDB	MySQL
Melbourne write	ms	369	249	2022
Melbourne read	ms	47	49	93
Melbourne read_hot	ms	24	43	95
Melbourne delete	ms	147	133	176
Sepang write	ms	965	1201	3213
Sepang read	ms	765	948	3455
Sepang read_hot	ms	789	859	3563
Sepang delete	ms	1384	1596	6214
Bahrain write	ms	1186	1387	6904
Bahrain query_indexed_string	ms	336	170	693
Bahrain query_string	ms	18064	39703	41243
Bahrain query_indexed_int	ms	104	134	678
Bahrain update	ms	191	87	159
Bahrain delete	ms	1215	729	6812
Imola retrieve	ms	198	194	4036
Barcelona write	ms	413	832	3191
Barcelona read	ms	119	160	1177
Barcelona query	ms	20	5169	101
Barcelona delete	ms	388	319	3287
Total	ms	26724	53962	87112

There are a few problems with the PolePosition test:

- HSQLDB uses in-memory tables by default while H2 uses persistent tables. The HSQLDB version included in PolePosition does not support changing this, so you need to replace poleposition-0.20/lib/hsqldb.jar with a newer version (for example hsqldb-1.8.0.7.jar), and then use the setting `hsqldb.connecturl=jdbc:hsqldb:file:data/hsqldb/dbbench2;hsqldb.default_table_type=cached;sql.enforce_size=true` in the file `Jdbc.properties`.
- HSQLDB keeps the database open between tests, while H2 closes the database (losing all the cache). To change that, use the database URL `jdbc:h2:file:data/h2/dbbench;DB_CLOSE_DELAY=-1`
- The amount of cache memory is quite important, specially for the PolePosition test. Unfortunately, the PolePosition test does not take this into account.

Database Performance Tuning

Keep Connections Open or Use a Connection Pool

If your application opens and closes connections a lot (for example, for each request), you should consider using a [connection pool](#). Opening a connection using `DriverManager.getConnection` is specially slow if the database is closed. By default the database is closed if the last connection is closed.

If you open and close connections a lot but don't want to use a connection pool, consider keeping a 'sentinel' connection open for as long as the application runs, or use delayed database closing. See also [Closing a database](#).

Use a Modern JVM

Newer JVMs are faster. Upgrading to the latest version of your JVM can provide a "free" boost to performance. Switching from the default Client JVM to the Server JVM using the `-server` command-line option improves performance at the cost of a slight increase in start-up time.

Virus Scanners

Some virus scanners scan files every time they are accessed. It is very important for performance that database files are not scanned for viruses. The database

engine never interprets the data stored in the files as programs, that means even if somebody would store a virus in a database file, this would be harmless (when the virus does not run, it cannot spread). Some virus scanners allow to exclude files by suffix. Ensure files ending with .db are not scanned.

Using the Trace Options

If the performance hot spots are in the database engine, in many cases the performance can be optimized by creating additional indexes, or changing the schema. Sometimes the application does not directly generate the SQL statements, for example if an O/R mapping tool is used. To view the SQL statements and JDBC API calls, you can use the trace options. For more information, see [Using the Trace Options](#).

Index Usage

This database uses indexes to improve the performance of SELECT, UPDATE, DELETE. If a column is used in the WHERE clause of a query, and if an index exists on this column, then the index can be used. Multi-column indexes are used if all or the first columns of the index are used. Both equality lookup and range scans are supported. Indexes are used to order result sets, but only if the condition uses the same index or no index at all. The results are sorted in memory if required. Indexes are created automatically for primary key and unique constraints. Indexes are also created for foreign key constraints, if required. For other columns, indexes need to be created manually using the CREATE INDEX statement.

Index Hints

If you have determined that H2 is not using the optimal index for your query, you can use index hints to force H2 to use specific indexes.

```
SELECT * FROM TEST USE INDEX (index_name_1, index_name_2) WHERE X=1
```

Only indexes in the list will be used when choosing an index to use on the given table. There is no significance to order in this list.

It is possible that no index in the list is chosen, in which case a full table scan will be used.

An empty list of index names forces a full table scan to be performed.

Each index in the list must exist.

How Data is Stored Internally

For persistent databases, if a table is created with a single column primary key of type BIGINT, INT, SMALLINT, TINYINT, then the data of the table is organized in this way. This is sometimes also called a "clustered index" or "index organized table".

H2 internally stores table data and indexes in the form of b-trees. Each b-tree stores entries as a list of unique keys (one or more columns) and data (zero or more columns). The table data is always organized in the form of a "data b-tree" with a single column key of type long. If a single column primary key of type BIGINT, INT, SMALLINT, TINYINT is specified when creating the table (or just after creating the table, but before inserting any rows), then this column is used as the key of the data b-tree. If no primary key has been specified, if the primary key column is of another data type, or if the primary key contains more than one column, then a hidden identity column of type BIGINT is added to the table, which is used as the key for the data b-tree. All other columns of the table are stored within the data area of this data b-tree (except for large BLOB, CLOB columns, which are stored externally).

For each additional index, one new "index b-tree" is created. The key of this b-tree consists of the indexed columns, plus the key of the data b-tree. If a primary key is created after the table has been created, or if the primary key contains multiple column, or if the primary key is not of the data types listed above, then the primary key is stored in a new index b-tree.

Optimizer

This database uses a cost based optimizer. For simple and queries and queries with medium complexity (less than 7 tables in the join), the expected cost (running time) of all possible plans is calculated, and the plan with the lowest

cost is used. For more complex queries, the algorithm first tries all possible combinations for the first few tables, and the remaining tables added using a greedy algorithm (this works well for most joins). Afterwards a genetic algorithm is used to test at most 2000 distinct plans. Only left-deep plans are evaluated.

Expression Optimization

After the statement is parsed, all expressions are simplified automatically if possible. Operations are evaluated only once if all parameters are constant. Functions are also optimized, but only if the function is constant (always returns the same result for the same parameter values). If the WHERE clause is always false, then the table is not accessed at all.

COUNT(*) Optimization

If the query only counts all rows of a table, then the data is not accessed. However, this is only possible if no WHERE clause is used, that means it only works for queries of the form `SELECT COUNT(*) FROM table`.

Updating Optimizer Statistics / Column Selectivity

When executing a query, at most one index per join can be used. If the same table is joined multiple times, for each join only one index is used (the same index could be used for both joins, or each join could use a different index). Example: for the query `SELECT * FROM TEST T1, TEST T2 WHERE T1.NAME='A' AND T2.ID=T1.ID`, two index can be used, in this case the index on NAME for T1 and the index on ID for T2.

If a table has multiple indexes, sometimes more than one index could be used. Example: if there is a table `TEST(ID, NAME, FIRSTNAME)` and an index on each column, then two indexes could be used for the query `SELECT * FROM TEST WHERE NAME='A' AND FIRSTNAME='B'`, the index on NAME or the index on FIRSTNAME. It is not possible to use both indexes at the same time. Which index is used depends on the selectivity of the column. The selectivity describes the 'uniqueness' of values in a column. A selectivity of 100 means each value appears only once, and a selectivity of 1 means the same value appears in many

or most rows. For the query above, the index on NAME should be used if the table contains more distinct names than first names.

The SQL statement ANALYZE can be used to automatically estimate the selectivity of the columns in the tables. This command should be run from time to time to improve the query plans generated by the optimizer.

In-Memory (Hash) Indexes

Using in-memory indexes, specially in-memory hash indexes, can speed up queries and data manipulation.

In-memory indexes are automatically used for in-memory databases, but can also be created for persistent databases using CREATE MEMORY TABLE. In many cases, the rows itself will also be kept in-memory. Please note this may cause memory problems for large tables.

In-memory hash indexes are backed by a hash table and are usually faster than regular indexes. However, hash indexes only supports direct lookup (WHERE ID = ?) but not range scan (WHERE ID < ?). To use hash indexes, use HASH as in: CREATE UNIQUE HASH INDEX and CREATE TABLE ...(ID INT PRIMARY KEY HASH,...).

Use Prepared Statements

If possible, use prepared statements with parameters.

Prepared Statements and IN(...)

Avoid generating SQL statements with a variable size IN(...) list. Instead, use a prepared statement with arrays as in the following example:

```
PreparedStatement prep = conn.prepareStatement(
    "SELECT * FROM TEST WHERE ID = ANY(?)");
prep setObject(1, new Long[] { 1L, 2L });
ResultSet rs = prep.executeQuery();
```


Optimization Examples

See `src/test/org/h2/samples/optimizations.sql` for a few examples of queries that benefit from special optimizations built into the database.

Cache Size and Type

By default the cache size of H2 is quite small. Consider using a larger cache size, or enable the second level soft reference cache. See also [Cache Settings](#).

Data Types

Each data type has different storage and performance characteristics:

- The DECIMAL/NUMERIC type is slower and requires more storage than the REAL and DOUBLE PRECISION types.
- Text types are slower to read, write, and compare than numeric types and generally require more storage.
- See [Large Objects](#) for information on BINARY vs. BLOB and VARCHAR vs. CLOB performance.
- Parsing and formatting takes longer for the TIME, DATE, and TIMESTAMP types than the numeric types.
- SMALLINT/TINYINT/BOOLEAN are not significantly smaller or faster to work with than INTEGER in most modes.

Sorted Insert Optimization

To reduce disk space usage and speed up table creation, an optimization for sorted inserts is available. When used, b-tree pages are split at the insertion point. To use this optimization, add SORTED before the SELECT statement:

```
CREATE TABLE TEST(ID INT PRIMARY KEY, NAME VARCHAR) AS
  SORTED SELECT X, SPACE(100) FROM SYSTEM_RANGE(1, 100);
INSERT INTO TEST
  SORTED SELECT X, SPACE(100) FROM SYSTEM_RANGE(101, 200);
```

Using the Built-In Profiler

A very simple Java profiler is built-in. To use it, use the following template:

```
import org.h2.util.Profiler;
Profiler prof = new Profiler();
prof.startCollecting();
// .... some long running process, at least a few seconds
prof.stopCollecting();
System.out.println(prof.getTop(3));
```

Application Profiling

Analyze First

Before trying to optimize performance, it is important to understand where the problem is (what part of the application is slow). Blind optimization or optimization based on guesses should be avoided, because usually it is not an efficient strategy. There are various ways to analyze an application. Sometimes two implementations can be compared using `System.currentTimeMillis()`. But this does not work for complex applications with many modules, and for memory problems.

A simple way to profile an application is to use the built-in profiling tool of java. Example:

```
java -Xrunhprof:cpu=samples,depth=16 com.acme.Test
```

Unfortunately, it is only possible to profile the application from start to end. Another solution is to create a number of full thread dumps. To do that, first run `jps -l` to get the process id, and then run `jstack <pid>` or `kill -QUIT <pid>` (Linux) or press `Ctrl+C` (Windows).

A simple profiling tool is included in H2. To use it, the application needs to be changed slightly. Example:

```
import org.h2.util;
...
Profiler profiler = new Profiler();
profiler.startCollecting();
// application code
```

```
System.out.println(profiler.getTop(3));
```

The profiler is built into the H2 Console tool, to analyze databases that open slowly. To use it, run the H2 Console, and then click on 'Test Connection'. Afterwards, click on "Test successful" and you get the most common stack traces, which helps to find out why it took so long to connect. You will only get the stack traces if opening the database took more than a few seconds.

Database Profiling

The ConvertTraceFile tool generates SQL statement statistics at the end of the SQL script file. The format used is similar to the profiling data generated when using `java -Xrunhprof`. For this to work, the trace level needs to be 2 or higher (`TRACE_LEVEL_FILE=2`). The easiest way to set the trace level is to append the setting to the database URL, for example: `jdbc:h2:~/test;TRACE_LEVEL_FILE=2` or `jdbc:h2:tcp://localhost/~/test;TRACE_LEVEL_FILE=2`. As an example, execute the following script using the H2 Console:

```
SET TRACE_LEVEL_FILE 2;
DROP TABLE IF EXISTS TEST;
CREATE TABLE TEST(ID INT PRIMARY KEY, NAME VARCHAR(255));
@LOOP 1000 INSERT INTO TEST VALUES(?, ?);
SET TRACE_LEVEL_FILE 0;
```

After running the test case, convert the `.trace.db` file using the ConvertTraceFile tool. The trace file is located in the same directory as the database file.

```
java -cp h2*.jar org.h2.tools.ConvertTraceFile
  -traceFile "~/test.trace.db" -script "~/test.sql"
```

The generated file `test.sql` will contain the SQL statements as well as the following profiling data (results vary):

```
-----
-- SQL Statement Statistics
-- time: total time in milliseconds (accumulated)
-- count: how many times the statement ran
```

```
-- result: total update count or row count
-----
-- self accu   time   count result sql
-- 62% 62%    158   1000   1000 INSERT INTO TEST VALUES(?, ?);
-- 37% 100%    93    1      0 CREATE TABLE TEST(ID INT PRIMARY KEY...
-- 0% 100%     0    1      0 DROP TABLE IF EXISTS TEST;
-- 0% 100%     0    1      0 SET TRACE_LEVEL_FILE 3;
```

Statement Execution Plans

The SQL statement EXPLAIN displays the indexes and optimizations the database uses for a statement. The following statements support EXPLAIN: SELECT, UPDATE, DELETE, MERGE, INSERT. The following query shows that the database uses the primary key index to search for rows:

```
EXPLAIN SELECT * FROM TEST WHERE ID=1;
SELECT
  TEST.ID,
  TEST.NAME
FROM PUBLIC.TEST
/* PUBLIC.PRIMARY_KEY_2: ID = 1 */
WHERE ID = 1
```

For joins, the tables in the execution plan are sorted in the order they are processed. The following query shows the database first processes the table INVOICE (using the primary key). For each row, it will additionally check that the value of the column AMOUNT is larger than zero, and for those rows the database will search in the table CUSTOMER (using the primary key). The query plan contains some redundancy so it is a valid statement.

```
CREATE TABLE CUSTOMER(ID IDENTITY, NAME VARCHAR);
CREATE TABLE INVOICE(ID IDENTITY,
  CUSTOMER_ID INT REFERENCES CUSTOMER(ID),
  AMOUNT NUMBER);

EXPLAIN SELECT I.ID, C.NAME FROM CUSTOMER C, INVOICE I
```

```
WHERE I.ID=10 AND AMOUNT>0 AND C.ID=I.CUSTOMER_ID;
```

```
SELECT
  I.ID,
  C.NAME
FROM PUBLIC.INVOICE I
  /* PUBLIC.PRIMARY_KEY_9: ID = 10 */
  /* WHERE (I.ID = 10)
    AND (AMOUNT > 0)
  */
INNER JOIN PUBLIC.CUSTOMER C
  /* PUBLIC.PRIMARY_KEY_5: ID = I.CUSTOMER_ID */
  ON 1=1
WHERE (C.ID = I.CUSTOMER_ID)
  AND ((I.ID = 10)
  AND (AMOUNT > 0))
```

Displaying the Scan Count

EXPLAIN ANALYZE additionally shows the scanned rows per table and pages read from disk per table or index. This will actually execute the query, unlike EXPLAIN which only prepares it. The following query scanned 1000 rows, and to do that had to read 85 pages from the data area of the table. Running the query twice will not list the pages read from disk, because they are now in the cache. The tableScan means this query doesn't use an index.

```
EXPLAIN ANALYZE SELECT * FROM TEST;
SELECT
  TEST.ID,
  TEST.NAME
FROM PUBLIC.TEST
  /* PUBLIC.TEST.tableScan */
  /* scanCount: 1000 */
/*
total: 85
TEST.TEST_DATA read: 85 (100%)
*/
```

The cache will prevent the pages are read twice. H2 reads all columns of the row unless only the columns in the index are read. Except for large CLOB and BLOB, which are not store in the table.

Special Optimizations

For certain queries, the database doesn't need to read all rows, or doesn't need to sort the result even if ORDER BY is used.

For queries of the form `SELECT COUNT(*), MIN(ID), MAX(ID) FROM TEST`, the query plan includes the line `/* direct lookup */` if the data can be read from an index.

For queries of the form `SELECT DISTINCT CUSTOMER_ID FROM INVOICE`, the query plan includes the line `/* distinct */` if there is an non-unique or multi-column index on this column, and if this column has a low selectivity.

For queries of the form `SELECT * FROM TEST ORDER BY ID`, the query plan includes the line `/* index sorted */` to indicate there is no separate sorting required. `/* index sorted: 2 of 3 columns */` indicates that only some columns are sorted with an index. An additional sorting is still required, but queries with the `FETCH (TOP, LIMIT)` clause may still stop their execution earlier. An index on (A ASC, B ASC) columns can be used for `ORDER BY A`, `ORDER BY A DESC`, `ORDER BY A, B` or `ORDER BY A DESC, B DESC`. With `ORDER BY A, B DESC` this index can only be used for ordering on the column A. If columns are nullable, order of nulls is also important. Index on (A ASC NULLS FIRST) cannot be used for `ORDER BY A ASC NULLS LAST`, but can be used for `ORDER BY A ASC NULLS FIRST` or `ORDER BY A DESC NULLS LAST`. When neither NULLS FIRST nor NULLS LAST is specified, a default is used, this default is controlled by [DEFAULT_NULL_ORDERING](#) setting.

For queries of the form `SELECT * FROM TEST GROUP BY ID ORDER BY ID`, the query plan includes the line `/* group sorted */` to indicate there is no separate sorting required.

How Data is Stored and How Indexes Work

Internally, each row in a table is identified by a unique number, the row id. The rows of a table are stored with the row id as the key. The row id is a number of type long. If a table has a single column primary key of type INT or BIGINT, then the value of this column is the row id, otherwise the database generates the row id automatically. There is a (non-standard) way to access the row id: using the `_ROWID_` pseudo-column:

```
CREATE TABLE ADDRESS(FIRST_NAME VARCHAR,  
    NAME VARCHAR, CITY VARCHAR, PHONE VARCHAR);  
INSERT INTO ADDRESS VALUES('John', 'Miller', 'Berne', '123 456 789');  
INSERT INTO ADDRESS VALUES('Philip', 'Jones', 'Berne', '123 012 345');  
SELECT _ROWID_, * FROM ADDRESS;
```

The data is stored in the database as follows:

<code>_ROWID_</code>	<code>FIRST_NAME</code>	<code>NAME</code>	<code>CITY</code>	<code>PHONE</code>
1	John	Miller	Berne	123 456 789
2	Philip	Jones	Berne	123 012 345

Access by row id is fast because the data is sorted by this key. Please note the row id is not available until after the row was added (that means, it can not be used in generated columns or constraints). If the query condition does not contain the row id (and if no other index can be used), then all rows of the table are scanned. A table scan iterates over all rows in the table, in the order of the row id. To find out what strategy the database uses to retrieve the data, use `EXPLAIN SELECT`:

```
SELECT * FROM ADDRESS WHERE NAME = 'Miller';  
  
EXPLAIN SELECT PHONE FROM ADDRESS WHERE NAME = 'Miller';  
SELECT  
    PHONE  
FROM PUBLIC.ADDRESS  
    /* PUBLIC.ADDRESS.tableScan */  
WHERE NAME = 'Miller';
```

Indexes

An index internally is basically just a table that contains the indexed column(s), plus the row id:

```
CREATE INDEX INDEX_PLACE ON ADDRESS(CITY, NAME, FIRST_NAME);
```

In the index, the data is sorted by the indexed columns. So this index contains the following data:

CITY	NAME	FIRST_NAME	_ROWID_
------	------	------------	---------

Berne	Jones	Philip	2
-------	-------	--------	---

Berne	Miller	John	1
-------	--------	------	---

When the database uses an index to query the data, it searches the index for the given data, and (if required) reads the remaining columns in the main data table (retrieved using the row id). An index on city, name, and first name (multi-column index) allows to quickly search for rows when the city, name, and first name are known. If only the city and name, or only the city is known, then this index is also used (so creating an additional index on just the city is not needed). This index is also used when reading all rows, sorted by the indexed columns. However, if only the first name is known, then this index is not used:

```
EXPLAIN SELECT PHONE FROM ADDRESS
  WHERE CITY = 'Berne' AND NAME = 'Miller'
    AND FIRST_NAME = 'John';
SELECT
  PHONE
FROM PUBLIC.ADDRESS
  /* PUBLIC.INDEX_PLACE: FIRST_NAME = 'John'
    AND CITY = 'Berne'
    AND NAME = 'Miller'
  */
WHERE (FIRST_NAME = 'John')
  AND ((CITY = 'Berne')
    AND (NAME = 'Miller'));
```

```
EXPLAIN SELECT PHONE FROM ADDRESS WHERE CITY = 'Berne';
```



```

SELECT
  PHONE
FROM PUBLIC.ADDRESS
  /* PUBLIC.INDEX_PLACE: CITY = 'Berne' */
WHERE CITY = 'Berne';

EXPLAIN SELECT * FROM ADDRESS ORDER BY CITY, NAME, FIRST_NAME;
SELECT
  ADDRESS.FIRST_NAME,
  ADDRESS.NAME,
  ADDRESS.CITY,
  ADDRESS.PHONE
FROM PUBLIC.ADDRESS
  /* PUBLIC.INDEX_PLACE */
ORDER BY 3, 2, 1
/* index sorted */;

EXPLAIN SELECT PHONE FROM ADDRESS WHERE FIRST_NAME = 'John';
SELECT
  PHONE
FROM PUBLIC.ADDRESS
  /* PUBLIC.ADDRESS.tableScan */
WHERE FIRST_NAME = 'John';

```

If your application often queries the table for a phone number, then it makes sense to create an additional index on it:

```
CREATE INDEX IDX_PHONE ON ADDRESS(PHONE);
```

This index contains the phone number, and the row id:

PHONE	_ROWID_
123 012 345	2
123 456 789	1

Using Multiple Indexes

Within a query, only one index per logical table is used. Using the condition
 PHONE = '123 567 789' OR CITY = 'Berne' would use a table scan instead of first

using the index on the phone number and then the index on the city. It makes sense to write two queries and combine them using UNION. In this case, each individual query uses a different index:

```
EXPLAIN SELECT NAME FROM ADDRESS WHERE PHONE = '123 567 789'  
UNION SELECT NAME FROM ADDRESS WHERE CITY = 'Berne';
```

```
(SELECT  
  NAME  
FROM PUBLIC.ADDRESS  
  /* PUBLIC.IDX_PHONE: PHONE = '123 567 789' */  
WHERE PHONE = '123 567 789')  
UNION  
(SELECT  
  NAME  
FROM PUBLIC.ADDRESS  
  /* PUBLIC.INDEX_PLACE: CITY = 'Berne' */  
WHERE CITY = 'Berne')
```

Fast Database Import

If you have to import a lot of rows, use a PreparedStatement or use CSV import. Please note that CREATE TABLE(...) ... AS SELECT ... is faster than CREATE TABLE(...); INSERT INTO ... SELECT

Advanced

Result Sets
Large Objects
Linked Tables
Spatial Features
Recursive Queries
Updatable Views
Transaction Isolation
Multi-Version Concurrency Control (MVCC)
Clustering / High Availability
Two Phase Commit
Compatibility
Keywords / Reserved Words
Standards Compliance
Run as Windows Service
ODBC Driver
ACID
Durability Problems
Using the Recover Tool
File Locking Protocols
Using Passwords
Password Hash
Protection against SQL Injection
Protection against Remote Access
Restricting Class Loading and Usage
Security Protocols
TLS Connections
Universally Unique Identifiers (UUID)
Settings Read from System Properties
Setting the Server Bind Address
Pluggable File System

[Split File System](#)

[Java Objects Serialization](#)

[Limits and Limitations](#)

[Glossary and Links](#)

Result Sets

Statements that Return a Result Set

The following statements return a result set: SELECT, TABLE, VALUES, EXPLAIN, CALL, SCRIPT, SHOW, HELP. EXECUTE may return either a result set or an update count. Result of a WITH statement depends on inner command. All other statements return an update count.

Limiting the Number of Rows

Before the result is returned to the application, all rows are read by the database. Server side cursors are not supported currently. If only the first few rows are interesting for the application, then the result set size should be limited to improve the performance. This can be done using FETCH in a query (example: SELECT * FROM TEST FETCH FIRST 100 ROWS ONLY), or by using Statement.setMaxRows(max).

Large Result Sets and External Sorting

For large result set, the result is buffered to disk. The threshold can be defined using the statement SET MAX_MEMORY_ROWS. If ORDER BY is used, the sorting is done using an external sort algorithm. In this case, each block of rows is sorted using quick sort, then written to disk; when reading the data, the blocks are merged together.

Large Objects

Storing and Reading Large Objects

If it is possible that the objects don't fit into memory, then the data type CLOB (for textual data) or BLOB (for binary data) should be used. For these data types, the objects are not fully read into memory, by using streams. To store a BLOB,

use `PreparedStatement.setBinaryStream`. To store a CLOB, use `PreparedStatement.setCharacterStream`. To read a BLOB, use `ResultSet.getBinaryStream`, and to read a CLOB, use `ResultSet.setCharacterStream`. When using the client/server mode, large BLOB and CLOB data is stored in a temporary file on the client side.

When to use CLOB/BLOB

By default, this database stores large LOB (CLOB and BLOB) objects separate from the main table data. Small LOB objects are stored in-place, the threshold can be set using [MAX_LENGTH_INPLACE_LOB](#), but there is still an overhead to use CLOB/BLOB. Because of this, BLOB and CLOB should never be used for columns with a maximum size below about 200 bytes. The best threshold depends on the use case; reading in-place objects is faster than reading from separate files, but slows down the performance of operations that don't involve this column.

Linked Tables

This database supports linked tables, which means tables that don't exist in the current database but are just links to another database. To create such a link, use the `CREATE LINKED TABLE` statement:

```
CREATE LINKED TABLE LINK('org.postgresql.Driver', 'jdbc:postgresql:test', 'sa', 'sa', 'TEST');
```

You can then access the table in the usual way. Whenever the linked table is accessed, the database issues specific queries over JDBC. Using the example above, if you issue the query `SELECT * FROM LINK WHERE ID=1`, then the following query is run against the PostgreSQL database: `SELECT * FROM TEST WHERE ID=?`. The same happens for insert and update statements. Only simple statements are executed against the target database, that means no joins (queries that contain joins are converted to simple queries). Prepared statements are used where possible.

To view the statements that are executed against the target table, set the trace level to 3.

If multiple linked tables point to the same database (using the same database URL), the connection is shared. To disable this, set the system property `h2.shareLinkedConnections=false`.

The statement [CREATE LINKED TABLE](#) supports an optional schema name parameter.

The following are not supported because they may result in a deadlock: creating a linked table to the same database, and creating a linked table to another database using the server mode if the other database is open in the same server (use the embedded mode instead).

Data types that are not supported in H2 are also not supported for linked tables, for example unsigned data types if the value is outside the range of the signed type. In such cases, the columns needs to be cast to a supported type.

Updatable Views

By default, views are not updatable. To make a view updatable, use an "instead of" trigger as follows:

```
CREATE TRIGGER TRIGGER_NAME
  INSTEAD OF INSERT, UPDATE, DELETE
  ON VIEW_NAME
  FOR EACH ROW CALL "com.acme.TriggerClassName";
```

Update the base table(s) within the trigger as required. For details, see the sample application `org.h2.samples.UpdatableView`.

Transaction Isolation

Please note that most data definition language (DDL) statements, such as "create table", commit the current transaction. See the [Commands](#) for details.

Transaction isolation is provided for all data manipulation language (DML) statements.

H2 supports read uncommitted, read committed, repeatable read, snapshot, and serializable (partially, see below) isolation levels:

- **Read uncommitted**

Dirty reads, non-repeatable reads, and phantom reads are possible. To enable, execute the SQL statement `SET SESSION CHARACTERISTICS AS TRANSACTION ISOLATION LEVEL READ UNCOMMITTED`

- **Read committed**

This is the default level. Dirty reads aren't possible; non-repeatable reads and phantom reads are possible. To enable, execute the SQL statement `SET SESSION CHARACTERISTICS AS TRANSACTION ISOLATION LEVEL READ COMMITTED`

- **Repeatable read**

Dirty reads and non-repeatable reads aren't possible, phantom reads are possible. To enable, execute the SQL statement `SET SESSION CHARACTERISTICS AS TRANSACTION ISOLATION LEVEL REPEATABLE READ`

- **Snapshot**

Dirty reads, non-repeatable reads, and phantom reads aren't possible. This isolation level is very expensive in databases with many tables. To enable, execute the SQL statement `SET SESSION CHARACTERISTICS AS TRANSACTION ISOLATION LEVEL SNAPSHOT`

- **Serializable**

Dirty reads, non-repeatable reads, and phantom reads aren't possible. Note that this isolation level in H2 currently doesn't ensure equivalence of concurrent and serializable execution of transactions that perform write operations. This isolation level is very expensive in databases with many tables. To enable, execute the SQL statement `SET SESSION CHARACTERISTICS AS TRANSACTION ISOLATION LEVEL SERIALIZABLE`

- **Dirty reads**

Means a connection can read uncommitted changes made by another connection.

Possible with: read uncommitted.

- **Non-repeatable reads**

A connection reads a row, another connection changes a row and commits, and the first connection re-reads the same row and gets the new result.

Possible with: read uncommitted, read committed.

- **Phantom reads**

A connection reads a set of rows using a condition, another connection inserts a row that falls in this condition and commits, then the first connection re-reads using the same condition and gets the new row.

Possible with: read uncommitted, read committed, repeatable read.

Multi-Version Concurrency Control (MVCC)

Insert and update operations only issue a shared lock on the table. An exclusive lock is still used when adding or removing columns or when dropping the table. Connections only 'see' committed data, and own changes. That means, if connection A updates a row but doesn't commit this change yet, connection B will see the old value. Only when the change is committed, the new value is visible by other connections (read committed). If multiple connections concurrently try to lock or update the same row, the database waits until it can apply the change, but at most until the lock timeout expires.

Lock Timeout

If a connection cannot get a lock on an object, the connection waits for some amount of time (the lock timeout). During this time, hopefully the connection holding the lock commits and it is then possible to get the lock. If this is not possible because the other connection does not release the lock for some time, the unsuccessful connection will get a lock timeout exception. The lock timeout can be set individually for each connection.

Clustering / High Availability

This database supports a simple clustering / high availability mechanism. The architecture is: two database servers run on two different computers, and on both computers is a copy of the same database. If both servers run, each

database operation is executed on both computers. If one server fails (power, hardware or network failure), the other server can still continue to work. From this point on, the operations will be executed only on one server until the other server is back up.

Clustering can only be used in the server mode (the embedded mode does not support clustering). The cluster can be re-created using the CreateCluster tool without stopping the remaining server. Applications that are still connected are automatically disconnected, however when appending `;AUTO_RECONNECT=TRUE`, they will recover from that.

To initialize the cluster, use the following steps:

- Create a database
- Use the CreateCluster tool to copy the database to another location and initialize the clustering. Afterwards, you have two databases containing the same data.
- Start two servers (one for each copy of the database)
- You are now ready to connect to the databases with the client application(s)

Using the CreateCluster Tool

To understand how clustering works, please try out the following example. In this example, the two databases reside on the same computer, but usually, the databases will be on different servers.

- Create two directories: server1, server2. Each directory will simulate a directory on a computer.
- Start a TCP server pointing to the first directory. You can do this using the command line:

```
java org.h2.tools.Server  
-tcp -tcpPort 9101  
-baseDir server1
```

- Start a second TCP server pointing to the second directory. This will simulate a server running on a second (redundant) computer. You can do this using the command line:

```
java org.h2.tools.Server  
-tcp -tcpPort 9102  
-baseDir server2
```

- Use the CreateCluster tool to initialize clustering. This will automatically create a new, empty database if it does not exist. Run the tool on the command line:

```
java org.h2.tools.CreateCluster  
-urlSource jdbc:h2:tcp://localhost:9101/~ /test  
-urlTarget jdbc:h2:tcp://localhost:9102/~ /test  
-user sa  
-serverList localhost:9101,localhost:9102
```

- You can now connect to the databases using an application or the H2 Console using the JDBC URL
jdbc:h2:tcp://localhost:9101,localhost:9102/~ /test
- If you stop a server (by killing the process), you will notice that the other machine continues to work, and therefore the database is still accessible.
- To restore the cluster, you first need to delete the database that failed, then restart the server that was stopped, and re-run the CreateCluster tool.

Detect Which Cluster Instances are Running

To find out which cluster nodes are currently running, execute the following SQL statement:

```
SELECT SETTING_VALUE FROM INFORMATION_SCHEMA.SETTINGS WHERE  
SETTING_NAME = 'CLUSTER'
```

If the result is " (two single quotes), then the cluster mode is disabled. Otherwise, the list of servers is returned, enclosed in single quote. Example: 'server1:9191,server2:9191'.

It is also possible to get the list of servers by using `Connection.getClientInfo()`.

The property list returned from `getClientInfo()` contains a `numServers` property that returns the number of servers that are in the connection list. To get the actual servers, `getClientInfo()` also has properties `server0..serverX`, where `serverX` is the number of servers minus 1.

Example: To get the 2nd server in the connection list one uses `getClientInfo('server1')`. **Note:** The `serverX` property only returns IP addresses and ports and not hostnames.

Clustering Algorithm and Limitations

Read-only queries are only executed against the first cluster node, but all other statements are executed against all nodes. There is currently no load balancing made to avoid problems with transactions. The following functions may yield different results on different cluster nodes and must be executed with care: `UUID()`, `RANDOM_UUID()`, `SECURE RAND()`, `SESSION_ID()`, `MEMORY_FREE()`, `MEMORY_USED()`, `CSVREAD()`, `CSVWRITE()`, `RAND()` [when not using a seed]. Those functions should not be used directly in modifying statements (for example `INSERT`, `UPDATE`, `MERGE`). However, they can be used in read-only statements and the result can then be used for modifying statements. Identity columns aren't supported. Instead, sequence values need to be manually requested and then used to insert data (using two statements).

When using the cluster modes, result sets are read fully in memory by the client, so that there is no problem if the server dies that executed the query. Result sets must fit in memory on the client side.

The SQL statement `SET AUTOCOMMIT FALSE` is not supported in the cluster mode. To disable autocommit, the method `Connection.setAutoCommit(false)` needs to be called.

It is possible that a transaction from one connection overtakes a transaction from a different connection. Depending on the operations, this might result in different results, for example when conditionally incrementing a value in a row.

Two Phase Commit

The two phase commit protocol is supported. 2-phase-commit works as follows:

- Autocommit needs to be switched off
- A transaction is started, for example by inserting a row
- The transaction is marked 'prepared' by executing the SQL statement `PREPARE COMMIT transactionName`
- The transaction can now be committed or rolled back
- If a problem occurs before the transaction was successfully committed or rolled back (for example because a network problem occurred), the transaction is in the state 'in-doubt'
- When re-connecting to the database, the in-doubt transactions can be listed with `SELECT * FROM INFORMATION_SCHEMA.IN_DOUBT`
- Each transaction in this list must now be committed or rolled back by executing `COMMIT TRANSACTION transactionName` or `ROLLBACK TRANSACTION transactionName`
- The database needs to be closed and re-opened to apply the changes

Compatibility

This database is (up to a certain point) compatible to other databases such as HSQLDB, MySQL and PostgreSQL. There are certain areas where H2 is incompatible.

Transaction Commit when Autocommit is On

At this time, this database engine commits a transaction (if autocommit is switched on) just before returning the result. For a query, this means the transaction is committed even before the application scans through the result set, and before the result set is closed. Other database engines may commit the transaction in this case when the result set is closed.

Keywords / Reserved Words

There is a list of keywords that can't be used as identifiers (table names, column names and so on), unless they are quoted (surrounded with double quotes). The following tokens are keywords in H2:

Keyword	H2	SQL Standard					
		2016	2011	2008	2003	1999	92
ALL	+	+	+	+	+	+	+
AND	+	+	+	+	+	+	+
ANY	+	+	+	+	+	+	+
ARRAY	+	+	+	+	+	+	
AS	+	+	+	+	+	+	+
ASYMMETRIC	+	+	+	+	+	NR	
AUTHORIZATION	+	+	+	+	+	+	+
BETWEEN	+	+	+	+	+	NR	+
BOTH	CS	+	+	+	+	+	+
CASE	+	+	+	+	+	+	+
CAST	+	+	+	+	+	+	+
CHECK	+	+	+	+	+	+	+
CONSTRAINT	+	+	+	+	+	+	+
CROSS	+	+	+	+	+	+	+
CURRENT_CATALOG	+	+	+	+			
CURRENT_DATE	+	+	+	+	+	+	+
CURRENT_PATH	+	+	+	+	+	+	
CURRENT_ROLE	+	+	+	+	+	+	
CURRENT_SCHEMA	+	+	+	+			
CURRENT_TIME	+	+	+	+	+	+	+
CURRENT_TIMESTAMP	+	+	+	+	+	+	+

Keyword	H2	SQL Standard					
		2016	2011	2008	2003	1999	92
CURRENT_USER	+	+	+	+	+	+	+
DAY	+	+	+	+	+	+	+
DEFAULT	+	+	+	+	+	+	+
DISTINCT	+	+	+	+	+	+	+
ELSE	+	+	+	+	+	+	+
END	+	+	+	+	+	+	+
EXCEPT	+	+	+	+	+	+	+
EXISTS	+	+	+	+	+	NR	+
FALSE	+	+	+	+	+	+	+
FETCH	+	+	+	+	+	+	+
FOR	+	+	+	+	+	+	+
FOREIGN	+	+	+	+	+	+	+
FROM	+	+	+	+	+	+	+
FULL	+	+	+	+	+	+	+
GROUP	+	+	+	+	+	+	+
GROUPS	CS	+	+				
HAVING	+	+	+	+	+	+	+
HOUR	+	+	+	+	+	+	+
IF	+						
ILIKE	CS						
IN	+	+	+	+	+	+	+
INNER	+	+	+	+	+	+	+
INTERSECT	+	+	+	+	+	+	+
INTERVAL	+	+	+	+	+	+	+

Keyword	H2	SQL Standard					
		2016	2011	2008	2003	1999	92
IS	+	+	+	+	+	+	+
JOIN	+	+	+	+	+	+	+
KEY	+	NR	NR	NR	NR	+	+
LEADING	CS	+	+	+	+	+	+
LEFT	+	+	+	+	+	+	+
LIKE	+	+	+	+	+	+	+
LIMIT	MS					+	
LOCALTIME	+	+	+	+	+	+	
LOCALTIMESTAMP	+	+	+	+	+	+	
MINUS	MS						
MINUTE	+	+	+	+	+	+	+
MONTH	+	+	+	+	+	+	+
NATURAL	+	+	+	+	+	+	+
NOT	+	+	+	+	+	+	+
NULL	+	+	+	+	+	+	+
OFFSET	+	+	+	+			
ON	+	+	+	+	+	+	+
OR	+	+	+	+	+	+	+
ORDER	+	+	+	+	+	+	+
OVER	CS	+	+	+	+		
PARTITION	CS	+	+	+	+		
PRIMARY	+	+	+	+	+	+	+
QUALIFY	+						
RANGE	CS	+	+	+	+		

Keyword	H2	SQL Standard					
		2016	2011	2008	2003	1999	92
REGEXP	CS						
RIGHT	+	+	+	+	+	+	+
ROW	+	+	+	+	+	+	
ROWNUM	+						
ROWS	CS	+	+	+	+	+	+
SECOND	+	+	+	+	+	+	+
SELECT	+	+	+	+	+	+	+
SESSION_USER	+	+	+	+	+	+	
SET	+	+	+	+	+	+	+
SOME	+	+	+	+	+	+	+
SYMMETRIC	+	+	+	+	+	NR	
SYSTEM_USER	+	+	+	+	+	+	+
TABLE	+	+	+	+	+	+	+
TO	+	+	+	+	+	+	+
TOP	MS CS						
TRAILING	CS	+	+	+	+	+	+
TRUE	+	+	+	+	+	+	+
UESCAPE	+	+	+	+	+		
UNION	+	+	+	+	+	+	+
UNIQUE	+	+	+	+	+	+	+
UNKNOWN	+	+	+	+	+	+	+
USER	+	+	+	+	+	+	+
USING	+	+	+	+	+	+	+
VALUE	+	+	+	+	+	+	+

Keyword	H2	SQL Standard					
		2016	2011	2008	2003	1999	92
VALUES	+	+	+	+	+	+	+
WHEN	+	+	+	+	+	+	+
WHERE	+	+	+	+	+	+	+
WINDOW	+	+	+	+	+		
WITH	+	+	+	+	+	+	+
YEAR	+	+	+	+	+	+	+
ROWID	+						

Mode-sensitive keywords (MS) are keywords only in some compatibility modes.

- LIMIT is a keywords only in Regular, Legacy, DB2, HSQLDB, MariaDB, MySQL, and PostgreSQL compatibility modes. It is an identifier in Strict, Derby, MSSQLServer, and Oracle compatibility modes.
- MINUS is a keyword only in Regular, Legacy, DB2, HSQLDB, and Oracle compatibility modes. It is an identifier in Strict, Derby, MSSQLServer, MariaDB, MySQL, and PostgreSQL compatibility modes.
- TOP is a context-sensitive keyword (can be either keyword or identifier) only in Regular, Legacy, HSQLDB, and MSSQLServer compatibility modes. It is an identifier unconditionally in Strict, Derby, DB2, MariaDB, MySQL, Oracle, and PostgreSQL compatibility modes.

Context-sensitive keywords (CS) can be used as identifiers in some places, but cannot be used as identifiers in others. Normal keywords (+) are always treated as keywords.

Most keywords in H2 are also reserved (+) or non-reserved (NR) words in the SQL Standard. Newer versions of H2 may have more keywords than older ones. Reserved words from the SQL Standard are potential candidates for keywords in future versions.

There is a compatibility setting [SET NON_KEYWORDS](#) that can be used as a temporary workaround for applications that use keywords as unquoted identifiers.

Standards Compliance

This database tries to be as much standard compliant as possible. For the SQL language, ANSI/ISO is the main standard. There are several versions that refer to the release date: SQL-92, SQL:1999, and SQL:2003. Unfortunately, the standard documentation is not freely available. Another problem is that important features are not standardized. Whenever this is the case, this database tries to be compatible to other databases.

Supported Character Sets, Character Encoding, and Unicode

H2 internally uses Unicode, and supports all character encoding systems and character sets supported by the virtual machine you use.

Run as Windows Service

Using a native wrapper / adapter, Java applications can be run as a Windows Service. There are various tools available to do that. The Java Service Wrapper from [Tanuki Software, Inc.](#) is included in the installation. Batch files are provided to install, start, stop and uninstall the H2 Database Engine Service. This service contains the TCP Server and the H2 Console web application. The batch files are located in the directory h2/service.

The service wrapper bundled with H2 is a 32-bit version. To use a 64-bit version of Windows (x64), you need to use a 64-bit version of the wrapper, for example the one from [Simon Krenger](#).

When running the database as a service, absolute path should be used. Using ~ in the database URL is problematic in this case, because it means to use the home directory of the current user. The service might run without or with the wrong user, so that the database files might end up in an unexpected place.

Install the Service

The service needs to be registered as a Windows Service first. To do that, double click on 1_install_service.bat. If successful, a command prompt window will pop up and disappear immediately. If not, a message will appear.

Start the Service

You can start the H2 Database Engine Service using the service manager of Windows, or by double clicking on 2_start_service.bat. Please note that the batch file does not print an error message if the service is not installed.

Connect to the H2 Console

After installing and starting the service, you can connect to the H2 Console application using a browser. Double clicking on 3_start_browser.bat to do that. The default port (8082) is hard coded in the batch file.

Stop the Service

To stop the service, double click on 4_stop_service.bat. Please note that the batch file does not print an error message if the service is not installed or started.

Uninstall the Service

To uninstall the service, double click on 5_uninstall_service.bat. If successful, a command prompt window will pop up and disappear immediately. If not, a message will appear.

Additional JDBC drivers

To use other databases (for example MySQL), the location of the JDBC drivers of those databases need to be added to the environment variables H2DRIVERS or CLASSPATH before installing the service. Multiple drivers can be set; each entry needs to be separated with a ; (Windows) or : (other operating systems). Spaces in the path names are supported. The settings must not be quoted.

ODBC Driver

This database does not come with its own ODBC driver at this time, but it supports the PostgreSQL network protocol. Therefore, the PostgreSQL ODBC driver can be used. Support for the PostgreSQL network protocol is quite new and should be viewed as experimental. It should not be used for production applications.

To use the PostgreSQL ODBC driver on 64 bit versions of Windows, first run `c:/windows/syswow64/odbcad32.exe`. At this point you set up your DSN just like you would on any other system. See also: [Re: ODBC Driver on Windows 64 bit](#)

ODBC Installation

First, the ODBC driver must be installed. Any recent PostgreSQL ODBC driver should work, however version 8.2 (psqlodbc-08_02*) or newer is recommended. The Windows version of the PostgreSQL ODBC driver is available at <https://www.postgresql.org/ftp/odbc/versions/msi/>.

Starting the Server

After installing the ODBC driver, start the H2 Server using the command line:

```
java -cp h2*.jar org.h2.tools.Server
```

The PG Server (PG for PostgreSQL protocol) is started as well. By default, databases are stored in the current working directory where the server is started. Use `-baseDir` to save databases in another directory, for example the user home directory:

```
java -cp h2*.jar org.h2.tools.Server -baseDir ~
```

The PG server can be started and stopped from within a Java application as follows:

```
Server server = Server.createPgServer("-baseDir", "~");
server.start();
...
server.stop();
```

By default, only connections from localhost are allowed. To allow remote connections, use `-pgAllowOthers` when starting the server.

To map an ODBC database name to a different JDBC database name, use the option `-key` when starting the server. Please note only one mapping is allowed. The following will map the ODBC database named TEST to the database URL `jdbc:h2:~/data/test;cipher=aes`:

```
java org.h2.tools.Server -pg -key TEST "~/data/test;cipher=aes"
```

ODBC Configuration

After installing the driver, a new Data Source must be added. In Windows, run `odbcad32.exe` to open the Data Source Administrator. Then click on 'Add...' and select the PostgreSQL Unicode driver. Then click 'Finish'. You will be able to change the connection properties. The property column represents the property key in the `odbc.ini` file (which may be different from the GUI).

Property	Example	Remarks
Data Source	H2 Test	The name of the ODBC Data Source
Database	~/ test;ifexist s=true	The database name. This can include connections settings. By default, the database is stored in the current working directory where the Server is started except when the <code>-baseDir</code> setting is used. The name must be at least 3 characters.
Servename	localhost	The server name or IP address. By default, only remote connections are allowed
Username	sa	The database user name.
SSL	false (disabled)	At this time, SSL is not supported.
Port	5435	The port where the PG Server is listening.
Password	sa	The database password.

To improve performance, please enable 'server side prepare' under Options / Datasource / Page 2 / Server side prepare.

Afterwards, you may use this data source.

PG Protocol Support Limitations

At this time, only a subset of the PostgreSQL network protocol is implemented. Also, there may be compatibility problems on the SQL level, with the catalog, or with text encoding. Problems are fixed as they are found. Currently, statements can not be canceled when using the PG protocol. Also, H2 does not provide index meta over ODBC.

PostgreSQL ODBC Driver Setup requires a database password; that means it is not possible to connect to H2 databases without password. This is a limitation of the ODBC driver.

Security Considerations

Currently, the PG Server does not support challenge response or encrypt passwords. This may be a problem if an attacker can listen to the data transferred between the ODBC driver and the server, because the password is readable to the attacker. Also, it is currently not possible to use encrypted SSL connections. Therefore the ODBC driver should not be used where security is important.

The first connection that opens a database using the PostgreSQL server needs to be an administrator user. Subsequent connections don't need to be opened by an administrator.

Using Microsoft Access

When using Microsoft Access to edit data in a linked H2 table, you may need to enable the following option: Tools - Options - Edit/Find - ODBC fields.

ACID

In the database world, ACID stands for:

- Atomicity: transactions must be atomic, meaning either all tasks are performed or none.
- Consistency: all operations must comply with the defined constraints.

- Isolation: transactions must be isolated from each other.
- Durability: committed transaction will not be lost.

Atomicity

Transactions in this database are always atomic.

Consistency

By default, this database is always in a consistent state. Referential integrity rules are enforced except when explicitly disabled.

Isolation

For H2, as with most other database systems, the default isolation level is 'read committed'. This provides better performance, but also means that transactions are not completely isolated. H2 supports the transaction isolation levels 'read uncommitted', 'read committed', 'repeatable read', and 'serializable'.

Durability

This database does not guarantee that all committed transactions survive a power failure. Tests show that all databases sometimes lose transactions on power failure (for details, see below). Where losing transactions is not acceptable, a laptop or UPS (uninterruptible power supply) should be used. If durability is required for all possible cases of hardware failure, clustering should be used, such as the H2 clustering mode.

Durability Problems

Complete durability means all committed transaction survive a power failure. Some databases claim they can guarantee durability, but such claims are wrong. A durability test was run against H2, HSQLDB, PostgreSQL, and Derby. All of those databases sometimes lose committed transactions. The test is included in the H2 download, see `org.h2.test.poweroff.Test`.

Ways to (Not) Achieve Durability

Making sure that committed transactions are not lost is more complicated than it seems first. To guarantee complete durability, a database must ensure that

the log record is on the hard drive before the commit call returns. To do that, databases use different methods. One is to use the 'synchronous write' file access mode. In Java, `RandomAccessFile` supports the modes `rws` and `rwd`:

- `rwd`: every update to the file's content is written synchronously to the underlying storage device.
- `rws`: in addition to `rwd`, every update to the metadata is written synchronously.

A test (`org.h2.test.poweroff.TestWrite`) with one of those modes achieves around 50 thousand write operations per second. Even when the operating system write buffer is disabled, the write rate is around 50 thousand operations per second. This feature does not force changes to disk because it does not flush all buffers. The test updates the same byte in the file again and again. If the hard drive was able to write at this rate, then the disk would need to make at least 50 thousand revolutions per second, or 3 million RPM (revolutions per minute). There are no such hard drives. The hard drive used for the test is about 7200 RPM, or about 120 revolutions per second. There is an overhead, so the maximum write rate must be lower than that.

Calling `fsync` flushes the buffers. There are two ways to do that in Java:

- `FileDescriptor.sync()`. The documentation says that this forces all system buffers to synchronize with the underlying device. This method is supposed to return after all in-memory modified copies of buffers associated with this file descriptor have been written to the physical medium.
- `FileChannel.force()`. This method is supposed to force any updates to this channel's file to be written to the storage device that contains it.

By default, MySQL calls `fsync` for each commit. When using one of those methods, only around 60 write operations per second can be achieved, which is consistent with the RPM rate of the hard drive used. Unfortunately, even when calling `FileDescriptor.sync()` or `FileChannel.force()`, data is not always persisted to the hard drive, because most hard drives do not obey `fsync()`: see [Your Hard](#)

[Drive Lies to You](#). In Mac OS X, fsync does not flush hard drive buffers. See [Bad fsync?](#). So the situation is confusing, and tests prove there is a problem.

Trying to flush hard drive buffers is hard, and if you do the performance is very bad. First you need to make sure that the hard drive actually flushes all buffers. Tests show that this can not be done in a reliable way. Then the maximum number of transactions is around 60 per second. Because of those reasons, the default behavior of H2 is to delay writing committed transactions.

In H2, after a power failure, a bit more than one second of committed transactions may be lost. To change the behavior, use SET WRITE_DELAY and CHECKPOINT SYNC. Most other databases support commit delay as well. In the performance comparison, commit delay was used for all databases that support it.

Running the Durability Test

To test the durability / non-durability of this and other databases, you can use the test application in the package org.h2.test.poweroff. Two computers with network connection are required to run this test. One computer just listens, while the test application is run (and power is cut) on the other computer. The computer with the listener application opens a TCP/IP port and listens for an incoming connection. The second computer first connects to the listener, and then created the databases and starts inserting records. The connection is set to 'autocommit', which means after each inserted record a commit is performed automatically. Afterwards, the test computer notifies the listener that this record was inserted successfully. The listener computer displays the last inserted record number every 10 seconds. Now, switch off the power manually, then restart the computer, and run the application again. You will find out that in most cases, none of the databases contains all the records that the listener computer knows about. For details, please consult the source code of the listener and test application.

Using the Recover Tool

Traditional 2-phase Recover

The Recover tool can be used to extract the contents of a database file, even if the database is corrupted. It also extracts the content of the transaction log and large objects (CLOB or BLOB). To run the tool, type on the command line:

```
java -cp h2*.jar org.h2.tools.Recover
```

For each database in the current directory, a text-dump file (*.mv.txt) and a SQL script file (*.h2.sql) will be created. Those uncompressed files are rather large, taking approximately twice the filespace of the database.

The SQL script file contains raw insert statements (for the data) and data definition (DDL) statements to recreate the schema of the database. This file can be executed using the RunScript tool or a [RUNSCRIPT](#) SQL statement. The script includes at least one CREATE USER statement. If you run the script against a database that was created with the same user, or if there are conflicting users, running the script will fail. Consider running the script against a database that was created with a username that is not in the script.

To verify the database can recover at any time, append ;RECOVER_TEST=64 to the database URL in your test environment. This will simulate an application crash after each 64 writes to the database file. A log file named databaseName.h2.db.log is created that lists the operations. The recovery is tested using an in-memory file system, that means it may require a larger heap setting.

Direct Recover

Since H2-2.3.239 there is an enhanced DirectRecover tool which features:

- Direct output into SQL-script (using a pipe)
- Compression using ZIP, GZIP, KANZI (when in classpath) or BZip2 (when in classpath)

Especially KANZI can be useful for very large databases on servers with many CPU cores as it yields in very small SQL script files. Its compression ratio beats BZip2 at a much faster speed.

```
# Database: testdb.mv.db (1.7GB)

# parallel compression using KANZI from https://github.com/flanglet/kanzi
java -Xmx8g -cp "h2-2.3.239-SNAPSHOT.jar:kanzi-2.4.0.jar"
org.h2.tools.DirectRecover -dir ~/ -db testdb -compress kanzi

# serial compression using BZip2 from
https://dlcdn.apache.org/commons/compress/binaries/
java -Xmx8g -cp "h2-2.3.239-SNAPSHOT.jar:commons-compress-1.28.0.jar"
org.h2.tools.DirectRecover -dir ~/ -db testdb -compress bzip2

# serial compression using GZIP w/o any additional libraries
java -Xmx8g -cp "h2-2.3.239-SNAPSHOT.jar" org.h2.tools.DirectRecover -dir ~/
-db testdb -compress gzip

# resulting SQL script files on a AMD Zen3 Ryzen5:
# KANZI: testdb.h2.sql.knz ( 99.7 MB in 106 secs)
# BZip2: testdb.h2.sql.bz2 (153.7 MB in 18 mins)
# GZip: testdb.h2.sql.gz (207.4 MB in 74 secs)
```

Since H2-2.3.239 the RunScript tool can read the KANZI or BZip2 compressed script files directly when the libraries are added to the classpath:

```
#KANZI from https://github.com/flanglet/kanzi
java -Xmx8G -cp "h2-2.3.239-SNAPSHOT.jar:kanzi-2.4.0.jar"
org.h2.tools.RunScript -url jdbc:h2:~/testdb -user sa -script ~/testdb.h2.sql.knz
-options "COMPRESSION kanzi"

#BZIP2 from https://dlcdn.apache.org/commons/compress/binaries/
java -Xmx8G -cp "h2-2.3.239-SNAPSHOT.jar:commons-compress-1.28.0.jar"
org.h2.tools.RunScript -url jdbc:h2:~/testdb -user sa -script ~/testdb.h2.sql.bz2
-options "COMPRESSION bzip2"
```

File Locking Protocols

Multiple concurrent connections to the same database are supported, however a database file can only be open for reading and writing (in embedded mode) by one process at the same time. Otherwise, the processes would overwrite each others data and corrupt the database file. To protect against this problem, whenever a database is opened, a lock file is created to signal other processes that the database is in use. If the database is closed, or if the process that opened the database stops normally, this lock file is deleted.

In special cases (if the process did not terminate normally, for example because there was a power failure), the lock file is not deleted by the process that created it. That means the existence of the lock file is not a safe protocol for file locking. However, this software uses a challenge-response protocol to protect the database files. There are two methods (algorithms) implemented to provide both security (that is, the same database files cannot be opened by two processes at the same time) and simplicity (that is, the lock file does not need to be deleted manually by the user). The two methods are 'file method' and 'socket methods'.

The file locking protocols (except the file locking method 'FS') have the following limitation: if a shared file system is used, and the machine with the lock owner is sent to sleep (standby or hibernate), another machine may take over. If the machine that originally held the lock wakes up, the database may become corrupt. If this situation can occur, the application must ensure the database is closed when the application is put to sleep.

File Locking Method 'File'

The default method for database file locking for version 1.3 and older is the 'File Method'. The algorithm is:

- If the lock file does not exist, it is created (using the atomic operation `File.createNewFile`). Then, the process waits a little bit (20 ms) and checks the file again. If the file was changed during this time, the operation is aborted. This protects against a race condition when one process deletes

the lock file just after another one create it, and a third process creates the file again. It does not occur if there are only two writers.

- If the file can be created, a random number is inserted together with the locking method ('file'). Afterwards, a watchdog thread is started that checks regularly (every second once by default) if the file was deleted or modified by another (challenger) thread / process. Whenever that occurs, the file is overwritten with the old data. The watchdog thread runs with high priority so that a change to the lock file does not get through undetected even if the system is very busy. However, the watchdog thread does use very little resources (CPU time), because it waits most of the time. Also, the watchdog only reads from the hard disk and does not write to it.
- If the lock file exists and was recently modified, the process waits for some time (up to two seconds). If it was still changed, an exception is thrown (database is locked). This is done to eliminate race conditions with many concurrent writers. Afterwards, the file is overwritten with a new version (challenge). After that, the thread waits for 2 seconds. If there is a watchdog thread protecting the file, he will overwrite the change and this process will fail to lock the database. However, if there is no watchdog thread, the lock file will still be as written by this thread. In this case, the file is deleted and atomically created again. The watchdog thread is started in this case and the file is locked.

This algorithm is tested with over 100 concurrent threads. In some cases, when there are many concurrent threads trying to lock the database, they block each other (meaning the file cannot be locked by any of them) for some time.

However, the file never gets locked by two threads at the same time. However using that many concurrent threads / processes is not the common use case. Generally, an application should throw an error to the user if it cannot open a database, and not try again in a (fast) loop.

File Locking Method 'Socket'

There is a second locking mechanism implemented, but disabled by default. To use it, append ;FILE_LOCK=SOCKET to the database URL. The algorithm is:

- If the lock file does not exist, it is created. Then a server socket is opened on a defined port, and kept open. The port and IP address of the process that opened the database is written into the lock file.
- If the lock file exists, and the lock method is 'file', then the software switches to the 'file' method.
- If the lock file exists, and the lock method is 'socket', then the process checks if the port is in use. If the original process is still running, the port is in use and this process throws an exception (database is in use). If the original process died (for example due to a power failure, or abnormal termination of the virtual machine), then the port was released. The new process deletes the lock file and starts again.

This method does not require a watchdog thread actively polling (reading) the same file every second. The problem with this method is, if the file is stored on a network share, two processes (running on different computers) could still open the same database files, if they do not have a direct TCP/IP connection.

File Locking Method 'FS'

This is the default mode for version 1.4 and newer. This database file locking mechanism uses native file system lock on the database file. No *.lock.db file is created in this case, and no background thread is started. This mechanism may not work on all systems as expected. Some systems allow to lock the same file multiple times within the same virtual machine, and on some system native file locking is not supported or files are not unlocked after a power failure.

To enable this feature, append ;FILE_LOCK=FS to the database URL.

This feature is relatively new. When using it for production, please ensure your system does in fact lock files as expected.

Using Passwords

Using Secure Passwords

Remember that weak passwords can be broken regardless of the encryption and security protocols. Don't use passwords that can be found in a dictionary. Appending numbers does not make passwords secure. A way to create good passwords that can be remembered is: take the first letters of a sentence, use upper and lower case characters, and creatively include special characters (but it's more important to use a long password than to use special characters).

Example:

i'sE2rtPiUKtT from the sentence it's easy to remember this password if you know the trick.

Passwords: Using Char Arrays instead of Strings

Java strings are immutable objects and cannot be safely 'destroyed' by the application. After creating a string, it will remain in the main memory of the computer at least until it is garbage collected. The garbage collection cannot be controlled by the application, and even if it is garbage collected the data may still remain in memory. It might also be possible that the part of memory containing the password is swapped to disk (if not enough main memory is available), which is a problem if the attacker has access to the swap file of the operating system.

It is a good idea to use char arrays instead of strings for passwords. Char arrays can be cleared (filled with zeros) after use, and therefore the password will not be stored in the swap file.

This database supports using char arrays instead of string to pass user and file passwords. The following code can be used to do that:

```
import java.sql.*;
import java.util.*;
public class Test {
    public static void main(String[] args) throws Exception {
        String url = "jdbc:h2:~/test";
```

```
Properties prop = new Properties();
prop.setProperty("user", "sa");
System.out.print("Password?");
char[] password = System.console().readPassword();
prop.put("password", password);
Connection conn = null;
try {
    conn = DriverManager.getConnection(url, prop);
} finally {
    Arrays.fill(password, (char) 0);
}
conn.close();
}
```

When using Swing, use `javax.swing.JPasswordField`.

Passing the User Name and/or Password in the URL

Instead of passing the user name as a separate parameter as in `Connection conn = DriverManager. getConnection("jdbc:h2:~/test", "sa", "123");` the user name (and/or password) can be supplied in the URL itself: `Connection conn = DriverManager. getConnection("jdbc:h2:~/test;USER=sa;PASSWORD=123");` The settings in the URL override the settings passed as a separate parameter.

Password Hash

Sometimes the database password needs to be stored in a configuration file (for example in the `web.xml` file). In addition to connecting with the plain text password, this database supports connecting with the password hash. This means that only the hash of the password (and not the plain text password) needs to be stored in the configuration file. This will only protect others from reading or re-constructing the plain text password (even if they have access to the configuration file); it does not protect others from accessing the database using the password hash.

To connect using the password hash instead of plain text password, append ;PASSWORD_HASH=TRUE to the database URL, and replace the password with the password hash. To calculate the password hash from a plain text password, run the following command within the H2 Console tool: @password_hash <upperCaseUserName> <password>. As an example, if the user name is sa and the password is test, run the command @password_hash SA test. Then use the resulting password hash as you would use the plain text password. When using an encrypted database, then the user password and file password need to be hashed separately. To calculate the hash of the file password, run: @password_hash file <filePassword>.

Protection against SQL Injection

What is SQL Injection

This database engine provides a solution for the security vulnerability known as 'SQL Injection'. Here is a short description of what SQL injection means. Some applications build SQL statements with embedded user input such as:

```
String sql = "SELECT * FROM USERS WHERE PASSWORD='"+pwd+"'";  
ResultSet rs = conn.createStatement().executeQuery(sql);
```

If this mechanism is used anywhere in the application, and user input is not correctly filtered or encoded, it is possible for a user to inject SQL functionality or statements by using specially built input such as (in this example) this password: ' OR '='. In this case the statement becomes:

```
SELECT * FROM USERS WHERE PASSWORD=" OR "=";
```

Which is always true no matter what the password stored in the database is. For more information about SQL Injection, see [Glossary and Links](#).

Disabling Literals

SQL Injection is not possible if user input is not directly embedded in SQL statements. A simple solution for the problem above is to use a prepared statement:

```
String sql = "SELECT * FROM USERS WHERE PASSWORD=?";
PreparedStatement prep = conn.prepareStatement(sql);
prep.setString(1, pwd);
ResultSet rs = prep.executeQuery();
```

This database provides a way to enforce usage of parameters when passing user input to the database. This is done by disabling embedded literals in SQL statements. To do this, execute the statement:

```
SET ALLOW_LITERALS NONE;
```

Afterwards, SQL statements with text and number literals are not allowed any more. That means, SQL statement of the form WHERE NAME='abc' or WHERE CustomerId=10 will fail. It is still possible to use prepared statements and parameters as described above. Also, it is still possible to generate SQL statements dynamically, and use the Statement API, as long as the SQL statements do not include literals. There is also a second mode where number literals are allowed: SET ALLOW_LITERALS NUMBERS. To allow all literals, execute SET ALLOW_LITERALS ALL (this is the default setting). Literals can only be enabled or disabled by an administrator.

Using Constants

Disabling literals also means disabling hard-coded 'constant' literals. This database supports defining constants using the CREATE CONSTANT command. Constants can be defined only when literals are enabled, but used even when literals are disabled. To avoid name clashes with column names, constants can be defined in other schemas:

```
CREATE SCHEMA CONST AUTHORIZATION SA;
CREATE CONSTANT CONST.ACTIVE VALUE 'Active';
CREATE CONSTANT CONST.INACTIVE VALUE 'Inactive';
SELECT * FROM USERS WHERE TYPE=CONST.ACTIVE;
```

Even when literals are enabled, it is better to use constants instead of hard-coded number or text literals in queries or views. With constants, typos are found at compile time, the source code is easier to understand and change.

Using the ZERO() Function

It is not required to create a constant for the number 0 as there is already a built-in function ZERO():

```
SELECT * FROM USERS WHERE LENGTH(PASSWORD)=ZERO();
```

Protection against Remote Access

By default this database does not allow connections from other machines when starting the H2 Console, the TCP server, or the PG server. Remote access can be enabled using the command line options `-webAllowOthers`, `-tcpAllowOthers`, `-pgAllowOthers`.

If you enable remote access using `-tcpAllowOthers` or `-pgAllowOthers`, please also consider using the options `-baseDir`, so that remote users can not create new databases or access existing databases with weak passwords. When using the option `-baseDir`, only databases within that directory may be accessed. Ensure the existing accessible databases are protected using strong passwords.

If you enable remote access using `-webAllowOthers`, please ensure the web server can only be accessed from trusted networks. If this option is specified, `-webExternalNames` should be also specified with comma-separated list of external names or addresses of this server. The options `-baseDir` don't protect access to the saved connection settings, or access to other databases accessible from the system.

Restricting Class Loading and Usage

By default there is no restriction on loading classes and executing Java code for admins. That means an admin may call system functions such as `System.setProperty` by executing:

```
CREATE ALIAS SET_PROPERTY FOR "java.lang.System.setProperty";
```

```
CALL SET_PROPERTY('abc', '1');  
CREATE ALIAS GET_PROPERTY FOR "java.lang.System.getProperty";  
CALL GET_PROPERTY('abc');
```

To restrict users (including admins) from loading classes and executing code, the list of allowed classes can be set in the system property `h2.allowedClasses` in the form of a comma separated list of classes or patterns (items ending with `*`). By default all classes are allowed. Example:

```
java -Dh2.allowedClasses=java.lang.Math,com.acme.*
```

This mechanism is used for all user classes, including database event listeners, trigger classes, user-defined functions, user-defined aggregate functions, and JDBC driver classes (with the exception of the H2 driver) when using the H2 Console.

Security Protocols

The following paragraphs document the security protocols used in this database. These descriptions are very technical and only intended for security experts that already know the underlying security primitives.

User Password Encryption

When a user tries to connect to a database, the combination of user name, @, and password are hashed using SHA-256, and this hash value is transmitted to the database. This step does not protect against an attacker that re-uses the value if he is able to listen to the (unencrypted) transmission between the client and the server. But, the passwords are never transmitted as plain text, even when using an unencrypted connection between client and server. That means if a user reuses the same password for different things, this password is still protected up to some point. See also 'RFC 2617 - HTTP Authentication: Basic and Digest Access Authentication' for more information.

When a new database or user is created, a new random salt value is generated. The size of the salt is 64 bits. Using the random salt reduces the risk of an

attacker pre-calculating hash values for many different (commonly used) passwords.

The combination of user-password hash value (see above) and salt is hashed using SHA-256. The resulting value is stored in the database. When a user tries to connect to the database, the database combines user-password hash value with the stored salt value and calculates the hash value. Other products use multiple iterations (hash the hash value again and again), but this is not done in this product to reduce the risk of denial of service attacks (where the attacker tries to connect with bogus passwords, and the server spends a lot of time calculating the hash value for each password). The reasoning is: if the attacker has access to the hashed passwords, he also has access to the data in plain text, and therefore does not need the password any more. If the data is protected by storing it on another computer and only accessible remotely, then the iteration count is not required at all.

File Encryption

The database files can be encrypted using the AES-128 algorithm.

When a user tries to connect to an encrypted database, the combination of file@ and the file password is hashed using SHA-256. This hash value is transmitted to the server.

When a new database file is created, a new cryptographically secure random salt value is generated. The size of the salt is 64 bits. The combination of the file password hash and the salt value is hashed 1024 times using SHA-256. The reason for the iteration is to make it harder for an attacker to calculate hash values for common passwords.

The resulting hash value is used as the key for the block cipher algorithm. Then, an initialization vector (IV) key is calculated by hashing the key again using SHA-256. This is to make sure the IV is unknown to the attacker. The reason for using a secret IV is to protect against watermark attacks.

Before saving a block of data (each block is 8 bytes long), the following operations are executed: first, the IV is calculated by encrypting the block

number with the IV key (using the same block cipher algorithm). This IV is combined with the plain text using XOR. The resulting data is encrypted using the AES-128 algorithm.

When decrypting, the operation is done in reverse. First, the block is decrypted using the key, and then the IV is calculated combined with the decrypted text using XOR.

Therefore, the block cipher mode of operation is CBC (cipher-block chaining), but each chain is only one block long. The advantage over the ECB (electronic codebook) mode is that patterns in the data are not revealed, and the advantage over multi block CBC is that flipped cipher text bits are not propagated to flipped plaintext bits in the next block.

Database encryption is meant for securing the database while it is not in use (stolen laptop and so on). It is not meant for cases where the attacker has access to files while the database is in use. When he has write access, he can for example replace pieces of files with pieces of older versions and manipulate data like this.

File encryption slows down the performance of the database engine. Compared to unencrypted mode, database operations take about 2.5 times longer using AES (embedded mode).

Wrong Password / User Name Delay

To protect against remote brute force password attacks, the delay after each unsuccessful login gets double as long. Use the system properties `h2.delayWrongPasswordMin` and `h2.delayWrongPasswordMax` to change the minimum (the default is 250 milliseconds) or maximum delay (the default is 4000 milliseconds, or 4 seconds). The delay only applies for those using the wrong password. Normally there is no delay for a user that knows the correct password, with one exception: after using the wrong password, there is a delay of up to (randomly distributed) the same delay as for a wrong password. This is to protect against parallel brute force attacks, so that an attacker needs to wait

for the whole delay. Delays are synchronized. This is also required to protect against parallel attacks.

There is only one exception message for both wrong user and for wrong password, to make it harder to get the list of user names. It is not possible from the stack trace to see if the user name was wrong or the password.

HTTPS Connections

The web server supports HTTP and HTTPS connections using `SSLServerSocket`. There is a default self-certified certificate to support an easy starting point, but custom certificates are supported as well.

TLS Connections

Remote TLS connections are supported using the Java Secure Socket Extension (`SSLServerSocket`, `SSLSocket`).

To use your own keystore, set the system properties `javax.net.ssl.keyStore` and `javax.net.ssl.keyStorePassword` before starting the H2 server and client. See also [Customizing the Default Key and Trust Stores, Store Types, and Store Passwords](#) for more information.

Universally Unique Identifiers (UUID)

This database supports UUIDs. Also supported is a function to create new UUIDs using a cryptographically strong pseudo random number generator.

With random UUIDs, the chance of two having the same value can be calculated using the probability theory. See also 'Birthday Paradox'.

RFC 9562-compliant randomly generated UUIDs with version 4 have 122 random bits. 4 bits are used for the version (Randomly generated UUID), and 2 bits for the variant (Leach-Salz). This database supports generating such UUIDs using the built-in function `RANDOM_UUID(4)`. Please note that indexes on UUIDs with this version may have a poor performance. Here is a small program to estimate the probability of having two identical UUIDs after generating a number of values:

```

public class Test {
    public static void main(String[] args) throws Exception {
        double x = Math.pow(2, 122);
        for (int i = 35; i < 62; i++) {
            double n = Math.pow(2, i);
            double p = 1 - Math.exp(-(n * n) / 2 / x);
            System.out.println("2^" + i + "=" + (1L << i) +
                " probability: 0" +
                String.valueOf(1 + p).substring(1));
        }
    }
}

```

Some values are:

Number of UIIs	Probability of Duplicates
$2^{36}=68'719'476'736$	0.000'000'000'000'000'4
$2^{41}=2'199'023'255'552$	0.000'000'000'000'000'4
$2^{46}=70'368'744'177'664$	0.000'000'000'000'4

To help non-mathematicians understand what those numbers mean, here a comparison: one's annual risk of being hit by a meteorite is estimated to be one chance in 17 billion, that means the probability is about 0.000'000'000'06.

RFC 9562-compliant time-ordered UUIDs with version 7 have layout optimized for database systems. They contain 48-bit number of milliseconds since midnight 1 Jan 1970 UTC with leap seconds excluded and additional 12-bit sub-millisecond timestamp fraction plus 62 random bits or 74 random bits without this fraction depending on implementation.

This database supports generating such UUIDs using the built-in function `RANDOM_UUID(7)`. This function produces 12-bit sub-millisecond timestamp fraction if high resolution timestamps are available in JVM and 62 pseudo random bits.

Spatial Features

H2 supports the geometry data type and spatial indexes. Here is an example SQL script to create a table with a spatial column and index:

```
CREATE TABLE GEO_TABLE(  
  GID BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
  THE_GEOM GEOMETRY);  
INSERT INTO GEO_TABLE(THE_GEOM) VALUES  
  ('POINT(500 505)'),  
  ('LINESTRING(550 551, 525 512, 565 566)'),  
  ('POLYGON ((550 521, 580 540, 570 564, 512 566, 550 521))');  
CREATE SPATIAL INDEX GEO_TABLE_SPATIAL_INDEX  
  ON GEO_TABLE(THE_GEOM);
```

To query the table using geometry envelope intersection, use the operation `&&`, as in PostGIS:

```
SELECT * FROM GEO_TABLE  
  WHERE THE_GEOM &&  
    'POLYGON ((490 490, 536 490, 536 515, 490 515, 490 490))';
```

You can verify that the spatial index is used using the "explain plan" feature:

```
EXPLAIN SELECT * FROM GEO_TABLE  
  WHERE THE_GEOM &&  
    'POLYGON ((490 490, 536 490, 536 515, 490 515, 490 490))';  
-- Result  
SELECT  
  "PUBLIC"."GEO_TABLE"."GID",  
  "PUBLIC"."GEO_TABLE"."THE_GEOM"  
FROM "PUBLIC"."GEO_TABLE"  
  /* PUBLIC.GEO_TABLE_SPATIAL_INDEX: THE_GEOM &&  
    GEOMETRY 'POLYGON ((490 490, 536 490, 536 515, 490 515, 490 490))' */  
WHERE "THE_GEOM" &&  
  GEOMETRY 'POLYGON ((490 490, 536 490, 536 515, 490 515, 490 490))'
```

For persistent databases, the spatial index is stored on disk; for in-memory databases, the index is kept in memory.

Recursive Queries

H2 has experimental support for recursive queries using so called "common table expressions" (CTE). Examples:

```
WITH RECURSIVE T(N) AS (  
  SELECT 1  
  UNION ALL  
  SELECT N+1 FROM T WHERE N<10  
)  
SELECT * FROM T;  
-- returns the values 1 .. 10
```

```
WITH RECURSIVE T(N) AS (  
  SELECT 1  
  UNION ALL  
  SELECT N*2 FROM T WHERE N<10  
)  
SELECT * FROM T;  
-- returns the values 1, 2, 4, 8, 16
```

```
CREATE TABLE FOLDER(ID INT PRIMARY KEY, NAME VARCHAR(255), PARENT  
INT);
```

```
INSERT INTO FOLDER VALUES(1, null, null), (2, 'src', 1),  
(3, 'main', 2), (4, 'org', 3), (5, 'test', 2);
```

```
WITH LINK(ID, NAME, LEVEL) AS (  
  SELECT ID, NAME, 0 FROM FOLDER WHERE PARENT IS NULL  
  UNION ALL  
  SELECT FOLDER.ID, COALESCE(LINK.NAME || '/', '') || FOLDER.NAME, LEVEL +  
1  
  FROM LINK INNER JOIN FOLDER ON LINK.ID = FOLDER.PARENT  
)  
SELECT NAME FROM LINK WHERE NAME IS NOT NULL ORDER BY ID;  
-- src  
-- src/main  
-- src/main/org
```

```
-- src/test
```

Limitations: Recursive queries need to be of the type UNION ALL, and the recursion needs to be on the second part of the query. No tables or views with the name of the table expression may exist. Different table expression names need to be used when using multiple distinct table expressions within the same transaction and for the same session. All columns of the table expression are of type VARCHAR, and may need to be cast to the required data type. Views with recursive queries are not supported. Subqueries and INSERT INTO ... FROM with recursive queries are not supported. Parameters are only supported within the last SELECT statement (a workaround is to use session variables like @start within the table expression). The syntax is:

```
WITH RECURSIVE recursiveQueryName(columnName, ...) AS (  
    nonRecursiveSelect  
    UNION ALL  
    recursiveSelect  
)  
select
```

Settings Read from System Properties

Some settings of the database can be set on the command line using -DpropertyName=value. It is usually not required to change those settings manually. The settings are case sensitive. Example:

```
java -Dh2.serverCachedObjects=256 org.h2.tools.Server
```

The current value of the settings can be read in the table INFORMATION_SCHEMA.SETTINGS.

For a complete list of settings, see [SysProperties](#).

Setting the Server Bind Address

Usually server sockets accept connections on any/all local addresses. This may be a problem on multi-homed hosts. To bind only to one address, use the

system property `h2.bindAddress`. This setting is used for both regular server sockets and for TLS server sockets. IPv4 and IPv6 address formats are supported.

Pluggable File System

This database supports a pluggable file system API. The file system implementation is selected using a file name prefix. Internally, the interfaces are very similar to the Java 7 NIO2 API. The following file systems are included:

- `file`: the default file system that uses `FileChannel`.
- `zip`: read-only zip-file based file system. Format: `zip:~/zipFileName!/fileName`.
- `split`: file system that splits files in 1 GB files (stackable with other file systems).
- `nioMapped`: file system that uses memory mapped files (faster in some operating systems). Please note that there currently is a file size limitation of 2 GB when using this file system. To work around this limitation, combine it with the split file system: `split:nioMapped:~/test`.
- `async`: experimental file system that uses `AsynchronousFileChannel` instead of `FileChannel` (faster in some operating systems).
- `memFS`: in-memory file system (slower than `mem`; experimental; mainly used for testing the database engine itself).
- `memLZF`: compressing in-memory file system (slower than `memFS` but uses less memory; experimental; mainly used for testing the database engine itself).
- `nioMemFS`: stores data outside of the VM's heap - useful for large memory DBs without incurring GC costs.
- `nioMemLZF`: stores compressed data outside of the VM's heap - useful for large memory DBs without incurring GC costs. Use `"nioMemLZF:12:"` to tweak the % of blocks that are stored uncompressed. If you size this to your working set correctly, compressed storage is roughly the same performance as uncompressed. The default value is 1%.

As an example, to use the async: file system use the following database URL:
jdbc:h2:async:~/test.

To register a new file system, extend the classes `org.h2.store.fs.FilePath`, `FileBase`, and call the method `FilePath.register` before using it.

For input streams (but not for random access files), URLs may be used in addition to the registered file systems. Example:

`jar:file:///c:/temp/example.zip!/org/example/nested.csv`. To read a stream from the classpath, use the prefix `classpath:`, as in `classpath:/org/h2/samples/newsfeed.sql`.

Split File System

The file system prefix `split:` is used to split logical files into multiple physical files, for example so that a database can get larger than the maximum file system size of the operating system. If the logical file is larger than the maximum file size, then the file is split as follows:

- `<fileName>` (first block, is always created)
- `<fileName>.1.part` (second block)

More physical files (`*.2.part`, `*.3.part`) are automatically created / deleted if needed. The maximum physical file size of a block is 2^{30} bytes, which is also called 1 GiB or 1 GB. However this can be changed if required, by specifying the block size in the file name. The file name format is: `split:<x>:<fileName>` where the file size per block is 2^x . For 1 MiB block sizes, use $x = 20$ (because 2^{20} is 1 MiB). The following file name means the logical file is split into 1 MiB blocks: `split:20:~/test.h2.db`. An example database URL for this case is `jdbc:h2:split:20:~/test`.

Java Objects Serialization

Java objects serialization is enabled by default for columns of type `OTHER`, using standard Java serialization/deserialization semantics.

To disable this feature set the system property `h2.serializeJavaObject=false` (default: `true`).

Serialization and deserialization of java objects is customizable both at system level and at database level providing a [JavaObjectSerializer](#) implementation:

- At system level set the system property `h2.javaObjectSerializer` with the Fully Qualified Name of the `JavaObjectSerializer` interface implementation. It will be used over the entire JVM session to (de)serialize java objects being stored in column of type `OTHER`. Example
`h2.javaObjectSerializer=com.acme.SerializerClassName.`
- At database level execute the SQL statement `SET JAVA_OBJECT_SERIALIZER 'com.acme.SerializerClassName'` or append
`;JAVA_OBJECT_SERIALIZER='com.acme.SerializerClassName'` to the database URL:
`jdbc:h2:~/test;JAVA_OBJECT_SERIALIZER='com.acme.SerializerClassName'.`

Please note that this SQL statement can only be executed before any tables are defined.

Limits and Limitations

This database has the following known limitations:

- Database file size limit: 4 TB (using the default page size of 2 KB) or higher (when using a larger page size). This limit is including CLOB and BLOB data.
- The maximum file size for FAT or FAT32 file systems is 4 GB. That means when using FAT or FAT32, the limit is 4 GB for the data. This is the limitation of the file system. The database does provide a workaround for this problem, it is to use the file name prefix split:. In that case files are split into files of 1 GB by default. An example database URL is:
`jdbc:h2:split:~/test.`
- The maximum number of rows per table is 2^{64} .
- The maximum number of open transactions is 65535.
- The maximum number of columns in a table or expressions in a SELECT statement is 16384. The actual possible number can be smaller if their definitions are too long.

- The maximum length of an identifier (table name, column name, and so on) is 256 characters.
- The maximum length of CHARACTER, CHARACTER VARYING and VARCHAR_IGNORECASE values and columns is 1_000_000_000 characters.
- The maximum length of BINARY, BINARY VARYING, JAVA_OBJECT, GEOMETRY, and JSON values and columns is 1_000_000_000 bytes.
- The maximum precision of NUMERIC and DECFLOAT values and columns is 100000.
- The maximum length of an ENUM value is 1048576 characters, the maximum number of ENUM values is 65536.
- The maximum cardinality of an ARRAY value or column is 65536.
- The maximum degree of a ROW value or column is 16384.
- The maximum index of parameter is 100000.
- Main memory requirements: The larger the database, the more main memory is required.
- Limit on the complexity of SQL statements. Very complex expressions may result in a stack overflow exception.
- There is no limit for the following entities, except the memory and storage capacity: maximum number of tables, indexes, triggers, and other database objects; maximum statement length, tables per statement; maximum rows per query; maximum indexes per table, lob columns per table, and so on; maximum row length, index row length, select row length.
- Querying from the metadata tables is slow if there are many tables (thousands).
- For other limitations on data types, see the data type documentation of this database.

Glossary and Links

Term	Description
AES-128	A block encryption algorithm. See also: Wikipedia: Advanced Encryption Standard

Birthday Paradox	Describes the higher than expected probability that two persons in a room have the same birthday. Also valid for randomly generated UUIDs. See also: Wikipedia: Birthday problem
Digest	Protocol to protect a password (but not to protect data). See also: RFC 2617: HTTP Digest Access Authentication
HTTPS	A protocol to provide security to HTTP connections. See also: RFC 2818: HTTP Over TLS
Modes of Operation	Wikipedia: Block cipher mode of operation
Salt	Random number to increase the security of passwords. See also: Wikipedia: Key derivation function
SHA-256	A cryptographic one-way hash function. See also: Wikipedia: Secure Hash Algorithms
SQL Injection	A security vulnerability where an application embeds SQL statements or expressions in user input. See also: Wikipedia: SQL injection
Watermark Attack	Security problem of certain encryption programs where the existence of certain data can be proven without decrypting. For more information, search in the internet for 'watermark attack cryptoloop'
SSL/TLS	Secure Sockets Layer / Transport Layer Security. See also: Java Secure Socket Extension (JSSE)

Commands

Index

Commands (Data Manipulation)

SELECT

INSERT

UPDATE

DELETE

BACKUP

CALL

EXECUTE IMMEDIATE

EXPLAIN

MERGE INTO

MERGE USING

RUNSCRIPT

SCRIPT

SHOW

Explicit table

Table value

WITH

Commands (Data Definition)

ALTER DOMAIN

ALTER DOMAIN ADD CONSTRAINT

ALTER DOMAIN DROP CONSTRAINT

ALTER DOMAIN RENAME

ALTER DOMAIN RENAME CONSTRAINT

ALTER INDEX RENAME

ALTER SCHEMA RENAME

ALTER SEQUENCE

ALTER TABLE ADD

ALTER TABLE ADD CONSTRAINT
ALTER TABLE RENAME CONSTRAINT
ALTER TABLE ALTER COLUMN
ALTER TABLE DROP COLUMN
ALTER TABLE DROP CONSTRAINT
ALTER TABLE SET
ALTER TABLE RENAME
ALTER USER ADMIN
ALTER USER RENAME
ALTER USER SET PASSWORD
ALTER VIEW RECOMPILE
ALTER VIEW RENAME
ANALYZE
COMMENT ON
CREATE AGGREGATE
CREATE ALIAS
CREATE CONSTANT
CREATE DOMAIN
CREATE INDEX
CREATE LINKED TABLE
CREATE ROLE
CREATE SCHEMA
CREATE SEQUENCE
CREATE TABLE
CREATE TRIGGER
CREATE USER
CREATE VIEW
CREATE MATERIALIZED VIEW
DROP AGGREGATE
DROP ALIAS
DROP ALL OBJECTS
DROP CONSTANT

DROP DOMAIN
DROP INDEX
DROP ROLE
DROP SCHEMA
DROP SEQUENCE
DROP TABLE
DROP TRIGGER
DROP USER
DROP VIEW
DROP MATERIALIZED VIEW
REFRESH MATERIALIZED VIEW
TRUNCATE TABLE
ALTER TYPE

Commands (Other)

CHECKPOINT
CHECKPOINT SYNC
COMMIT
COMMIT TRANSACTION
GRANT RIGHT
GRANT ALTER ANY SCHEMA
GRANT ROLE
HELP
PREPARE COMMIT
REVOKE RIGHT
REVOKE ALTER ANY SCHEMA
REVOKE ROLE
ROLLBACK
ROLLBACK TRANSACTION
SAVEPOINT
SET @
SET ALLOW_LITERALS
SET AUTOCOMMIT

SET CACHE_SIZE
SET CLUSTER
SET BUILTIN_ALIAS_OVERRIDE
SET CATALOG
SET COLLATION
SET DATABASE_EVENT_LISTENER
SET DB_CLOSE_DELAY
SET DEFAULT_LOCK_TIMEOUT
SET DEFAULT_NULL_ORDERING
SET DEFAULT_TABLE_TYPE
SET EXCLUSIVE
SET IGNORECASE
SET IGNORE_CATALOGS
SET JAVA_OBJECT_SERIALIZER
SET LAZY_QUERY_EXECUTION
SET LOCK_MODE
SET LOCK_TIMEOUT
SET MAX_LENGTH_INPLACE_LOB
SET MAX_LOG_SIZE
SET MAX_MEMORY_ROWS
SET MAX_MEMORY_UNDO
SET MAX_OPERATION_MEMORY
SET MODE
SET NON_KEYWORDS
SET OPTIMIZE_REUSE_RESULTS
SET PASSWORD
SET QUERY_STATISTICS
SET QUERY_STATISTICS_MAX_ENTRIES
SET QUERY_TIMEOUT
SET REFERENTIAL_INTEGRITY
SET RETENTION_TIME
SET SALT_HASH

SET SCHEMA
SET SCHEMA_SEARCH_PATH
SET SESSION CHARACTERISTICS
SET THROTTLE
SET TIME_ZONE
SET TRACE_LEVEL
SET TRACE_MAX_FILE_SIZE
SET TRUNCATE_LARGE_LENGTH
SET VARIABLE_BINARY
SET WRITE_DELAY
SHUTDOWN

Details

Non-standard syntax is marked in green. Compatibility-only non-standard syntax is marked in red, don't use it unless you need it for compatibility with other databases or old versions of H2.

Commands (Data Manipulation)

SELECT

```
SELECT [ DISTINCT [ ON ( expression [,...] ) ] | ALL ]  
selectExpression [,...]  
[ FROM tableExpression [,...] ]  
[ WHERE expression ]  
[ GROUP BY groupingElement [,...] ] [ HAVING expression ]  
[ WINDOW { { windowName AS windowSpecification } [,...] } ]  
[ QUALIFY expression ]  
[ { UNION [ ALL ] | EXCEPT | INTERSECT } query ]  
[ ORDER BY selectOrder [,...] ]  
[ OFFSET expression { ROW | ROWS } ]  
[ FETCH { FIRST | NEXT } [ expression [ PERCENT ] ] { ROW | ROWS }  
{ ONLY | WITH TIES } ]  
[ FOR UPDATE [ NOWAIT | WAIT secondsNumeric | SKIP LOCKED ] ]
```

Selects data from a table or multiple tables.

Command is executed in the following logical order:

1. Data is taken from table value expressions that are specified in the FROM clause, joins are executed. If FROM clause is not specified a single row is constructed.
2. WHERE filters rows. Aggregate or window functions are not allowed in this clause.
3. GROUP BY groups the result by the given expression(s). If GROUP BY clause is not specified, but non-window aggregate functions are used or HAVING is specified all rows are grouped together.
4. Aggregate functions are evaluated.
5. HAVING filters rows after grouping and evaluation of aggregate functions. Non-window aggregate functions are allowed in this clause.
6. Window functions are evaluated.
7. QUALIFY filters rows after evaluation of window functions. Aggregate and window functions are allowed in this clause.
8. DISTINCT removes duplicates. If DISTINCT ON is used only the specified expressions are checked for duplicates; ORDER BY clause, if any, is used to determine preserved rows. First row in each DISTINCT ON group is preserved. In absence of ORDER BY preserved rows are not determined, database may choose any row from each DISTINCT ON group.
9. UNION, EXCEPT, and INTERSECT combine the result of this query with the results of another query. INTERSECT has higher precedence than UNION and EXCEPT. Operators with equal precedence are evaluated from left to right.
10. ORDER BY sorts the result by the given column(s) or expression(s).
11. Number of rows in output can be limited with OFFSET and FETCH clauses. OFFSET specifies how many rows to skip. Please note that queries with high offset values can be slow. FETCH FIRST/NEXT limits the number of rows returned by the query. If PERCENT is specified number of rows is specified as a percent of the total number of rows and should be an integer value between 0

and 100 inclusive. WITH TIES can be used only together with ORDER BY and means that all additional rows that have the same sorting position as the last row will be also returned.

WINDOW clause specifies window definitions for window functions and window aggregate functions. This clause can be used to reuse the same definition in multiple functions.

If FOR UPDATE is specified, the tables or rows are locked for writing. If some rows are locked by another session, this query will wait some time for release of these locks, unless NOWAIT or SKIP LOCKED is specified. If SKIP LOCKED is specified, these locked rows will be excluded from result of this query. If NOWAIT is specified, presence of these rows will stop execution of this query immediately. If WAIT with timeout is specified and some rows are locked by another session, this timeout will be used instead of default timeout for this session. Please note that with current implementation the timeout doesn't limit execution time of the whole query, it only limits wait time for completion of particular transaction that holds a lock on a row selected by this query.

This clause is not allowed in DISTINCT queries and in queries with non-window aggregates, GROUP BY, or HAVING clauses. Only the selected rows are locked as in an UPDATE statement. Rows from the right side of a left join and from the left side of a right join, including nested joins, aren't locked. Locking behavior for rows that were excluded from result using OFFSET / FETCH / LIMIT / TOP or QUALIFY is undefined, to avoid possible locking of excessive rows try to filter out unneeded rows with the WHERE criteria when possible. Rows are processed one by one. Each row is read, tested with WHERE criteria, locked, read again and re-tested, because its value may be changed by concurrent transaction before lock acquisition. Note that new uncommitted rows from other transactions are not visible unless read uncommitted isolation level is used and therefore cannot be selected and locked. Modified uncommitted rows from other transactions that satisfy the WHERE criteria cause this SELECT to wait for commit or rollback of those transactions.

Example:

```

SELECT * FROM TEST;
SELECT * FROM TEST ORDER BY NAME;
SELECT ID, COUNT(*) FROM TEST GROUP BY ID;
SELECT NAME, COUNT(*) FROM TEST GROUP BY NAME HAVING COUNT(*) > 2;
SELECT 'ID' COL, MAX(ID) AS MAX FROM TEST UNION SELECT 'NAME',
MAX(NAME) FROM TEST;
SELECT * FROM TEST OFFSET 1000 ROWS FETCH FIRST 1000 ROWS ONLY;
SELECT A, B FROM TEST ORDER BY A FETCH FIRST 10 ROWS WITH TIES;
SELECT * FROM (SELECT ID, COUNT(*) FROM TEST
    GROUP BY ID UNION SELECT NULL, COUNT(*) FROM TEST)
    ORDER BY 1 NULLS LAST;
SELECT DISTINCT C1, C2 FROM TEST;
SELECT DISTINCT ON(C1) C1, C2 FROM TEST ORDER BY C1;
SELECT ID, V FROM TEST WHERE ID IN (1, 2, 3) FOR UPDATE WAIT 0.5;

```

INSERT

```

INSERT INTO [schemaName.]tableName [ ( columnName [,...] ) ]
{ [ overrideClause ] { insertValues | [ DIRECT ] query } }
| DEFAULT VALUES

```

Inserts a new row / new rows into a table.

If column names aren't specified a list of all visible columns in the table is assumed.

When using DIRECT, then the results from the query are directly applied in the target table without any intermediate step.

Example:

```
INSERT INTO TEST VALUES(1, 'Hello')
```

UPDATE

```

UPDATE [schemaName.]tableName [ [ AS ] newTableAlias ] SET setClauseList
[ WHERE expression ] [ ORDER BY sortSpecificationList ]
FETCH { FIRST | NEXT } [ expression ] { ROW | ROWS } ONLY

```


Updates data in a table. ORDER BY is supported for MySQL compatibility, but it is ignored. If FETCH is specified, at most the specified number of rows are updated (no limit if null or smaller than zero).

Example:

```
UPDATE TEST SET NAME='Hi' WHERE ID=1;
```

```
UPDATE PERSON P SET NAME=(SELECT A.NAME FROM ADDRESS A WHERE  
A.ID=P.ID);
```

DELETE

```
DELETE FROM [schemaName.]tableName  
[ WHERE expression ]  
FETCH { FIRST | NEXT } [ expression ] { ROW | ROWS } ONLY
```

Deletes rows from a table. If FETCH is specified, at most the specified number of rows are deleted (no limit if null or smaller than zero).

Example:

```
DELETE FROM TEST WHERE ID=2
```

BACKUP

```
BACKUP TO fileNameString
```

Backs up the database files to a .zip file. Objects are not locked, but the backup is transactionally consistent because the transaction log is also copied. Admin rights are required to execute this command.

Example:

```
BACKUP TO 'backup.zip'
```

CALL

```
CALL expression
```

Calculates a simple expression. This statement returns a result set with one row, except if the called function returns a result set itself. If the called function returns an array, then each element in this array is returned as a column.

Example:

```
CALL 15*25
```

EXECUTE IMMEDIATE

```
EXECUTE IMMEDIATE sqlString
```

Dynamically prepares and executes the SQL command specified as a string. Query commands may not be used.

Example:

```
EXECUTE IMMEDIATE 'ALTER TABLE TEST DROP CONSTRAINT ' ||  
    QUOTE_IDENT((SELECT CONSTRAINT_NAME  
        FROM INFORMATION_SCHEMA.TABLE_CONSTRAINTS  
        WHERE TABLE_SCHEMA = 'PUBLIC' AND TABLE_NAME = 'TEST'  
        AND CONSTRAINT_TYPE = 'UNIQUE'));
```

EXPLAIN

```
EXPLAIN { [ PLAN FOR ] | ANALYZE }  
{ query | insert | update | delete | mergeInto | mergeUsing }
```

Shows the execution plan for a statement. When using EXPLAIN ANALYZE, the statement is actually executed, and the query plan will include the actual row scan count for each table.

Example:

```
EXPLAIN SELECT * FROM TEST WHERE ID=1
```

MERGE INTO

```
MERGE INTO [schemaName.]tableName [ ( columnName [...]) ]  
[ KEY ( columnName [...]) ]  
{ insertValues | query }
```

Updates existing rows, and insert rows that don't exist. If no key column is specified, the primary key columns are used to find the row. If more than one row per new row is affected, an exception is thrown.

Example:

```
MERGE INTO TEST KEY(ID) VALUES(2, 'World')
```

MERGE USING

```
MERGE INTO [schemaName.]targetTableName [ [AS] targetAlias]  
USING tableExpression  
ON expression  
mergeWhenClause [...]
```

Updates or deletes existing rows, and insert rows that don't exist.

The ON clause specifies the matching column expression.

Different rows from a source table may not match with the same target row (this is not ensured by H2 if target table is an updatable view). One source row may be matched with multiple target rows.

If statement doesn't need a source table a DUAL table can be substituted.

Example:

```
MERGE INTO TARGET_TABLE AS T USING SOURCE_TABLE AS S  
  ON T.ID = S.ID  
  WHEN MATCHED AND T.COL2 <> 'FINAL' THEN  
    UPDATE SET T.COL1 = S.COL1  
  WHEN MATCHED AND T.COL2 = 'FINAL' THEN  
    DELETE  
  WHEN NOT MATCHED THEN  
    INSERT (ID, COL1, COL2) VALUES(S.ID, S.COL1, S.COL2);  
MERGE INTO TARGET_TABLE AS T USING (SELECT * FROM SOURCE_TABLE) AS S  
  ON T.ID = S.ID  
  WHEN MATCHED AND T.COL2 <> 'FINAL' THEN  
    UPDATE SET T.COL1 = S.COL1
```

```

WHEN MATCHED AND T.COL2 = 'FINAL' THEN
    DELETE
WHEN NOT MATCHED THEN
    INSERT VALUES (S.ID, S.COL1, S.COL2);
MERGE INTO TARGET T USING (VALUES (1, 4), (2, 15)) S(ID, V)
    ON T.ID = S.ID
    WHEN MATCHED THEN UPDATE SET V = S.V
    WHEN NOT MATCHED THEN INSERT VALUES (S.ID, S.V);
MERGE INTO TARGET_TABLE USING DUAL ON ID = 1
    WHEN NOT MATCHED THEN INSERT VALUES (1, 'Test')
    WHEN MATCHED THEN UPDATE SET NAME = 'Test';

```

RUNSCRIPT

```

RUNSCRIPT FROM fileNameString scriptCompressionEncryption
[ CHARSET charsetString ]
{ [ QUIRKS_MODE ] [ VARIABLE_BINARY ] | FROM_1X }

```

Runs a SQL script from a file. The script is a text file containing SQL statements; each statement must end with ';'. This command can be used to restore a database from a backup. The password must be in single quotes; it is case sensitive and can contain spaces.

Instead of a file name, a URL may be used. To read a stream from the classpath, use the prefix 'classpath:'. See the [Pluggable File System](#) section.

The compression algorithm must match the one used when creating the script. Instead of a file, a URL may be used.

If QUIRKS_MODE is specified, the various compatibility quirks for scripts from older versions of H2 are enabled. Use this clause when you import script that was generated by H2 1.4.200 or an older version into more recent version.

If VARIABLE_BINARY is specified, the BINARY data type will be parsed as VARBINARY. Use this clause when you import script that was generated by H2 1.4.200 or an older version into more recent version.

If FROM_1X is specified, quirks for scripts exported from H2 1.*.* are enabled. Use this flag to populate a new database with the data exported from 1.*.* versions of H2. This flag also enables QUIRKS_MODE and VARIABLE_BINARY implicitly.

Admin rights are required to execute this command.

Example:

```
RUNSCRIPT FROM 'backup.sql'
```

```
RUNSCRIPT FROM 'classpath:/com/acme/test.sql'
```

```
RUNSCRIPT FROM 'dump_from_1_4_200.sql' FROM_1X
```

SCRIPT

```
SCRIPT { [ NODATA ] | [ SIMPLE ] [ COLUMNS ] }  
[ NOPASSWORDS ] [ NOSETTINGS ]  
[ DROP ] [ BLOCKSIZE blockSizeInt ]  
[ TO fileNameString scriptCompressionEncryption  
[ CHARSET charsetString ] ]  
[ TABLE [schemaName.]tableName [, ...] ]  
[ SCHEMA schemaName [, ...] ]
```

Creates a SQL script from the database.

NODATA will not emit INSERT statements. SIMPLE does not use multi-row insert statements. COLUMNS includes column name lists into insert statements. If the DROP option is specified, drop statements are created for tables, views, and sequences. If the block size is set, CLOB and BLOB values larger than this size are split into separate blocks. BLOCKSIZE is used when writing out LOB data, and specifies the point at the values transition from being inserted as inline values, to be inserted using out-of-line commands. NOSETTINGS turns off dumping the database settings (the SET XXX commands)

If no 'TO fileName' clause is specified, the script is returned as a result set. This command can be used to create a backup of the database. For long term storage, it is more portable than copying the database files.

If a 'TO fileName' clause is specified, then the whole script (including insert statements) is written to this file, and a result set without the insert statements is returned.

The password must be in single quotes; it is case sensitive and can contain spaces.

This command locks objects while it is running. Admin rights are required to execute this command.

When using the TABLE or SCHEMA option, only the selected table(s) / schema(s) are included.

Example:

```
SCRIPT NODATA
```

SHOW

```
SHOW { SCHEMAS | TABLES [ FROM schemaName ] |  
COLUMNS FROM tableName [ FROM schemaName ] }
```

Lists the schemas, tables, or the columns of a table.

Example:

```
SHOW TABLES
```

Explicit table

```
TABLE [schemaName.]tableName  
[ ORDER BY selectOrder [,...] ]  
[ OFFSET expression { ROW | ROWS } ]  
[ FETCH { FIRST | NEXT } [ expression [ PERCENT ] ] { ROW | ROWS }  
{ ONLY | WITH TIES } ]
```

Selects data from a table.

This command is an equivalent to `SELECT * FROM tableName`. See [SELECT](#) command for description of ORDER BY, OFFSET, and FETCH.

Example:

TABLE TEST;

TABLE TEST ORDER BY ID FETCH FIRST ROW ONLY;

Table value

```
VALUES rowValueExpression [...]  
[ ORDER BY selectOrder [...] ]  
[ OFFSET expression { ROW | ROWS } ]  
[ FETCH { FIRST | NEXT } [ expression [ PERCENT ] ] { ROW | ROWS }  
{ ONLY | WITH TIES } ]
```

A list of rows that can be used like a table. See [SELECT](#) command for description of ORDER BY, OFFSET, and FETCH. The column list of the resulting table is C1, C2, and so on.

Example:

```
VALUES (1, 'Hello'), (2, 'World');
```

WITH

```
WITH [ RECURSIVE ] { name [( columnName [...] ) ] AS ( query ) [...] }  
query
```

Can be used to create a recursive or non-recursive query (common table expression). For recursive queries the first select has to be a UNION. One or more common table entries can be referred to by name. Column name declarations are optional - the column names will be inferred from the named select queries.

Example:

```
WITH RECURSIVE cte(n) AS (  
    SELECT 1  
    UNION ALL  
    SELECT n + 1  
    FROM cte  
    WHERE n < 100  
)
```

```
SELECT sum(n) FROM cte;
```

Example 2:

```
WITH cte1 AS (  
    SELECT 1 AS FIRST_COLUMN  
) , cte2 AS (  
    SELECT FIRST_COLUMN+1 AS FIRST_COLUMN FROM cte1  
)  
SELECT sum(FIRST_COLUMN) FROM cte2;
```

Commands (Data Definition)

ALTER DOMAIN

```
ALTER DOMAIN [ IF EXISTS ] [schemaName.]domainName  
{ SET DEFAULT expression }  
| { DROP DEFAULT }  
| { SET ON UPDATE expression }  
| { DROP ON UPDATE }
```

Changes the default or on update expression of a domain. Schema owner rights are required to execute this command.

SET DEFAULT changes the default expression of a domain.

DROP DEFAULT removes the default expression of a domain. Old expression is copied into domains and columns that use this domain and don't have an own default expression.

SET ON UPDATE changes the expression that is set on update if value for this domain is not specified in update statement.

DROP ON UPDATE removes the expression that is set on update of a column with this domain. Old expression is copied into domains and columns that use this domain and don't have an own on update expression.

This command commits an open transaction in this connection.

Example:


```
ALTER DOMAIN D1 SET DEFAULT "";
ALTER DOMAIN D1 DROP DEFAULT;
ALTER DOMAIN D1 SET ON UPDATE CURRENT_TIMESTAMP;
ALTER DOMAIN D1 DROP ON UPDATE;
```

ALTER DOMAIN ADD CONSTRAINT

```
ALTER DOMAIN [ IF EXISTS ] [schemaName.]domainName
ADD [ constraintNameDefinition ]
CHECK (condition) [ CHECK | NOCHECK ]
```

Adds a constraint to a domain. Schema owner rights are required to execute this command. This command commits an open transaction in this connection.

Example:

```
ALTER DOMAIN D ADD CONSTRAINT D_POSITIVE CHECK (VALUE > 0)
```

ALTER DOMAIN DROP CONSTRAINT

```
ALTER DOMAIN [ IF EXISTS ] [schemaName.]domainName
DROP CONSTRAINT [ IF EXISTS ] [schemaName.]constraintName
```

Removes a constraint from a domain. Schema owner rights are required to execute this command. This command commits an open transaction in this connection.

Example:

```
ALTER DOMAIN D DROP CONSTRAINT D_POSITIVE
```

ALTER DOMAIN RENAME

```
ALTER DOMAIN [ IF EXISTS ] [schemaName.]domainName RENAME TO
newName
```

Renames a domain. Schema owner rights are required to execute this command. This command commits an open transaction in this connection.

Example:

ALTER DOMAIN TEST RENAME TO MY_TYPE

ALTER DOMAIN RENAME CONSTRAINT

```
ALTER DOMAIN [ IF EXISTS ] [schemaName.]domainName  
RENAME CONSTRAINT [schemaName.]oldConstraintName  
TO newConstraintName
```

Renames a constraint. This command commits an open transaction in this connection.

Example:

ALTER DOMAIN D RENAME CONSTRAINT FOO TO BAR

ALTER INDEX RENAME

```
ALTER INDEX [ IF EXISTS ] [schemaName.]indexName RENAME TO  
newIndexName
```

Renames an index. This command commits an open transaction in this connection.

Example:

ALTER INDEX IDXNAME RENAME TO IDX_TEST_NAME

ALTER SCHEMA RENAME

```
ALTER SCHEMA [ IF EXISTS ] schemaName RENAME TO newSchemaName
```

Renames a schema. Schema admin rights are required to execute this command. This command commits an open transaction in this connection.

Example:

ALTER SCHEMA TEST RENAME TO PRODUCTION

ALTER SEQUENCE

```
ALTER SEQUENCE [ IF EXISTS ] [schemaName.]sequenceName  
alterSequenceOption [...]
```

Changes the parameters of a sequence. Schema owner rights are required to execute this command. This command does not commit the current transaction; however the new value is used by other transactions immediately, and rolling back this command has no effect.

Example:

```
ALTER SEQUENCE SEQ_ID RESTART WITH 1000
```

ALTER TABLE ADD

```
ALTER TABLE [ IF EXISTS ] [schemaName.]tableName ADD [ COLUMN ]
{ [ IF NOT EXISTS ] columnName columnDefinition [ USING
initialValueExpression ]
| ( ( { columnName columnDefinition | tableConstraintDefinition } [...]) ) }
[ { { BEFORE | AFTER } columnName } | FIRST ]
```

Adds a new column to a table. This command commits an open transaction in this connection.

If USING is specified the provided expression is used to generate initial value of the new column for each row. The expression may reference existing columns of the table. Otherwise the DEFAULT expression is used, if any. If neither USING nor DEFAULT are specified, the NULL is used.

Example:

```
ALTER TABLE TEST ADD CREATEDATE TIMESTAMP
```

ALTER TABLE ADD CONSTRAINT

```
ALTER TABLE [ IF EXISTS ] tableName ADD tableConstraintDefinition [ CHECK |
NOCHECK ]
```

Adds a constraint to a table. If NOCHECK is specified, existing rows are not checked for consistency (the default is to check consistency for existing rows). The required indexes are automatically created if they don't exist yet. It is not possible to disable checking for unique constraints. This command commits an open transaction in this connection.

Example:

```
ALTER TABLE TEST ADD CONSTRAINT NAME_UNIQUE UNIQUE(NAME)
```

ALTER TABLE RENAME CONSTRAINT

```
ALTER TABLE [ IF EXISTS ] [schemaName.]tableName  
RENAME CONSTRAINT [schemaName.]oldConstraintName  
TO newConstraintName
```

Renames a constraint. This command commits an open transaction in this connection.

Example:

```
ALTER TABLE TEST RENAME CONSTRAINT FOO TO BAR
```

ALTER TABLE ALTER COLUMN

```
ALTER TABLE [ IF EXISTS ] [schemaName.]tableName  
ALTER COLUMN [ IF EXISTS ] columnName  
{ { columnNameDefinition }  
| { RENAME TO name }  
| SET GENERATED { ALWAYS | BY DEFAULT } [ alterIdentityColumnOption [...] ]  
| alterIdentityColumnOption [...]  
| DROP IDENTITY  
| { SELECTIVITY int }  
| { SET DEFAULT expression }  
| { DROP DEFAULT }  
| DROP EXPRESSION  
| { SET ON UPDATE expression }  
| { DROP ON UPDATE }  
| { SET DEFAULT ON NULL }  
| { DROP DEFAULT ON NULL }  
| { SET NOT NULL }  
| { DROP NOT NULL } | { SET NULL }  
| { SET DATA TYPE dataTypeOrDomain [ USING newValueExpression ] }  
| { SET { VISIBLE | INVISIBLE } } }
```

Changes the data type of a column, rename a column, change the identity value, or change the selectivity.

Changing the data type fails if the data can not be converted.

SET GENERATED ALWAYS, SET GENERATED BY DEFAULT, or identity options convert the column into identity column (if it wasn't an identity column) and set new values of specified options for its sequence.

DROP IDENTITY removes identity status of a column.

SELECTIVITY sets the selectivity (1-100) for a column. Setting the selectivity to 0 means the default value. Selectivity is used by the cost based optimizer to calculate the estimated cost of an index. Selectivity 100 means values are unique, 10 means every distinct value appears 10 times on average.

SET DEFAULT changes the default value of a column. This command doesn't affect generated and identity columns.

DROP DEFAULT removes the default value of a column.

DROP EXPRESSION converts generated column into base column.

SET ON UPDATE changes the value that is set on update if value for this column is not specified in update statement. This command doesn't affect generated and identity columns.

DROP ON UPDATE removes the value that is set on update of a column.

SET DEFAULT ON NULL makes NULL value work as DEFAULT value is assignments to this column.

DROP DEFAULT ON NULL makes NULL value work as NULL value in assignments to this column.

SET NOT NULL sets a column to not allow NULL. Rows may not contain NULL in this column.

DROP NOT NULL and SET NULL set a column to allow NULL. The column may not be part of a primary key and may not be an identity column.

SET DATA TYPE changes the data type of a column, for each row old value is converted to this data type unless USING is specified with a custom expression. USING expression may reference previous value of the modified column by its name and values of other columns.

SET INVISIBLE makes the column hidden, i.e. it will not appear in SELECT * results. SET VISIBLE has the reverse effect.

This command commits an open transaction in this connection.

Example:

```
ALTER TABLE TEST ALTER COLUMN NAME CLOB;  
ALTER TABLE TEST ALTER COLUMN NAME RENAME TO TEXT;  
ALTER TABLE TEST ALTER COLUMN ID RESTART WITH 10000;  
ALTER TABLE TEST ALTER COLUMN NAME SELECTIVITY 100;  
ALTER TABLE TEST ALTER COLUMN NAME SET DEFAULT "";  
ALTER TABLE TEST ALTER COLUMN NAME SET NOT NULL;  
ALTER TABLE TEST ALTER COLUMN NAME SET NULL;  
ALTER TABLE TEST ALTER COLUMN NAME SET VISIBLE;  
ALTER TABLE TEST ALTER COLUMN NAME SET INVISIBLE;
```

ALTER TABLE DROP COLUMN

```
ALTER TABLE [ IF EXISTS ] [schemaName.]tableName  
DROP [ COLUMN ] [ IF EXISTS ]  
{ ( columnName [,...] ) } | columnName [...]
```

Removes column(s) from a table. This command commits an open transaction in this connection.

Example:

```
ALTER TABLE TEST DROP COLUMN NAME  
ALTER TABLE TEST DROP COLUMN (NAME1, NAME2)
```

ALTER TABLE DROP CONSTRAINT

```
ALTER TABLE [ IF EXISTS ] [schemaName.]tableName DROP
```

```
CONSTRAINT [ IF EXISTS ] [schemaName.]constraintName [ RESTRICT |  
CASCADE ] | { PRIMARY KEY }
```

Removes a constraint or a primary key from a table. If CASCADE is specified, unique or primary key constraint is dropped together with all referential constraints that reference the specified constraint. This command commits an open transaction in this connection.

Example:

```
ALTER TABLE TEST DROP CONSTRAINT UNIQUE_NAME RESTRICT
```

ALTER TABLE SET

```
ALTER TABLE [ IF EXISTS ] [schemaName.]tableName  
SET REFERENTIAL_INTEGRITY  
{ FALSE | TRUE } [ CHECK | NOCHECK ]
```

Disables or enables referential integrity checking for a table. This command can be used inside a transaction. Enabling referential integrity does not check existing data, except if CHECK is specified. Use SET REFERENTIAL_INTEGRITY to disable it for all tables; the global flag and the flag for each table are independent.

This command commits an open transaction in this connection.

Example:

```
ALTER TABLE TEST SET REFERENTIAL_INTEGRITY FALSE
```

ALTER TABLE RENAME

```
ALTER TABLE [ IF EXISTS ] [schemaName.]tableName RENAME TO newName
```

Renames a table. This command commits an open transaction in this connection.

Example:

```
ALTER TABLE TEST RENAME TO MY_DATA
```

ALTER USER ADMIN

```
ALTER USER userName ADMIN { TRUE | FALSE }
```

Switches the admin flag of a user on or off.

Only unquoted or uppercase user names are allowed. Admin rights are required to execute this command. This command commits an open transaction in this connection.

Example:

```
ALTER USER TOM ADMIN TRUE
```

ALTER USER RENAME

```
ALTER USER userName RENAME TO newUserNames
```

Renames a user. After renaming a user, the password becomes invalid and needs to be changed as well.

Only unquoted or uppercase user names are allowed. Admin rights are required to execute this command. This command commits an open transaction in this connection.

Example:

```
ALTER USER TOM RENAME TO THOMAS
```

ALTER USER SET PASSWORD

```
ALTER USER userName SET { PASSWORD string | SALT bytes HASH bytes }
```

Changes the password of a user. Only unquoted or uppercase user names are allowed. The password must be enclosed in single quotes. It is case sensitive and can contain spaces. The salt and hash values are hex strings.

Admin rights are required to execute this command. This command commits an open transaction in this connection.

Example:


```
ALTER USER SA SET PASSWORD 'rioyxlgd'
```

ALTER VIEW RECOMPILE

```
ALTER VIEW [ IF EXISTS ] [schemaName.]viewName RECOMPILE
```

Recompiles a view after the underlying tables have been changed or created. Schema owner rights are required to execute this command. This command is used for views created using CREATE FORCE VIEW. This command commits an open transaction in this connection.

Example:

```
ALTER VIEW ADDRESS_VIEW RECOMPILE
```

ALTER VIEW RENAME

```
ALTER VIEW [ IF EXISTS ] [schemaName.]viewName RENAME TO newName
```

Renames a view. Schema owner rights are required to execute this command. This command commits an open transaction in this connection.

Example:

```
ALTER VIEW TEST RENAME TO MY_VIEW
```

ANALYZE

```
ANALYZE [ TABLE [schemaName.]tableName ] [ SAMPLE_SIZE rowCountInt ]
```

Updates the selectivity statistics of tables. If no table name is given, all tables are analyzed. The selectivity is used by the cost based optimizer to select the best index for a given query. If no sample size is set, up to 10000 rows per table are read. The value 0 means all rows are read. The selectivity can be set manually using ALTER TABLE ALTER COLUMN SELECTIVITY. Manual values are overwritten by this statement. The selectivity is available in the INFORMATION_SCHEMA.COLUMNS table.

This command commits an open transaction in this connection.

Example:

ANALYZE SAMPLE_SIZE 1000

COMMENT ON

```
COMMENT ON  
{ { COLUMN [schemaName.]tableName.columnName }  
| { { TABLE | VIEW | CONSTANT | CONSTRAINT | ALIAS | INDEX | ROLE  
| SCHEMA | SEQUENCE | TRIGGER | USER | DOMAIN }  
[schemaName.]objectName } }  
IS expression
```

Sets the comment of a database object. Use NULL or empty string to remove the comment.

Admin rights are required to execute this command if object is a USER or ROLE. Schema owner rights are required to execute this command for all other types of objects. This command commits an open transaction in this connection.

Example:

```
COMMENT ON TABLE TEST IS 'Table used for testing'
```

CREATE AGGREGATE

```
CREATE AGGREGATE [ IF NOT EXISTS ] [schemaName.]aggregateName FOR  
classNameString
```

Creates a new user-defined aggregate function. The method name must be the full qualified class name. The class must implement the interface org.h2.api.Aggregate or org.h2.api.AggregateFunction.

Admin rights are required to execute this command. This command commits an open transaction in this connection.

Example:

```
CREATE AGGREGATE SIMPLE_MEDIAN FOR 'com.acme.db.Median'
```

CREATE ALIAS

```
CREATE ALIAS [ IF NOT EXISTS ] [schemaName.]functionAliasName
```

```
[ DETERMINISTIC ]  
{ FOR classAndMethodString | AS sourceCodeString }
```

Creates a new function alias. If this is a ResultSet returning function, by default the return value is cached in a local temporary file.

DETERMINISTIC - Deterministic functions must always return the same value for the same parameters.

The method name must be the full qualified class and method name, and may optionally include the parameter classes as in `java.lang.Integer.parseInt(java.lang.String, int)`. The class and the method must both be public, and the method must be static. The class must be available in the classpath of the database engine (when using the server mode, it must be in the classpath of the server).

When defining a function alias with source code, the Sun javac is compiler is used if the file `tools.jar` is in the classpath. If not, javac is run as a separate process. Only the source code is stored in the database; the class is compiled each time the database is re-opened. Source code is usually passed as dollar quoted text to avoid escaping problems. If import statements are used, then the tag `@CODE` must be added before the method.

If the method throws an `SQLException`, it is directly re-thrown to the calling application; all other exceptions are first converted to a `SQLException`.

If the first parameter of the Java function is a `java.sql.Connection`, then a connection to the database is provided. This connection must not be closed. If the class contains multiple methods with the given name but different parameter count, all methods are mapped.

Admin rights are required to execute this command. This command commits an open transaction in this connection.

If you have the Groovy jar in your classpath, it is also possible to write methods using Groovy.

Example:

```

CREATE ALIAS MY_SQRT FOR 'java.lang.Math.sqrt';
CREATE ALIAS MY_ROUND FOR 'java.lang.Math.round(double)';
CREATE ALIAS GET_SYSTEM_PROPERTY FOR 'java.lang.System.getProperty';
CALL GET_SYSTEM_PROPERTY('java.class.path');
CALL GET_SYSTEM_PROPERTY('com.acme.test', 'true');
CREATE ALIAS REVERSE AS 'String reverse(String s) { return new
StringBuilder(s).reverse().toString(); }';
CALL REVERSE('Test');
CREATE ALIAS tr AS '@groovy.transform.CompileStatic
    static String tr(String str, String sourceSet, String replacementSet){
        return str.tr(sourceSet, replacementSet);
    }
'

```

CREATE CONSTANT

```

CREATE CONSTANT [ IF NOT EXISTS ] [schemaName.]constantName
VALUE expression

```

Creates a new constant. Schema owner rights are required to execute this command. This command commits an open transaction in this connection.

Example:

```
CREATE CONSTANT ONE VALUE 1
```

CREATE DOMAIN

```

CREATE DOMAIN [ IF NOT EXISTS ] [schemaName.]domainName
[ AS ] dataTypeOrDomain
[ DEFAULT expression ]
[ ON UPDATE expression ]
[ COMMENT expression ]
[ CHECK (condition) ] [...]

```

Creates a new domain to define a set of permissible values. Schema owner rights are required to execute this command. Domains can be used as data

types. The domain constraints must evaluate to TRUE or to UNKNOWN. In the conditions, the term VALUE refers to the value being tested.

This command commits an open transaction in this connection.

Example:

```
CREATE DOMAIN EMAIL AS VARCHAR(255) CHECK (POSITION('@', VALUE) > 1)
```

CREATE INDEX

```
CREATE [ UNIQUE [ nullsDistinct ] | SPATIAL ] INDEX  
[ [ IF NOT EXISTS ] [schemaName.]indexName ]  
ON [schemaName.]tableName ( indexColumn [,...] )  
[ INCLUDE ( indexColumn [,...] ) ]
```

Creates a new index. This command commits an open transaction in this connection.

INCLUDE clause may only be specified for UNIQUE indexes. With this clause additional columns are included into index, but aren't used in unique checks. If nulls distinct clause is not specified, the default is NULLS DISTINCT, excluding some compatibility modes.

Spatial indexes are supported only on GEOMETRY columns. They may contain only one column and are used by the [spatial overlapping operator](#).

Example:

```
CREATE INDEX IDXNAME ON TEST(NAME)
```

CREATE LINKED TABLE

```
CREATE [ FORCE ] [ [ GLOBAL | LOCAL ] TEMPORARY ]  
LINKED TABLE [ IF NOT EXISTS ]  
[schemaName.]tableName ( driverString, urlString, userString, passwordString,  
[ originalSchemaString, ] originalTableString )  
[ EMIT UPDATES | READONLY ] [ FETCH_SIZE sizeInt ] [AUTOCOMMIT ON|OFF]
```

Creates a table link to an external table. The driver name may be empty if the driver is already loaded. If the schema name is not set, only one table with that name may exist in the target database.

FORCE - Create the LINKED TABLE even if the remote database/table does not exist.

EMIT UPDATES - Usually, for update statements, the old rows are deleted first and then the new rows are inserted. It is possible to emit update statements (except on rollback), however in this case multi-row unique key updates may not always work. Linked tables to the same database share one connection.

READONLY - is set, the remote table may not be updated. This is enforced by H2.

FETCH_SIZE - the number of rows fetched, a hint with non-negative number of rows to fetch from the external table at once, may be ignored by the driver of external database. 0 is default and means no hint. The value is passed to `java.sql.Statement.setFetchSize()` method.

AUTOCOMMIT - is set to ON, the auto-commit mode is enable. OFF is disable. The value is passed to `java.sql.Connection.setAutoCommit()` method.

If the connection to the source database is lost, the connection is re-opened (this is a workaround for MySQL that disconnects after 8 hours of inactivity by default).

If a query is used instead of the original table name, the table is read only.

Queries must be enclosed in parenthesis: (SELECT * FROM ORDERS).

To use JNDI to get the connection, the driver class must be a `javax.naming.Context` (for example `javax.naming.InitialContext`), and the URL must be the resource name (for example `java:comp/env/jdbc/Test`).

Admin rights are required to execute this command. This command commits an open transaction in this connection.

Example:

```
CREATE LINKED TABLE LINK('org.h2.Driver', 'jdbc:h2:./test2',
    'sa', 'sa', 'TEST');
CREATE LINKED TABLE LINK('', 'jdbc:h2:./test2', 'sa', 'sa',
    '(SELECT * FROM TEST WHERE ID>0)');
CREATE LINKED TABLE LINK('javax.naming.InitialContext',
    'java:comp/env/jdbc/Test', NULL, NULL,
    '(SELECT * FROM TEST WHERE ID>0)');
```

CREATE ROLE

```
CREATE ROLE [ IF NOT EXISTS ] newRoleName
```

Creates a new role. This command commits an open transaction in this connection.

Example:

```
CREATE ROLE READONLY
```

CREATE SCHEMA

```
CREATE SCHEMA [ IF NOT EXISTS ]
{ name [ AUTHORIZATION ownerName ] | [ AUTHORIZATION ownerName ] }
[ WITH tableEngineParamName [,...] ]
```

Creates a new schema. Schema admin rights are required to execute this command.

If schema name is not specified, the owner name is used as a schema name. If schema name is specified, but no owner is specified, the current user is used as an owner.

Schema owners can create, rename, and drop objects in the schema. Schema owners can drop the schema itself, but cannot rename it. Some objects may still require admin rights for their creation, see documentation of their CREATE statements for details.

Optional table engine parameters are used when CREATE TABLE command is run on this schema without having its engine params set.

This command commits an open transaction in this connection.

Example:

```
CREATE SCHEMA TEST_SCHEMA AUTHORIZATION SA
```

CREATE SEQUENCE

```
CREATE SEQUENCE [ IF NOT EXISTS ] [schemaName.]sequenceName  
[ { AS dataType | sequenceOption } [...] ]
```

Creates a new sequence. Schema owner rights are required to execute this command.

The data type of a sequence must be a numeric type, the default is BIGINT. Sequence can produce only integer values. For TINYINT the allowed values are between -128 and 127. For SMALLINT the allowed values are between -32768 and 32767. For INTEGER the allowed values are between -2147483648 and 2147483647. For BIGINT the allowed values are between -9223372036854775808 and 9223372036854775807. For NUMERIC and DECFLOAT the allowed values depend on precision, but cannot exceed the range of BIGINT data type (from -9223372036854775808 to 9223372036854775807); the scale of NUMERIC must be 0. For REAL the allowed values are between -16777216 and 16777216. For DOUBLE PRECISION the allowed values are between -9007199254740992 and 9007199254740992.

Used values are never re-used, even when the transaction is rolled back.

This command commits an open transaction in this connection.

Example:

```
CREATE SEQUENCE SEQ_ID;  
CREATE SEQUENCE SEQ2 AS INTEGER START WITH 10;
```

CREATE TABLE

```
CREATE [ CACHED | MEMORY ] [ { TEMP } | [ GLOBAL | LOCAL ] TEMPORARY ]  
TABLE [ IF NOT EXISTS ] [schemaName.]tableName  
[ ( { columnName [columnDefinition] | tableConstraintDefinition } [,...] ) ]
```



```
[ ENGINE tableEngineName ]  
[ WITH tableEngineParamName [...] ]  
[ NOT PERSISTENT ] [ TRANSACTIONAL ]  
[ AS ( query ) [ WITH [ NO ] DATA ] ]
```

Creates a new table.

Admin rights are required to execute this command if and only if ENGINE option is used or custom default table engine is configured in the database. Schema owner rights or ALTER ANY SCHEMA rights are required for creation of regular tables and GLOBAL TEMPORARY tables.

Cached tables (the default for regular tables) are persistent, and the number of rows is not limited by the main memory. Memory tables (the default for temporary tables) are persistent, but the index data is kept in main memory, that means memory tables should not get too large.

Temporary tables are deleted when closing or opening a database. Temporary tables can be global (accessible by all connections) or local (only accessible by the current connection). The default for temporary tables is global. Indexes of temporary tables are kept fully in main memory, unless the temporary table is created using CREATE CACHED TABLE.

The ENGINE option is only required when custom table implementations are used. The table engine class must implement the interface `org.h2.api.TableEngine`. Any table engine parameters are passed down in the `tableEngineParams` field of the `CreateTableData` object.

Either ENGINE, or WITH (table engine params), or both may be specified. If ENGINE is not specified in CREATE TABLE, then the engine specified by DEFAULT_TABLE_ENGINE option of database params is used.

Tables with the NOT PERSISTENT modifier are kept fully in memory, and all rows are lost when the database is closed.

The column definitions are optional if a query is specified. In that case the column list of the query is used. If the query is specified its results are inserted into created table unless WITH NO DATA is specified.

This command commits an open transaction, except when using TRANSACTIONAL (only supported for temporary tables).

Example:

```
CREATE TABLE TEST(ID INT PRIMARY KEY, NAME VARCHAR(255))
```

CREATE TRIGGER

```
CREATE TRIGGER [ IF NOT EXISTS ] [schemaName.]triggerName  
{ BEFORE | AFTER | INSTEAD OF }  
{ INSERT | UPDATE | DELETE | { SELECT | ROLLBACK } }  
[,...] ON [schemaName.]tableName [ FOR EACH { ROW | STATEMENT } ]  
[ QUEUE int ] [ NOWAIT ]  
{ CALL triggeredClassNameString | AS sourceCodeString }
```

Creates a new trigger. Admin rights are required to execute this command.

The trigger class must be public and implement org.h2.api.Trigger. Inner classes are not supported. The class must be available in the classpath of the database engine (when using the server mode, it must be in the classpath of the server).

The sourceCodeString must define a single method with no parameters that returns org.h2.api.Trigger. See CREATE ALIAS for requirements regarding the compilation. Alternatively, javax.script.ScriptEngineManager can be used to create an instance of org.h2.api.Trigger. Currently JavaScript (included in older JREs or provided by org.graalvm.js:js-scriptengine library in newer JREs) and ruby (with JRuby) are supported. In that case the source must begin respectively with //javascript or #ruby.

BEFORE triggers are called after data conversion is made, default values are set, null and length constraint checks have been made; but before other constraints have been checked. If there are multiple triggers, the order in which they are called is undefined.

ROLLBACK can be specified in combination with INSERT, UPDATE, and DELETE. Only row based AFTER trigger can be called on ROLLBACK. Exceptions that occur within such triggers are ignored. As the operations that occur within a

trigger are part of the transaction, ROLLBACK triggers are only required if an operation communicates outside of the database.

INSTEAD OF triggers are implicitly row based and behave like BEFORE triggers. Only the first such trigger is called. Such triggers on views are supported. They can be used to make views updatable. These triggers on INSERT and UPDATE must update the passed new row to values that were actually inserted by the trigger; they are used for **FINAL TABLE** and for retrieval of generated keys.

A BEFORE SELECT trigger is fired just before the database engine tries to read from the table. The trigger can be used to update a table on demand. The trigger is called with both 'old' and 'new' set to null.

The MERGE statement will call both INSERT and UPDATE triggers. Not supported are SELECT triggers with the option FOR EACH ROW, and AFTER SELECT triggers.

Committing or rolling back a transaction within a trigger is not allowed, except for SELECT triggers.

By default a trigger is called once for each statement, without the old and new rows. FOR EACH ROW triggers are called once for each inserted, updated, or deleted row.

QUEUE is implemented for syntax compatibility with HSQL and has no effect.

The trigger need to be created in the same schema as the table. The schema name does not need to be specified when creating the trigger.

This command commits an open transaction in this connection.

Example:

```
CREATE TRIGGER TRIG_INS BEFORE INSERT ON TEST FOR EACH ROW CALL  
'MyTrigger';
```

```
CREATE TRIGGER TRIG_SRC BEFORE INSERT ON TEST AS
```

```
    'org.h2.api.Trigger create() { return new MyTrigger("constructorParam"); }';
```

```
CREATE TRIGGER TRIG_JS BEFORE INSERT ON TEST AS '//javascript  
return new (Java.type("org.example.MyTrigger"))("constructorParam");';
```

```
CREATE TRIGGER TRIG_RUBY BEFORE INSERT ON TEST AS '#ruby
Java::MyPackage::MyTrigger.new("constructorParam");'
```

CREATE USER

```
CREATE USER [ IF NOT EXISTS ] newUserName
{ PASSWORD string | SALT bytes HASH bytes } [ ADMIN ]
```

Creates a new user. For compatibility, only unquoted or uppercase user names are allowed. The password must be in single quotes. It is case sensitive and can contain spaces. The salt and hash values are hex strings.

Admin rights are required to execute this command. This command commits an open transaction in this connection.

Example:

```
CREATE USER GUEST PASSWORD 'abc'
```

CREATE VIEW

```
CREATE [ OR REPLACE ] [ FORCE ]
VIEW [ IF NOT EXISTS ] [schemaName.]viewName
[ ( columnName [...] ) ] AS query
```

Creates a new view. If the force option is used, then the view is created even if the underlying table(s) don't exist. Schema owner rights are required to execute this command.

If the OR REPLACE clause is used an existing view will be replaced, and any dependent views will not need to be recreated. If dependent views will become invalid as a result of the change an error will be generated, but this error can be ignored if the FORCE clause is also used.

Views are not updatable except when using 'instead of' triggers.

This command commits an open transaction in this connection.

Example:

```
CREATE VIEW TEST_VIEW AS SELECT * FROM TEST WHERE ID < 100
```

CREATE MATERIALIZED VIEW

```
CREATE [ OR REPLACE ]  
MATERIALIZED VIEW [ IF NOT EXISTS ] [schemaName.]viewName  
[ ( columnName [,...] ) ] AS query
```

Creates a new materialized view. Schema owner rights are required to execute this command.

If the OR REPLACE clause is used an existing view will be replaced.

Views are not updatable except using REFRESH MATERIALIZED VIEW.

This command commits an open transaction in this connection.

Example:

```
CREATE MATERIALIZED VIEW TEST_VIEW AS SELECT * FROM TEST WHERE ID <  
100
```

DROP AGGREGATE

```
DROP AGGREGATE [ IF EXISTS ] aggregateName
```

Drops an existing user-defined aggregate function. Schema owner rights are required to execute this command.

This command commits an open transaction in this connection.

Example:

```
DROP AGGREGATE SIMPLE_MEDIAN
```

DROP ALIAS

```
DROP ALIAS [ IF EXISTS ] [schemaName.]aliasName
```

Drops an existing function alias. Schema owner rights are required to execute this command.

This command commits an open transaction in this connection.

Example:

DROP ALIAS MY_SQRT

DROP ALL OBJECTS

DROP ALL OBJECTS [DELETE FILES]

Drops all existing views, tables, sequences, schemas, function aliases, roles, user-defined aggregate functions, domains, and users (except the current user). If DELETE FILES is specified, the database files will be removed when the last user disconnects from the database. Warning: this command can not be rolled back.

Admin rights are required to execute this command.

Example:

DROP ALL OBJECTS

DROP CONSTANT

DROP CONSTANT [IF EXISTS] [schemaName.]constantName

Drops a constant. Schema owner rights are required to execute this command. This command commits an open transaction in this connection.

Example:

DROP CONSTANT ONE

DROP DOMAIN

DROP DOMAIN [IF EXISTS] [schemaName.]domainName [RESTRICT | CASCADE]

Drops a data type (domain). Schema owner rights are required to execute this command.

The command will fail if it is referenced by a column or another domain (the default). Column descriptors are replaced with original definition of specified domain if the CASCADE clause is used. Default and on update expressions are copied into domains and columns that use this domain and don't have own

expressions. Domain constraints are copied into domains that use this domain and to columns (as check constraints) that use this domain. This command commits an open transaction in this connection.

Example:

```
DROP DOMAIN EMAIL
```

DROP INDEX

```
DROP INDEX [ IF EXISTS ] [schemaName.]indexName
```

Drops an index. This command commits an open transaction in this connection.

Example:

```
DROP INDEX IF EXISTS IDXNAME
```

DROP ROLE

```
DROP ROLE [ IF EXISTS ] roleName
```

Drops a role. Admin rights are required to execute this command. This command commits an open transaction in this connection.

Example:

```
DROP ROLE READONLY
```

DROP SCHEMA

```
DROP SCHEMA [ IF EXISTS ] schemaName [ RESTRICT | CASCADE ]
```

Drops a schema. Schema owner rights are required to execute this command. The command will fail if objects in this schema exist and the RESTRICT clause is used (the default). All objects in this schema are dropped as well if the CASCADE clause is used. This command commits an open transaction in this connection.

Example:

```
DROP SCHEMA TEST_SCHEMA
```

DROP SEQUENCE

```
DROP SEQUENCE [ IF EXISTS ] [schemaName.]sequenceName
```

Drops a sequence. Schema owner rights are required to execute this command. This command commits an open transaction in this connection.

Example:

```
DROP SEQUENCE SEQ_ID
```

DROP TABLE

```
DROP TABLE [ IF EXISTS ] [schemaName.]tableName [...]  
[ RESTRICT | CASCADE ]
```

Drops an existing table, or a list of tables. The command will fail if dependent objects exist and the RESTRICT clause is used (the default). All dependent views and constraints are dropped as well if the CASCADE clause is used. This command commits an open transaction in this connection.

Example:

```
DROP TABLE TEST
```

DROP TRIGGER

```
DROP TRIGGER [ IF EXISTS ] [schemaName.]triggerName
```

Drops an existing trigger. This command commits an open transaction in this connection.

Example:

```
DROP TRIGGER TRIG_INS
```

DROP USER

```
DROP USER [ IF EXISTS ] userName
```

Drops a user. The current user cannot be dropped. For compatibility, only unquoted or uppercase user names are allowed.

Admin rights are required to execute this command. This command commits an open transaction in this connection.

Example:

```
DROP USER TOM
```

DROP VIEW

```
DROP VIEW [ IF EXISTS ] [schemaName.]viewName [ RESTRICT | CASCADE ]
```

Drops an existing view. Schema owner rights are required to execute this command. All dependent views are dropped as well if the CASCADE clause is used (the default). The command will fail if dependent views exist and the RESTRICT clause is used. This command commits an open transaction in this connection.

Example:

```
DROP VIEW TEST_VIEW
```

DROP MATERIALIZED VIEW

```
DROP MATERIALIZED VIEW [ IF EXISTS ] [schemaName.]viewName
```

Drops an existing materialized view. Schema owner rights are required to execute this command. This command commits an open transaction in this connection.

Example:

```
DROP MATERIALIZED VIEW TEST_VIEW
```

REFRESH MATERIALIZED VIEW

```
REFRESH MATERIALIZED VIEW [schemaName.]viewName
```

Recreates an existing materialized view. Schema owner rights are required to execute this command. This command commits an open transaction in this connection.

Example:

```
REFRESH MATERIALIZED VIEW TEST_VIEW
```

TRUNCATE TABLE

```
TRUNCATE TABLE [schemaName.]tableName [ [ CONTINUE | RESTART ]  
IDENTITY ]
```

Removes all rows from a table. Unlike DELETE FROM without where clause, this command can not be rolled back. This command is faster than DELETE without where clause. Only regular data tables without foreign key constraints can be truncated (except if referential integrity is disabled for this database or for this table). Linked tables can't be truncated. If RESTART IDENTITY is specified next values for identity columns are restarted.

This command commits an open transaction in this connection.

Example:

```
TRUNCATE TABLE TEST
```

ALTER TYPE

```
ALTER TYPE [schemaName.]enumName ADD VALUE string
```

Adds new value to enum.

Duplicate and empty values are not permitted. The maximum number of values is 65536. The maximum allowed length of complete data type definition with all values is 1,000,000,000 characters.

Example:

```
ALTER TYPE card_suit ADD VALUE 'clubs'
```

Commands (Other)

CHECKPOINT

```
CHECKPOINT
```

Flushes the data to disk.

Admin rights are required to execute this command.

Example:

```
CHECKPOINT
```

CHECKPOINT SYNC

```
CHECKPOINT SYNC
```

Flushes the data to disk and forces all system buffers be written to the underlying device.

Admin rights are required to execute this command.

Example:

```
CHECKPOINT SYNC
```

COMMIT

```
COMMIT [ WORK ]
```

Commits a transaction.

Example:

```
COMMIT
```

COMMIT TRANSACTION

```
COMMIT TRANSACTION transactionName
```

Sets the resolution of an in-doubt transaction to 'commit'.

Admin rights are required to execute this command. This command is part of the 2-phase-commit protocol.

Example:

```
COMMIT TRANSACTION XID_TEST
```

GRANT RIGHT

```
GRANT { { SELECT | INSERT | UPDATE | DELETE } [,...] | ALL [ PRIVILEGES ] } ON  
{ { SCHEMA schemaName } | { [ TABLE ] [schemaName.]tableName [,...] } }  
TO { PUBLIC | userName | roleName }
```

Grants rights for a table to a user or role.

Schema owner rights are required to execute this command. This command commits an open transaction in this connection.

Example:

```
GRANT SELECT ON TEST TO READONLY
```

GRANT ALTER ANY SCHEMA

```
GRANT ALTER ANY SCHEMA TO userName
```

Grant schema admin rights to a user.

Schema admin can create, rename, or drop schemas and also has schema owner rights in every schema.

Admin rights are required to execute this command. This command commits an open transaction in this connection.

Example:

```
GRANT ALTER ANY SCHEMA TO Bob
```

GRANT ROLE

```
GRANT { roleName [,...] } TO { PUBLIC | userName | roleName }
```

Grants a role to a user or role.

Admin rights are required to execute this command. This command commits an open transaction in this connection.

Example:

```
GRANT READONLY TO PUBLIC
```

HELP

```
HELP [ anything [...] ]
```

Displays the help pages of SQL commands or keywords.

Example:

```
HELP SELECT
```

PREPARE COMMIT

```
PREPARE COMMIT newTransactionName
```

Prepares committing a transaction. This command is part of the 2-phase-commit protocol.

Example:

```
PREPARE COMMIT XID_TEST
```

REVOKE RIGHT

```
REVOKE { { SELECT | INSERT | UPDATE | DELETE } [,...] | ALL [ PRIVILEGES ] } ON  
{ { SCHEMA schemaName } | { [ TABLE ] [ schemaName.]tableName [,...] } }  
FROM { PUBLIC | userName | roleName }
```

Removes rights for a table from a user or role.

Schema owner rights are required to execute this command. This command commits an open transaction in this connection.

Example:

```
REVOKE SELECT ON TEST FROM READONLY
```

REVOKE ALTER ANY SCHEMA

```
REVOKE ALTER ANY SCHEMA FROM userName
```

Removes schema admin rights from a user.

Admin rights are required to execute this command. This command commits an open transaction in this connection.

Example:

```
GRANT ALTER ANY SCHEMA TO Bob
```

REVOKE ROLE

```
REVOKE { roleName [,...] } FROM { PUBLIC | userName | roleName }
```

Removes a role from a user or role.

Admin rights are required to execute this command. This command commits an open transaction in this connection.

Example:

```
REVOKE READONLY FROM TOM
```

ROLLBACK

```
ROLLBACK [ WORK ] [ TO SAVEPOINT savepointName ]
```

Rolls back a transaction. If a savepoint name is used, the transaction is only rolled back to the specified savepoint.

Example:

```
ROLLBACK
```

ROLLBACK TRANSACTION

```
ROLLBACK TRANSACTION transactionName
```

Sets the resolution of an in-doubt transaction to 'rollback'.

Admin rights are required to execute this command. This command is part of the 2-phase-commit protocol.

Example:

```
ROLLBACK TRANSACTION XID_TEST
```

SAVEPOINT

```
SAVEPOINT savepointName
```

Create a new savepoint. See also ROLLBACK. Savepoints are only valid until the transaction is committed or rolled back.

Example:

```
SAVEPOINT HALF_DONE
```

SET @

```
SET @variableName [ = ] expression
```

Updates a user-defined variable. Variables are not persisted and session scoped, that means only visible from within the session in which they are defined. This command does not commit a transaction, and rollback does not affect it.

Example:

```
SET @TOTAL=0
```

SET ALLOW_LITERALS

```
SET ALLOW_LITERALS { NONE | ALL | NUMBERS }
```

This setting can help solve the SQL injection problem. By default, text and number literals are allowed in SQL statements. However, this enables SQL injection if the application dynamically builds SQL statements. SQL injection is not possible if user data is set using parameters ('?').

NONE means literals of any kind are not allowed, only parameters and constants are allowed. NUMBERS mean only numerical and boolean literals are allowed. ALL means all literals are allowed (default).

See also CREATE CONSTANT.

Admin rights are required to execute this command, as it affects all connections. This command commits an open transaction in this connection.

This setting is persistent. This setting can be appended to the database URL:
jdbc:h2:./test;ALLOW_LITERALS=NONE

Example:

```
SET ALLOW_LITERALS NONE
```

SET AUTOCOMMIT

```
SET AUTOCOMMIT { TRUE | ON | FALSE | OFF }
```

Switches auto commit on or off. This setting can be appended to the database URL: jdbc:h2:./test;AUTOCOMMIT=OFF - however this will not work as expected when using a connection pool (the connection pool manager will re-enable autocommit when returning the connection to the pool, so autocommit will only be disabled the first time the connection is used).

Example:

```
SET AUTOCOMMIT OFF
```

SET CACHE_SIZE

```
SET CACHE_SIZE int
```

Sets the size of the cache in KB (each KB being 1024 bytes) for the current database. The default is 65536 per available GB of RAM, i.e. 64 MB per GB. The value is rounded to the next higher power of two. Depending on the virtual machine, the actual memory required may be higher.

This setting is persistent and affects all connections as there is only one cache per database. Using a very small value (specially 0) will reduce performance a lot. This setting only affects the database engine (the server in a client/server environment; in embedded mode, the database engine is in the same process as the application). It has no effect for in-memory databases.

Admin rights are required to execute this command, as it affects all connections. This command commits an open transaction in this connection.

This setting is persistent. This setting can be appended to the database URL:
jdbc:h2:./test;CACHE_SIZE=8192

Example:

```
SET CACHE_SIZE 8192
```

SET CLUSTER

```
SET CLUSTER serverListString
```

This command should not be used directly by an application, the statement is executed automatically by the system. The behavior may change in future releases. Sets the cluster server list. An empty string switches off the cluster mode. Switching on the cluster mode requires admin rights, but any user can switch it off (this is automatically done when the client detects the other server is not responding).

This command is effective immediately, but does not commit an open transaction.

Example:

```
SET CLUSTER "
```

SET BUILTIN_ALIAS_OVERRIDE

```
SET BUILTIN_ALIAS_OVERRIDE { TRUE | FALSE }
```

Allows the overriding of the builtin system date/time functions for unit testing purposes.

Admin rights are required to execute this command. This command commits an open transaction in this connection.

Example:

```
SET BUILTIN_ALIAS_OVERRIDE TRUE
```

SET CATALOG

```
SET CATALOG { catalogString | { catalogName } }
```

This command has no effect if the specified name matches the name of the database, otherwise it throws an exception.

This command does not commit a transaction.

Example:

```
SET CATALOG 'DB'
```

```
SET CATALOG DB_NAME
```

SET COLLATION

```
SET [ DATABASE ] COLLATION  
{ OFF | collationName  
[ STRENGTH { PRIMARY | SECONDARY | TERTIARY | IDENTICAL } ] }
```

Sets the collation used for comparing strings. This command can only be executed if there are no tables defined. See `java.text.Collator` for details about the supported collations and the STRENGTH (PRIMARY is usually case- and umlaut-insensitive; SECONDARY is case-insensitive but umlaut-sensitive; TERTIARY is both case- and umlaut-sensitive; IDENTICAL is sensitive to all differences and only affects ordering).

The ICU4J collator is used if it is in the classpath. It is also used if the collation name starts with ICU4J_ (in that case, the ICU4J must be in the classpath, otherwise an exception is thrown). The default collator is used if the collation name starts with DEFAULT_ (even if ICU4J is in the classpath). The charset collator is used if the collation name starts with CHARSET_ (e.g. CHARSET_CP500). This collator sorts strings according to the binary representation in the given charset.

Admin rights are required to execute this command. This command commits an open transaction in this connection. This setting is persistent. This setting can be appended to the database URL: `jdbc:h2:./test;COLLATION='ENGLISH'`

Example:

```
SET COLLATION ENGLISH
```

```
SET COLLATION CHARSET_CP500
```

SET DATABASE_EVENT_LISTENER

SET DATABASE_EVENT_LISTENER [classNameString](#)

Sets the event listener class. An empty string (") means no listener should be used. This setting is not persistent.

Admin rights are required to execute this command, except if it is set when opening the database (in this case it is reset just after opening the database).

This setting can be appended to the database URL:

```
jdbc:h2:./test;DATABASE_EVENT_LISTENER='sample.MyListener'
```

Example:

```
SET DATABASE_EVENT_LISTENER 'sample.MyListener'
```

SET DB_CLOSE_DELAY

SET DB_CLOSE_DELAY [int](#)

Sets the delay for closing a database if all connections are closed. The value -1 means the database is never closed until the close delay is set to some other value or SHUTDOWN is called. The value 0 means no delay (default; the database is closed if the last connection to it is closed). Values 1 and larger mean the number of seconds the database is left open after closing the last connection.

If the application exits normally or System.exit is called, the database is closed immediately, even if a delay is set.

Admin rights are required to execute this command, as it affects all connections. This command commits an open transaction in this connection.

This setting is persistent. This setting can be appended to the database URL:

```
jdbc:h2:./test;DB_CLOSE_DELAY=-1
```

Example:

```
SET DB_CLOSE_DELAY -1
```

SET DEFAULT_LOCK_TIMEOUT

```
SET DEFAULT_LOCK_TIMEOUT int
```

Sets the default lock timeout (in milliseconds) in this database that is used for the new sessions. The default value for this setting is 1000 (one second).

Admin rights are required to execute this command, as it affects all connections. This command commits an open transaction in this connection. This setting is persistent.

Example:

```
SET DEFAULT_LOCK_TIMEOUT 5000
```

SET DEFAULT_NULL_ORDERING

```
SET DEFAULT_NULL_ORDERING { LOW | HIGH | FIRST | LAST }
```

Changes the default ordering of NULL values. This setting affects new indexes without explicit NULLS FIRST or NULLS LAST columns, and ordering clauses of other commands without explicit null ordering. This setting doesn't affect ordering of NULL values inside ARRAY or ROW values (ARRAY[NULL] is always considered as smaller than ARRAY[1] during sorting).

LOW is the default one, NULL values are considered as smaller than other values during sorting.

With HIGH default ordering NULL values are considered as larger than other values during sorting.

With FIRST default ordering NULL values are sorted before other values, no matter if ascending or descending order is used.

With LAST default ordering NULL values are sorted after other values, no matter if ascending or descending order is used.

Please note that FIRST and LAST make impossible to use an index on (A ASC) for ORDER BY A DESC if column is nullable.

This setting is not persistent, but indexes are persisted with explicit NULLS FIRST or NULLS LAST ordering and aren't affected by changes in this setting. Admin rights are required to execute this command, as it affects all connections. This command commits an open transaction in this connection. This setting can be appended to the database URL:

```
jdbc:h2:./test;DEFAULT_NULL_ORDERING=HIGH
```

Example:

```
SET DEFAULT_NULL_ORDERING HIGH
```

SET DEFAULT_TABLE_TYPE

```
SET DEFAULT_TABLE_TYPE { MEMORY | CACHED }
```

Sets the default table storage type that is used when creating new tables. Memory tables are kept fully in the main memory (including indexes), however the data is still stored in the database file. The size of memory tables is limited by the memory. The default is CACHED.

Admin rights are required to execute this command, as it affects all connections. This command commits an open transaction in this connection. This setting is persistent. It has no effect for in-memory databases.

Example:

```
SET DEFAULT_TABLE_TYPE MEMORY
```

SET EXCLUSIVE

```
SET EXCLUSIVE { 0 | 1 | 2 }
```

Switched the database to exclusive mode (1, 2) and back to normal mode (0).

In exclusive mode, new connections are rejected, and operations by other connections are paused until the exclusive mode is disabled. When using the value 1, existing connections stay open. When using the value 2, all existing connections are closed (and current transactions are rolled back) except the connection that executes SET EXCLUSIVE. Only the connection that set the

exclusive mode can disable it. When the connection is closed, it is automatically disabled.

Admin rights are required to execute this command. This command commits an open transaction in this connection.

Example:

```
SET EXCLUSIVE 1
```

SET IGNORECASE

```
SET IGNORECASE { TRUE | FALSE }
```

If IGNORECASE is enabled, text columns in newly created tables will be case-insensitive. Already existing tables are not affected. The effect of case-insensitive columns is similar to using a collation with strength PRIMARY. Case-insensitive columns are compared faster than when using a collation. String literals and parameters are however still considered case sensitive even if this option is set.

Admin rights are required to execute this command, as it affects all connections. This command commits an open transaction in this connection. This setting is persistent. This setting can be appended to the database URL: `jdbc:h2:./test;IGNORECASE=TRUE`

Example:

```
SET IGNORECASE TRUE
```

SET IGNORE_CATALOGS

```
SET IGNORE_CATALOGS { TRUE | FALSE }
```

If IGNORE_CATALOGS is enabled, catalog names in front of schema names will be ignored. This can be used if multiple catalogs used by the same connections must be simulated. Caveat: if both catalogs contain schemas of the same name and if those schemas contain objects of the same name, this will lead to errors,

when trying to manage, access or change these objects. This setting can be appended to the database URL: jdbc:h2:./test;IGNORE_CATALOGS=TRUE

Example:

```
SET IGNORE_CATALOGS TRUE
```

SET JAVA_OBJECT_SERIALIZER

```
SET JAVA_OBJECT_SERIALIZER { null | className }
```

Sets the object used to serialize and deserialize java objects being stored in column of type OTHER. The serializer class must be public and implement org.h2.api.JavaObjectSerializer. Inner classes are not supported. The class must be available in the classpath of the database engine (when using the server mode, it must be both in the classpath of the server and the client). This command can only be executed if there are no tables defined.

Admin rights are required to execute this command. This command commits an open transaction in this connection. This setting is persistent. This setting can be appended to the database URL:

```
jdbc:h2:./test;JAVA_OBJECT_SERIALIZER='com.acme.SerializerClassName'
```

Example:

```
SET JAVA_OBJECT_SERIALIZER 'com.acme.SerializerClassName'
```

SET LAZY_QUERY_EXECUTION

```
SET LAZY_QUERY_EXECUTION int
```

Sets the lazy query execution mode. The values 0, 1 are supported.

If true, then large results are retrieved in chunks.

Note that not all queries support this feature, queries which do not are processed normally.

This command does not commit a transaction, and rollback does not affect it. This setting can be appended to the database URL:

```
jdbc:h2:./test;LAZY_QUERY_EXECUTION=1
```

Example:

```
SET LAZY_QUERY_EXECUTION 1
```

SET LOCK_MODE

SET LOCK_MODE *int*

Sets the lock mode. The values 0, 1, 2, and 3 are supported. The default is 3. This setting affects all connections.

The value 0 means no locking (should only be used for testing). Please note that using SET LOCK_MODE 0 while at the same time using multiple connections may result in inconsistent transactions.

The value 3 means row-level locking for write operations.

The values 1 and 2 have the same effect as 3.

Admin rights are required to execute this command, as it affects all connections. This command commits an open transaction in this connection. This setting is persistent. This setting can be appended to the database URL:
jdbc:h2:./test;LOCK_MODE=0

Example:

```
SET LOCK_MODE 0
```

SET LOCK_TIMEOUT

SET LOCK_TIMEOUT *int*

Sets the lock timeout (in milliseconds) for the current session. The default value for this setting is 1000 (one second).

This command does not commit a transaction, and rollback does not affect it. This setting can be appended to the database URL:

```
jdbc:h2:./test;LOCK_TIMEOUT=10000
```

Example:

```
SET LOCK_TIMEOUT 1000
```


SET MAX_LENGTH_INPLACE_LOB

```
SET MAX_LENGTH_INPLACE_LOB int
```

Sets the maximum size of an in-place LOB object.

This is the maximum length of an LOB that is stored with the record itself, and the default value is 256.

Admin rights are required to execute this command, as it affects all connections. This command commits an open transaction in this connection. This setting is persistent.

Example:

```
SET MAX_LENGTH_INPLACE_LOB 128
```

SET MAX_LOG_SIZE

```
SET MAX_LOG_SIZE int
```

Sets the maximum size of the transaction log, in megabytes. If the log is larger, and if there is no open transaction, the transaction log is truncated. If there is an open transaction, the transaction log will continue to grow however. The default max size is 16 MB. This setting has no effect for in-memory databases.

Admin rights are required to execute this command, as it affects all connections. This command commits an open transaction in this connection. This setting is persistent.

Example:

```
SET MAX_LOG_SIZE 2
```

SET MAX_MEMORY_ROWS

```
SET MAX_MEMORY_ROWS int
```

The maximum number of rows in a result set that are kept in-memory. If more rows are read, then the rows are buffered to disk. The default is 40000 per GB of available RAM.

Admin rights are required to execute this command, as it affects all connections. This command commits an open transaction in this connection. This setting is persistent. It has no effect for in-memory databases.

Example:

```
SET MAX_MEMORY_ROWS 1000
```

SET MAX_MEMORY_UNDO

```
SET MAX_MEMORY_UNDO int
```

The maximum number of undo records per a session that are kept in-memory. If a transaction is larger, the records are buffered to disk. The default value is 50000. Changes to tables without a primary key can not be buffered to disk. This setting is not supported when using multi-version concurrency.

Admin rights are required to execute this command, as it affects all connections. This command commits an open transaction in this connection. This setting is persistent. It has no effect for in-memory databases.

Example:

```
SET MAX_MEMORY_UNDO 1000
```

SET MAX_OPERATION_MEMORY

```
SET MAX_OPERATION_MEMORY int
```

Sets the maximum memory used for large operations (delete and insert), in bytes. Operations that use more memory are buffered to disk, slowing down the operation. The default max size is 100000. 0 means no limit.

This setting is not persistent. Admin rights are required to execute this command, as it affects all connections. It has no effect for in-memory databases. This setting can be appended to the database URL:

```
jdbc:h2:./test;MAX_OPERATION_MEMORY=10000
```

Example:

```
SET MAX_OPERATION_MEMORY 0
```

SET MODE

```
SET MODE { REGULAR | STRICT | LEGACY | DB2 | DERBY | HSQLDB |  
MSSQLSERVER | MYSQL | ORACLE | POSTGRESQL }
```

Changes to another database compatibility mode. For details, see [Compatibility Modes](#).

This setting is not persistent. Admin rights are required to execute this command, as it affects all connections. This command commits an open transaction in this connection. This setting can be appended to the database URL: jdbc:h2:./test;MODE=MYSQL

Example:

```
SET MODE HSQLDB
```

SET NON_KEYWORDS

```
SET NON_KEYWORDS [ name [...]]
```

Converts the specified tokens from keywords to plain identifiers for the current session. This setting may break some commands and should be used with caution and only when necessary. Use [quoted identifiers](#) instead of this setting if possible.

This command does not commit a transaction, and rollback does not affect it. This setting can be appended to the database URL:

```
jdbc:h2:./test;NON_KEYWORDS=KEY,VALUE
```

Example:

```
SET NON_KEYWORDS KEY, VALUE
```

SET OPTIMIZE_REUSE_RESULTS

```
SET OPTIMIZE_REUSE_RESULTS { 0 | 1 }
```

Enabled (1) or disabled (0) the result reuse optimization. If enabled, subqueries and views used as subqueries are only re-run if the data in one of the tables was changed. This option is enabled by default.

Admin rights are required to execute this command, as it affects all connections. This command commits an open transaction in this connection. This setting can be appended to the database URL:
jdbc:h2:./test;OPTIMIZE_REUSE_RESULTS=0

Example:

```
SET OPTIMIZE_REUSE_RESULTS 0
```

SET PASSWORD

```
SET PASSWORD string
```

Changes the password of the current user. The password must be in single quotes. It is case sensitive and can contain spaces.

This command commits an open transaction in this connection.

Example:

```
SET PASSWORD 'abcstzri!.5'
```

SET QUERY_STATISTICS

```
SET QUERY_STATISTICS { TRUE | FALSE }
```

Disabled or enables query statistics gathering for the whole database. The statistics are reflected in the INFORMATION_SCHEMA.QUERY_STATISTICS meta-table.

This setting is not persistent. This command commits an open transaction in this connection. Admin rights are required to execute this command, as it affects all connections.

Example:

```
SET QUERY_STATISTICS FALSE
```

SET QUERY_STATISTICS_MAX_ENTRIES

```
SET QUERY_STATISTICS int
```

Set the maximum number of entries in query statistics meta-table. Default value is 100.

This setting is not persistent. This command commits an open transaction in this connection. Admin rights are required to execute this command, as it affects all connections.

Example:

```
SET QUERY_STATISTICS_MAX_ENTRIES 500
```

SET QUERY_TIMEOUT

```
SET QUERY_TIMEOUT int
```

Set the query timeout of the current session to the given value. The timeout is in milliseconds. All kinds of statements will throw an exception if they take longer than the given value. The default timeout is 0, meaning no timeout.

This command does not commit a transaction, and rollback does not affect it.

Example:

```
SET QUERY_TIMEOUT 10000
```

SET REFERENTIAL_INTEGRITY

```
SET REFERENTIAL_INTEGRITY { TRUE | FALSE }
```

Disabled or enables referential integrity checking for the whole database. Enabling it does not check existing data. Use ALTER TABLE SET to disable it only for one table.

This setting is not persistent. This command commits an open transaction in this connection. Admin rights are required to execute this command, as it affects all connections.

Example:

```
SET REFERENTIAL_INTEGRITY FALSE
```

SET RETENTION_TIME

```
SET RETENTION_TIME int
```

How long to retain old, persisted data, in milliseconds. The default is 45000 (45 seconds), 0 means overwrite data as early as possible. It is assumed that a file system and hard disk will flush all write buffers within this time. Using a lower value might be dangerous, unless the file system and hard disk flush the buffers earlier. To manually flush the buffers, use CHECKPOINT SYNC, however please note that according to various tests this does not always work as expected depending on the operating system and hardware.

Admin rights are required to execute this command, as it affects all connections. This command commits an open transaction in this connection. This setting is persistent. This setting can be appended to the database URL: jdbc:h2:./test;RETENTION_TIME=0

Example:

```
SET RETENTION_TIME 0
```

SET SALT HASH

```
SET SALT bytes HASH bytes
```

Sets the password salt and hash for the current user. The password must be in single quotes. It is case sensitive and can contain spaces.

This command commits an open transaction in this connection.

Example:

```
SET SALT '00' HASH '1122'
```

SET SCHEMA

```
SET SCHEMA { schemaString | { schemaName } }
```

Changes the default schema of the current connection. The default schema is used in statements where no schema is set explicitly. The default schema for new connections is PUBLIC.

This command does not commit a transaction, and rollback does not affect it. This setting can be appended to the database URL: jdbc:h2:./test;SCHEMA=ABC

Example:

```
SET SCHEMA 'PUBLIC'
```

```
SET SCHEMA INFORMATION_SCHEMA
```

SET SCHEMA_SEARCH_PATH

```
SET SCHEMA_SEARCH_PATH schemaName [...]
```

Changes the schema search path of the current connection. The default schema is used in statements where no schema is set explicitly. The default schema for new connections is PUBLIC.

This command does not commit a transaction, and rollback does not affect it. This setting can be appended to the database URL:

```
jdbc:h2:./test;SCHEMA_SEARCH_PATH=ABC,DEF
```

Example:

```
SET SCHEMA_SEARCH_PATH INFORMATION_SCHEMA, PUBLIC
```

SET SESSION CHARACTERISTICS

```
SET SESSION CHARACTERISTICS AS TRANSACTION ISOLATION LEVEL  
{ READ UNCOMMITTED | READ COMMITTED | REPEATABLE READ |  
SERIALIZABLE }
```

Changes the transaction isolation level of the current session. The actual support of isolation levels depends on the database engine.

This command commits an open transaction in this session.

Example:

SET SESSION CHARACTERISTICS AS TRANSACTION ISOLATION LEVEL
SERIALIZABLE

SET THROTTLE

```
SET THROTTLE int
```

Sets the throttle for the current connection. The value is the number of milliseconds delay after each 50 ms. The default value is 0 (throttling disabled).

This command does not commit a transaction, and rollback does not affect it. This setting can be appended to the database URL: jdbc:h2:./test;THROTTLE=50

Example:

```
SET THROTTLE 200
```

SET TIME ZONE

```
SET TIME ZONE { LOCAL | intervalHourToMinute | { intervalHourToSecond | string } }
```

Sets the current time zone for the session.

This command does not commit a transaction, and rollback does not affect it.

This setting can be appended to the database URL: jdbc:h2:./test;TIME
ZONE='1:00'

Time zone offset used for [CURRENT_TIME](#), [CURRENT_TIMESTAMP](#), [CURRENT_DATE](#), [LOCALTIME](#), and [LOCALTIMESTAMP](#) is adjusted, so these functions will return new values based on the same UTC timestamp after execution of this command.

Example:

```
SET TIME ZONE LOCAL
```

```
SET TIME ZONE '-5:00'
```

```
SET TIME ZONE INTERVAL '1:00' HOUR TO MINUTE
```

```
SET TIME ZONE 'Europe/London'
```


SET TRACE_LEVEL

```
SET { TRACE_LEVEL_FILE | TRACE_LEVEL_SYSTEM_OUT } int
```

Sets the trace level for file the file or system out stream. Levels are: 0=off, 1=error, 2=info, 3=debug. The default level is 1 for file and 0 for system out. To use SLF4J, append ;TRACE_LEVEL_FILE=4 to the database URL when opening the database.

This setting is not persistent. Admin rights are required to execute this command, as it affects all connections. This command does not commit a transaction, and rollback does not affect it. This setting can be appended to the database URL: jdbc:h2:./test;TRACE_LEVEL_SYSTEM_OUT=3

Example:

```
SET TRACE_LEVEL_SYSTEM_OUT 3
```

SET TRACE_MAX_FILE_SIZE

```
SET TRACE_MAX_FILE_SIZE int
```

Sets the maximum trace file size. If the file exceeds the limit, the file is renamed to .old and a new file is created. If another .old file exists, it is deleted. The default max size is 16 MB.

This setting is persistent. Admin rights are required to execute this command, as it affects all connections. This command commits an open transaction in this connection. This setting can be appended to the database URL:

```
jdbc:h2:./test;TRACE_MAX_FILE_SIZE=3
```

Example:

```
SET TRACE_MAX_FILE_SIZE 10
```

SET TRUNCATE_LARGE_LENGTH

```
SET TRUNCATE_LARGE_LENGTH { TRUE | FALSE }
```

If TRUE is specified, the CHARACTER, CHARACTER VARYING, VARCHAR_IGNORECASE, BINARY, BINARY_VARYING, JAVA_OBJECT and JSON data types with too large length will be treated as these data types with maximum allowed length instead. By default, or if FALSE is specified, such definitions throw an exception. This setting can be used for compatibility with definitions from older versions of H2.

This setting can be appended to the database URL:

```
jdbc:h2:./test;TRUNCATE_LARGE_LENGTH=TRUE
```

Example:

```
SET TRUNCATE_LARGE_LENGTH TRUE
```

SET VARIABLE_BINARY

```
SET VARIABLE_BINARY { TRUE | FALSE }
```

If TRUE is specified, the BINARY data type will be parsed as VARBINARY in the current session. It can be used for compatibility with older versions of H2.

This setting can be appended to the database URL:

```
jdbc:h2:./test;VARIABLE_BINARY=TRUE
```

Example:

```
SET VARIABLE_BINARY TRUE
```

SET WRITE_DELAY

```
SET WRITE_DELAY int
```

Set the maximum delay between a commit and flushing the log, in milliseconds. This setting is persistent. The default is 500 ms.

Admin rights are required to execute this command, as it affects all connections. This command commits an open transaction in this connection.

This setting can be appended to the database URL:

```
jdbc:h2:./test;WRITE_DELAY=0
```

Example:

SET WRITE_DELAY 2000

SHUTDOWN

SHUTDOWN [IMMEDIATELY | COMPACT | DEFrag]

This statement closes all open connections to the database and closes the database. This command is usually not required, as the database is closed automatically when the last connection to it is closed.

If no option is used, then the database is closed normally. All connections are closed, open transactions are rolled back.

SHUTDOWN COMPACT fully compacts the database (re-creating the database may further reduce the database size). If the database is closed normally (using SHUTDOWN or by closing all connections), then the database is also compacted, but only for at most the time defined by the database setting `h2.maxCompactTime` in milliseconds (see there).

SHUTDOWN IMMEDIATELY closes the database files without any cleanup and without compacting.

SHUTDOWN DEFrag is currently equivalent to COMPACT.

Admin rights are required to execute this command.

Example:

SHUTDOWN COMPACT

Functions

Index

Numeric Functions

ABS

ACOS

ASIN

ATAN

COS

COSH

COT

SIN

SINH

TAN

TANH

ATAN2

BITAND

BITOR

BITXOR

BITNOT

BITNAND

BITNOR

BITXNOR

BITGET

BITCOUNT

LSHIFT

RSHIFT

ULSHIFT

URSHIFT

ROTATELEFT

ROTATERIGHT

MOD
CEIL
DEGREES
EXP
FLOOR
LN
LOG
LOG10
ORA_HASH
RADIANS
SQRT
PI
POWER
RAND
RANDOM_UUID
ROUND
SECURE_RAND
SIGN
ENCRYPT
DECRYPT
HASH
TRUNC
COMPRESS
EXPAND
ZERO
GCD
LCM

String Functions

ASCII
BIT_LENGTH
CHAR_LENGTH
OCTET_LENGTH

CHAR
CONCAT
CONCAT_WS
DIFFERENCE
HEXTORAW
RAWTOHEX
INSERT Function
LOWER
UPPER
LEFT
RIGHT
LOCATE
LPAD
RPAD
LTRIM
RTRIM
BTRIM
TRIM
REGEXP_REPLACE
REGEXP_LIKE
REGEXP_SUBSTR
REPEAT
REPLACE
SOUNDEX
SPACE
STRINGDECODE
STRINGENCODE
STRINGTOUTF8
SUBSTRING
UTF8TOSTRING
QUOTE_IDENT
XMLATTR

XMLNODE
XMLCOMMENT
XMLCDATA
XMLSTARTDOC
XMLTEXT
TO_CHAR
TRANSLATE

Time and Date Functions

CURRENT_DATE
CURRENT_TIME
CURRENT_TIMESTAMP
LOCALTIME
LOCALTIMESTAMP
DATEADD
DATEDIFF
DATE_TRUNC
LAST_DAY
DAYNAME
DAY_OF_MONTH
DAY_OF_WEEK
ISO_DAY_OF_WEEK
DAY_OF_YEAR
EXTRACT
FORMATDATETIME
HOUR
MINUTE
MONTH
MONTHNAME
PARSEDATETIME
QUARTER
SECOND
WEEK

ISO_WEEK

YEAR

ISO_YEAR

System Functions

ABORT_SESSION

ARRAY_GET

CARDINALITY

ARRAY_CONTAINS

ARRAY_CAT

ARRAY_APPEND

ARRAY_MAX_CARDINALITY

TRIM_ARRAY

ARRAY_SLICE

AUTOCOMMIT

CANCEL_SESSION

CASEWHEN Function

COALESCE

CONVERT

CURRVAL

CSVWRITE

CURRENT_SCHEMA

CURRENT_CATALOG

DATABASE_PATH

DATA_TYPE_SQL

DB_OBJECT_ID

DB_OBJECT_SQL

DB_OBJECT_SIZE

DB_OBJECT_TOTAL_SIZE

DB_OBJECT_APPROXIMATE_SIZE

DB_OBJECT_APPROXIMATE_TOTAL_SIZE

DECODE

DISK_SPACE_USED

SIGNAL
ESTIMATED_ENVELOPE
FILE_READ
FILE_WRITE
GREATEST
LEAST
LOCK_MODE
LOCK_TIMEOUT
MEMORY_FREE
MEMORY_USED
NEXTVAL
NULLIF
NVL2
READONLY
ROWNUM
SESSION_ID
SET
TRANSACTION_ID
TRUNCATE_VALUE
CURRENT_PATH
CURRENT_ROLE
CURRENT_USER
H2VERSION

JSON Functions

JSON_OBJECT
JSON_ARRAY

Table Functions

CSVREAD
LINK_SCHEMA
TABLE
UNNEST

Details

Non-standard syntax is marked in green. Compatibility-only non-standard syntax is marked in red, don't use it unless you need it for compatibility with other databases or old versions of H2.

Numeric Functions

ABS

```
ABS( { numeric | interval } )
```

Returns the absolute value of a specified value. The returned value is of the same data type as the parameter.

Note that TINYINT, SMALLINT, INT, and BIGINT data types cannot represent absolute values of their minimum negative values, because they have more negative values than positive. For example, for INT data type allowed values are from -2147483648 to 2147483647. ABS(-2147483648) should be 2147483648, but this value is not allowed for this data type. It leads to an exception. To avoid it cast argument of this function to a higher data type.

Example:

```
ABS(I)
```

```
ABS(CAST(I AS BIGINT))
```

ACOS

```
ACOS(numeric)
```

Calculate the arc cosine.

Argument must be between -1 and 1 inclusive.

This function returns a double precision value.

Example:

```
ACOS(D)
```

ASIN

ASIN([numeric](#))

Calculate the arc sine.

Argument must be between -1 and 1 inclusive.

This function returns a double precision value.

Example:

ASIN(D)

ATAN

ATAN([numeric](#))

Calculate the arc tangent.

This function returns a double precision value.

Example:

ATAN(D)

COS

COS([numeric](#))

Calculate the trigonometric cosine.

This function returns a double precision value.

Example:

COS(ANGLE)

COSH

COSH([numeric](#))

Calculate the hyperbolic cosine.

This function returns a double precision value.

Example:

COSH(X)

COT

COT(numeric)

Calculate the trigonometric cotangent (1/TAN(ANGLE)).

This function returns a double precision value.

Example:

COT(ANGLE)

SIN

SIN(numeric)

Calculate the trigonometric sine.

This function returns a double precision value.

Example:

SIN(ANGLE)

SINH

SINH(numeric)

Calculate the hyperbolic sine.

This function returns a double precision value.

Example:

SINH(ANGLE)

TAN

TAN(numeric)

Calculate the trigonometric tangent.

This function returns a double precision value.

Example:

TAN(ANGLE)

TANH

```
TANH(numeric)
```

Calculate the hyperbolic tangent.

This function returns a double precision value.

Example:

TANH(X)

ATAN2

```
ATAN2(numeric, numeric)
```

Calculate the angle when converting the rectangular coordinates to polar coordinates.

This function returns a double precision value.

Example:

ATAN2(X, Y)

BITAND

```
BITAND(expression, expression)
```

The bitwise AND operation. Arguments should have TINYINT, SMALLINT, INTEGER, BIGINT, BINARY, or BINARY VARYING data type. This function returns result of the same data type.

For aggregate function see [BIT_AND_AGG](#).

Example:

BITAND(A, B)

BITOR

```
BITOR(expression, expression)
```

The bitwise OR operation. Arguments should have TINYINT, SMALLINT, INTEGER, BIGINT, BINARY, or BINARY VARYING data type. This function returns result of the same data type.

For aggregate function see [BIT_OR_AGG](#).

Example:

```
BITOR(A, B)
```

BITXOR

```
BITXOR(expression, expression)
```

Arguments should have TINYINT, SMALLINT, INTEGER, BIGINT, BINARY, or BINARY VARYING data type. This function returns result of the same data type.

For aggregate function see [BIT_XOR_AGG](#).

Example:

The bitwise XOR operation.

BITNOT

```
BITNOT(expression)
```

The bitwise NOT operation. Argument should have TINYINT, SMALLINT, INTEGER, BIGINT, BINARY, or BINARY VARYING data type. This function returns result of the same data type.

Example:

```
BITNOT(A)
```

BITNAND

```
BITNAND(expression, expression)
```

The bitwise NAND operation equivalent to BITNOT(BITAND(expression, expression)). Arguments should have TINYINT, SMALLINT, INTEGER, BIGINT, BINARY, or BINARY VARYING data type. This function returns result of the same data type.

For aggregate function see [BIT_NAND_AGG](#).

Example:

```
BITNAND(A, B)
```

BITNOR

```
BITNOR(expression, expression)
```

The bitwise NOR operation equivalent to BITNOT(BITOR(expression, expression)). Arguments should have TINYINT, SMALLINT, INTEGER, BIGINT, BINARY, or BINARY VARYING data type. This function returns result of the same data type.

For aggregate function see [BIT_NOR_AGG](#).

Example:

```
BITNOR(A, B)
```

BITXNOR

```
BITXNOR(expression, expression)
```

The bitwise XNOR operation equivalent to BITNOT(BITXOR(expression, expression)). Arguments should have TINYINT, SMALLINT, INTEGER, BIGINT, BINARY, or BINARY VARYING data type. This function returns result of the same data type.

For aggregate function see [BIT_XNOR_AGG](#).

Example:

```
BITXNOR(A, B)
```

BITGET

```
BITGET(expression, long)
```

Returns true if and only if the first argument has a bit set in the position specified by the second parameter. The first argument should have TINYINT, SMALLINT, INTEGER, BIGINT, BINARY, or BINARY VARYING data type. This method returns a boolean. The second argument is zero-indexed; the least significant bit has position 0.

Example:

```
BITGET(A, 1)
```

BITCOUNT

```
BITCOUNT(expression)
```

Returns count of set bits in the specified value. Value should have TINYINT, SMALLINT, INTEGER, BIGINT, BINARY, or BINARY VARYING data type. This method returns a long.

Example:

```
BITCOUNT(A)
```

LSHIFT

```
LSHIFT(expression, long)
```

The bitwise signed left shift operation. Shifts the first argument by the number of bits given by the second argument. Argument should have TINYINT, SMALLINT, INTEGER, BIGINT, BINARY, or BINARY VARYING data type. This function returns result of the same data type.

If number of bits is negative, a signed right shift is performed instead. For numeric values a sign bit is used for left-padding (with negative offset). If number of bits is equal to or larger than number of bits in value all bits are pushed out from the value. For binary string arguments signed and unsigned shifts return the same results.

Example:

LSHIFT(A, B)

RSHIFT

```
RSHIFT(expression, long)
```

The bitwise signed right shift operation. Shifts the first argument by the number of bits given by the second argument. Argument should have TINYINT, SMALLINT, INTEGER, BIGINT, BINARY, or BINARY VARYING data type. This function returns result of the same data type.

If number of bits is negative, a signed left shift is performed instead. For numeric values a sign bit is used for left-padding (with positive offset). If number of bits is equal to or larger than number of bits in value all bits are pushed out from the value. For binary string arguments signed and unsigned shifts return the same results.

Example:

RSHIFT(A, B)

ULSHIFT

```
ULSHIFT(expression, long)
```

The bitwise unsigned left shift operation. Shifts the first argument by the number of bits given by the second argument. Argument should have TINYINT, SMALLINT, INTEGER, BIGINT, BINARY, or BINARY VARYING data type. This function returns result of the same data type.

If number of bits is negative, an unsigned right shift is performed instead. If number of bits is equal to or larger than number of bits in value all bits are pushed out from the value. For binary string arguments signed and unsigned shifts return the same results.

Example:

ULSHIFT(A, B)

URSHIFT

```
URSHIFT(expression, long)
```

The bitwise unsigned right shift operation. Shifts the first argument by the number of bits given by the second argument. Argument should have TINYINT, SMALLINT, INTEGER, BIGINT, BINARY, or BINARY VARYING data type. This function returns result of the same data type.

If number of bits is negative, an unsigned left shift is performed instead. If number of bits is equal to or larger than number of bits in value all bits are pushed out from the value. For binary string arguments signed and unsigned shifts return the same results.

Example:

```
URSHIFT(A, B)
```

ROTATELEFT

```
ROTATELEFT(expression, long)
```

The bitwise left rotation operation. Rotates the first argument by the number of bits given by the second argument. Argument should have TINYINT, SMALLINT, INTEGER, BIGINT, BINARY, or BINARY VARYING data type. This function returns result of the same data type.

Example:

```
ROTATELEFT(A, B)
```

ROTATERIGHT

```
ROTATERIGHT(expression, long)
```

The bitwise right rotation operation. Rotates the first argument by the number of bits given by the second argument. Argument should have TINYINT, SMALLINT, INTEGER, BIGINT, BINARY, or BINARY VARYING data type. This function returns result of the same data type.

Example:

ROTATERIGHT(A, B)

MOD

```
MOD(dividendNumeric, divisorNumeric)
```

The modulus expression.

Result has the same type as divisor. Result is NULL if either of arguments is NULL. If divisor is 0, an exception is raised. Result has the same sign as dividend or is equal to 0.

Usually arguments should have scale 0, but it isn't required by H2.

Example:

MOD(A, B)

CEIL

```
{ CEIL | CEILING } (numeric)
```

Returns the smallest integer value that is greater than or equal to the argument. This method returns value of the same type as argument, but with scale set to 0 and adjusted precision, if applicable.

Example:

CEIL(A)

DEGREES

```
DEGREES(numeric)
```

See also Java Math.toDegrees. This method returns a double.

Example:

DEGREES(A)

EXP

```
EXP(numeric)
```

See also Java Math.exp. This method returns a double.

Example:

```
EXP(A)
```

FLOOR

```
FLOOR(numeric)
```

Returns the largest integer value that is less than or equal to the argument. This method returns value of the same type as argument, but with scale set to 0 and adjusted precision, if applicable.

Example:

```
FLOOR(A)
```

LN

```
LN(numeric)
```

Calculates the natural (base e) logarithm as a double value. Argument must be a positive numeric value.

Example:

```
LN(A)
```

LOG

```
LOG({baseNumeric, numeric | {numeric}})
```

Calculates the logarithm with specified base as a double value. Argument and base must be positive numeric values. Base cannot be equal to 1.

The default base is e (natural logarithm), in the PostgreSQL mode the default base is base 10. In MSSQLServer mode the optional base is specified after the argument.

Single-argument variant of LOG function is deprecated, use [LN](#) or [LOG10](#) instead.

Example:

LOG(2, A)

LOG10

```
LOG10(numeric)
```

Calculates the base 10 logarithm as a double value. Argument must be a positive numeric value.

Example:

LOG10(A)

ORA_HASH

```
ORA_HASH(expression [, bucketLong [, seedLong]])
```

Computes a hash value. Optional bucket argument determines the maximum returned value. This argument should be between 0 and 4294967295, default is 4294967295. Optional seed argument is combined with the given expression to return the different values for the same expression. This argument should be between 0 and 4294967295, default is 0. This method returns a long value between 0 and the specified or default bucket value inclusive.

Example:

ORA_HASH(A)

RADIANS

```
RADIANS(numeric)
```

See also Java Math.toRadians. This method returns a double.

Example:

RADIANS(A)

SQRT

```
SQRT(numeric)
```

See also Java Math.sqrt. This method returns a double.

Example:

```
SQRT(A)
```

PI

```
PI()
```

See also Java Math.PI. This method returns a double.

Example:

```
PI()
```

POWER

```
POWER(numeric, numeric)
```

See also Java Math.pow. This method returns a double.

Example:

```
POWER(A, B)
```

RAND

```
{ RAND | RANDOM } ( [ int ] )
```

Calling the function without parameter returns the next a pseudo random number. Calling it with an parameter seeds the session's random number generator. This method returns a double between 0 (including) and 1 (excluding).

Example:

```
RAND()
```

RANDOM_UUID

```
RANDOM_UUID([versionInt]) | UUID()
```

Returns a new RFC 9562-compliant UUID with the specified version. If version is not specified, a default version will be used. Current default is 4, but it may be changed in future versions of H2.

Version 4 is a UUID with 122 pseudo random bits. Please note that using an index on randomly generated data will result on poor performance once there are millions of rows in a table. The reason is that the cache behavior is very bad with randomly distributed data. This is a problem for any database system.

Version 7 is a time-ordered UUID value with layout optimized for database systems. It contains 48-bit number of milliseconds seconds since midnight 1 Jan 1970 UTC with leap seconds excluded, additional 12-bit sub-millisecond timestamp fraction if available, and 62 pseudo random bits.

Example:

```
RANDOM_UUID(7)
```

```
RANDOM_UUID()
```

ROUND

```
ROUND(numeric [, digitsInt])
```

Rounds to a number of fractional digits. This method returns value of the same type as argument, but with adjusted precision and scale, if applicable.

Example:

```
ROUND(N, 2)
```

SECURE_RANDOM

```
SECURE_RANDOM(int)
```

Generates a number of cryptographically secure random numbers. This method returns bytes.

Example:

```
CALL SECURE RAND(16)
```

SIGN

```
SIGN( { numeric | interval } )
```

Returns -1 if the value is smaller than 0, 0 if zero or NaN, and otherwise 1.

Example:

```
SIGN(N)
```

ENCRYPT

```
ENCRYPT(algorithmString, keyBytes, dataBytes)
```

Encrypts data using a key. The supported algorithm is AES. The block size is 16 bytes. This method returns bytes.

Example:

```
CALL ENCRYPT('AES', '00', STRINGTOUTF8('Test'))
```

DECRYPT

```
DECRYPT(algorithmString, keyBytes, dataBytes)
```

Decrypts data using a key. The supported algorithm is AES. The block size is 16 bytes. This method returns bytes.

Example:

```
CALL TRIM(CHAR(0) FROM UTF8TOSTRING(  
  DECRYPT('AES', '00', '3fabb4de8f1ee2e97d7793bab2db1116')))
```

HASH

```
HASH(algorithmString, expression [, iterationInt])
```

Calculate the hash value using an algorithm, and repeat this process for a number of iterations.

This function supports MD5, SHA-1, SHA-224, SHA-256, SHA-384, SHA-512, SHA3-224, SHA3-256, SHA3-384, and SHA3-512 algorithms. SHA-224, SHA-384, and SHA-512 may be unavailable in some JREs.

MD5 and SHA-1 algorithms should not be considered as secure.

If this function is used to encrypt a password, a random salt should be concatenated with a password and this salt and result of the function should be stored to prevent a rainbow table attack and number of iterations should be large enough to slow down a dictionary or a brute force attack.

This method returns bytes.

Example:

```
CALL HASH('SHA-256', 'Text', 1000)
```

```
CALL HASH('SHA3-256', X'0102')
```

TRUNC

```
{ TRUNC | TRUNCATE } ( { {numeric [, digitsInt] }  
| { timestamp | timestampWithTimeZone | date | timestampString } } )
```

When a numeric argument is specified, truncates it to a number of digits (to the next value closer to 0) and returns value of the same type as argument, but with adjusted precision and scale, if applicable.

This function with datetime or string argument is deprecated, use [DATE_TRUNC](#) instead. When used with a timestamp, truncates the timestamp to a date (day) value and returns a timestamp with or without time zone depending on type of the argument. When used with a date, returns a timestamp at start of this date. When used with a timestamp as string, truncates the timestamp to a date (day) value and returns a timestamp without time zone.

Example:

```
TRUNCATE(N, 2)
```

COMPRESS

```
COMPRESS(dataBytes [, algorithmString])
```

Compresses the data using the specified compression algorithm. Supported algorithms are: LZF (faster but lower compression; default), and DEFLATE (higher compression). Compression does not always reduce size. Very small objects and objects with little redundancy may get larger. This method returns bytes.

Example:

```
COMPRESS(STRINGTOUTF8('Test'))
```

EXPAND

```
EXPAND(bytes)
```

Expands data that was compressed using the COMPRESS function. This method returns bytes.

Example:

```
UTF8TOSTRING(EXPAND(COMPRESS(STRINGTOUTF8('Test'))))
```

ZERO

```
ZERO()
```

Returns the value 0. This function can be used even if numeric literals are disabled.

Example:

```
ZERO()
```

GCD

```
GCD(numeric, numeric [...])
```

Returns the greatest common divisor of specified arguments. Arguments must have TINYINT, SMALLINT, INTEGER, BIGINT, or NUMERIC data type with scale 0. This function returns result of NUMERIC data type with scale 0.

For aggregate function see [GCD_AGG](#).

Example:

GCD(A, B)

LCM

```
LCM(numeric, numeric [...])
```

Returns the least common multiple of specified arguments. Arguments must have TINYINT, SMALLINT, INTEGER, BIGINT, or NUMERIC data type with scale 0. This function returns result of NUMERIC data type with scale 0.

For aggregate function see [LCM_AGG](#).

Example:

LCM(A, B)

String Functions

ASCII

```
ASCII(string)
```

Returns the ASCII value of the first character in the string. This method returns an int.

Example:

ASCII('Hi')

BIT_LENGTH

```
BIT_LENGTH(bytes)
```

Returns the number of bits in a binary string. This method returns a long.

Example:

BIT_LENGTH(NAME)

CHAR_LENGTH

```
{ CHAR_LENGTH | CHARACTER_LENGTH | { LENGTH } } ( string )
```

Returns the number of characters in a character string. This method returns a long.

Example:

CHAR_LENGTH(NAME)

OCTET_LENGTH

```
OCTET_LENGTH(bytes)
```

Returns the number of bytes in a binary string. This method returns a long.

Example:

OCTET_LENGTH(NAME)

CHAR

```
{ CHAR | CHR } ( int )
```

Returns the character that represents the ASCII value. This method returns a string.

Example:

CHAR(65)

CONCAT

```
CONCAT(string, string [...])
```

Combines strings. Unlike with the operator ||, NULL parameters are ignored, and do not cause the result to become NULL. If all parameters are NULL the result is an empty string. This method returns a string.

Example:

```
CONCAT(NAME, '!')
```

CONCAT_WS

```
CONCAT_WS(separatorString, string, string [...])
```

Combines strings with separator. If separator is NULL it is treated like an empty string. Other NULL parameters are ignored. Remaining non-NULL parameters, if any, are concatenated with the specified separator. If there are no remaining parameters the result is an empty string. This method returns a string.

Example:

```
CONCAT_WS(',', NAME, '!')
```

DIFFERENCE

```
DIFFERENCE(string, string)
```

Returns the difference between the sounds of two strings. The difference is calculated as a number of matched characters in the same positions in SOUNDEX representations of arguments. This method returns an int between 0 and 4 inclusive, or null if any of its parameters is null. Note that value of 0 means that strings are not similar to each other. Value of 4 means that strings are fully similar to each other (have the same SOUNDEX representation).

Example:

```
DIFFERENCE(T1.NAME, T2.NAME)
```

HEXTORAW

```
HEXTORAW(string)
```

Converts a hex representation of a string to a string. 4 hex characters per string character are used.

Example:

HEXTORAW(DATA)

RAWTOHEX

```
RAWTOHEX({string | bytes})
```

Converts a string or bytes to the hex representation. 4 hex characters per string character are used. This method returns a string.

Example:

RAWTOHEX(DATA)

INSERT Function

```
INSERT(originalString, startInt, lengthInt, addString)
```

Inserts a additional string into the original string at a specified start position. The length specifies the number of characters that are removed at the start position in the original string. This method returns a string.

Example:

INSERT(NAME, 1, 1, '')

LOWER

```
{ LOWER | { LCASE } }( string )
```

Converts a string to lowercase.

Example:

LOWER(NAME)

UPPER

```
{ UPPER | { UCASE } }( string )
```

Converts a string to uppercase.

Example:

UPPER(NAME)

LEFT

```
LEFT(string, int)
```

Returns the leftmost number of characters.

Example:

```
LEFT(NAME, 3)
```

RIGHT

```
RIGHT(string, int)
```

Returns the rightmost number of characters.

Example:

```
RIGHT(NAME, 3)
```

LOCATE

```
{ LOCATE(searchString, string [, startInt]) }  
| { INSTR(string, searchString, [, startInt]) }  
| { POSITION(searchString, string) }
```

Returns the location of a search string in a string. If a start position is used, the characters before it are ignored. If position is negative, the rightmost location is returned. 0 is returned if the search string is not found. Please note this function is case sensitive, even if the parameters are not.

Example:

```
LOCATE('.', NAME)
```

LPAD

```
LPAD(string, int[, paddingString])
```

Left pad the string to the specified length. If the length is shorter than the string, it will be truncated at the end. If the padding string is not set, spaces will be used.

Example:

```
LPAD(AMOUNT, 10, '*')
```

RPAD

```
RPAD(string, int[, paddingString])
```

Right pad the string to the specified length. If the length is shorter than the string, it will be truncated. If the padding string is not set, spaces will be used.

Example:

```
RPAD(TEXT, 10, '-')
```

LTRIM

```
LTRIM(string [, charactersToTrimString])
```

Removes all leading spaces or other specified characters from a string, multiple characters can be specified.

Example:

```
LTRIM(NAME)
```

```
LTRIM(NAME, ' _~');
```

RTRIM

```
RTRIM(string [, charactersToTrimString])
```

Removes all trailing spaces or other specified characters from a string, multiple characters can be specified.

Example:

```
RTRIM(NAME)
```

```
RTRIM(NAME, ' _~');
```

BTRIM

```
BTRIM(string [, charactersToTrimString])
```


Removes all leading and trailing spaces or other specified characters from a string, multiple characters can be specified.

Example:

```
BTRIM(NAME)
```

```
BTRIM(NAME, ' _~');
```

TRIM

```
TRIM ( [ [ LEADING | TRAILING | BOTH ] [ characterToTrimString ] FROM ]  
string )
```

Removes all leading spaces, trailing spaces, or spaces at both ends from a string. If character to trim is specified, these characters are removed instead of spaces, only one character can be specified. To trim multiple different characters use [LTRIM](#), [RTRIM](#), or [BTRIM](#).

If neither LEADING, TRAILING, nor BOTH are specified, BOTH is implicit.

Example:

```
TRIM(NAME)
```

```
TRIM(LEADING FROM NAME)
```

```
TRIM(BOTH ' _' FROM NAME)
```

REGEXP_REPLACE

```
REGEXP_REPLACE(inputString, regexString, replacementString [, flagsString])
```

Replaces each substring that matches a regular expression. For details, see the Java `String.replaceAll()` method. If any parameter is null (except optional `flagsString` parameter), the result is null.

Flags values are limited to 'i', 'c', 'n', 'm'. Other symbols cause exception.

Multiple symbols could be used in one `flagsString` parameter (like 'im'). Later flags override first ones, for example 'ic' is equivalent to case sensitive matching 'c'.

'i' enables case insensitive matching (`Pattern.CASE_INSENSITIVE`)

'c' disables case insensitive matching (Pattern.CASE_INSENSITIVE)

'n' allows the period to match the newline character (Pattern.DOTALL)

'm' enables multiline mode (Pattern.MULTILINE)

Example:

```
REGEXP_REPLACE('Hello World', ' +', ' ')
```

```
REGEXP_REPLACE('Hello WWWWorld', 'w+', 'W', 'i')
```

REGEXP_LIKE

```
REGEXP_LIKE(inputString, regexString [, flagsString])
```

Matches string to a regular expression. For details, see the Java Matcher.find() method. If any parameter is null (except optional flagsString parameter), the result is null.

Flags values are limited to 'i', 'c', 'n', 'm'. Other symbols cause exception.

Multiple symbols could be used in one flagsString parameter (like 'im'). Later flags override first ones, for example 'ic' is equivalent to case sensitive matching 'c'.

'i' enables case insensitive matching (Pattern.CASE_INSENSITIVE)

'c' disables case insensitive matching (Pattern.CASE_INSENSITIVE)

'n' allows the period to match the newline character (Pattern.DOTALL)

'm' enables multiline mode (Pattern.MULTILINE)

Example:

```
REGEXP_LIKE('Hello World', '[A-Z ]*', 'i')
```

REGEXP_SUBSTR

```
REGEXP_SUBSTR(inputString, regexString [, positionInt, occurrenceInt,  
flagsString, groupInt])
```

Matches string to a regular expression and returns the matched substring. For details, see the java.util.regex.Pattern and related functionality.

The parameter position specifies where in inputString the match should start. Occurrence indicates which occurrence of pattern in inputString to search for. Flags values are limited to 'i', 'c', 'n', 'm'. Other symbols cause exception. Multiple symbols could be used in one flagsString parameter (like 'im'). Later flags override first ones, for example 'ic' is equivalent to case sensitive matching 'c'.

'i' enables case insensitive matching (Pattern.CASE_INSENSITIVE)

'c' disables case insensitive matching (Pattern.CASE_INSENSITIVE)

'n' allows the period to match the newline character (Pattern.DOTALL)

'm' enables multiline mode (Pattern.MULTILINE)

If the pattern has groups, the group parameter can be used to specify which group to return.

Example:

```
REGEXP_SUBSTR('2020-10-01', '\d{4}')
```

```
REGEXP_SUBSTR('2020-10-01', '(\d{4})-(\d{2})-(\d{2})', 1, 1, NULL, 2)
```

REPEAT

```
REPEAT(string, int)
```

Returns a string repeated some number of times.

Example:

```
REPEAT(NAME || ' ', 10)
```

REPLACE

```
REPLACE(string, searchString [, replacementString])
```

Replaces all occurrences of a search string in a text with another string. If no replacement is specified, the search string is removed from the original string. If any parameter is null, the result is null.

Example:

REPLACE(NAME, ' ')

SOUNDEX

SOUNDEX(string)

Returns a four character upper-case code representing the sound of a string as pronounced in English. This method returns a string, or null if parameter is null. See <https://en.wikipedia.org/wiki/Soundex> for more information.

Example:

SOUNDEX(NAME)

SPACE

SPACE(int)

Returns a string consisting of a number of spaces.

Example:

SPACE(80)

STRINGDECODE

STRINGDECODE(string)

Converts a encoded string using the Java string literal encoding format. Special characters are \b, \t, \n, \f, \r, \", \\, \<octal>, \u<unicode>. This method returns a string.

Example:

CALL STRINGENCODE(STRINGDECODE('Lines 1\nLine 2'))

STRINGENCODE

STRINGENCODE(string)

Encodes special characters in a string using the Java string literal encoding format. Special characters are \b, \t, \n, \f, \r, \", \\, \<octal>, \u<unicode>. This method returns a string.

Example:

```
CALL STRINGENCODE(STRINGDECODE('Lines 1\nLine 2'))
```

STRINGTOUTF8

```
STRINGTOUTF8(string)
```

Encodes a string to a byte array using the UTF8 encoding format. This method returns bytes.

Example:

```
CALL UTF8TOSTRING(STRINGTOUTF8('This is a test'))
```

SUBSTRING

```
SUBSTRING ( {string | bytes} FROM startInt [ FOR lengthInt ] )  
| { { SUBSTRING | SUBSTR } ( {string | bytes}, startInt [, lengthInt ] ) }
```

Returns a substring of a string starting at a position. If the start index is negative, then the start index is relative to the end of the string. The length is optional.

Example:

```
CALL SUBSTRING('[Hello]' FROM 2 FOR 5);
```

```
CALL SUBSTRING('hour' FROM 2);
```

UTF8TOSTRING

```
UTF8TOSTRING(bytes)
```

Decodes a byte array in the UTF8 format to a string.

Example:

```
CALL UTF8TOSTRING(STRINGTOUTF8('This is a test'))
```

QUOTE_IDENT

```
QUOTE_IDENT(string)
```

Quotes the specified identifier. Identifier is surrounded by double quotes. If identifier contains double quotes they are repeated twice.

Example:

```
QUOTE_IDENT('Column 1')
```

XMLATTR

```
XMLATTR(nameString, valueString)
```

Creates an XML attribute element of the form name=value. The value is encoded as XML text. This method returns a string.

Example:

```
CALL XMLNODE('a', XMLATTR('href', 'https://h2database.com'))
```

XMLNODE

```
XMLNODE(elementString [, attributesString [, contentString [, indentBoolean]]])
```

Create an XML node element. An empty or null attribute string means no attributes are set. An empty or null content string means the node is empty. The content is indented by default if it contains a newline. This method returns a string.

Example:

```
CALL XMLNODE('a', XMLATTR('href', 'https://h2database.com'), 'H2')
```

XMLCOMMENT

```
XMLCOMMENT(commentString)
```

Creates an XML comment. Two dashes (--) are converted to - -. This method returns a string.

Example:

CALL XMLCOMMENT('Test')

XMLCDATA

XMLCDATA(valueString)

Creates an XML CDATA element. If the value contains]]>, an XML text element is created instead. This method returns a string.

Example:

CALL XMLCDATA('data')

XMLSTARTDOC

XMLSTARTDOC()

Returns the XML declaration. The result is always <?xml version=1.0?>.

Example:

CALL XMLSTARTDOC()

XMLTEXT

XMLTEXT(valueString [, escapeNewlineBoolean])

Creates an XML text element. If enabled, newline and linefeed is converted to an XML entity (&#). This method returns a string.

Example:

CALL XMLTEXT('test')

TO_CHAR

TO_CHAR(value [, formatString[, nlsParamString]])

Oracle-compatible TO_CHAR function that can format a timestamp, a number, or text.

Example:

CALL TO_CHAR(TIMESTAMP '2010-01-01 00:00:00', 'DD MON, YYYY')

TRANSLATE

```
TRANSLATE(value, searchString, replacementString)
```

Oracle-compatible TRANSLATE function that replaces a sequence of characters in a string with another set of characters.

Example:

```
CALL TRANSLATE('Hello world', 'eo', 'EO')
```

Time and Date Functions

CURRENT_DATE

```
CURRENT_DATE
```

Returns the current date.

These functions return the same value within a transaction (default) or within a command depending on database mode.

[SET TIME ZONE](#) command reevaluates the value for these functions using the same original UTC timestamp of transaction.

Example:

```
CURRENT_DATE
```

CURRENT_TIME

```
CURRENT_TIME [ (int) ]
```

Returns the current time with time zone. If fractional seconds precision is specified it should be from 0 to 9, 0 is default. The specified value can be used only to limit precision of a result. The actual maximum available precision depends on operating system and JVM and can be 3 (milliseconds) or higher. Higher precision is not available before Java 9.

This function returns the same value within a transaction (default) or within a command depending on database mode.

SET TIME ZONE command reevaluates the value for this function using the same original UTC timestamp of transaction.

Example:

CURRENT_TIME

CURRENT_TIME(9)

CURRENT_TIMESTAMP

CURRENT_TIMESTAMP [([int](#))]

Returns the current timestamp with time zone. Time zone offset is set to a current time zone offset. If fractional seconds precision is specified it should be from 0 to 9, 6 is default. The specified value can be used only to limit precision of a result. The actual maximum available precision depends on operating system and JVM and can be 3 (milliseconds) or higher. Higher precision is not available before Java 9.

This function returns the same value within a transaction (default) or within a command depending on database mode.

SET TIME ZONE command reevaluates the value for this function using the same original UTC timestamp of transaction.

Example:

CURRENT_TIMESTAMP

CURRENT_TIMESTAMP(9)

LOCALTIME

LOCALTIME [([int](#))]

Returns the current time without time zone. If fractional seconds precision is specified it should be from 0 to 9, 0 is default. The specified value can be used only to limit precision of a result. The actual maximum available precision depends on operating system and JVM and can be 3 (milliseconds) or higher. Higher precision is not available before Java 9.

These functions return the same value within a transaction (default) or within a command depending on database mode.

[SET TIME ZONE](#) command reevaluates the value for these functions using the same original UTC timestamp of transaction.

Example:

LOCALTIME

LOCALTIME(9)

LOCALTIMESTAMP

```
LOCALTIMESTAMP [ (int) ]
```

Returns the current timestamp without time zone. If fractional seconds precision is specified it should be from 0 to 9, 6 is default. The specified value can be used only to limit precision of a result. The actual maximum available precision depends on operating system and JVM and can be 3 (milliseconds) or higher. Higher precision is not available before Java 9.

The returned value has date and time without time zone information. If time zone has DST transitions the returned values are ambiguous during transition from DST to normal time. For absolute timestamps use the

[CURRENT_TIMESTAMP](#) function and [TIMESTAMP WITH TIME ZONE](#) data type.

These functions return the same value within a transaction (default) or within a command depending on database mode.

[SET TIME ZONE](#) reevaluates the value for these functions using the same original UTC timestamp of transaction.

Example:

LOCALTIMESTAMP

LOCALTIMESTAMP(9)

DATEADD

```
{ DATEADD | TIMESTAMPADD } (datetimeField, addIntLong, dateAndTime)
```

Adds units to a date-time value. The `datetimeField` indicates the unit. Use negative values to subtract units. `addIntLong` may be a long value when manipulating milliseconds, microseconds, or nanoseconds otherwise its range is restricted to `int`. This method returns a value with the same type as specified value if unit is compatible with this value. If specified field is a HOUR, MINUTE, SECOND, MILLISECOND, etc and value is a DATE value `DATEADD` returns combined `TIMESTAMP`. Fields DAY, MONTH, YEAR, WEEK, etc are not allowed for TIME values. Fields `TIMEZONE_HOUR`, `TIMEZONE_MINUTE`, and `TIMEZONE_SECOND` are only allowed for `TIMESTAMP WITH TIME ZONE` values.

Example:

```
DATEADD(MONTH, 1, DATE '2001-01-31')
```

DATEDIFF

```
{ DATEDIFF | TIMESTAMPDIFF } (datetimeField, aDateAndTime, bDateAndTime)
```

Returns the number of crossed unit boundaries between two date/time values. This method returns a long. The `datetimeField` indicates the unit. Only `TIMEZONE_HOUR`, `TIMEZONE_MINUTE`, and `TIMEZONE_SECOND` fields use the time zone offset component. With all other fields if date/time values have time zone offset component it is ignored.

Example:

```
DATEDIFF(YEAR, T1.CREATED, T2.CREATED)
```

DATE_TRUNC

```
DATE_TRUNC(datetimeField, dateAndTime)
```

Truncates the specified date-time value to the specified field.

Example:

```
DATE_TRUNC(DAY, TIMESTAMP '2010-01-03 10:40:00');
```

LAST_DAY

```
LAST_DAY(date | timestamp | timestampWithTimeZone | string)
```

Returns the last day of the month that contains the specified date-time value. This function returns a date.

Example:

```
LAST_DAY(DAY, DATE '2020-02-05');
```

DAYNAME

```
DAYNAME(dateAndTime)
```

Returns the name of the day (in English).

Example:

```
DAYNAME(CREATED)
```

DAY_OF_MONTH

```
DAY_OF_MONTH({dateAndTime | interval})
```

Returns the day of the month (1-31).

This function is deprecated, use [EXTRACT](#) instead of it.

Example:

```
DAY_OF_MONTH(CREATED)
```

DAY_OF_WEEK

```
DAY_OF_WEEK(dateAndTime)
```

Returns the day of the week (1-7), locale-specific.

This function is deprecated, use [EXTRACT](#) instead of it.

Example:

```
DAY_OF_WEEK(CREATED)
```

ISO_DAY_OF_WEEK

```
ISO_DAY_OF_WEEK(dateAndTime)
```

Returns the ISO day of the week (1 means Monday).

This function is deprecated, use [EXTRACT](#) instead of it.

Example:

```
ISO_DAY_OF_WEEK(CREATED)
```

DAY_OF_YEAR

```
DAY_OF_YEAR({dateAndTime | interval})
```

Returns the day of the year (1-366).

This function is deprecated, use [EXTRACT](#) instead of it.

Example:

```
DAY_OF_YEAR(CREATED)
```

EXTRACT

```
EXTRACT ( datetimeField FROM { dateAndTime | interval })
```

Returns a value of the specific time unit from a date/time value. This method returns a numeric value with EPOCH field and an int for all other fields.

Example:

```
EXTRACT(SECOND FROM CURRENT_TIMESTAMP)
```

FORMATDATETIME

```
FORMATDATETIME ( dateAndTime, formatString  
[, localeString [, timeZoneString ] ] )
```

Formats a date, time or timestamp as a string. The most important format characters are: y year, M month, d day, H hour, m minute, s second. For details

of the format, see `java.time.format.DateTimeFormatter`. Allowed format characters depend on data type of passed date/time value.

If `timeZoneString` is specified, it is used in formatted string if `formatString` has time zone. For `TIME` and `TIME WITH TIME ZONE` values the specified time zone must have a fixed offset.

If `TIME WITH TIME ZONE` is passed and `timeZoneString` is specified, the time is converted to the specified time zone offset and its UTC value is preserved. If `TIMESTAMP WITH TIME ZONE` is passed and `timeZoneString` is specified, the timestamp is converted to the specified time zone and its UTC value is preserved.

This method returns a string.

See also [cast specification](#).

Example:

```
CALL FORMATDATETIME(TIMESTAMP '2001-02-03 04:05:06',  
    'EEE, d MMM yyyy HH:mm:ss z', 'en', 'GMT')
```

HOUR

`HOUR({dateAndTime | interval})`

Returns the hour (0-23) from a date/time value.

This function is deprecated, use [EXTRACT](#) instead of it.

Example:

```
HOUR(CREATED)
```

MINUTE

`MINUTE({dateAndTime | interval})`

Returns the minute (0-59) from a date/time value.

This function is deprecated, use [EXTRACT](#) instead of it.

Example:

MINUTE(CREATED)

MONTH

```
MONTH({dateAndTime | interval})
```

Returns the month (1-12) from a date/time value.

This function is deprecated, use [EXTRACT](#) instead of it.

Example:

MONTH(CREATED)

MONTHNAME

```
MONTHNAME(dateAndTime)
```

Returns the name of the month (in English).

Example:

MONTHNAME(CREATED)

PARSEDATETIME

```
PARSEDATETIME(string, formatString  
[, localeString [, timeZoneString]])
```

Parses a string and returns a `TIMESTAMP WITH TIME ZONE` value. The most important format characters are: y year, M month, d day, H hour, m minute, s second. For details of the format, see `java.time.format.DateTimeFormatter`.

If `timeZoneString` is specified, it is used as default.

See also [cast specification](#).

Example:

```
CALL PARSEDATETIME('Sat, 3 Feb 2001 03:05:06 GMT',  
  'EEE, d MMM yyyy HH:mm:ss z', 'en', 'GMT')
```

QUARTER

QUARTER(dateAndTime)

Returns the quarter (1-4) from a date/time value.

This function is deprecated, use [EXTRACT](#) instead of it.

Example:

QUARTER(CREATED)

SECOND

SECOND(dateAndTime)

Returns the second (0-59) from a date/time value.

This function is deprecated, use [EXTRACT](#) instead of it.

Example:

SECOND(CREATED | interval)

WEEK

WEEK(dateAndTime)

Returns the week (1-53) from a date/time value.

This function is deprecated, use [EXTRACT](#) instead of it.

This function uses the current system locale.

Example:

WEEK(CREATED)

ISO_WEEK

ISO_WEEK(dateAndTime)

Returns the ISO week (1-53) from a date/time value.

This function is deprecated, use [EXTRACT](#) instead of it.

This function uses the ISO definition when first week of year should have at least four days and week is started with Monday.

Example:

```
ISO_WEEK(CREATED)
```

YEAR

```
YEAR({dateAndTime | interval})
```

Returns the year from a date/time value.

This function is deprecated, use [EXTRACT](#) instead of it.

Example:

```
YEAR(CREATED)
```

ISO_YEAR

```
ISO_YEAR(dateAndTime)
```

Returns the ISO week year from a date/time value.

This function is deprecated, use [EXTRACT](#) instead of it.

Example:

```
ISO_YEAR(CREATED)
```

System Functions

ABORT_SESSION

```
ABORT_SESSION(sessionInt)
```

Cancels the currently executing statement of another session. Closes the session and releases the allocated resources. Returns true if the session was closed, false if no session with the given id was found.

If a client was connected while its session was aborted it will see an error.

Admin rights are required to execute this command.

Example:

```
CALL ABORT_SESSION(3)
```

ARRAY_GET

```
ARRAY_GET(arrayExpression, indexExpression)
```

Returns element at the specified 1-based index from an array.

This function is deprecated, use [array element reference] (https://www.h2database.com/html/#array_element_reference) instead of it.

Returns NULL if array or index is NULL.

Example:

```
CALL ARRAY_GET(ARRAY['Hello', 'World'], 2)
```

CARDINALITY

```
{ CARDINALITY | { ARRAY_LENGTH } }(arrayExpression)
```

Returns the length of an array or JSON array. Returns NULL if the specified array is NULL.

Example:

```
CALL CARDINALITY(ARRAY['Hello', 'World'])
```

```
CALL CARDINALITY(JSON '[1, 2, 3]')
```

ARRAY_CONTAINS

```
ARRAY_CONTAINS(arrayExpression, value)
```

Returns a boolean TRUE if the array contains the value or FALSE if it does not contain it. Returns NULL if the specified array is NULL.

Example:

```
CALL ARRAY_CONTAINS(ARRAY['Hello', 'World'], 'Hello')
```

ARRAY_CAT

```
ARRAY_CAT(arrayExpression, arrayExpression)
```

Returns the concatenation of two arrays.

This function is deprecated, use || instead of it.

Returns NULL if any parameter is NULL.

Example:

```
CALL ARRAY_CAT(ARRAY[1, 2], ARRAY[3, 4])
```

ARRAY_APPEND

```
ARRAY_APPEND(arrayExpression, value)
```

Append an element to the end of an array.

This function is deprecated, use || instead of it.

Returns NULL if any parameter is NULL.

Example:

```
CALL ARRAY_APPEND(ARRAY[1, 2], 3)
```

ARRAY_MAX_CARDINALITY

```
ARRAY_MAX_CARDINALITY(arrayExpression)
```

Returns the maximum allowed array cardinality (length) of the declared data type of argument.

Example:

```
SELECT ARRAY_MAX_CARDINALITY(COL1) FROM TEST FETCH FIRST ROW ONLY;
```

TRIM_ARRAY

```
TRIM_ARRAY(arrayExpression, int)
```

Removes the specified number of elements from the end of the array.

Returns NULL if second parameter is NULL or if first parameter is NULL and second parameter is not negative. Throws exception if second parameter is negative or larger than number of elements in array. Otherwise returns the truncated array.

Example:

```
CALL TRIM_ARRAY(ARRAY[1, 2, 3, 4], 1)
```

ARRAY_SLICE

```
ARRAY_SLICE(arrayExpression, lowerBoundInt, upperBoundInt)
```

Returns elements from the array as specified by the lower and upper bound parameters. Both parameters are inclusive and the first element has index 1, i.e. ARRAY_SLICE(a, 2, 2) has only the second element. Returns NULL if any parameter is NULL or if an index is out of bounds.

Example:

```
CALL ARRAY_SLICE(ARRAY[1, 2, 3, 4], 1, 3)
```

AUTOCOMMIT

```
AUTOCOMMIT()
```

Returns true if auto commit is switched on for this session.

Example:

```
AUTOCOMMIT()
```

CANCEL_SESSION

```
CANCEL_SESSION(sessionInt)
```

Cancels the currently executing statement of another session. Returns true if the statement was canceled, false if the session is closed or no statement is currently executing.

Admin rights are required to execute this command.

Example:

```
CANCEL_SESSION(3)
```

CASEWHEN Function

```
CASEWHEN(boolean, aValue, bValue)
```

Returns 'aValue' if the boolean expression is true, otherwise 'bValue'.

This function is deprecated, use [CASE](#) instead of it.

Example:

```
CASEWHEN(ID=1, 'A', 'B')
```

COALESCE

```
{ COALESCE | { NVL } } (aValue, bValue [...])  
| IFNULL(aValue, bValue)
```

Returns the first value that is not null.

Example:

```
COALESCE(A, B, C)
```

CONVERT

```
CONVERT(value, dataTypeOrDomain)
```

Converts a value to another data type.

This function is deprecated, use [CAST](#) instead of it.

Example:

```
CONVERT(NAME, INT)
```

CURRVAL

```
CURRVAL( [ schemaNameString, ] sequenceString )
```

Returns the latest generated value of the sequence for the current session. Current value may only be requested after generation of the sequence value in the current session. This method exists only for compatibility, when it isn't required use `CURRENT VALUE FOR sequenceName` instead. If the schema name is not set, the current schema is used. When sequence is not found, the uppercase name is also checked. This method returns a long.

Example:

```
CURRVAL('TEST_SEQ')
```

CSVWRITE

```
CSVWRITE ( fileNameString, queryString [, csvOptions [, lineSepString]] )
```

Writes a CSV (comma separated values). The file is overwritten if it exists. If only a file name is specified, it will be written to the current working directory. For each parameter, NULL means the default value should be used. The default charset is the default value for this system, and the default field separator is a comma.

The values are converted to text using the default string representation; if another conversion is required you need to change the select statement accordingly. The parameter nullString is used when writing NULL (by default nothing is written when NULL appears). The default line separator is the default value for this system (system property line.separator).

The returned value is the number of rows written. Admin rights are required to execute this command.

Example:

```
CALL CSVWRITE('data/test.csv', 'SELECT * FROM TEST');
CALL CSVWRITE('data/test2.csv', 'SELECT * FROM TEST', 'charset=UTF-8
fieldSeparator=|');
-- Write a tab-separated file
CALL CSVWRITE('data/test.tsv', 'SELECT * FROM TEST', 'charset=UTF-8
fieldSeparator=' || CHAR(9));
```

CURRENT_SCHEMA

CURRENT_SCHEMA | SCHEMA()

Returns the name of the default schema for this session.

Example:

```
CALL CURRENT_SCHEMA
```

CURRENT_CATALOG

CURRENT_CATALOG | DATABASE()

Returns the name of the database.

Example:

```
CALL CURRENT_CATALOG
```

DATABASE_PATH

DATABASE_PATH()

Returns the directory of the database files and the database name, if it is file based. Returns NULL otherwise.

Example:

```
CALL DATABASE_PATH();
```

DATA_TYPE_SQL

DATA_TYPE_SQL
(objectSchemaString, objectNameString, objectTypeString,
typeIdentifierString)

Returns SQL representation of data type of the specified constant, domain, table column, routine result or argument.

For constants object type is 'CONSTANT' and type identifier is the value of INFORMATION_SCHEMA.CONSTANTS.DTD_IDENTIFIER.

For domains object type is 'DOMAIN' and type identifier is the value of INFORMATION_SCHEMA.DOMAINS.DTD_IDENTIFIER.

For columns object type is 'TABLE' and type identifier is the value of INFORMATION_SCHEMA.COLUMNS.DTD_IDENTIFIER.

For routines object name is the value of INFORMATION_SCHEMA.ROUTINES.SPECIFIC_NAME, object type is 'ROUTINE', and type identifier is the value of INFORMATION_SCHEMA.ROUTINES.DTD_IDENTIFIER for data type of the result and the value of INFORMATION_SCHEMA.PARAMETERS.DTD_IDENTIFIER for data types of arguments. Aggregate functions aren't supported by this function, because their data type isn't statically known.

This function returns NULL if any argument is NULL, object type is not valid, or object isn't found.

Example:

```
DATA_TYPE_SQL('PUBLIC', 'C', 'CONSTANT', 'TYPE')
```

```
DATA_TYPE_SQL('PUBLIC', 'D', 'DOMAIN', 'TYPE')
```

```
DATA_TYPE_SQL('PUBLIC', 'T', 'TABLE', '1')
```

```
DATA_TYPE_SQL('PUBLIC', 'R_1', 'ROUTINE', 'RESULT')
```

```
DATA_TYPE_SQL('PUBLIC', 'R_1', 'ROUTINE', '1')
```

```
COALESCE(
```

```
    QUOTE_IDENT(DOMAIN_SCHEMA) || '.' || QUOTE_IDENT(DOMAIN_NAME),
```

```
    DATA_TYPE_SQL(TABLE_SCHEMA, TABLE_NAME, 'TABLE', DTD_IDENTIFIER))
```

DB_OBJECT_ID

```
DB_OBJECT_ID({{'ROLE'|'SETTING'|'SCHEMA'|'USER'}, objectNameString  
| {'CONSTANT'|'CONSTRAINT'|'DOMAIN'|'INDEX'|'ROUTINE'|'SEQUENCE'  
| 'SYNONYM'|'TABLE'|'TRIGGER'}, schemaNameString, objectNameString })
```

Returns internal identifier of the specified database object as integer value or NULL if object doesn't exist.

Admin rights are required to execute this function.

Example:

```
CALL DB_OBJECT_ID('ROLE', 'MANAGER');  
CALL DB_OBJECT_ID('TABLE', 'PUBLIC', 'MY_TABLE');
```

DB_OBJECT_SQL

```
DB_OBJECT_SQL({{'ROLE'|'SETTING'|'SCHEMA'|'USER'}, objectNameString  
| {'CONSTANT'|'CONSTRAINT'|'DOMAIN'|'INDEX'|'ROUTINE'|'SEQUENCE'  
| 'SYNONYM'|'TABLE'|'TRIGGER'}, schemaNameString, objectNameString })
```

Returns internal SQL definition of the specified database object or NULL if object doesn't exist or it is a system object without SQL definition.

This function should not be used to analyze structure of the object by machine code. Internal SQL representation may contain undocumented non-standard clauses and may be different in different versions of H2. Objects are described in the INFORMATION_SCHEMA in machine-readable way.

Admin rights are required to execute this function.

Example:

```
CALL DB_OBJECT_SQL('ROLE', 'MANAGER');  
CALL DB_OBJECT_SQL('TABLE', 'PUBLIC', 'MY_TABLE');
```

DB_OBJECT_SIZE

```
DB_OBJECT_SIZE({'INDEX'|'TABLE'}, schemaNameString, objectNameString)
```

Returns the approximate amount of space used by the specified table (excluding its indexes) or index. Only size of version used by the current transaction is estimated. Size of large LOBs currently is not included into estimation. This function may be expensive since it has to load every page in the table or index. Use [DB_OBJECT_APPROXIMATE_SIZE](#) for a faster coarse approximation.

Example:

```
CALL DB_OBJECT_SIZE('TABLE', 'PUBLIC', 'MY_TABLE');
```

DB_OBJECT_TOTAL_SIZE

```
DB_OBJECT_TOTAL_SIZE('TABLE', schemaNameString, objectNameString)
```

Returns the approximate amount of space used by the specified table and all its indexes. Only size of version used by the current transaction is estimated. Size of large LOBs currently is not included into estimation. This function may be expensive since it has to load every page in the table and its indexes. Use [DB_OBJECT_APPROXIMATE_TOTAL_SIZE](#) for a faster coarse approximation.

Example:

```
CALL DB_OBJECT_TOTAL_SIZE('TABLE', 'PUBLIC', 'MY_TABLE');
```

DB_OBJECT_APPROXIMATE_SIZE

```
DB_OBJECT_APPROXIMATE_SIZE({'INDEX' | 'TABLE'}, schemaNameString,  
objectNameString)
```

Returns the coarse approximate amount of space used by the specified table (excluding its indexes) or index. Only size of version used by the current transaction is estimated. Size of large LOBs currently is not included into estimation.

Example:

```
CALL DB_OBJECT_APPROXIMATE_SIZE('TABLE', 'PUBLIC', 'MY_TABLE');
```

DB_OBJECT_APPROXIMATE_TOTAL_SIZE

```
DB_OBJECT_APPROXIMATE_TOTAL_SIZE('TABLE', schemaNameString,  
objectNameString)
```

Returns the coarse approximate amount of space used by the specified table and all its indexes. Only size of version used by the current transaction is estimated. Size of large LOBs currently is not included into estimation.

Example:

```
CALL DB_OBJECT_APPROXIMATE_TOTAL_SIZE('TABLE', 'PUBLIC', 'MY_TABLE');
```

DECODE

```
DECODE(value, whenValue, thenValue [...])
```

Returns the first matching value. NULL is considered to match NULL. If no match was found, then NULL or the last parameter (if the parameter count is even) is returned. This function is provided for Oracle compatibility, use [CASE](#) instead of it.

Example:

```
CALL DECODE(RAND()>0.5, 0, 'Red', 1, 'Black');
```

DISK_SPACE_USED

```
DISK_SPACE_USED(tableNameString)
```

Returns the approximate amount of space used by the table specified. Only size of version used by the current transaction is estimated. Does not currently take into account indexes or LOB's. This function may be expensive since it has to load every page in the table. This function is deprecated, use [DB_OBJECT_SIZE](#) instead of it.

Example:

```
CALL DISK_SPACE_USED('my_table');
```

SIGNAL

```
SIGNAL(sqlStateString, messageString)
```

Throw an SQLException with the passed SQLState and reason.

Example:

```
CALL SIGNAL('23505', 'Duplicate user ID: ' || user_id);
```

ESTIMATED_ENVELOPE

```
ESTIMATED_ENVELOPE(tableNameString, columnNameString)
```

Returns the estimated minimum bounding box that encloses all specified GEOMETRY values. Only 2D coordinate plane is supported. NULL values are ignored. Column must have a spatial index. This function is fast, but estimation may include uncommitted data (including data from other transactions), may return approximate bounds, or be different with actual value due to other reasons. Use with caution. If estimation is not available this function returns NULL. For accurate and reliable result use ESTIMATE aggregate function instead.

Example:

```
CALL ESTIMATED_ENVELOPE('MY_TABLE', 'GEOMETRY_COLUMN');
```

FILE_READ

```
FILE_READ(fileNameString [,encodingString])
```

Returns the contents of a file. If only one parameter is supplied, the data are returned as a BLOB. If two parameters are used, the data is returned as a CLOB (text). The second parameter is the character set to use, NULL meaning the default character set for this system.

File names and URLs are supported. To read a stream from the classpath, use the prefix classpath:.

Admin rights are required to execute this command.

Example:

```
SELECT LENGTH(FILE_READ('~/.h2.server.properties')) LEN;  
SELECT FILE_READ('http://localhost:8182/stylesheet.css', NULL) CSS;
```

FILE_WRITE

```
FILE_WRITE(blobValue, fileNameString)
```

Write the supplied parameter into a file. Return the number of bytes written.

Write access to folder, and admin rights are required to execute this command.

Example:

```
SELECT FILE_WRITE('Hello world', '/tmp/hello.txt')) LEN;
```

GREATEST

```
GREATEST(aValue, bValue [...]) [{RESPECT|IGNORE} NULLS]
```

Returns the largest value or NULL if any value is NULL or the largest value cannot be determined. For example, ROW (NULL, 1) is neither equal to nor smaller than nor larger than ROW (1, 1). If IGNORE NULLS is specified, NULL values are ignored.

Example:

```
CALL GREATEST(1, 2, 3);
```

LEAST

```
LEAST(aValue, bValue [...]) [{RESPECT|IGNORE} NULLS]
```

Returns the smallest value or NULL if any value is NULL or the smallest value cannot be determined. For example, ROW (NULL, 1) is neither equal to nor smaller than nor larger than ROW (1, 1). If IGNORE NULLS is specified, NULL values are ignored.

Example:

```
CALL LEAST(1, 2, 3);
```

LOCK_MODE

```
LOCK_MODE()
```

Returns the current lock mode. See SET LOCK_MODE. This method returns an int.

Example:

```
CALL LOCK_MODE();
```

LOCK_TIMEOUT

```
LOCK_TIMEOUT()
```

Returns the lock timeout of the current session (in milliseconds).

Example:

```
LOCK_TIMEOUT()
```

MEMORY_FREE

```
MEMORY_FREE()
```

Returns the free memory in KB (where 1024 bytes is a KB). This method returns a long. The garbage collector is run before returning the value. Admin rights are required to execute this command.

Example:

```
MEMORY_FREE()
```

MEMORY_USED

```
MEMORY_USED()
```

Returns the used memory in KB (where 1024 bytes is a KB). This method returns a long. The garbage collector is run before returning the value. Admin rights are required to execute this command.

Example:

```
MEMORY_USED()
```

NEXTVAL

```
NEXTVAL ( [ schemaNameString, ] sequenceString )
```

Increments the sequence and returns its value. The current value of the sequence and the last identity in the current session are updated with the generated value. Used values are never re-used, even when the transaction is rolled back. This method exists only for compatibility, it's recommended to use the standard `NEXT VALUE FOR sequenceName` instead. If the schema name is not set, the current schema is used. When sequence is not found, the uppercase name is also checked. This method returns a long.

Example:

```
NEXTVAL('TEST_SEQ')
```

NULLIF

```
NULLIF(aValue, bValue)
```

Returns NULL if 'a' is equal to 'b', otherwise 'a'.

Example:

```
NULLIF(A, B)
```

```
A / NULLIF(B, 0)
```

NVL2

```
NVL2(testValue, aValue, bValue)
```

If the test value is null, then 'b' is returned. Otherwise, 'a' is returned. The data type of the returned value is the data type of 'a' if this is a text type.

This function is provided for Oracle compatibility, use [CASE](#) or [COALESCE](#) instead of it.

Example:

```
NVL2(X, 'not null', 'null')
```

READONLY

```
READONLY()
```

Returns true if the database is read-only.

Example:

```
READONLY()
```

ROWNUM

```
ROWNUM()
```

Returns the number of the current row. This method returns a long value. It is supported for SELECT statements, as well as for DELETE and UPDATE. The first row has the row number 1, and is calculated before ordering and grouping the result set, but after evaluating index conditions (even when the index conditions are specified in an outer query). Use the [ROW_NUMBER\(\) OVER \(\)](#) function to get row numbers after grouping or in specified order.

Example:

```
SELECT ROWNUM(), * FROM TEST;  
SELECT ROWNUM(), * FROM (SELECT * FROM TEST ORDER BY NAME);  
SELECT ID FROM (SELECT T.*, ROWNUM AS R FROM TEST T) WHERE R BETWEEN  
2 AND 3;
```

SESSION_ID

[SESSION_ID\(\)](#)

Returns the unique session id number for the current database connection. This id stays the same while the connection is open. This method returns an int. The database engine may re-use a session id after the connection is closed.

Example:

```
CALL SESSION_ID()
```

SET

[SET\(@variableName, value\)](#)

Updates a variable with the given value. The new value is returned. When used in a query, the value is updated in the order the rows are read. When used in a subquery, not all rows might be read depending on the query plan. This can be used to implement running totals / cumulative sums.

Example:

```
SELECT X, SET(@I, COALESCE(@I, 0)+X) RUNNING_TOTAL FROM  
SYSTEM_RANGE(1, 10)
```


TRANSACTION_ID

TRANSACTION_ID()

Returns the current transaction id for this session. This method returns NULL if there is no uncommitted change, or if the database is not persisted. Otherwise a value of the following form is returned: logFileId-position-sessionId. This method returns a string. The value is unique across database restarts (values are not re-used).

Example:

```
CALL TRANSACTION_ID()
```

TRUNCATE_VALUE

TRUNCATE_VALUE(value, precisionInt, forceBoolean)

Truncate a value to the required precision. If force flag is set to FALSE fixed precision values are not truncated. The method returns a value with the same data type as the first parameter.

Example:

```
CALL TRUNCATE_VALUE(X, 10, TRUE);
```

CURRENT_PATH

CURRENT_PATH

Returns the comma-separated list of quoted schema names where user-defined functions are searched when they are referenced without the schema name.

Example:

```
CURRENT_PATH
```

CURRENT_ROLE

CURRENT_ROLE

Returns the name of the PUBLIC role.

Example:

CURRENT_ROLE

CURRENT_USER

CURRENT_USER | SESSION_USER | SYSTEM_USER | USER

Returns the name of the current user of this session.

Example:

CURRENT_USER

H2VERSION

H2VERSION()

Returns the H2 version as a String.

Example:

H2VERSION()

JSON Functions

JSON_OBJECT

```
JSON_OBJECT(  
  [{[KEY] string VALUE expression} | {string : expression}} [,...] ]  
  [ { NULL | ABSENT } ON NULL ]  
  [ { WITH | WITHOUT } UNIQUE KEYS ]  
)
```

Returns a JSON object constructed from the specified properties. If ABSENT ON NULL is specified properties with NULL value are not included in the object. If WITH UNIQUE KEYS is specified the constructed object is checked for uniqueness of keys, nested objects, if any, are checked too.

Example:

JSON_OBJECT('id': 100, 'name': 'Joe', 'groups': '[2,5]' FORMAT JSON);

JSON_ARRAY

```
JSON_ARRAY(  
[expression [...]] | {(query) [FORMAT JSON]}  
[ { NULL | ABSENT } ON NULL ]  
)
```

Returns a JSON array constructed from the specified values or from the specified single-column subquery. If NULL ON NULL is specified NULL values are included in the array.

Example:

```
JSON_ARRAY(10, 15, 20);  
JSON_ARRAY(JSON_DATA_A FORMAT JSON, JSON_DATA_B FORMAT JSON);  
JSON_ARRAY((SELECT J FROM PROPS) FORMAT JSON);
```

Table Functions

CSVREAD

```
CSVREAD(fileNameString [, columnsString [, csvOptions ] ] )
```

Returns the result set of reading the CSV (comma separated values) file. For each parameter, NULL means the default value should be used.

If the column names are specified (a list of column names separated with the `fieldSeparator`), those are used, otherwise (or if they are set to NULL) the first line of the file is interpreted as the column names. In that case, column names that contain no special characters (only letters, '_', and digits; similar to the rule for Java identifiers) are processed in the same way as unquoted identifiers and therefore case of characters may be changed. Other column names are processed as quoted identifiers and case of characters is preserved. To preserve the case of column names unconditionally use [caseSensitiveColumnNames](#) option.

The default charset is the default value for this system, and the default field separator is a comma. Missing unquoted values as well as data that matches nullString is parsed as NULL. All columns are of type VARCHAR.

The BOM (the byte-order-mark) character 0xfeff at the beginning of the file is ignored.

This function can be used like a table: `SELECT * FROM CSVREAD(...)`.

Instead of a file, a URL may be used, for example

`jar:file:///c:/temp/example.zip!/org/example/nested.csv`. To read a stream from the classpath, use the prefix `classpath:.` To read from HTTP, use the prefix `http:` (as in a browser).

For performance reason, `CSVREAD` should not be used inside a join. Instead, import the data first (possibly into a temporary table) and then use the table.

Admin rights are required to execute this command.

Example:

```
SELECT * FROM CSVREAD('test.csv');
```

```
-- Read a file containing the columns ID, NAME with
```

```
SELECT * FROM CSVREAD('test2.csv', 'ID | NAME', 'charset=UTF-8  
fieldSeparator=|');
```

```
SELECT * FROM CSVREAD('data/test.csv', null, 'rowSeparator=');
```

```
-- Read a tab-separated file
```

```
SELECT * FROM CSVREAD('data/test.tsv', null, 'rowSeparator=' || CHAR(9));
```

```
SELECT "Last Name" FROM CSVREAD('address.csv');
```

```
SELECT "Last Name" FROM CSVREAD('classpath:/org/acme/data/address.csv');
```

LINK_SCHEMA

```
LINK_SCHEMA (targetSchemaString, driverString, urlString,  
userString, passwordString, sourceSchemaString)
```

Creates table links for all tables in a schema. If tables with the same name already exist, they are dropped first. The target schema is created automatically if it does not yet exist. The driver name may be empty if the driver is already

loaded. The list of tables linked is returned in the form of a result set. Admin rights are required to execute this command.

Example:

```
SELECT * FROM LINK_SCHEMA('TEST2', '', 'jdbc:h2:./test2', 'sa', 'sa', 'PUBLIC');
```

TABLE

```
{ TABLE | TABLE_DISTINCT }  
( { name dataTypeOrDomain = {array | rowValueExpression} } [...] )
```

Returns the result set. TABLE_DISTINCT removes duplicate rows.

Example:

```
SELECT * FROM TABLE(V INT = ARRAY[1, 2]);  
SELECT * FROM TABLE(ID INT=(1, 2), NAME VARCHAR=('Hello', 'World'));
```

UNNEST

```
UNNEST(arrayExpression, [...]) [WITH ORDINALITY]
```

Returns the result set. Number of columns is equal to number of arguments, plus one additional column with row number if WITH ORDINALITY is specified. Number of rows is equal to length of longest specified array. If multiple arguments are specified and they have different length, cells with missing values will contain null values.

Example:

```
SELECT * FROM UNNEST(ARRAY['a', 'b', 'c']);  
SELECT * FROM UNNEST(JSON '["a", "b", "c"]');
```

Aggregate Functions

Index

General Aggregate Functions

AVG

MAX

MIN

SUM

EVERY

ANY

COUNT

STDDEV_POP

STDDEV_SAMP

VAR_POP

VAR_SAMP

ANY_VALUE

BIT_AND_AGG

BIT_OR_AGG

BIT_XOR_AGG

BIT_NAND_AGG

BIT_NOR_AGG

BIT_XNOR_AGG

ENVELOPE

GCD_AGG

LCM_AGG

Binary Set Functions

COVAR_POP

COVAR_SAMP

CORR

REGR_SLOPE

REGR_INTERCEPT
REGR_COUNT
REGR_R2
REGR_AVGX
REGR_AVGY
REGR_SXX
REGR_SYY
REGR_SXY

Ordered Aggregate Functions

LISTAGG
ARRAY_AGG

Hypothetical Set Functions

RANK aggregate
DENSE_RANK aggregate
PERCENT_RANK aggregate
CUME_DIST aggregate

Inverse Distribution Functions

PERCENTILE_CONT
PERCENTILE_DISC
MEDIAN
MODE

JSON Aggregate Functions

JSON_OBJECTAGG
JSON_ARRAYAGG

Details

Non-standard syntax is marked in green. Compatibility-only non-standard syntax is marked in red, don't use it unless you need it for compatibility with other databases or old versions of H2.

General Aggregate Functions

AVG

```
AVG ( [ DISTINCT|ALL ] { numeric | interval } )  
[FILTER (WHERE expression)] [OVER windowNameOrSpecification]
```

The average (mean) value. If no rows are selected, the result is NULL. Aggregates are only allowed in select statements.

The data type of result is DOUBLE PRECISION for TINYINT, SMALLINT, INTEGER, and REAL arguments, NUMERIC with additional 10 decimal digits of precision and scale for BIGINT and NUMERIC arguments; DECFLOAT with additional 10 decimal digits of precision for DOUBLE PRECISION and DECFLOAT arguments; INTERVAL with the same leading field precision, all additional smaller datetime units in interval qualifier, and the maximum scale for INTERVAL arguments.

Example:

AVG(X)

MAX

```
MAX(value)  
[FILTER (WHERE expression)] [OVER windowNameOrSpecification]
```

The highest value. If no rows are selected, the result is NULL. Aggregates are only allowed in select statements. The returned value is of the same data type as the parameter.

Example:

MAX(NAME)

MIN

```
MIN(value)  
[FILTER (WHERE expression)] [OVER windowNameOrSpecification]
```


The lowest value. If no rows are selected, the result is NULL. Aggregates are only allowed in select statements. The returned value is of the same data type as the parameter.

Example:

MIN(NAME)

SUM

```
SUM( [ DISTINCT|ALL ] { numeric | interval | { boolean } } )  
[FILTER (WHERE expression)] [OVER windowNameOrSpecification]
```

The sum of all values. If no rows are selected, the result is NULL. Aggregates are only allowed in select statements.

The data type of result is BIGINT for BOOLEAN, TINYINT, SMALLINT, and INTEGER arguments; NUMERIC with additional 10 decimal digits of precision for BIGINT and NUMERIC arguments; DOUBLE PRECISION for REAL arguments, DECFLOAT with additional 10 decimal digits of precision for DOUBLE PRECISION and DECFLOAT arguments; INTERVAL with maximum precision and the same interval qualifier and scale for INTERVAL arguments.

Example:

SUM(X)

EVERY

```
{EVERY | {BOOL_AND}}(boolean)  
[FILTER (WHERE expression)] [OVER windowNameOrSpecification]
```

Returns true if all expressions are true. If no rows are selected, the result is NULL. Aggregates are only allowed in select statements.

Example:

EVERY(ID>10)

ANY

```
{ANY|SOME| {BOOL_OR}}(boolean)  
[FILTER (WHERE expression)] [OVER windowNameOrSpecification]
```

Returns true if any expression is true. If no rows are selected, the result is NULL. Aggregates are only allowed in select statements.

Note that if ANY or SOME aggregate function is placed on the right side of comparison operation or distinct predicate and argument of this function is a subquery additional parentheses around aggregate function are required, otherwise it will be parsed as quantified predicate.

Example:

```
ANY(NAME LIKE 'W%')  
A = (ANY((SELECT B FROM T)))
```

COUNT

```
COUNT( { * | { [ DISTINCT|ALL ] expression } } )  
[FILTER (WHERE expression)] [OVER windowNameOrSpecification]
```

The count of all row, or of the non-null values. This method returns a long. If no rows are selected, the result is 0. Aggregates are only allowed in select statements.

Example:

```
COUNT(*)
```

STDDEV_POP

```
STDDEV_POP( [ DISTINCT|ALL ] numeric )  
[FILTER (WHERE expression)] [OVER windowNameOrSpecification]
```

The population standard deviation. This method returns a double. If no rows are selected, the result is NULL. Aggregates are only allowed in select statements.

Example:

STDDEV_POP(X)

STDDEV_SAMP

```
STDDEV_SAMP( [ DISTINCT | ALL ] numeric )  
[FILTER (WHERE expression)] [OVER windowNameOrSpecification]
```

The sample standard deviation. This method returns a double. If less than two rows are selected, the result is NULL. Aggregates are only allowed in select statements.

Example:

STDDEV(X)

VAR_POP

```
VAR_POP( [ DISTINCT | ALL ] numeric )  
[FILTER (WHERE expression)] [OVER windowNameOrSpecification]
```

The population variance (square of the population standard deviation). This method returns a double. If no rows are selected, the result is NULL. Aggregates are only allowed in select statements.

Example:

VAR_POP(X)

VAR_SAMP

```
VAR_SAMP( [ DISTINCT | ALL ] numeric )  
[FILTER (WHERE expression)] [OVER windowNameOrSpecification]
```

The sample variance (square of the sample standard deviation). This method returns a double. If less than two rows are selected, the result is NULL. Aggregates are only allowed in select statements.

Example:

VAR_SAMP(X)

ANY_VALUE

```
ANY_VALUE( [ DISTINCT | ALL ] value )  
[FILTER (WHERE expression)] [OVER windowNameOrSpecification]
```

Returns any non-NULL value from aggregated values. If no rows are selected, the result is NULL. This function uses the same pseudo random generator as [RAND\(\)](#) function.

If DISTINCT is specified, each distinct value will be returned with approximately the same probability as other distinct values. If it isn't specified, more frequent values will be returned with higher probability than less frequent.

Aggregates are only allowed in select statements.

Example:

ANY_VALUE(X)

BIT_AND_AGG

```
{BIT_AND_AGG} | {BIT_AND}(expression)  
[FILTER (WHERE expression)] [OVER windowNameOrSpecification]
```

The bitwise AND of all non-null values. If no rows are selected, the result is NULL. Aggregates are only allowed in select statements.

For non-aggregate function see [BITAND](#).

Example:

BIT_AND_AGG(X)

BIT_OR_AGG

```
{BIT_OR_AGG} | {BIT_OR}(expression)  
[FILTER (WHERE expression)] [OVER windowNameOrSpecification]
```

The bitwise OR of all non-null values. If no rows are selected, the result is NULL. Aggregates are only allowed in select statements.

For non-aggregate function see [BITOR](#).

Example:

BIT_OR_AGG(X)

BIT_XOR_AGG

```
BIT_XOR_AGG( [ DISTINCT | ALL ] expression)  
[FILTER (WHERE expression)] [OVER windowNameOrSpecification]
```

The bitwise XOR of all non-null values. If no rows are selected, the result is NULL. Aggregates are only allowed in select statements.

For non-aggregate function see [BITXOR](#).

Example:

BIT_XOR_AGG(X)

BIT_NAND_AGG

```
BIT_NAND_AGG(expression)  
[FILTER (WHERE expression)] [OVER windowNameOrSpecification]
```

The bitwise NAND of all non-null values. If no rows are selected, the result is NULL. Aggregates are only allowed in select statements.

For non-aggregate function see [BITNAND](#).

Example:

BIT_NAND_AGG(X)

BIT_NOR_AGG

```
BIT_NOR_AGG(expression)  
[FILTER (WHERE expression)] [OVER windowNameOrSpecification]
```

The bitwise NOR of all non-null values. If no rows are selected, the result is NULL. Aggregates are only allowed in select statements.

For non-aggregate function see [BITNOR](#).

Example:

BIT_NOR_AGG(X)

BIT_XNOR_AGG

```
BIT_XNOR_AGG( [ DISTINCT | ALL ] expression)  
[FILTER (WHERE expression)] [OVER windowNameOrSpecification]
```

The bitwise XNOR of all non-null values. If no rows are selected, the result is NULL. Aggregates are only allowed in select statements.

For non-aggregate function see [BITXNOR](#).

Example:

BIT_XNOR_AGG(X)

ENVELOPE

```
ENVELOPE( value )  
[FILTER (WHERE expression)] [OVER windowNameOrSpecification]
```

Returns the minimum bounding box that encloses all specified GEOMETRY values. Only 2D coordinate plane is supported. NULL values are ignored in the calculation. If no rows are selected, the result is NULL. Aggregates are only allowed in select statements.

Example:

ENVELOPE(X)

GCD_AGG

```
GCD_AGG(numeric)  
[FILTER (WHERE expression)] [OVER windowNameOrSpecification]
```

Returns the greatest common divisor of all values. Arguments must have TINYINT, SMALLINT, INTEGER, BIGINT, or NUMERIC data type with scale 0. This function returns result of NUMERIC data type with scale 0. NULL values are ignored in the calculation. If no rows are selected, the result is NULL. Aggregates are only allowed in select statements.

For non-aggregate function see [GCD](#).

Example:

GCD_AGG(V)

LCM_AGG

```
LCM_AGG(numeric)
[FILTER (WHERE expression)] [OVER windowNameOrSpecification]
```

Returns the least common multiple of all values. Arguments must have TINYINT, SMALLINT, INTEGER, BIGINT, or NUMERIC data type with scale 0. This function returns result of NUMERIC data type with scale 0. NULL values are ignored in the calculation. If no rows are selected, the result is NULL. Aggregates are only allowed in select statements.

For non-aggregate function see [LCM](#).

Example:

LCM_AGG(V)

Binary Set Functions

COVAR_POP

```
COVAR_POP(dependentExpression, independentExpression)
[FILTER (WHERE expression)] [OVER windowNameOrSpecification]
```

The population covariance. This method returns a double. Rows in which either argument is NULL are ignored in the calculation. If no rows are selected, the result is NULL. Aggregates are only allowed in select statements.

Example:

COVAR_POP(Y, X)

COVAR_SAMP

```
COVAR_SAMP(dependentExpression, independentExpression)
[FILTER (WHERE expression)] [OVER windowNameOrSpecification]
```

The sample covariance. This method returns a double. Rows in which either argument is NULL are ignored in the calculation. If less than two rows are selected, the result is NULL. Aggregates are only allowed in select statements.

Example:

```
COVAR_SAMP(Y, X)
```

CORR

```
CORR(dependentExpression, independentExpression)  
[FILTER (WHERE expression)] [OVER windowNameOrSpecification]
```

Pearson's correlation coefficient. This method returns a double. Rows in which either argument is NULL are ignored in the calculation. If no rows are selected, the result is NULL. Aggregates are only allowed in select statements.

Example:

```
CORR(Y, X)
```

REGR_SLOPE

```
REGR_SLOPE(dependentExpression, independentExpression)  
[FILTER (WHERE expression)] [OVER windowNameOrSpecification]
```

The slope of the line. This method returns a double. Rows in which either argument is NULL are ignored in the calculation. If no rows are selected, the result is NULL. Aggregates are only allowed in select statements.

Example:

```
REGR_SLOPE(Y, X)
```

REGR_INTERCEPT

```
REGR_INTERCEPT(dependentExpression, independentExpression)  
[FILTER (WHERE expression)] [OVER windowNameOrSpecification]
```


The y-intercept of the regression line. This method returns a double. Rows in which either argument is NULL are ignored in the calculation. If no rows are selected, the result is NULL. Aggregates are only allowed in select statements.

Example:

```
REGR_INTERCEPT(Y, X)
```

REGR_COUNT

```
REGR_COUNT(dependentExpression, independentExpression)  
[FILTER (WHERE expression)] [OVER windowNameOrSpecification]
```

Returns the number of rows in the group. This method returns a long. Rows in which either argument is NULL are ignored in the calculation. If no rows are selected, the result is 0. Aggregates are only allowed in select statements.

Example:

```
REGR_COUNT(Y, X)
```

REGR_R2

```
REGR_R2(dependentExpression, independentExpression)  
[FILTER (WHERE expression)] [OVER windowNameOrSpecification]
```

The coefficient of determination. This method returns a double. Rows in which either argument is NULL are ignored in the calculation. If no rows are selected, the result is NULL. Aggregates are only allowed in select statements.

Example:

```
REGR_R2(Y, X)
```

REGR_AVGX

```
REGR_AVGX(dependentExpression, independentExpression)  
[FILTER (WHERE expression)] [OVER windowNameOrSpecification]
```

The average (mean) value of dependent expression. Rows in which either argument is NULL are ignored in the calculation. If no rows are selected, the

result is NULL. For details about the data type see [AVG](#). Aggregates are only allowed in select statements.

Example:

```
REGR_AVGX(Y, X)
```

REGR_AVGY

```
REGR_AVGY(dependentExpression, independentExpression)  
[FILTER (WHERE expression)] [OVER windowNameOrSpecification]
```

The average (mean) value of independent expression. Rows in which either argument is NULL are ignored in the calculation. If no rows are selected, the result is NULL. For details about the data type see [AVG](#). Aggregates are only allowed in select statements.

Example:

```
REGR_AVGY(Y, X)
```

REGR_SXX

```
REGR_SXX(dependentExpression, independentExpression)  
[FILTER (WHERE expression)] [OVER windowNameOrSpecification]
```

The the sum of squares of independent expression. Rows in which either argument is NULL are ignored in the calculation. If no rows are selected, the result is NULL. Aggregates are only allowed in select statements.

Example:

```
REGR_SXX(Y, X)
```

REGR_SYY

```
REGR_SYY(dependentExpression, independentExpression)  
[FILTER (WHERE expression)] [OVER windowNameOrSpecification]
```

The the sum of squares of dependent expression. Rows in which either argument is NULL are ignored in the calculation. If no rows are selected, the result is NULL. Aggregates are only allowed in select statements.

Example:

REGR_SYY(Y, X)

REGR_SXY

```
REGR_SXY(dependentExpression, independentExpression)
[FILTER (WHERE expression)] [OVER windowNameOrSpecification]
```

The the sum of products independent expression times dependent expression. Rows in which either argument is NULL are ignored in the calculation. If no rows are selected, the result is NULL. Aggregates are only allowed in select statements.

Example:

REGR_SXY(Y, X)

Ordered Aggregate Functions

LISTAGG

```
LISTAGG ( [ DISTINCT|ALL ] string [, separatorString]
[ ON OVERFLOW { ERROR
| TRUNCATE [ filterString ] { WITH | WITHOUT } COUNT } ] )
withinGroupSpecification
[FILTER (WHERE expression)] [OVER windowNameOrSpecification]
```

Concatenates strings with a separator. The default separator is a ',' (without space). This method returns a string. NULL values are ignored in the calculation, COALESCE can be used to replace them. If no rows are selected, the result is NULL.

If ON OVERFLOW TRUNCATE is specified, values that don't fit into returned string are truncated and replaced with filter string placeholder ('...' by default)

and count of truncated elements in parentheses. If WITHOUT COUNT is specified, count of truncated elements is not appended.

Aggregates are only allowed in select statements.

Example:

```
LISTAGG(NAME, ', ' ) WITHIN GROUP (ORDER BY ID)
LISTAGG(COALESCE(NAME, 'null'), ', ' ) WITHIN GROUP (ORDER BY ID)
LISTAGG(ID, ', ' ) WITHIN GROUP (ORDER BY ID) OVER (ORDER BY ID)
LISTAGG(ID, ';' ON OVERFLOW TRUNCATE 'etc' WITHOUT COUNT) WITHIN
GROUP (ORDER BY ID)
```

ARRAY_AGG

```
ARRAY_AGG ( [ DISTINCT | ALL ] value
[ ORDER BY sortSpecificationList ] )
[FILTER (WHERE expression)] [OVER windowNameOrSpecification]
```

Aggregate the value into an array. This method returns an array. NULL values are included in the array, FILTER clause can be used to exclude them. If no rows are selected, the result is NULL. If ORDER BY is not specified order of values is not determined. When this aggregate is used with OVER clause that contains ORDER BY subclause it does not enforce exact order of values. This aggregate needs additional own ORDER BY clause to make it deterministic. Aggregates are only allowed in select statements.

Example:

```
ARRAY_AGG(NAME ORDER BY ID)
ARRAY_AGG(NAME ORDER BY ID) FILTER (WHERE NAME IS NOT NULL)
ARRAY_AGG(ID ORDER BY ID) OVER (ORDER BY ID)
```

Hypothetical Set Functions

RANK aggregate

```
RANK(value [,...])
withinGroupSpecification
```

```
[FILTER (WHERE expression)] [OVER windowNameOrSpecification]
```

Returns the rank of the hypothetical row in specified collection of rows. The rank of a row is the number of rows that precede this row plus 1. If two or more rows have the same values in ORDER BY columns, these rows get the same rank from the first row with the same values. It means that gaps in ranks are possible.

See [RANK](#) for a window function with the same name.

Example:

```
SELECT RANK(5) WITHIN GROUP (ORDER BY V) FROM TEST;
```

DENSE_RANK aggregate

```
DENSE_RANK(value [...])  
withinGroupSpecification  
[FILTER (WHERE expression)] [OVER windowNameOrSpecification]
```

Returns the dense rank of the hypothetical row in specified collection of rows. The rank of a row is the number of groups of rows with the same values in ORDER BY columns that precede group with this row plus 1. If two or more rows have the same values in ORDER BY columns, these rows get the same rank. Gaps in ranks are not possible.

See [DENSE_RANK](#) for a window function with the same name.

Example:

```
SELECT DENSE_RANK(5) WITHIN GROUP (ORDER BY V) FROM TEST;
```

PERCENT_RANK aggregate

```
PERCENT_RANK(value [...])  
withinGroupSpecification  
[FILTER (WHERE expression)] [OVER windowNameOrSpecification]
```

Returns the relative rank of the hypothetical row in specified collection of rows. The relative rank is calculated as $(\text{RANK} - 1) / (\text{NR} - 1)$, where RANK is a rank of

the row and NR is a total number of rows in the collection including hypothetical row.

See [PERCENT_RANK](#) for a window function with the same name.

Example:

```
SELECT PERCENT_RANK(5) WITHIN GROUP (ORDER BY V) FROM TEST;
```

CUME_DIST aggregate

```
CUME_DIST(value [...])  
withinGroupSpecification  
[FILTER (WHERE expression)] [OVER windowNameOrSpecification]
```

Returns the relative rank of the hypothetical row in specified collection of rows. The relative rank is calculated as NP / NR where NP is a number of rows that precede the current row or have the same values in ORDER BY columns and NR is a total number of rows in the collection including hypothetical row.

See [CUME_DIST](#) for a window function with the same name.

Example:

```
SELECT CUME_DIST(5) WITHIN GROUP (ORDER BY V) FROM TEST;
```

Inverse Distribution Functions

PERCENTILE_CONT

```
PERCENTILE_CONT(numeric) WITHIN GROUP (ORDER BY sortSpecification)  
[FILTER (WHERE expression)] [OVER windowNameOrSpecification]
```

Return percentile of values from the group with interpolation. Interpolation is only supported for numeric, date-time, and interval data types. Argument must be between 0 and 1 inclusive. Argument must be the same for all rows in the same group. If argument is NULL, the result is NULL. NULL values are ignored in the calculation. If no rows are selected, the result is NULL. Aggregates are only allowed in select statements.

Example:

PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY V)

PERCENTILE_DISC

```
PERCENTILE_DISC(numeric) WITHIN GROUP (ORDER BY sortSpecification)
[FILTER (WHERE expression)] [OVER windowNameOrSpecification]
```

Return percentile of values from the group. Interpolation is not performed. Argument must be between 0 and 1 inclusive. Argument must be the same for all rows in the same group. If argument is NULL, the result is NULL. NULL values are ignored in the calculation. If no rows are selected, the result is NULL.

Aggregates are only allowed in select statements.

Example:

PERCENTILE_DISC(0.5) WITHIN GROUP (ORDER BY V)

MEDIAN

```
MEDIAN( [ DISTINCT | ALL ] value )
[FILTER (WHERE expression)] [OVER windowNameOrSpecification]
```

The value separating the higher half of a values from the lower half. Returns the middle value or an interpolated value between two middle values if number of values is even. Interpolation is only supported for numeric, date-time, and interval data types. NULL values are ignored in the calculation. If no rows are selected, the result is NULL. Aggregates are only allowed in select statements.

Example:

MEDIAN(X)

MODE

```
{ MODE() WITHIN GROUP (ORDER BY sortSpecification) }
| { MODE( value [ ORDER BY sortSpecification ] ) }
[FILTER (WHERE expression)] [OVER windowNameOrSpecification]
```

Returns the value that occurs with the greatest frequency. If there are multiple values with the same frequency only one value will be returned. In this situation

value will be chosen based on optional ORDER BY clause that should specify exactly the same expression as argument of this function. Use ascending order to get smallest value or descending order to get largest value from multiple values with the same frequency. If this clause is not specified the exact chosen value is not determined in this situation. NULL values are ignored in the calculation. If no rows are selected, the result is NULL. Aggregates are only allowed in select statements.

Example:

MODE() WITHIN GROUP (ORDER BY X)

JSON Aggregate Functions

JSON_OBJECTAGG

```
JSON_OBJECTAGG(  
  {[KEY] string VALUE value} | {string : value}  
  [ { NULL | ABSENT } ON NULL ]  
  [ { WITH | WITHOUT } UNIQUE KEYS ]  
)  
[FILTER (WHERE expression)] [OVER windowNameOrSpecification]
```

Aggregates the keys with values into a JSON object. If ABSENT ON NULL is specified properties with NULL value are not included in the object. If WITH UNIQUE KEYS is specified the constructed object is checked for uniqueness of keys, nested objects, if any, are checked too. If no values are selected, the result is SQL NULL value.

Example:

JSON_OBJECTAGG(NAME: VAL);

JSON_OBJECTAGG(KEY NAME VALUE VAL);

JSON_ARRAYAGG

```
JSON_ARRAYAGG( [ DISTINCT | ALL ] expression  
  [ ORDER BY sortSpecificationList ]  
  [ { NULL | ABSENT } ON NULL ] )
```


[FILTER (WHERE [expression](#))] [OVER [windowNameOrSpecification](#)]

Aggregates the values into a JSON array. If NULL ON NULL is specified NULL values are included in the array. If no values are selected, the result is SQL NULL value.

Example:

JSON_ARRAYAGG(NUMBER)

Window Functions

Index

Row Number Function

[ROW_NUMBER](#)

Rank Functions

[RANK](#)

[DENSE_RANK](#)

[PERCENT_RANK](#)

[CUME_DIST](#)

Lead or Lag Functions

[LEAD](#)

[LAG](#)

Nth Value Functions

[FIRST_VALUE](#)

[LAST_VALUE](#)

[NTH_VALUE](#)

Other Window Functions

[NTILE](#)

[RATIO_TO_REPORT](#)

Details

Non-standard syntax is marked in green. Compatibility-only non-standard syntax is marked in red, don't use it unless you need it for compatibility with other databases or old versions of H2.

Row Number Function

ROW_NUMBER

`ROW_NUMBER() OVER windowNameOrSpecification`

Returns the number of the current row starting with 1. Window frame clause is not allowed for this function.

Window functions in H2 may require a lot of memory for large queries.

Example:

```
SELECT ROW_NUMBER() OVER (), * FROM TEST;  
SELECT ROW_NUMBER() OVER (ORDER BY ID), * FROM TEST;  
SELECT ROW_NUMBER() OVER (PARTITION BY CATEGORY ORDER BY ID), * FROM  
TEST;
```

Rank Functions

RANK

`RANK() OVER windowNameOrSpecification`

Returns the rank of the current row. The rank of a row is the number of rows that precede this row plus 1. If two or more rows have the same values in ORDER BY columns, these rows get the same rank from the first row with the same values. It means that gaps in ranks are possible. This function requires window order clause. Window frame clause is not allowed for this function.

Window functions in H2 may require a lot of memory for large queries.

See [RANK aggregate](#) for a hypothetical set function with the same name.

Example:

```
SELECT RANK() OVER (ORDER BY ID), * FROM TEST;  
SELECT RANK() OVER (PARTITION BY CATEGORY ORDER BY ID), * FROM TEST;
```

DENSE_RANK

DENSE_RANK() OVER [windowNameOrSpecification](#)

Returns the dense rank of the current row. The rank of a row is the number of groups of rows with the same values in ORDER BY columns that precede group with this row plus 1. If two or more rows have the same values in ORDER BY columns, these rows get the same rank. Gaps in ranks are not possible. This function requires window order clause. Window frame clause is not allowed for this function.

Window functions in H2 may require a lot of memory for large queries.

See [DENSE_RANK aggregate](#) for a hypothetical set function with the same name.

Example:

```
SELECT DENSE_RANK() OVER (ORDER BY ID), * FROM TEST;  
SELECT DENSE_RANK() OVER (PARTITION BY CATEGORY ORDER BY ID), * FROM  
TEST;
```

PERCENT_RANK

PERCENT_RANK() OVER [windowNameOrSpecification](#)

Returns the relative rank of the current row. The relative rank is calculated as $(\text{RANK} - 1) / (\text{NR} - 1)$, where RANK is a rank of the row and NR is a number of rows in window partition with this row. Note that result is always 0 if window order clause is not specified. Window frame clause is not allowed for this function.

Window functions in H2 may require a lot of memory for large queries.

See [PERCENT_RANK aggregate](#) for a hypothetical set function with the same name.

Example:

```
SELECT PERCENT_RANK() OVER (ORDER BY ID), * FROM TEST;  
SELECT PERCENT_RANK() OVER (PARTITION BY CATEGORY ORDER BY ID), *  
FROM TEST;
```

CUME_DIST

CUME_DIST() OVER [windowNameOrSpecification](#)

Returns the relative rank of the current row. The relative rank is calculated as NP / NR where NP is a number of rows that precede the current row or have the same values in ORDER BY columns and NR is a number of rows in window partition with this row. Note that result is always 1 if window order clause is not specified. Window frame clause is not allowed for this function.

Window functions in H2 may require a lot of memory for large queries.

See [CUME_DIST aggregate](#) for a hypothetical set function with the same name.

Example:

```
SELECT CUME_DIST() OVER (ORDER BY ID), * FROM TEST;  
SELECT CUME_DIST() OVER (PARTITION BY CATEGORY ORDER BY ID), * FROM  
TEST;
```

Lead or Lag Functions

LEAD

LEAD([value](#) [, [offsetInt](#) [, [defaultValue](#)]]) [{RESPECT|IGNORE} NULLS]
OVER [windowNameOrSpecification](#)

Returns the value in a next row with specified offset relative to the current row. Offset must be non-negative. If IGNORE NULLS is specified rows with null values in selected expression are skipped. If number of considered rows is less than specified relative number this function returns NULL or the specified default value, if any. If offset is 0 the value from the current row is returned unconditionally. This function requires window order clause. Window frame clause is not allowed for this function.

Window functions in H2 may require a lot of memory for large queries.

Example:

```
SELECT LEAD(X) OVER (ORDER BY ID), * FROM TEST;  
SELECT LEAD(X, 2, 0) IGNORE NULLS OVER (  
    PARTITION BY CATEGORY ORDER BY ID  
) , * FROM TEST;
```

LAG

```
LAG(value [, offsetInt [, defaultValue]]) [{RESPECT | IGNORE} NULLS]  
OVER windowNameOrSpecification
```

Returns the value in a previous row with specified offset relative to the current row. Offset must be non-negative. If IGNORE NULLS is specified rows with null values in selected expression are skipped. If number of considered rows is less than specified relative number this function returns NULL or the specified default value, if any. If offset is 0 the value from the current row is returned unconditionally. This function requires window order clause. Window frame clause is not allowed for this function.

Window functions in H2 may require a lot of memory for large queries.

Example:

```
SELECT LAG(X) OVER (ORDER BY ID), * FROM TEST;  
SELECT LAG(X, 2, 0) IGNORE NULLS OVER (  
    PARTITION BY CATEGORY ORDER BY ID  
) , * FROM TEST;
```

Nth Value Functions

FIRST_VALUE

```
FIRST_VALUE(value) [{RESPECT | IGNORE} NULLS]  
OVER windowNameOrSpecification
```

Returns the first value in a window. If IGNORE NULLS is specified null values are skipped and the function returns first non-null value, if any.

Window functions in H2 may require a lot of memory for large queries.

Example:

```
SELECT FIRST_VALUE(X) OVER (ORDER BY ID), * FROM TEST;  
SELECT FIRST_VALUE(X) IGNORE NULLS OVER (PARTITION BY CATEGORY ORDER  
BY ID), * FROM TEST;
```

LAST_VALUE

`LAST_VALUE(value) [{RESPECT|IGNORE} NULLS]
OVER windowNameOrSpecification`

Returns the last value in a window. If IGNORE NULLS is specified null values are skipped and the function returns last non-null value before them, if any; if there is no non-null value it returns NULL. Note that the last value is actually a value in the current group of rows if window order clause is specified and window frame clause is not specified.

Window functions in H2 may require a lot of memory for large queries.

Example:

```
SELECT LAST_VALUE(X) OVER (ORDER BY ID), * FROM TEST;  
SELECT LAST_VALUE(X) IGNORE NULLS OVER (  
    PARTITION BY CATEGORY ORDER BY ID  
    RANGE BETWEEN CURRENT ROW AND UNBOUNDED FOLLOWING  
) , * FROM TEST;
```

NTH_VALUE

`NTH_VALUE(value, nInt) [FROM {FIRST|LAST}] [{RESPECT|IGNORE} NULLS]
OVER windowNameOrSpecification`

Returns the value in a row with a specified relative number in a window.

Relative row number must be positive. If FROM LAST is specified rows are counted backwards from the last row. If IGNORE NULLS is specified rows with null values

in selected expression are skipped. If number of considered rows is less than specified relative number this function returns NULL. Note that the last row is actually a last row in the current group of rows if window order clause is specified and window frame clause is not specified.

Window functions in H2 may require a lot of memory for large queries.

Example:

```
SELECT NTH_VALUE(X) OVER (ORDER BY ID), * FROM TEST;  
SELECT NTH_VALUE(X) IGNORE NULLS OVER (  
    PARTITION BY CATEGORY ORDER BY ID  
    RANGE BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING  
) , * FROM TEST;
```

Other Window Functions

NTILE

NTILE(long) OVER windowNameOrSpecification

Distributes the rows into a specified number of groups. Number of groups should be a positive long value. NTILE returns the 1-based number of the group to which the current row belongs. First groups will have more rows if number of rows is not divisible by number of groups. For example, if 5 rows are distributed into 2 groups this function returns 1 for the first 3 row and 2 for the last 2 rows. This function requires window order clause. Window frame clause is not allowed for this function.

Window functions in H2 may require a lot of memory for large queries.

Example:

```
SELECT NTILE(10) OVER (ORDER BY ID), * FROM TEST;  
SELECT NTILE(5) OVER (PARTITION BY CATEGORY ORDER BY ID), * FROM TEST;
```

RATIO_TO_REPORT

RATIO_TO_REPORT(value)

OVER `windowNameOrSpecification`

Returns the ratio of a value to the sum of all values. If argument is NULL or sum of all values is 0, then the value of function is NULL. Window ordering and window frame clauses are not allowed for this function.

Window functions in H2 may require a lot of memory for large queries.

Example:

```
SELECT X, RATIO_TO_REPORT(X) OVER (PARTITION BY CATEGORY), CATEGORY  
FROM TEST;
```

Data Types

Index

CHARACTER

CHARACTER VARYING

CHARACTER LARGE OBJECT

VARCHAR_IGNORECASE

BINARY

BINARY VARYING

BINARY LARGE OBJECT

BOOLEAN

TINYINT

SMALLINT

INTEGER

BIGINT

NUMERIC

REAL

DOUBLE PRECISION

DECFLOAT

DATE

TIME

TIME WITH TIME ZONE

TIMESTAMP

TIMESTAMP WITH TIME ZONE

INTERVAL

JAVA_OBJECT

ENUM

GEOMETRY

JSON

UUID

ARRAY

ROW

Details

Non-standard syntax is marked in green. Compatibility-only non-standard syntax is marked in red, don't use it unless you need it for compatibility with other databases or old versions of H2.

CHARACTER

```
{ CHARACTER | CHAR | NATIONAL { CHARACTER | CHAR } | NCHAR }  
[ ( lengthInt [CHARACTERS|OCTETS] ) ]
```

A Unicode String of fixed length.

Length, if any, should be specified in characters, CHARACTERS and OCTETS units have no effect in H2. The allowed length is from 1 to 1,000,000,000 characters. If length is not specified, 1 character is used by default.

The whole text is kept in memory when using this data type. For variable-length strings use [CHARACTER VARYING](#) data type instead. For large text data [CHARACTER LARGE OBJECT](#) should be used; see there for details.

Too short strings are right-padded with space characters. Too long strings are truncated by CAST specification and rejected by column assignment.

Two CHARACTER strings of different length are considered as equal if all additional characters in the longer string are space characters.

See also [string](#) literal grammar. Mapped to java.lang.String.

Example:

CHARACTER

CHAR(10)

CHARACTER VARYING

```
{ { CHARACTER | CHAR } VARYING  
| VARCHAR  
| { NATIONAL { CHARACTER | CHAR } | NCHAR } VARYING  
| { VARCHAR\_CASESENSITIVE } }
```

```
[ ( lengthInt [CHARACTERS | OCTETS] ) ]
```

A Unicode String. Use two single quotes (") to create a quote.

The allowed length is from 1 to 1,000,000,000 characters. The length is a size constraint; only the actual data is persisted. Length, if any, should be specified in characters, CHARACTERS and OCTETS units have no effect in H2.

The whole text is loaded into memory when using this data type. For large text data [CHARACTER LARGE OBJECT](#) should be used; see there for details.

See also [string](#) literal grammar. Mapped to java.lang.String.

Example:

CHARACTER VARYING(100)

VARCHAR(255)

CHARACTER LARGE OBJECT

```
{ { CHARACTER | CHAR } LARGE OBJECT | CLOB  
| { NATIONAL CHARACTER | NCHAR } LARGE OBJECT | NCLOB }  
[ ( lengthLong [K|M|G|T|P] [CHARACTERS | OCTETS]) ]
```

CHARACTER LARGE OBJECT is intended for very large Unicode character string values. Unlike when using [CHARACTER VARYING](#), large CHARACTER LARGE OBJECT values are not kept fully in-memory; instead, they are streamed. CHARACTER LARGE OBJECT should be used for documents and texts with arbitrary size such as XML or HTML documents, text files, or memo fields of unlimited size. Use PreparedStatement.setCharacterStream to store values. See also [Large Objects](#) section.

CHARACTER VARYING should be used for text with relatively short average size (for example shorter than 200 characters). Short CHARACTER LARGE OBJECT values are stored inline, but there is an overhead compared to CHARACTER VARYING.

Length, if any, should be specified in characters, CHARACTERS and OCTETS units have no effect in H2.

Mapped to java.sql.Clob (java.io.Reader is also supported).

Example:

CHARACTER LARGE OBJECT
CLOB(10K)

VARCHAR_IGNORECASE

```
VARCHAR_IGNORECASE  
[ ( lengthInt [CHARACTERS|OCTETS] ) ]
```

Same as VARCHAR, but not case sensitive when comparing. Stored in mixed case.

The allowed length is from 1 to 1,000,000,000 characters. The length is a size constraint; only the actual data is persisted. Length, if any, should be specified in characters, CHARACTERS and OCTETS units have no effect in H2.

The whole text is loaded into memory when using this data type. For large text data CLOB should be used; see there for details.

See also [string](#) literal grammar. Mapped to java.lang.String.

Example:

VARCHAR_IGNORECASE

BINARY

```
BINARY [ ( lengthInt ) ]
```

Represents a binary string (byte array) of fixed predefined length.

The allowed length is from 1 to 1,000,000,000 bytes. If length is not specified, 1 byte is used by default.

The whole binary string is kept in memory when using this data type. For variable-length binary strings use [BINARY VARYING](#) data type instead. For large binary data [BINARY LARGE OBJECT](#) should be used; see there for details.

Too short binary string are right-padded with zero bytes. Too long binary strings are truncated by CAST specification and rejected by column assignment. Binary strings of different length are considered as not equal to each other. See also [bytes](#) literal grammar. Mapped to `byte[]`.

Example:

`BINARY`

`BINARY(1000)`

BINARY VARYING

```
{ BINARY VARYING | VARBINARY }  
[ ( lengthInt ) ]
```

Represents a byte array.

The allowed length is from 1 to 1,000,000,000 bytes. The length is a size constraint; only the actual data is persisted.

The whole binary string is kept in memory when using this data type. For large binary data [BINARY LARGE OBJECT](#) should be used; see there for details.

See also [bytes](#) literal grammar. Mapped to `byte[]`.

Example:

`BINARY VARYING(100)`

`VARBINARY(1000)`

BINARY LARGE OBJECT

```
{ BINARY LARGE OBJECT | BLOB }  
[ ( lengthLong [K|M|G|T|P]) ]
```

BINARY LARGE OBJECT is intended for very large binary values such as files or images. Unlike when using [BINARY VARYING](#), large objects are not kept fully in-memory; instead, they are streamed. Use `PreparedStatement.setBinaryStream` to store values. See also [CHARACTER LARGE OBJECT](#) and [Large Objects](#) section.

Mapped to java.sql.Blob (java.io.InputStream is also supported).

Example:

BINARY LARGE OBJECT

BLOB(10K)

BOOLEAN

BOOLEAN

Possible values: TRUE, FALSE, and UNKNOWN (NULL).

See also [boolean](#) literal grammar. Mapped to java.lang.Boolean.

Example:

BOOLEAN

TINYINT

TINYINT

Possible values are: -128 to 127.

See also [integer](#) literal grammar.

In JDBC this data type is mapped to java.lang.Integer. java.lang.Byte is also supported.

In org.h2.api.Aggregate, org.h2.api.AggregateFunction, and org.h2.api.Trigger this data type is mapped to java.lang.Byte.

Example:

TINYINT

SMALLINT

SMALLINT

Possible values: -32768 to 32767.

See also [integer](#) literal grammar.

In JDBC this data type is mapped to `java.lang.Integer`. `java.lang.Short` is also supported.

In `org.h2.api.Aggregate`, `org.h2.api.AggregateFunction`, and `org.h2.api.Trigger` this data type is mapped to `java.lang.Short`.

Example:

SMALLINT

INTEGER

INTEGER | INT

Possible values: -2147483648 to 2147483647.

See also [integer](#) literal grammar. Mapped to `java.lang.Integer`.

Example:

INTEGER

INT

BIGINT

BIGINT

Possible values: -9223372036854775808 to 9223372036854775807.

See also [long](#) literal grammar. Mapped to `java.lang.Long`.

Example:

BIGINT

NUMERIC

{ NUMERIC | DECIMAL | DEC } [([precisionInt](#) [, [scaleInt](#)])]

Data type with fixed decimal precision and scale. This data type is recommended for storing currency values.

If precision is specified, it must be from 1 to 100000. If scale is specified, it must be from 0 to 100000, 0 is default.

See also [numeric](#) literal grammar. Mapped to java.math.BigDecimal.

Example:

NUMERIC(20, 2)

REAL

REAL | FLOAT ([precisionInt](#))

A single precision floating point number. Should not be used to represent currency values, because of rounding problems. Precision value for FLOAT type name should be from 1 to 24.

See also [numeric](#) literal grammar. Mapped to java.lang.Float.

Example:

REAL

DOUBLE PRECISION

DOUBLE PRECISION | FLOAT [([precisionInt](#))]

A double precision floating point number. Should not be used to represent currency values, because of rounding problems. If precision value is specified for FLOAT type name, it should be from 25 to 53.

See also [numeric](#) literal grammar. Mapped to java.lang.Double.

Example:

DOUBLE PRECISION

DECFLOAT

DECFLOAT [([precisionInt](#))]

Decimal floating point number. This data type is not recommended to represent currency values, because of variable scale.

If precision is specified, it must be from 1 to 100000.

See also [numeric](#) literal grammar. Mapped to `java.math.BigDecimal`. There are three special values: 'Infinity', '-Infinity', and 'NaN'. These special values can't be read or set as `BigDecimal` values, but they can be read or set using `java.lang.String`, `float`, or `double`.

Example:

DECFLOAT

DECFLOAT(20)

DATE

DATE

The date data type. The proleptic Gregorian calendar is used.

See also [date](#) literal grammar.

In JDBC this data type is mapped to `java.sql.Date`, with the time set to 00:00:00 (or to the next possible time if midnight doesn't exist for the given date and time zone due to a daylight saving change). `java.time.LocalDate` is also supported and recommended.

In `org.h2.api.Aggregate`, `org.h2.api.AggregateFunction`, and `org.h2.api.Trigger` this data type is mapped to `java.time.LocalDate`.

If your time zone had LMT (local mean time) in the past and you use such old dates (depends on the time zone, usually 100 or more years ago), don't use `java.sql.Date` to read and write them.

If you deal with very old dates (before 1582-10-15) note that `java.sql.Date` uses a mixed Julian/Gregorian calendar, `java.util.GregorianCalendar` can be configured to proleptic Gregorian with `setGregorianChange(new java.util.Date(Long.MIN_VALUE))` and used to read or write fields of dates.

Example:

DATE

TIME

```
TIME [ ( precisionInt ) ] [ WITHOUT TIME ZONE ]
```

The time data type. The format is hh:mm:ss[.nnnnnnnnnn]. If fractional seconds precision is specified it should be from 0 to 9, 0 is default.

See also [time](#) literal grammar.

In JDBC this data type is mapped to java.sql.Time. java.time.LocalTime is also supported and recommended.

In org.h2.api.Aggregate, org.h2.api.AggregateFunction, and org.h2.api.Trigger this data type is mapped to java.time.LocalTime.

Use java.time.LocalTime or String instead of java.sql.Time when non-zero precision is needed. Cast from higher fractional seconds precision to lower fractional seconds precision performs round half up; if result of rounding is higher than maximum supported value 23:59:59.999999999 the value is rounded down instead. The CAST operation to TIMESTAMP and TIMESTAMP WITH TIME ZONE data types uses the [CURRENT_DATE](#) for date fields.

Example:

TIME

TIME(9)

TIME WITH TIME ZONE

```
TIME [ ( precisionInt ) ] WITH TIME ZONE
```

The time with time zone data type. If fractional seconds precision is specified it should be from 0 to 9, 0 is default.

See also [time with time zone](#) literal grammar. Mapped to java.time.OffsetTime. Cast from higher fractional seconds precision to lower fractional seconds

precision performs round half up; if result of rounding is higher than maximum supported value 23:59:59.999999999 the value is rounded down instead. The CAST operation to TIMESTAMP and TIMESTAMP WITH TIME ZONE data types uses the [CURRENT_DATE](#) for date fields.

Example:

TIME WITH TIME ZONE

TIME(9) WITH TIME ZONE

TIMESTAMP

```
TIMESTAMP [ ( precisionInt ) ] [ WITHOUT TIME ZONE ]
```

The timestamp data type. The proleptic Gregorian calendar is used. If fractional seconds precision is specified it should be from 0 to 9, 6 is default.

This data type holds the local date and time without time zone information. It cannot distinguish timestamps near transitions from DST to normal time. For absolute timestamps use the [TIMESTAMP WITH TIME ZONE](#) data type instead.

See also [timestamp](#) literal grammar.

In JDBC this data type is mapped to `java.sql.Timestamp` (`java.util.Date` may be used too). `java.time.LocalDateTime` is also supported and recommended.

In `org.h2.api.Aggregate`, `org.h2.api.AggregateFunction`, and `org.h2.api.Trigger` this data type is mapped to `java.time.LocalDateTime`.

If your time zone had LMT (local mean time) in the past and you use such old dates (depends on the time zone, usually 100 or more years ago), don't use `java.sql.Timestamp` and `java.util.Date` to read and write them.

If you deal with very old dates (before 1582-10-15) note that `java.sql.Timestamp` and `java.util.Date` use a mixed Julian/Gregorian calendar, `java.util.GregorianCalendar` can be configured to proleptic Gregorian with `setGregorianChange(new java.util.Date(Long.MIN_VALUE))` and used to read or write fields of timestamps.

Cast from higher fractional seconds precision to lower fractional seconds precision performs round half up.

Example:

TIMESTAMP

TIMESTAMP(9)

TIMESTAMP WITH TIME ZONE

```
TIMESTAMP [ ( precisionInt ) ] WITH TIME ZONE
```

The timestamp with time zone data type. The proleptic Gregorian calendar is used. If fractional seconds precision is specified it should be from 0 to 9, 6 is default.

See also [timestamp with time zone](#) literal grammar. Mapped to `java.time.OffsetDateTime`, `java.time.ZonedDateTime` and `java.time.Instant` are also supported.

Values of this data type are compared by UTC values. It means that 2010-01-01 10:00:00+01 is greater than 2010-01-01 11:00:00+03.

Conversion to TIMESTAMP uses time zone offset to get UTC time and converts it to local time using the system time zone. Conversion from TIMESTAMP does the same operations in reverse and sets time zone offset to offset of the system time zone. Cast from higher fractional seconds precision to lower fractional seconds precision performs round half up.

Example:

TIMESTAMP WITH TIME ZONE

TIMESTAMP(9) WITH TIME ZONE

INTERVAL

```
intervalYearType | intervalMonthType | intervalDayType  
| intervalHourType | intervalMinuteType | intervalSecondType  
| intervalYearToMonthType | intervalDayToHourType
```

```
| intervalDayToMinuteType | intervalDayToSecondType  
| intervalHourToMinuteType | intervalHourToSecondType  
| intervalMinuteToSecondType
```

Interval data type. There are two classes of intervals. Year-month intervals can store years and months. Day-time intervals can store days, hours, minutes, and seconds. Year-month intervals are comparable only with another year-month intervals. Day-time intervals are comparable only with another day-time intervals.

Mapped to `org.h2.api.Interval`.

Example:

INTERVAL DAY TO SECOND

JAVA_OBJECT

```
{ JAVA_OBJECT | OBJECT | OTHER } [ ( lengthInt ) ]
```

This type allows storing serialized Java objects. Internally, a byte array with serialized form is used. The allowed length is from 1 (useful only with custom serializer) to 1,000,000,000 bytes. The length is a size constraint; only the actual data is persisted.

Serialization and deserialization is done on the client side only with two exclusions described below. Deserialization is only done when `getObject` is called. Java operations cannot be executed inside the database engine for security reasons. Use `PreparedStatement.setObject` with `Types.JAVA_OBJECT` or `H2Type.JAVA_OBJECT` as a third argument to store values.

If Java method alias has Object parameter(s), values are deserialized during invocation of this method on the server side.

If a [linked table](#) has a column with `Types.JAVA_OBJECT` JDBC data type and its database is not an another H2, Java objects need to be serialized and deserialized during interaction between H2 and database that owns the table on the server side of H2.

This data type needs special attention in secure environments.

Mapped to java.lang.Object (or any subclass).

Example:

```
JAVA_OBJECT
```

```
JAVA_OBJECT(10000)
```

ENUM

```
ENUM (string [, ... ])
```

A type with enumerated values. Mapped to java.lang.String.

Duplicate and empty values are not permitted. The maximum number of values is 65536. The maximum allowed length of complete data type definition with all values is 1,000,000,000 characters.

Example:

```
ENUM('clubs', 'diamonds', 'hearts', 'spades')
```

GEOMETRY

```
GEOMETRY  
[( { GEOMETRY |  
  { POINT  
    | LINESTRING  
    | POLYGON  
    | MULTIPOINT  
    | MULTILINESTRING  
    | MULTIPOLYGON  
    | GEOMETRYCOLLECTION } [Z | M | ZM]}  
[, sridInt] )]
```

A spatial geometry type. If additional constraints are not specified this type accepts all supported types of geometries. A constraint with required geometry type and dimension system can be set by specifying name of the type and dimension system. A whitespace between them is optional. 2D dimension

system does not have a name and assumed if only a geometry type name is specified. POINT means 2D point, POINT Z or POINTZ means 3D point. GEOMETRY constraint means no restrictions on type or dimension system of geometry. A constraint with required spatial reference system identifier (SRID) can be set by specifying this identifier.

Mapped to `org.locationtech.jts.geom.Geometry` if JTS library is in classpath and to `java.lang.String` otherwise. May be represented in textual format using the WKT (well-known text) or EWKT (extended well-known text) format. Values are stored internally in EWKB (extended well-known binary) format, the maximum allowed length is 1,000,000,000 bytes. Only a subset of EWKB and EWKT features is supported. Supported objects are POINT, LINESTRING, POLYGON, MULTIPOINT, MULTILINESTRING, MULTIPOLYGON, and GEOMETRYCOLLECTION. Supported dimension systems are 2D (XY), Z (XYZ), M (XYM), and ZM (XYZM). SRID (spatial reference system identifier) is supported.

Use a quoted string containing a WKT/EWKT formatted string or `PreparedStatement.setObject()` to store values, and `ResultSet.getObject(..)` or `ResultSet.getString(..)` to retrieve the values.

Example:

GEOMETRY

GEOMETRY(POINT)

GEOMETRY(POINT Z)

GEOMETRY(POINT Z, 4326)

GEOMETRY(GEOMETRY, 4326)

JSON

JSON [(lengthInt)]

A RFC 8259-compliant JSON text.

See also [json](#) literal grammar. Mapped to `byte[]`. The allowed length is from 1 to 1,000,000,000 bytes. The length is a size constraint; only the actual data is persisted.

To set a JSON value with `java.lang.String` in a `PreparedStatement` use a `FORMAT JSON` data format (`INSERT INTO TEST(ID, DATA) VALUES (?, ? FORMAT JSON)`) or use `setObject(parameter, jsonText, H2Type.JSON)` instead of `setString()`.

Without the data format `VARCHAR` values are converted to JSON string values.

SQL/JSON null value JSON 'null' is distinct from the SQL null value `NULL`.

Order of object members is preserved as is. Duplicate object member names are allowed.

Example:

JSON

UUID

UUID

RFC 9562-compliant universally unique identifier. This is a 128 bit value. To store values, use `PreparedStatement.setBytes`, `setString`, or `setObject(uuid)` (where `uuid` is a `java.util.UUID`). `ResultSet.getObject` will return a `java.util.UUID`.

Please note that using an index on randomly generated data will result on poor performance once there are millions of rows in a table. The reason is that the cache behavior is very bad with randomly distributed data. This is a problem for any database system. To avoid this problem use UUID version 7 values.

For details, see the documentation of `java.util.UUID`.

Example:

UUID

ARRAY

`baseDataType` ARRAY ['[' `maximumCardinalityInt` ']']

A data type for array of values. Base data type specifies the data type of elements. Array may have `NULL` elements. Maximum cardinality, if any,

specifies maximum allowed number of elements in the array. The allowed cardinality is from 0 to 65536 elements.

See also [array](#) literal grammar. Mapped to `java.lang.Object[]` (arrays of any non-primitive type are also supported).

Use `PreparedStatement.setArray(..)` or `PreparedStatement.setObject(.., new Object[] {..})` to store values, and `ResultSet.getObject(..)` or `ResultSet.getArray(..)` to retrieve the values.

Example:

BOOLEAN ARRAY

VARCHAR(100) ARRAY

INTEGER ARRAY[10]

ROW

```
ROW (fieldName dataType [,...])
```

A row value data type. This data type should not be normally used as data type of a column.

See also [row value expression](#) grammar. Mapped to `java.sql.ResultSet`.

Example:

ROW(A INT, B VARCHAR(10))

Interval Data Types

INTERVAL YEAR

```
INTERVAL YEAR [ ( precisionInt ) ]
```

Interval data type. If precision is specified it should be from 1 to 18, 2 is default.

See also [year interval](#) literal grammar. Mapped to `org.h2.api.Interval`.
`java.time.Period` is also supported.

Example:

INTERVAL YEAR

INTERVAL MONTH

```
INTERVAL MONTH [ ( precisionInt ) ]
```

Interval data type. If precision is specified it should be from 1 to 18, 2 is default.

See also [month interval](#) literal grammar. Mapped to `org.h2.api.Interval`.
`java.time.Period` is also supported.

Example:

INTERVAL MONTH

INTERVAL DAY

```
INTERVAL DAY [ ( precisionInt ) ]
```

Interval data type. If precision is specified it should be from 1 to 18, 2 is default.

See also [day interval](#) literal grammar. Mapped to `org.h2.api.Interval`.
`java.time.Duration` is also supported.

Example:

INTERVAL DAY

INTERVAL HOUR

```
INTERVAL HOUR [ ( precisionInt ) ]
```

Interval data type. If precision is specified it should be from 1 to 18, 2 is default.

See also [hour interval](#) literal grammar. Mapped to `org.h2.api.Interval`.
`java.time.Duration` is also supported.

Example:

INTERVAL HOUR

INTERVAL MINUTE

```
INTERVAL MINUTE [ ( precisionInt ) ]
```

Interval data type. If precision is specified it should be from 1 to 18, 2 is default.

See also [minute interval](#) literal grammar. Mapped to `org.h2.api.Interval`.
`java.time.Duration` is also supported.

Example:

INTERVAL MINUTE

INTERVAL SECOND

```
INTERVAL SECOND [ ( precisionInt [, fractionalPrecisionInt ] ) ]
```

Interval data type. If precision is specified it should be from 1 to 18, 2 is default.
If fractional seconds precision is specified it should be from 0 to 9, 6 is default.

See also [second interval](#) literal grammar. Mapped to `org.h2.api.Interval`.
`java.time.Duration` is also supported.

Example:

INTERVAL SECOND

INTERVAL YEAR TO MONTH

```
INTERVAL YEAR [ ( precisionInt ) ] TO MONTH
```

Interval data type. If leading field precision is specified it should be from 1 to 18, 2 is default.

See also [year to month interval](#) literal grammar. Mapped to `org.h2.api.Interval`.
`java.time.Period` is also supported.

Example:

INTERVAL YEAR TO MONTH

INTERVAL DAY TO HOUR

```
INTERVAL DAY [ ( precisionInt ) ] TO HOUR
```

Interval data type. If leading field precision is specified it should be from 1 to 18, 2 is default.

See also [day to hour interval](#) literal grammar. Mapped to `org.h2.api.Interval`.
`java.time.Duration` is also supported.

Example:

INTERVAL DAY TO HOUR

INTERVAL DAY TO MINUTE

```
INTERVAL DAY [ ( precisionInt ) ] TO MINUTE
```

Interval data type. If leading field precision is specified it should be from 1 to 18, 2 is default.

See also [day to minute interval](#) literal grammar. Mapped to `org.h2.api.Interval`.
`java.time.Duration` is also supported.

Example:

INTERVAL DAY TO MINUTE

INTERVAL DAY TO SECOND

```
INTERVAL DAY [ ( precisionInt ) ] TO SECOND [ ( fractionalPrecisionInt ) ]
```

Interval data type. If leading field precision is specified it should be from 1 to 18, 2 is default. If fractional seconds precision is specified it should be from 0 to 9, 6 is default.

See also [day to second interval](#) literal grammar. Mapped to `org.h2.api.Interval`.
`java.time.Duration` is also supported.

Example:

INTERVAL DAY TO SECOND

INTERVAL HOUR TO MINUTE

```
INTERVAL HOUR [ ( precisionInt ) ] TO MINUTE
```

Interval data type. If leading field precision is specified it should be from 1 to 18, 2 is default.

See also [hour to minute interval](#) literal grammar. Mapped to `org.h2.api.Interval`. `java.time.Duration` is also supported.

Example:

INTERVAL HOUR TO MINUTE

INTERVAL HOUR TO SECOND

```
INTERVAL HOUR [ ( precisionInt ) ] TO SECOND [ ( fractionalPrecisionInt ) ]
```

Interval data type. If leading field precision is specified it should be from 1 to 18, 2 is default. If fractional seconds precision is specified it should be from 0 to 9, 6 is default.

See also [hour to second interval](#) literal grammar. Mapped to `org.h2.api.Interval`. `java.time.Duration` is also supported.

Example:

INTERVAL HOUR TO SECOND

INTERVAL MINUTE TO SECOND

```
INTERVAL MINUTE [ ( precisionInt ) ] TO SECOND [ ( fractionalPrecisionInt ) ]
```

Interval data type. If leading field precision is specified it should be from 1 to 18, 2 is default. If fractional seconds precision is specified it should be from 0 to 9, 6 is default.

See also [minute to second interval](#) literal grammar. Mapped to `org.h2.api.Interval`. `java.time.Duration` is also supported.

Example:

INTERVAL MINUTE TO SECOND

SQL Grammar

Index

Literals

Value

Approximate numeric

Array

Boolean

Bytes

Date

Date and time

Dollar Quoted String

Exact numeric

Hex Number

Octal Number

Binary Number

Int

GEOMETRY

JSON

Long

Null

Number

Numeric

String

UUID

Time

Time with time zone

Timestamp

Timestamp with time zone

Interval

INTERVAL YEAR

INTERVAL MONTH
INTERVAL DAY
INTERVAL HOUR
INTERVAL MINUTE
INTERVAL SECOND
INTERVAL YEAR TO MONTH
INTERVAL DAY TO HOUR
INTERVAL DAY TO MINUTE
INTERVAL DAY TO SECOND
INTERVAL HOUR TO MINUTE
INTERVAL HOUR TO SECOND
INTERVAL MINUTE TO SECOND

Datetime fields

Datetime field
Year field
Month field
Day of month field
Hour field
Minute field
Second field
Timezone hour field
Timezone minute field
Timezone second field
Millennium field
Century field
Decade field
Quarter field
Millisecond field
Microsecond field
Nanosecond field
Day of year field
ISO day of week field

ISO week field

ISO week year field

Day of week field

Week field

Week year field

Epoch field

Other Grammar

Alias

And Condition

Array element reference

Field reference

Array value constructor by query

Case expression

Simple case

Searched case

Cast specification

Cipher

Column Definition

Column Constraint Definition

Comment

Bracketed comment

Compare

Condition

Condition Right Hand Side

Comparison Right Hand Side

Quantified Comparison Right Hand Side

Null Predicate Right Hand Side

Distinct Predicate Right Hand Side

Quantified Distinct Predicate Right Hand Side

Boolean Test Right Hand Side

Type Predicate Right Hand Side

JSON Predicate Right Hand Side

Between Predicate Right Hand Side
In Predicate Right Hand Side
Like Predicate Right Hand Side
Regex Predicate Right Hand Side
Nulls Distinct
Table Constraint Definition
Constraint Name Definition
Csv Options
Data Change Delta Table
Data Type or Domain
Data Type
Predefined Type
Digit
Expression
Factor
Grouping element
Hex
Index Column
Insert values
Interval qualifier
Join specification
Merge when clause
Merge when matched clause
Merge when not matched clause
Name
Operand
Override clause
Query
Quoted Name
Referential Constraint
References Specification
Referential Triggered Action

Referential Action
Script Compression Encryption
Select order
Row value expression
Select Expression
Sequence value expression
Sequence option
Alter sequence option
Alter identity column option
Basic sequence option
Set clause list
Sort specification
Sort specification list
Summand
Table Expression
Update target
Within group specification
Wildcard expression
Window name or specification
Window specification
Window frame
Window frame preceding
Window frame bound
Term
Time zone
Column

Details

Non-standard syntax is marked in green. Compatibility-only non-standard syntax is marked in red, don't use it unless you need it for compatibility with other databases or old versions of H2.

Literals

Value

```
string | { dollarQuotedString } | numeric | dateAndTime | boolean | bytes  
| interval | array | { geometry | json | uuid } | null
```

A literal value of any data type, or null.

Example:

10

Approximate numeric

```
[ + | - ] { { number [ . [ number ] ] } | { . number } }  
E [ + | - ] expNumber
```

An approximate numeric value. Approximate numeric values have **DECFLOAT** data type. To define a **DOUBLE PRECISION** value, use CAST(X AS DOUBLE PRECISION). To define a **REAL** value, use CAST(X AS REAL). There are some special REAL, DOUBLE PRECISION, and DECFLOAT values: to represent positive infinity, use CAST('Infinity' AS dataType); for negative infinity, use CAST('-Infinity' AS dataType); for NaN (not a number), use CAST('NaN' AS dataType).

Example:

-1.4e-10

1.111_111E3

CAST(1e2 AS REAL)

CAST('NaN' AS DOUBLE PRECISION)

Array

```
ARRAY '[' [ expression [,...] ] '']'
```

An array of values.

Example:

ARRAY[1, 2]

ARRAY[1]

ARRAY[]

Boolean

TRUE | FALSE | UNKNOWN

A boolean value. UNKNOWN is a NULL value with the boolean data type.

Example:

TRUE

Bytes

X'hex' ['hex' [...]]

A binary string value. The hex value is not case sensitive and may contain space characters as separators. If there are more than one group of quoted hex values, groups must be separated with whitespace.

Example:

X''

X'01FF'

X'01 bc 2a'

X'01' '02'

Date

DATE '[-]yyyy-MM-dd'

A date literal.

Example:

DATE '2004-12-31'

Date and time

date | time | timeWithTimeZone | timestamp | timestampWithTimeZone

A literal value of any date-time data type.

Example:

TIMESTAMP '1999-01-31 10:00:00'

Dollar Quoted String

```
$$anything$$
```

A string starts and ends with two dollar signs. Two dollar signs are not allowed within the text. A whitespace is required before the first set of dollar signs. No escaping is required within the text.

Example:

\$\$John's car\$\$

Exact numeric

```
[ + | - ] { { number [ . number ] } | { . number } }
```

An exact numeric value. Exact numeric values with dot have **NUMERIC** data type, values without dot small enough to fit into **INTEGER** data type have this type, larger values small enough to fit into **BIGINT** data type have this type, others also have **NUMERIC** data type.

Example:

-1600.05

134_518.235_114

Hex Number

```
[ + | - ] { 0x | 0X } { [ _ ] { digit | a-f | A-F } [...] } [...]
```

A number written in hexadecimal notation.

Example:

0xff

0x_ABCD_1234

Octal Number

```
[+|-]{0o|0O}{ { 0-7 } [...] } [...]
```

A number written in octal notation.

Example:

0o664

0o_123_777

Binary Number

```
[+|-]{0b|0B}{ { 0-1 } [...] } [...]
```

A number written in binary notation.

Example:

0b101

0b_01010101_10101010

Int

```
[ + | - ] number
```

The maximum integer number is 2147483647, the minimum is -2147483648.

Example:

10

65_536

GEOMETRY

```
GEOMETRY { bytes | string }
```

A binary string or character string with GEOMETRY object.

A binary string should contain Well-known Binary Representation (WKB) from OGC 06-103r4. Dimension system marks may be specified either in both OGC WKB or in PostGIS EWKB formats. Optional SRID from EWKB may be specified.

POINT EMPTY stored with NaN values as specified in OGC 12-128r15 is supported.

A character string should contain Well-known Text Representation (WKT) from OGC 06-103r4 with optional SRID from PostGIS EWKT extension.

Example:

```
GEOMETRY 'GEOMETRYCOLLECTION (POINT (1 2))'
```

```
GEOMETRY X'00000000013ff00000000000003ff0000000000000'
```

JSON

```
JSON { bytes | string }
```

A binary or character string with a RFC 8259-compliant JSON text and data format. JSON text is parsed into internal representation. Order of object members is preserved as is. Duplicate object member names are allowed.

Example:

```
JSON '{"id":10,"name":"What"s this?"}'
```

```
JSON '[1, '2]';
```

```
JSON X'7472' '7565'
```

Long

```
[ + | - ] number
```

Long numbers are between -9223372036854775808 and 9223372036854775807.

Example:

```
100000
```

```
1_000_000_000
```

Null

```
NULL
```


NULL is a value without data type and means 'unknown value'.

Example:

NULL

Number

```
digit [ [ ] digit [...] ] [...]
```

The maximum length of the number depends on the data type used.

Example:

100

10_000

Numeric

```
exactNumeric | approximateNumeric | int | long | hexNumber | octalNumber  
| binaryNumber
```

The data type of a numeric literal is the one of numeric data types, such as NUMERIC, DECFLOAT, BIGINT, or INTEGER depending on format and value.

An explicit CAST can be used to change the data type.

Example:

-1600.05

CAST(0 AS DOUBLE PRECISION)

-1.4e-10

999_999_999.999_999

String

```
[N]'anything' [...]  
| U&{'anything' [...]} [ UESCAPE 'anything' ]
```

A character string literal starts and ends with a single quote. Two single quotes can be used to create a single quote inside a string. Prefix N means a national

character string literal; H2 does not distinguish regular and national character string literals in any way, this prefix has no effect in H2.

String literals starting with U& are Unicode character string literals. All character string literals in H2 may have Unicode characters, but Unicode character string literals may contain Unicode escape sequences \0000 or \+000000, where \ is an escape character, 0000 and 000000 are Unicode character codes in hexadecimal notation. Optional UESCAPE clause may be used to specify another escape character, with exception for single quote, double quote, plus sign, and hexadecimal digits (0-9, a-f, and A-F). By default the backslash is used. Two escape characters can be used to include a single character inside a string. Two single quotes can be used to create a single quote inside a string.

Example:

```
'John"s car'
```

```
'A' 'B' 'C'
```

```
U&'W\00f6rter ' '\ \+01f600 /'
```

```
U&'|00a1' UESCAPE '|'
```

UUID

```
UUID '{ digit | a-f | A-F | - } [...]'
```

A UUID literal. Must contain 32 hexadecimal digits. Digits may be separated with - signs.

Example:

```
UUID '12345678-1234-1234-1234-123456789ABC'
```

Time

```
TIME [ WITHOUT TIME ZONE ] 'hh:mm:ss[.nnnnnnnnnn]'
```

A time literal. A value is between 0:00:00 and 23:59:59.999999999 and has nanosecond resolution.

Example:

TIME '23:59:59'

Time with time zone

```
TIME WITH TIME ZONE 'hh:mm:ss[.nnnnnnnnnn]{ { Z } | { - | + }  
timeZoneOffsetString}'
```

A time with time zone literal. A value is between 0:00:00 and 23:59:59.999999999 and has nanosecond resolution.

Example:

TIME WITH TIME ZONE '23:59:59+01'

TIME WITH TIME ZONE '10:15:30.334-03:30'

TIME WITH TIME ZONE '0:00:00Z'

Timestamp

```
TIMESTAMP [ WITHOUT TIME ZONE ] '[-]yyyy-MM-dd hh:mm:ss[.nnnnnnnnnn]
```

A timestamp literal.

Example:

TIMESTAMP '2005-12-31 23:59:59'

Timestamp with time zone

```
TIMESTAMP WITH TIME ZONE '[-]yyyy-MM-dd hh:mm:ss[.nnnnnnnnnn]  
[ { Z } | { - | + } timeZoneOffsetString | { timeZoneNameString } ]'
```

A timestamp with time zone literal. If name of time zone is specified it will be converted to time zone offset.

Example:

TIMESTAMP WITH TIME ZONE '2005-12-31 23:59:59Z'

TIMESTAMP WITH TIME ZONE '2005-12-31 23:59:59-10:00'

TIMESTAMP WITH TIME ZONE '2005-12-31 23:59:59.123+05'

TIMESTAMP WITH TIME ZONE '2005-12-31 23:59:59.123456789 Europe/London'

Interval

[intervalYear](#) | [intervalMonth](#) | [intervalDay](#) | [intervalHour](#) | [intervalMinute](#)
| [intervalSecond](#) | [intervalYearToMonth](#) | [intervalDayToHour](#)
| [intervalDayToMinute](#) | [intervalDayToSecond](#) | [intervalHourToMinute](#)
| [intervalHourToSecond](#) | [intervalMinuteToSecond](#)

An interval literal.

Example:

INTERVAL '1-2' YEAR TO MONTH

INTERVAL YEAR

INTERVAL [- | +] '[- | +][yearInt](#)' YEAR [([precisionInt](#))]

An INTERVAL YEAR literal. If precision is specified it should be from 1 to 18.

Example:

INTERVAL '10' YEAR

INTERVAL MONTH

INTERVAL [- | +] '[- | +][monthInt](#)' MONTH [([precisionInt](#))]

An INTERVAL MONTH literal. If precision is specified it should be from 1 to 18.

Example:

INTERVAL '10' MONTH

INTERVAL DAY

INTERVAL [- | +] '[- | +][dayInt](#)' DAY [([precisionInt](#))]

An INTERVAL DAY literal. If precision is specified it should be from 1 to 18.

Example:

INTERVAL '10' DAY

INTERVAL HOUR

```
INTERVAL [-|+] '[-|+]hourInt' HOUR [ ( precisionInt ) ]
```

An INTERVAL HOUR literal. If precision is specified it should be from 1 to 18.

Example:

INTERVAL '10' HOUR

INTERVAL MINUTE

```
INTERVAL [-|+] '[-|+]minuteInt' MINUTE [ ( precisionInt ) ]
```

An INTERVAL MINUTE literal. If precision is specified it should be from 1 to 18.

Example:

INTERVAL '10' MINUTE

INTERVAL SECOND

```
INTERVAL [-|+] '[-|+]secondInt[.nnnnnnnnnn]'  
SECOND [ ( precisionInt [, fractionalPrecisionInt ] ) ]
```

An INTERVAL SECOND literal. If precision is specified it should be from 1 to 18. If fractional seconds precision is specified it should be from 0 to 9.

Example:

INTERVAL '10.123' SECOND

INTERVAL YEAR TO MONTH

```
INTERVAL [-|+] '[-|+]yearInt-monthInt' YEAR [ ( precisionInt ) ] TO MONTH
```

An INTERVAL YEAR TO MONTH literal. If leading field precision is specified it should be from 1 to 18.

Example:

INTERVAL '1-6' YEAR TO MONTH

INTERVAL DAY TO HOUR

```
INTERVAL [-|+] '[-|+]dayInt hoursInt' DAY [ ( precisionInt ) ] TO HOUR
```

An INTERVAL DAY TO HOUR literal. If leading field precision is specified it should be from 1 to 18.

Example:

```
INTERVAL '10 11' DAY TO HOUR
```

INTERVAL DAY TO MINUTE

```
INTERVAL [-|+] '[-|+]dayInt hh:mm' DAY [ ( precisionInt ) ] TO MINUTE
```

An INTERVAL DAY TO MINUTE literal. If leading field precision is specified it should be from 1 to 18.

Example:

```
INTERVAL '10 11:12' DAY TO MINUTE
```

INTERVAL DAY TO SECOND

```
INTERVAL [-|+] '[-|+]dayInt hh:mm:ss[.nnnnnnnnnn]' DAY [ ( precisionInt ) ]  
TO SECOND [ ( fractionalPrecisionInt ) ]
```

An INTERVAL DAY TO SECOND literal. If leading field precision is specified it should be from 1 to 18. If fractional seconds precision is specified it should be from 0 to 9.

Example:

```
INTERVAL '10 11:12:13.123' DAY TO SECOND
```

INTERVAL HOUR TO MINUTE

```
INTERVAL [-|+] '[-|+]hh:mm' HOUR [ ( precisionInt ) ] TO MINUTE
```

An INTERVAL HOUR TO MINUTE literal. If leading field precision is specified it should be from 1 to 18.

Example:

INTERVAL '10:11' HOUR TO MINUTE

INTERVAL HOUR TO SECOND

```
INTERVAL [-|+] '[-|+]hh:mm:ss[.nnnnnnnnnn]' HOUR [ ( precisionInt ) ]  
TO SECOND [ ( fractionalPrecisionInt ) ]
```

An INTERVAL HOUR TO SECOND literal. If leading field precision is specified it should be from 1 to 18. If fractional seconds precision is specified it should be from 0 to 9.

Example:

INTERVAL '10:11:12.123' HOUR TO SECOND

INTERVAL MINUTE TO SECOND

```
INTERVAL [-|+] '[-|+]mm:ss[.nnnnnnnnnn]' MINUTE [ ( precisionInt ) ]  
TO SECOND [ ( fractionalPrecisionInt ) ]
```

An INTERVAL MINUTE TO SECOND literal. If leading field precision is specified it should be from 1 to 18. If fractional seconds precision is specified it should be from 0 to 9.

Example:

INTERVAL '11:12.123' MINUTE TO SECOND

Datetime fields

Datetime field

```
yearField | monthField | dayOfMonthField  
| hourField | minuteField | secondField  
| timezoneHourField | timezoneMinuteField  
| { timezoneSecondField  
| millenniumField | centuryField | decadeField  
| quarterField  
| millisecondField | microsecondField | nanosecondField  
| dayOfYearField  
| isoDayOfWeekField | isoWeekField | isoWeekYearField
```

```
| dayOfWeekField | weekField | weekYearField  
| epochField }
```

Fields for EXTRACT, DATEADD, DATEDIFF, and DATE_TRUNC functions.

Example:

YEAR

Year field

```
YEAR | { YYYY | YY | SQL_TSI_YEAR }
```

Year.

Example:

YEAR

Month field

```
MONTH | { MM | M | SQL_TSI_MONTH }
```

Month (1-12).

Example:

MONTH

Day of month field

```
DAY | { DD | D | SQL_TSI_DAY }
```

Day of month (1-31).

Example:

DAY

Hour field

```
HOUR | { HH | SQL_TSI_HOUR }
```

Hour (0-23).

Example:

HOUR

Minute field

MINUTE | { MI | N | SQL_TSI_MINUTE }

Minute (0-59).

Example:

MINUTE

Second field

SECOND | { SS | S | SQL_TSI_SECOND }

Second (0-59).

Example:

SECOND

Timezone hour field

TIMEZONE_HOUR

Timezone hour (from -18 to +18).

Example:

TIMEZONE_HOUR

Timezone minute field

TIMEZONE_MINUTE

Timezone minute (from -59 to +59).

Example:

TIMEZONE_MINUTE

Timezone second field

TIMEZONE_SECOND

Timezone second (from -59 to +59). Local mean time (LMT) used in the past may have offsets with seconds. Standard time doesn't use such offsets.

Example:

TIMEZONE_SECOND

Millennium field

MILLENNIUM

Century, or one thousand years (2001-01-01 to 3000-12-31).

Example:

MILLENNIUM

Century field

CENTURY

Century, or one hundred years (2001-01-01 to 2100-12-31).

Example:

CENTURY

Decade field

DECADE

Decade, or ten years (2020-01-01 to 2029-12-31).

Example:

DECADE

Quarter field

QUARTER

Quarter (1-4).

Example:

QUARTER

Millisecond field

{ **MILLISECOND** } | { **MS** }

Millisecond (0-999).

Example:

MILLISECOND

Microsecond field

{ **MICROSECOND** } | { **MCS** }

Microsecond (0-999999).

Example:

MICROSECOND

Nanosecond field

{ **NANOSECOND** } | { **NS** }

Nanosecond (0-999999999).

Example:

NANOSECOND

Day of year field

{ **DAYOFYEAR** | **DAY_OF_YEAR** } | { **DOY** | **DY** }

Day of year (1-366).

Example:

DAYOFYEAR

ISO day of week field

`{ ISO_DAY_OF_WEEK } | { ISODOW }`

ISO day of week (1-7). Monday is 1.

Example:

ISO_DAY_OF_WEEK

ISO week field

`ISO_WEEK`

ISO week of year (1-53). ISO definition is used when first week of year should have at least four days and week is started with Monday.

Example:

ISO_WEEK

ISO week year field

`{ ISO_WEEK_YEAR } | { ISO_YEAR | ISOYEAR }`

Returns the ISO week-based year from a date/time value.

Example:

ISO_WEEK_YEAR

Day of week field

`{ DAY_OF_WEEK | DAYOFWEEK } | { DOW }`

Day of week (1-7), locale-specific.

Example:

DAY_OF_WEEK

Week field

`{ WEEK } | { WW | W | SQL_TSI_WEEK }`

Week of year (1-53) using local rules.

Example:

WEEK

Week year field

```
{ WEEK_YEAR }
```

Returns the week-based year (locale-specific) from a date/time value.

Example:

WEEK_YEAR

Epoch field

```
EPOCH
```

For TIMESTAMP values number of seconds since 1970-01-01 00:00:00 in local time zone. For TIMESTAMP WITH TIME ZONE values number of seconds since 1970-01-01 00:00:00 in UTC time zone. For DATE values number of seconds since 1970-01-01. For TIME values number of seconds since midnight.

Example:

EPOCH

Other Grammar

Alias

```
name
```

An alias is a name that is only valid in the context of the statement.

Example:

A

And Condition

```
condition [ { AND condition } [...] ]
```

Value or condition.

Example:

ID=1 AND NAME='Hi'

Array element reference

```
{ array | json } [' indexInt ']
```

Returns array element at specified 1-based index. Returns NULL if array or json is null, index is null, or element with specified index isn't found in JSON.

Example:

A[2]

M[5][8]

Field reference

```
(expression).fieldName
```

Returns field value from the row value or JSON value. Returns NULL if value is null or field with specified name isn't found in JSON. Expression on the left must be enclosed in parentheses if it is an identifier (column name), in other cases they aren't required.

Example:

(R).FIELD1

(TABLE1.COLUMN2).FIELD.SUBFIELD

JSON '{"a": 1, "b": 2}'. "b"

Array value constructor by query

```
ARRAY (query)
```

Collects values from the subquery into array.

The subquery should have exactly one column. Number of elements in the returned array is the number of rows in the subquery. NULL values are included into array.

Example:

```
ARRAY(SELECT * FROM SYSTEM_RANGE(1, 10));
```

Case expression

[simpleCase](#) | [searchedCase](#)

Performs conditional evaluation of expressions.

Example:

```
CASE A WHEN 'a' THEN 1 ELSE 2 END
```

```
CASE WHEN V > 10 THEN 1 WHEN V < 0 THEN 2 END
```

```
CASE WHEN A IS NULL THEN 'Null' ELSE 'Not null' END
```

Simple case

```
CASE expression  
{ WHEN { expression | conditionRightHandSide } [...] THEN expression } [...]  
[ ELSE expression ] END
```

Returns then expression from the first when clause where one of its operands was evaluated to TRUE for the case expression. If there are no such clauses, returns else expression or NULL if it is absent.

Plain expressions are tested for equality with the case expression, NULL is not equal to NULL. Right sides of conditions are evaluated with the case expression on the left side.

Example:

```
CASE CNT WHEN IS NULL THEN 'Null' WHEN 0 THEN 'No' WHEN 1 THEN 'One'  
WHEN 2, 3 THEN 'Few' ELSE 'Some' END
```

Searched case

```
CASE { WHEN expression THEN expression } [...]  
[ ELSE expression ] END
```

Returns the first expression where the condition is true. If no else part is specified, return NULL.

Example:

```
CASE WHEN CNT<10 THEN 'Low' ELSE 'High' END
```

```
CASE WHEN A IS NULL THEN 'Null' ELSE 'Not null' END
```

Cast specification

```
CAST(value AS dataTypeOrDomain [ FORMAT templateString ])
```

Converts a value to another data type. The following conversion rules are used: When converting a number to a boolean, 0 is false and every other value is true. When converting a boolean to a number, false is 0 and true is 1. When converting a number to a number of another type, the value is checked for overflow. When converting a string to binary, UTF-8 encoding is used. Note that some data types may need explicitly specified precision to avoid overflow or rounding.

Template may only be specified for casts from datetime data types to character string data types and for casts from character string data types to datetime data types.

'-', ':', '/', ',', '"', ';', ':' and ' ' (space) characters can be used as delimiters.

Y, YY, YYYY, YYYY represent last 1, 2, 3, or 4 digits of year. YYYY, if delimited, can also be used to parse any year, including negative years. When a year is parsed with Y, YY, or YYYY pattern missing leading digits are filled using digits from the current year.

RR and RRRR have the same meaning as YY and YYYY for formatting. When a year is parsed with RR, the resulting year is within current year - 49 years and

current year + 50 years in H2, other database systems may use different range of years.

MM represent a month.

DD represent a day of month.

DDD represent a day of year, if this pattern is specified, MM and DD may not be specified.

HH24 represent an hour (from 0 to 23).

HH and HH12 represent an hour (from 1 to 12), this pattern may only be used together with A.M. or P.M. pattern. These patterns may not be used together with HH24.

MI represent minutes.

SS represent seconds of minute.

SSSSS represent seconds of day, this pattern may not be used together with HH24, HH (HH12), A.M. (P.M.), MI or SS pattern.

FF1, FF2, ..., FF9 represent fractional seconds.

TZH, TZM and TZH represent hours, minutes and seconds of time zone offset.

Multiple patterns for the same datetime field may not be specified.

If year is not specified, current year is used. If month is not specified, current month is used. If day is not specified, 1 is used.

If some fields of time or time zone are not specified, 0 is used.

Example:

```
CAST(NAME AS INT);
```

```
CAST(TIMESTAMP '2010-01-01 10:40:00.123456' AS TIME(6));
```

```
CAST('12:00:00 P.M.' AS TIME FORMAT 'HH:MI:SS A.M.');
```

Cipher

AES

Only the algorithm AES (AES-128) is supported currently.

Example:

AES

Column Definition

```
dataTypeOrDomain [ VISIBLE | INVISIBLE ]  
[ { DEFAULT expression  
  | GENERATED ALWAYS AS (generatedColumnExpression)  
  | GENERATED {ALWAYS | BY DEFAULT} AS IDENTITY [(sequenceOption [...])}] ]  
[ ON UPDATE expression ]  
[ DEFAULT ON NULL ]  
[ SELECTIVITY selectivityInt ] [ COMMENT expression ]  
[ columnConstraintDefinition ] [...]
```

The default expression is used if no explicit value was used when adding a row and when DEFAULT value was specified in an update command.

A column is either a generated column or a base column. The generated column has a generated column expression. The generated column expression is evaluated and assigned whenever the row changes. This expression may reference base columns of the table, but may not reference other data. The value of the generated column cannot be set explicitly. Generated columns may not have DEFAULT or ON UPDATE expressions.

On update column expression is used if row is updated, at least one column has a new value that is different from its previous value and value for this column is not set explicitly in update statement.

Identity column is a column generated with a sequence. The column declared as the identity column with IDENTITY data type or with IDENTITY () clause is implicitly the primary key column of this table. GENERATED ALWAYS AS IDENTITY, GENERATED BY DEFAULT AS IDENTITY, and AUTO_INCREMENT clauses do not create the primary key constraint automatically. GENERATED ALWAYS AS IDENTITY clause indicates that column can only be generated by the sequence, its value cannot be set explicitly. Identity column has implicit NOT

NULL constraint. Identity column may not have DEFAULT or ON UPDATE expressions.

DEFAULT ON NULL makes NULL value work as DEFAULT value is assignments to this column.

The invisible column will not be displayed as a result of SELECT * query. Otherwise, it works as normal column.

Column constraint definitions are not supported for ALTER statements.

Example:

```
CREATE TABLE TEST(ID INT PRIMARY KEY,  
    NAME VARCHAR(255) DEFAULT " NOT NULL);  
CREATE TABLE TEST(ID BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    QUANTITY INT, PRICE NUMERIC(10, 2),  
    AMOUNT NUMERIC(20, 2) GENERATED ALWAYS AS (QUANTITY * PRICE));
```

Column Constraint Definition

```
[ constraintNameDefinition ]  
NOT NULL | PRIMARY KEY | UNIQUE [ nullsDistinct ] | referencesSpecification  
| CHECK (condition)
```

NOT NULL disallows NULL value for a column.

PRIMARY KEY and UNIQUE require unique values. PRIMARY KEY also disallows NULL values and marks the column as a primary key. UNIQUE constraint allows NULL values, if nulls distinct clause is not specified, the default is NULLS DISTINCT, excluding some compatibility modes.

Referential constraint requires values that exist in other column (usually in another table).

Check constraint require a specified condition to return TRUE or UNKNOWN (NULL). It can reference columns of the table, and can reference objects that exist while the statement is executed. Conditions are only checked when a row is added or modified in the table where the constraint exists.

Example:

NOT NULL

PRIMARY KEY

UNIQUE

REFERENCES T2(ID)

CHECK (VALUE > 0)

Comment

```
bracketedComment | -- anything | // anything
```

Comments can be used anywhere in a command and are ignored by the database. Line comments -- and // end with a newline.

Example:

```
-- comment
```

```
/* comment */
```

Bracketed comment

```
/* [ [ bracketedComment ] [ anything ] [...] ] */
```

Comments can be used anywhere in a command and are ignored by the database. Bracketed comments /* */ can be nested and can be multiple lines long.

Example:

```
/* comment */
```

```
/* comment /* nested comment */ comment */
```

Compare

```
<> | <= | >= | = | < | > | { != } | &&
```

Comparison operator. The operator != is the same as <>. The operator && means overlapping; it can only be used with geometry types.

Example:

<>

Condition

```
operand [ conditionRightHandSide ]  
| NOT condition  
| EXISTS ( query )  
| UNIQUE [ nullsDistinct ] ( query )  
| INTERSECTS (operand, operand)
```

Boolean value or condition.

NOT condition negates the result of subcondition and returns TRUE, FALSE, or UNKNOWN (NULL).

EXISTS predicate tests whether the result of the specified subquery is not empty and returns TRUE or FALSE.

UNIQUE predicate tests absence of duplicate rows in the specified subquery and returns TRUE or FALSE. If nulls distinct clause is not specified, NULLS DISTINCT is implicit.

INTERSECTS checks whether 2D bounding boxes of specified geometries intersect with each other and returns TRUE or FALSE.

Example:

ID <> 2

NOT(A OR B)

EXISTS (SELECT NULL FROM TEST T WHERE T.GROUP_ID = P.ID)

UNIQUE (SELECT A, B FROM TEST T WHERE T.CATEGORY = CAT)

INTERSECTS(GEOM1, GEOM2)

Condition Right Hand Side

```
comparisonRightHandSide  
| quantifiedComparisonRightHandSide  
| nullPredicateRightHandSide  
| distinctPredicateRightHandSide  
| quantifiedDistinctPredicateRightHandSide
```

- | `booleanTestRightHandSide`
- | `typePredicateRightHandSide`
- | `jsonPredicateRightHandSide`
- | `betweenPredicateRightHandSide`
- | `inPredicateRightHandSide`
- | `likePredicateRightHandSide`
- | `regexpPredicateRightHandSide`

The right hand side of a condition.

Example:

> 10

IS NULL

IS NOT NULL

IS NOT DISTINCT FROM B

IS OF (DATE, TIMESTAMP, TIMESTAMP WITH TIME ZONE)

IS JSON OBJECT WITH UNIQUE KEYS

LIKE 'Jo%'

Comparison Right Hand Side

`compare operand`

Right side of comparison predicates.

Example:

> 10

Quantified Comparison Right Hand Side

`compare { ALL | ANY | SOME } ( { query | { array } } )`

Right side of quantified comparison predicates.

Quantified comparison predicate ALL returns TRUE if specified comparison operation between left side of condition and each row from a subquery or each element of array returns TRUE, including case when there are no rows

(elements). ALL predicate returns FALSE if at least one such comparison returns FALSE. Otherwise it returns UNKNOWN.

Quantified comparison predicates ANY and SOME return TRUE if specified comparison operation between left side of condition and at least one row from a subquery or at least one element of array returns TRUE. ANY and SOME predicates return FALSE if all such comparisons return FALSE. Otherwise they return UNKNOWN.

Note that these predicates have priority over ANY and SOME aggregate functions with subquery on the right side. Use parentheses around aggregate function.

If version with array is required and this array is returned from a subquery, wrap this subquery with a cast to distinguish this operation from standard quantified comparison predicate with a query.

Example:

```
< ALL(SELECT V FROM TEST)
= ANY(ARRAY_COLUMN)
= ANY(CAST((SELECT ARRAY_COLUMN FROM OTHER_TABLE WHERE ID = 5) AS
INTEGER ARRAY))
```

Null Predicate Right Hand Side

IS [NOT] NULL

Right side of null predicate.

Check whether the specified value(s) are NULL values. To test multiple values a row value must be specified. IS NULL returns TRUE if and only if all values are NULL values; otherwise it returns FALSE. IS NOT NULL returns TRUE if and only if all values are not NULL values; otherwise it returns FALSE.

Example:

IS NULL

Distinct Predicate Right Hand Side

```
IS [ NOT ] [ DISTINCT FROM ] operand
```

Right side of distinct predicate.

Distinct predicate is null-safe, meaning NULL is considered the same as NULL, and the condition never evaluates to UNKNOWN.

Example:

```
IS NOT DISTINCT FROM OTHER
```

Quantified Distinct Predicate Right Hand Side

```
IS [ NOT ] [ DISTINCT FROM ] { ALL | ANY | SOME } ( { query | array } )
```

Right side of quantified distinct predicate.

Quantified distinct predicate is null-safe, meaning NULL is considered the same as NULL, and the condition never evaluates to UNKNOWN.

Quantified distinct predicate ALL returns TRUE if specified distinct predicate between left side of condition and each row from a subquery or each element of array returns TRUE, including case when there are no rows. Otherwise it returns FALSE.

Quantified distinct predicates ANY and SOME return TRUE if specified distinct predicate between left side of condition and at least one row from a subquery or at least one element of array returns TRUE. Otherwise they return FALSE.

Note that these predicates have priority over ANY and SOME aggregate functions with subquery on the right side. Use parentheses around aggregate function.

If version with array is required and this array is returned from a subquery, wrap this subquery with a cast to distinguish this operation from quantified comparison predicate with a query.

Example:

IS DISTINCT FROM ALL(SELECT V FROM TEST)
IS NOT DISTINCT FROM ANY(ARRAY_COLUMN)
IS NOT DISTINCT FROM ANY(CAST((SELECT ARRAY_COLUMN FROM
OTHER_TABLE WHERE ID = 5) AS INTEGER ARRAY))

Boolean Test Right Hand Side

```
IS [ NOT ] { TRUE | FALSE | UNKNOWN }
```

Right side of boolean test.

Checks whether the specified value is (not) TRUE, FALSE, or UNKNOWN (NULL) and return TRUE or FALSE. This test is null-safe.

Example:

IS TRUE

Type Predicate Right Hand Side

```
IS [ NOT ] OF (dataType [...])
```

Right side of type predicate.

Checks whether the data type of the specified operand is one of the specified data types. Some data types have multiple names, these names are considered as equal here. Domains and their base data types are currently not distinguished from each other. Precision and scale are also ignored. If operand is NULL, the result is UNKNOWN.

Example:

IS OF (INTEGER, BIGINT)

JSON Predicate Right Hand Side

```
IS [ NOT ] JSON [ VALUE | ARRAY | OBJECT | SCALAR ]  
[ [ WITH | WITHOUT ] UNIQUE [ KEYS ] ]
```

Right side of JSON predicate.

Checks whether value of the specified string, binary data, or a JSON is a valid JSON. If ARRAY, OBJECT, or SCALAR is specified, only JSON items of the specified type are considered as valid. If WITH UNIQUE [KEYS] is specified only JSON with unique keys is considered as valid. This predicate isn't null-safe, it returns UNKNOWN if operand is NULL.

Example:

IS JSON OBJECT WITH UNIQUE KEYS

Between Predicate Right Hand Side

```
[ NOT ] BETWEEN [ ASYMMETRIC | SYMMETRIC ] operand AND operand
```

Right side of between predicate.

Checks whether the value is within the range inclusive. V BETWEEN [ASYMMETRIC] A AND B is equivalent to $A \leq V \text{ AND } V \leq B$. V BETWEEN SYMMETRIC A AND B is equivalent to $A \leq V \text{ AND } V \leq B \text{ OR } A \geq V \text{ AND } V \geq B$.

Example:

BETWEEN LOW AND HIGH

In Predicate Right Hand Side

```
[ NOT ] IN ( { query | expression [,...] } )
```

Right side of in predicate.

Checks presence of value in the specified list of values or in result of the specified query.

Returns TRUE if row value on the left side is equal to one of values on the right side, FALSE if all comparison operations were evaluated to FALSE or right side has no values, and UNKNOWN otherwise.

This operation is logically equivalent to OR between comparison operations comparing left side and each value from the right side.

Example:

IN (A, B, C)

IN (SELECT V FROM TEST)

Like Predicate Right Hand Side

```
[ NOT ] { LIKE | { ILIKE } } operand [ ESCAPE string ]
```

Right side of like predicate.

The wildcards characters are _ (any one character) and % (any characters). The database uses an index when comparing with LIKE except if the operand starts with a wildcard. To search for the characters % and _, the characters need to be escaped. The default escape character is \ (backslash). To select no escape character, use ESCAPE '' (empty string). At most one escape character is allowed. Each character that follows the escape character in the pattern needs to match exactly. Patterns that end with an escape character are invalid and the expression returns NULL.

ILIKE does a case-insensitive compare.

Example:

LIKE 'a%'

Regexp Predicate Right Hand Side

```
{ [ NOT ] REGEXP operand }
```

Right side of Regexp predicate.

Regular expression matching is used. See `Java Matcher.find` for details.

Example:

REGEXP '[a-z]'

Nulls Distinct

```
NULLS { DISTINCT | NOT DISTINCT | { ALL DISTINCT } }
```

Are nulls distinct for unique constraint, index, or predicate.

If NULLS DISTINCT is specified, rows with null value in any column are distinct. If NULLS ALL DISTINCT is specified, rows with null value in all columns are distinct. If NULLS NOT DISTINCT is specified, null values are identical.

Treatment of null values inside composite data types is not affected.

Example:

NULLS DISTINCT

NULLS NOT DISTINCT

Table Constraint Definition

```
[ constraintNameDefinition ]  
{ PRIMARY KEY [ HASH ] ( columnName [,...] ) }  
| UNIQUE [ nullsDistinct ] ( { columnName [,...] | VALUE } )  
| referentialConstraint  
| CHECK (condition)
```

Defines a constraint.

PRIMARY KEY and UNIQUE require unique values. PRIMARY KEY also disallows NULL values and marks the column as a primary key, a table can have only one primary key. UNIQUE constraint supports NULL values and rows with NULL value in any column are considered as unique. UNIQUE constraint allows NULL values, if nulls distinct clause is not specified, the default is NULLS DISTINCT, excluding some compatibility modes. UNIQUE (VALUE) creates a unique constraint on entire row, excluding invisible columns; but if new columns will be added to the table, they will not be included into this constraint.

Referential constraint requires values that exist in other column(s) (usually in another table).

Check constraint requires a specified condition to return TRUE or UNKNOWN (NULL). It can reference columns of the table, and can reference objects that exist while the statement is executed. Conditions are only checked when a row is added or modified in the table where the constraint exists.

Example:

PRIMARY KEY(ID, NAME)

Constraint Name Definition

```
CONSTRAINT [ IF NOT EXISTS ] newConstraintName
```

Defines a constraint name.

Example:

```
CONSTRAINT CONST_ID
```

Csv Options

```
charsetString [, fieldSepString [, fieldDelimString [, escString [, nullString]]]  
| optionString
```

Optional parameters for CSVREAD and CSVWRITE. Instead of setting the options one by one, all options can be combined into a space separated key-value pairs, as follows: STRINGDECODE('charset=UTF-8 escape=\" fieldDelimiter=\"

fieldSeparator=, ' | | 'lineComment=# lineSeparator=\n null= rowSeparator=').

The following options are supported:

caseSensitiveColumnNames (true or false; disabled by default),

charset (for example 'UTF-8'),

escape (the character that escapes the field delimiter),

fieldDelimiter (a double quote by default),

fieldSeparator (a comma by default),

lineComment (disabled by default),

lineSeparator (the line separator used for writing; ignored for reading),

null Support reading existing CSV files that contain explicit null delimiters. Note that an empty, unquoted values are also treated as null.

quotedNulls (quotes the nullString. true or false; disabled by default),

preserveWhitespace (true or false; disabled by default),

writeColumnHeader (true or false; enabled by default).

For a newline or other special character, use STRINGDECODE as in the example above. A space needs to be escaped with a backslash ('\ '), and a backslash needs to be escaped with another backslash ('\\'). All other characters are not to be escaped, that means newline and tab characters are written as such.

Example:

```
CALL CSVWRITE('test2.csv', 'SELECT * FROM TEST', 'charset=UTF-8  
fieldSeparator=|');
```

Data Change Delta Table

```
{ OLD | NEW | FINAL } TABLE  
( { insert | update | delete | { mergeInto } | mergeUsing } )
```

Executes the inner data change command and returns old, new, or final rows.

OLD is not allowed for INSERT command. It returns old rows.

NEW and FINAL are not allowed for DELETE command.

NEW returns new rows after evaluation of default expressions, but before execution of triggers.

FINAL returns new rows after execution of triggers.

Example:

```
SELECT ID FROM FINAL TABLE (INSERT INTO TEST (A, B) VALUES (1, 2))
```

Data Type or Domain

```
dataType | [schemaName.]domainName
```

A data type or domain name.

Example:

INTEGER

MY_DOMAIN

Data Type

[predefinedType](#) | [arrayType](#) | [rowType](#)

A data type.

Example:

INTEGER

Predefined Type

[characterType](#) | [characterVaryingType](#) | [characterLargeObjectType](#)
| [binaryType](#) | [binaryVaryingType](#) | [binaryLargeObjectType](#)
| [booleanType](#)
| [smallintType](#) | [integerType](#) | [bigintType](#)
| [numericType](#) | [realType](#) | [doublePrecisionType](#) | [decfloatType](#)
| [dateType](#) | [timeType](#) | [timeWithTimeZoneType](#)
| [timestampType](#) | [timestampWithTimeZoneType](#)
| [intervalType](#)
| { [tinyintType](#) | [javaObjectType](#) | [enumType](#)
| [geometryType](#) | [jsonType](#) | [uuidType](#) }

A predefined data type.

Example:

INTEGER

Digit

0-9

A digit.

Example:

0

Expression

[andCondition](#) [{ OR [andCondition](#) } [...]]

Value or condition.

Example:

ID=1 OR NAME='Hi'

Factor

```
term [ { { * | / | { % } } term } [...]
```

A value or a numeric factor.

Example:

ID * 10

Grouping element

```
expression | (expression [, ...]) | ()
```

A grouping element of GROUP BY clause.

Example:

A

(B, C)

()

Hex

```
[' ' [...]] { { digit | a-f | A-F } [' ' [...]] { digit | a-f | A-F } [' ' [...]] } [...]
```

The hexadecimal representation of a number or of bytes with optional space characters. Two hexadecimal digit characters are one byte.

Example:

cafe

11 22 33

a b c d

Index Column

```
columnName [ ASC | DESC ] [ NULLS { FIRST | LAST } ]
```

Indexes this column in ascending or descending order. Usually it is not required to specify the order; however doing so will speed up large queries that order the column in the same way.

Example:

NAME

Insert values

```
VALUES { DEFAULT | expression | [ROW] ({DEFAULT | expression} [...]) }, [...]
```

Values for INSERT statement.

Example:

VALUES (1, 'Test')

Interval qualifier

```
YEAR [(precisionInt)] [ TO MONTH ]  
| MONTH [(precisionInt)]  
| DAY [(precisionInt)] [ TO { HOUR | MINUTE | SECOND [(scaleInt)] } ]  
| HOUR [(precisionInt)] [ TO { MINUTE | SECOND [(scaleInt)] } ]  
| MINUTE [(precisionInt)] [ TO SECOND [(scaleInt)] ]  
| SECOND [(precisionInt [, scaleInt])]
```

An interval qualifier.

Example:

DAY TO SECOND

Join specification

```
ON expression | USING (columnName [...])
```

Specifies a join condition or column names.

Example:

```
ON B.ID = A.PARENT_ID  
USING (ID)
```

Merge when clause

[mergeWhenMatchedClause](#) | [mergeWhenNotMatchedClause](#)

WHEN MATCHED or WHEN NOT MATCHED clause for MERGE USING command.

Example:

```
WHEN MATCHED THEN DELETE
```

Merge when matched clause

```
WHEN MATCHED [ AND expression ] THEN  
UPDATE SET setClauseList | DELETE
```

WHEN MATCHED clause for MERGE USING command.

Updates or deletes rows in a target table.

Example:

```
WHEN MATCHED THEN UPDATE SET NAME = S.NAME  
WHEN MATCHED THEN DELETE
```

Merge when not matched clause

```
WHEN NOT MATCHED [ AND expression ] THEN INSERT  
[ ( columnName [,...] ) ]  
[ overrideClause ]  
VALUES ({DEFAULT | expression} [,...])
```

WHEN NOT MATCHED clause for MERGE USING command.

Inserts rows into a target table.

If column names aren't specified a list of all visible columns in the target table is assumed.

Example:

WHEN NOT MATCHED THEN INSERT (ID, NAME) VALUES (S.ID, S.NAME)

Name

```
{ { A-Z | _ } [ { A-Z | _ | 0-9 } [...] ] } | quotedName
```

With default settings unquoted names are converted to upper case. The maximum name length is 256 characters.

Identifiers in H2 are case sensitive by default. Because unquoted names are converted to upper case, they can be written in any case anyway. When both quoted and unquoted names are used for the same identifier the quoted names must be written in upper case. Identifiers with lowercase characters can be written only as a quoted name, they aren't accessible with unquoted names.

If DATABASE_TO_UPPER setting is set to FALSE the unquoted names aren't converted to upper case.

If DATABASE_TO_LOWER setting is set to TRUE the unquoted names are converted to lower case instead.

If CASE_INSENSITIVE_IDENTIFIERS setting is set to TRUE all identifiers are case insensitive.

Example:

TEST

Operand

```
summand [ { | | summand } [...] ]
```

Performs the concatenation of character string, binary string, or array values. In the default mode, the result is NULL if either parameter is NULL. In compatibility modes result of string concatenation with NULL parameter can be different.

Example:

```
'Hi' || 'Eva'  
X'AB' || X'CD'  
ARRAY[1, 2] || 3  
1 || ARRAY[2, 3]  
ARRAY[1, 2] || ARRAY[3, 4]
```

Override clause

```
OVERRIDING { USER | SYSTEM } VALUE
```

If OVERRIDING USER VALUE is specified, INSERT statement ignores the provided value for identity column and generates a new one instead.

If OVERRIDING SYSTEM VALUE is specified, INSERT statement assigns the provided value to identity column.

If neither clauses are specified, INSERT statement assigns the provided value to GENERATED BY DEFAULT AS IDENTITY column, but throws an exception if value is specified for GENERATED ALWAYS AS IDENTITY column.

Example:

```
OVERRIDING SYSTEM VALUE  
OVERRIDING USER VALUE
```

Query

```
select | explicitTable | tableValue
```

A query, such as SELECT, explicit table, or table value.

Example:

```
SELECT ID FROM TEST;  
TABLE TEST;  
VALUES (1, 2), (3, 4);
```

Quoted Name

```
"anything"
```

```
| U&"anything" [ UESCAPE 'anything' ]
```

Case of characters in quoted names is preserved as is. Such names can contain spaces. The maximum name length is 256 characters. Two double quotes can be used to create a single double quote inside an identifier. With default settings identifiers in H2 are case sensitive.

Identifiers starting with U& are Unicode identifiers. All identifiers in H2 may have Unicode characters, but Unicode identifiers may contain Unicode escape sequences \0000 or \+000000, where \ is an escape character, 0000 and 000000 are Unicode character codes in hexadecimal notation. Optional UESCAPE clause may be used to specify another escape character, with exception for single quote, double quote, plus sign, and hexadecimal digits (0-9, a-f, and A-F). By default the backslash is used. Two escape characters can be used to include a single character inside an Unicode identifier. Two double quotes can be used to create a single double quote inside an Unicode identifier.

Example:

```
"FirstName"
```

```
U&"\00d6ffnungszeit"
```

```
U&"/00d6ffnungszeit" UESCAPE '/'
```

Referential Constraint

```
FOREIGN KEY ( columnName [,...] ) referencesSpecification
```

Defines a referential constraint.

Example:

```
FOREIGN KEY(ID) REFERENCES TEST(ID)
```

References Specification

```
REFERENCES [ refTableName ] [ ( refColumnName [,...] ) ]  
[ referentialTriggeredAction ]
```

Defines a referential specification of a referential constraint. If the table name is not specified, then the same table is referenced. NO ACTION is the default action, but currently it works like stricter RESTRICT. If the referenced columns are not specified, then the primary key columns are used. Referential constraint requires an existing unique or primary key constraint on referenced columns, this constraint must include all referenced columns in any order and must not include any other columns. Some tables may not be referenced, such as metadata tables.

Example:

```
REFERENCES TEST(ID)
```

Referential Triggered Action

```
ON UPDATE referentialAction [ ON DELETE referentialAction ]  
| ON DELETE referentialAction [ ON UPDATE referentialAction ]
```

Referential actions triggered during unique key updates and row deletions in the referenced table.

Example:

```
ON UPDATE CASCADE ON DELETE CASCADE
```

Referential Action

```
CASCADE | RESTRICT | NO ACTION | SET { DEFAULT | NULL }
```

The action CASCADE will cause conflicting rows in the referencing (child) table to be deleted or updated. NO ACTION is the default action, but because this database does not support deferred checking, RESTRICT and NO ACTION will both throw an exception immediately if the constraint is violated. The action SET DEFAULT will set the column in the referencing (child) table to the default value, while SET NULL will set it to NULL.

Example:

CASCADE

SET NULL

Script Compression Encryption

```
[ COMPRESSION { DEFLATE | LZF | ZIP | GZIP } ]  
[ CIPHER cipher PASSWORD string ]
```

The compression and encryption algorithm to use for script files. When using encryption, only DEFLATE and LZF are supported. LZF is faster but uses more space.

Example:

COMPRESSION LZF

Select order

```
{ expression | { int } } [ ASC | DESC ] [ NULLS { FIRST | LAST } ]
```

Sorts the result by the given column number, or by an expression. If the expression is a single parameter, then the value is interpreted as a column number. Negative column numbers reverse the sort order.

Example:

NAME DESC NULLS LAST

Row value expression

```
ROW (expression, [...])  
| ( [ expression, expression [...] ] )  
| expression
```

A row value expression.

Example:

ROW (1)

(1, 2)

1

Select Expression

```
wildcardExpression | expression [ [ AS ] columnAlias ]
```

An expression in a SELECT statement.

Example:

```
ID AS DOCUMENT_ID
```

Sequence value expression

```
{ NEXT | { CURRENT } } VALUE FOR [schemaName.]sequenceName
```

The next or current value of a sequence.

When the next value is requested the sequence is incremented and the current value of the sequence and the last identity in the current session are updated with the generated value. The next value of the sequence is generated only once for each processed row. If this expression is used multiple times with the same sequence it returns the same value within a processed row. Used values are never re-used, even when the transaction is rolled back.

Current value may only be requested after generation of the sequence value in the current session. It returns the latest generated value for the current session.

If a single command contains next and current value expressions for the same sequence there is no guarantee that the next value expression will be evaluated before the evaluation of current value expression.

Example:

```
NEXT VALUE FOR SEQ1
```

```
CURRENT VALUE FOR SCHEMA2.SEQ2
```

Sequence option

```
START WITH long  
| { RESTART WITH long }  
| basicSequenceOption
```


Option of a sequence.

START WITH is used to set the initial value of the sequence. If initial value is not defined, MINVALUE for incrementing sequences and MAXVALUE for decrementing sequences is used.

RESTART is used to immediately restart the sequence with the specified value.

Example:

```
START WITH 10000  
NO CACHE
```

Alter sequence option

```
{ START WITH long }  
| RESTART [ WITH long ]  
| basicSequenceOption
```

Option of a sequence.

START WITH is used to change the initial value of the sequence. It does not affect the current value of the sequence, it only changes the preserved initial value that is used for simple RESTART without a value.

RESTART is used to restart the sequence from its initial value or with the specified value.

Example:

```
START WITH 10000  
NO CACHE
```

Alter identity column option

```
{ START WITH long }  
| RESTART [ WITH long ]  
| SET basicSequenceOption
```

Option of an identity column.

START WITH is used to set or change the initial value of the sequence. START WITH does not affect the current value of the sequence, it only changes the preserved initial value that is used for simple RESTART without a value.

RESTART is used to restart the sequence from its initial value or with the specified value.

Example:

```
START WITH 10000  
SET NO CACHE
```

Basic sequence option

```
INCREMENT BY long  
| MINVALUE long | NO MINVALUE | { NOMINVALUE }  
| MAXVALUE long | NO MAXVALUE | { NOMAXVALUE }  
| CYCLE | NO CYCLE | { EXHAUSTED } | { NOCYCLE }  
| { CACHE long } | { NO CACHE } | { NOCACHE }
```

Basic option of a sequence.

INCREMENT BY specifies the step of the sequence, may be positive or negative, but may not be zero. The default is 1.

MINVALUE and MAXVALUE specify the bounds of the sequence.

Sequences with CYCLE option start the generation again from MINVALUE (incrementing sequences) or MAXVALUE (decrementing sequences) instead of exhausting with an error. Sequences with EXHAUSTED option can't return values until they will be restarted.

The CACHE option sets the number of pre-allocated numbers. If the system crashes without closing the database, at most this many numbers are lost. The default cache size is 32 if sequence has enough range of values. NO CACHE option or the cache size 1 or lower disable the cache. If CACHE option is specified, it cannot be larger than the total number of values that sequence can produce within a cycle.

Example:

MAXVALUE 100000

CYCLE

NO CACHE

Set clause list

```
{ { updateTarget = { DEFAULT | expression } }  
| { ( updateTarget [,...] ) = { rowValueExpression | ( query ) } } } [,...]
```

List of SET clauses.

Each column may be specified only once in update targets.

Example:

NAME = 'Test', PRICE = 2

(A, B) = (1, 2)

(A, B) = (1, 2), C = 3

(A, B) = (SELECT X, Y FROM OTHER T2 WHERE T1.ID = T2.ID)

Sort specification

```
expression [ ASC | DESC ] [ NULLS { FIRST | LAST } ]
```

Sorts the result by an expression.

Example:

X ASC NULLS FIRST

Sort specification list

```
sortSpecification [,...]
```

Sorts the result by expressions.

Example:

V

A, B DESC NULLS FIRST

Summand

```
factor [ { { + | - } factor } [...] ]
```

A value or a numeric sum.

Please note the text concatenation operator is ||.

Example:

ID + 20

Table Expression

```
{ [ schemaName. ] tableName  
| ( query )  
| unnest  
| table  
| dataChangeDeltaTable }  
[ [ AS ] newTableAlias [ ( columnName [...] ) ] ]  
[ USE INDEX ( [ indexName [...] ] ) ]  
[ { { LEFT | RIGHT } [ OUTER ] | [ INNER ] | CROSS | NATURAL }  
JOIN tableExpression [ joinSpecification ] ]
```

Joins a table. The join specification is not supported for cross and natural joins. A natural join is an inner join, where the condition is automatically on the columns with the same name.

Example:

TEST1 AS T1 LEFT JOIN TEST2 AS T2 ON T1.ID = T2.PARENT_ID

Update target

```
columnName [ '[' int ']' [...] ]
```

Column or element of a column of ARRAY data type.

If array indexes are specified, column must have a compatible ARRAY data type and updated rows may not have NULL values in this column. It means for C[2]

[3] both C and C[2] may not be NULL. Too short arrays are expanded, missing elements are set to NULL.

Example:

A

B[1]

C[2][3]

Within group specification

```
WITHIN GROUP (ORDER BY sortSpecificationList)
```

Group specification for ordered set functions.

Example:

```
WITHIN GROUP (ORDER BY ID DESC)
```

Wildcard expression

```
[[schemaName.]tableAlias.]*  
EXCEPT ([[schemaName.]tableAlias.]columnName, [...]])
```

A wildcard expression in a SELECT statement. A wildcard expression represents all visible columns. Some columns can be excluded with optional EXCEPT clause.

Example:

*

* EXCEPT (DATA)

Window name or specification

```
windowName | windowSpecification
```

A window name or inline specification for a window function or aggregate.

Window functions in H2 may require a lot of memory for large queries.

Example:

W1

(ORDER BY ID)

Window specification

```
([existingWindowName]  
[PARTITION BY expression [,...]] [ORDER BY sortSpecificationList]  
[windowFrame])
```

A window specification for a window, window function or aggregate.

If name of an existing window is specified its clauses are used by default.

Optional window partition clause separates rows into independent partitions. Each partition is processed separately. If this clause is not present there is one implicit partition with all rows.

Optional window order clause specifies order of rows in the partition. If some rows have the same order position they are considered as a group of rows in optional window frame clause.

Optional window frame clause specifies which rows are processed by a window function, see its documentation for a more details.

Example:

()

(W1 ORDER BY ID)

(PARTITION BY CATEGORY)

(PARTITION BY CATEGORY ORDER BY NAME, ID)

(ORDER BY Y RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
EXCLUDE TIES)

Window frame

```
ROWS | RANGE | GROUP  
{windowFramePreceding | BETWEEN windowFrameBound AND  
windowFrameBound}  
[EXCLUDE {CURRENT ROW | GROUP | TIES | NO OTHERS}]
```

A window frame clause. May be specified only for aggregates and FIRST_VALUE(), LAST_VALUE(), and NTH_VALUE() window functions.

If this clause is not specified for an aggregate or window function that supports this clause the default window frame depends on window order clause. If window order clause is also not specified the default window frame contains all the rows in the partition. If window order clause is specified the default window frame contains all preceding rows and all rows from the current group.

Window frame unit determines how rows or groups of rows are selected and counted. If ROWS is specified rows are not grouped in any way and relative numbers of rows are used in bounds. If RANGE is specified rows are grouped according window order clause, preceding and following values mean the difference between value in the current row and in the target rows, and CURRENT ROW in bound specification means current group of rows. If GROUPS is specified rows are grouped according window order clause, preceding and following values means relative number of groups of rows, and CURRENT ROW in bound specification means current group of rows.

If only window frame preceding clause is specified it is treated as BETWEEN windowFramePreceding AND CURRENT ROW.

Optional window frame exclusion clause specifies rows that should be excluded from the frame. EXCLUDE CURRENT ROW excludes only the current row regardless the window frame unit. EXCLUDE GROUP excludes the whole current group of rows, including the current row. EXCLUDE TIES excludes the current group of rows, but not the current row. EXCLUDE NO OTHERS is default and it does not exclude anything.

Example:

ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW EXCLUDE TIES

Window frame preceding

UNBOUNDED PRECEDING | **value** PRECEDING | CURRENT ROW

A window frame preceding clause. If value is specified it should not be negative.

Example:

UNBOUNDED PRECEDING

1 PRECEDING

CURRENT ROW

Window frame bound

```
UNBOUNDED PRECEDING | value PRECEDING | CURRENT ROW  
| value FOLLOWING | UNBOUNDED FOLLOWING
```

A window frame bound clause. If value is specified it should not be negative.

Example:

UNBOUNDED PRECEDING

UNBOUNDED FOLLOWING

1 FOLLOWING

CURRENT ROW

Term

```
{ value  
| column  
| ?[ int ]  
| sequenceValueExpression  
| function  
| { - | + } term  
| ( expression )  
| arrayElementReference  
| fieldReference  
| ( query )  
| caseExpression  
| castSpecification  
| userDefinedFunctionName }  
[ timeZone | intervalQualifier ]
```

A value. Parameters can be indexed, for example ?1 meaning the first parameter.

Interval qualifier may only be specified for a compatible value or for a subtraction operation between two datetime values. The subtraction operation ignores the leading field precision of the qualifier.

Example:

'Hello'

Time zone

```
AT { TIME ZONE { intervalHourToMinute | intervalHourToSecond | { string } } |  
LOCAL }
```

A time zone. Converts the timestamp with or without time zone into timestamp with time zone at specified time zone. If a day-time interval is specified as a time zone, it may not have fractional seconds and must be between -18 to 18 hours inclusive.

Example:

AT LOCAL

AT TIME ZONE '2'

AT TIME ZONE '-6:00'

AT TIME ZONE INTERVAL '10:00' HOUR TO MINUTE

AT TIME ZONE INTERVAL '10:00:00' HOUR TO SECOND

AT TIME ZONE 'UTC'

AT TIME ZONE 'Europe/London'

Column

```
[[schemaName.]tableAlias.] { columnName | { _ROWID_ } }
```

A column name with optional table alias and schema. `_ROWID_` can be used to access unique row identifier.

Example:

ID

System Tables

Index

Information Schema

CHECK_CONSTRAINTS

COLLATIONS

COLUMNS

COLUMN_PRIVILEGES

CONSTANTS

CONSTRAINT_COLUMN_USAGE

DOMAINS

DOMAIN_CONSTRAINTS

ELEMENT_TYPES

ENUM_VALUES

FIELDS

INDEXES

INDEX_COLUMNS

INFORMATION_SCHEMA_CATALOG_NAME

IN_DOUBT

KEY_COLUMN_USAGE

LOCKS

PARAMETERS

QUERY_STATISTICS

REFERENTIAL_CONSTRAINTS

RIGHTS

ROLES

ROUTINES

SCHEMATA

SEQUENCES

SESSIONS

SESSION_STATE

SETTINGS

SYNONYMS

TABLES

TABLE_CONSTRAINTS

TABLE_PRIVILEGES

TRIGGERS

USERS

VIEWS

Range Table

Information Schema

The system tables and views in the schema INFORMATION_SCHEMA contain the meta data of all tables, views, domains, and other objects in the database as well as the current settings. This documentation describes the default new version of INFORMATION_SCHEMA for H2 2.0. Old TCP clients (1.4.200 and below) see the legacy version of INFORMATION_SCHEMA, because they can't work with the new one. The legacy version is not documented.

CHECK_CONSTRAINTS

Contains CHECK clauses of check and domain constraints.

CONSTRAINT_CATALOG	CHARACTER VARYING
The catalog (database name).	
CONSTRAINT_SCHEMA	CHARACTER VARYING
The schema of the constraint.	
CONSTRAINT_NAME	CHARACTER VARYING
The name of the constraint.	
CHECK_CLAUSE	CHARACTER VARYING
The SQL of CHECK clause.	

COLLATIONS

Contains available collations.

COLLATION_CATALOG	CHARACTER VARYING
The catalog (database name) for character string data types.	
COLLATION_SCHEMA	CHARACTER VARYING
The name of public schema for character string data types.	
COLLATION_NAME	CHARACTER VARYING
The name of collation for character string data types.	
PAD_ATTRIBUTE	CHARACTER VARYING
'NO PAD'.	
LANGUAGE_TAG	CHARACTER VARYING
The language tag.	

COLUMNS

Contains information about columns of tables.

TABLE_CATALOG	CHARACTER VARYING
The catalog (database name).	
TABLE_SCHEMA	CHARACTER VARYING
The schema of the table.	
TABLE_NAME	CHARACTER VARYING
The name of the table.	
COLUMN_NAME	CHARACTER VARYING
The name of the column.	
ORDINAL_POSITION	INTEGER
The ordinal position (1-based).	
COLUMN_DEFAULT	CHARACTER VARYING
The SQL of DEFAULT expression, if any.	
IS_NULLABLE	CHARACTER VARYING
Whether column may contain NULL value ('YES' or 'NO').	

DATA_TYPE	CHARACTER VARYING
The SQL data type name.	
CHARACTER_MAXIMUM_LENGTH	BIGINT
The maximum length in characters for character string data types. For binary string data types contains the same value as CHARACTER_OCTET_LENGTH.	
CHARACTER_OCTET_LENGTH	BIGINT
The maximum length in bytes for binary string data types. For character string data types contains the same value as CHARACTER_MAXIMUM_LENGTH.	
NUMERIC_PRECISION	INTEGER
The precision for numeric data types.	
NUMERIC_PRECISION_RADIX	INTEGER
The radix of precision (2 or 10) for numeric data types.	
NUMERIC_SCALE	INTEGER
The scale for numeric data types.	
DATETIME_PRECISION	INTEGER
The fractional seconds precision for datetime data types.	
INTERVAL_TYPE	CHARACTER VARYING
The data type of interval qualifier for interval data types.	
INTERVAL_PRECISION	INTEGER
The leading field precision for interval data types.	
CHARACTER_SET_CATALOG	CHARACTER VARYING
The catalog (database name) for character string data types.	
CHARACTER_SET_SCHEMA	CHARACTER VARYING
The name of public schema for character string data types.	
CHARACTER_SET_NAME	CHARACTER VARYING
The 'Unicode' for character string data types.	
COLLATION_CATALOG	CHARACTER VARYING

The catalog (database name) for character string data types.	
COLLATION_SCHEMA	CHARACTER VARYING
The name of public schema for character string data types.	
COLLATION_NAME	CHARACTER VARYING
The name of collation for character string data types.	
DOMAIN_CATALOG	CHARACTER VARYING
The catalog for columns with domain.	
DOMAIN_SCHEMA	CHARACTER VARYING
The schema of domain for columns with domain.	
DOMAIN_NAME	CHARACTER VARYING
The name of domain for columns with domain.	
MAXIMUM_CARDINALITY	INTEGER
The maximum cardinality for array data types.	
DTD_IDENTIFIER	CHARACTER VARYING
The data type identifier to read additional information from INFORMATION_SCHEMA.ELEMENT_TYPES for array data types, INFORMATION_SCHEMA.ENUM_VALUES for ENUM data type, and INFORMATION_SCHEMA.FIELDS for row value data types.	
IS_IDENTITY	CHARACTER VARYING
Whether column is an identity column ('YES' or 'NO').	
IDENTITY_GENERATION	CHARACTER VARYING
Identity generation ('ALWAYS' or 'BY DEFAULT') for identity columns.	
IDENTITY_START	BIGINT
The initial start value for identity columns.	
IDENTITY_INCREMENT	BIGINT
The increment value for identity columns.	
IDENTITY_MAXIMUM	BIGINT

The maximum value for identity columns.	
IDENTITY_MINIMUM	BIGINT
The minimum value for identity columns.	
IDENTITY_CYCLE	CHARACTER VARYING
Whether identity values are cycled ('YES' or 'NO') for identity columns.	
IS_GENERATED	CHARACTER VARYING
Whether column is an generated column ('ALWAYS' or 'NEVER')	
GENERATION_EXPRESSION	CHARACTER VARYING
The SQL of GENERATED ALWAYS AS expression for generated columns.	
DECLARED_DATA_TYPE	CHARACTER VARYING
The declared SQL data type name for numeric data types.	
DECLARED_NUMERIC_PRECISION	INTEGER
The declared precision, if any, for numeric data types.	
DECLARED_NUMERIC_SCALE	INTEGER
The declared scale, if any, for numeric data types.	
GEOMETRY_TYPE	CHARACTER VARYING
The geometry type constraint, if any, for geometry data types.	
GEOMETRY_SRID	INTEGER
The geometry SRID (Spatial Reference Identifier) constraint, if any, for geometry data types.	
IDENTITY_BASE	BIGINT
The current base value for identity columns.	
IDENTITY_CACHE	BIGINT
The cache size for identity columns.	
COLUMN_ON_UPDATE	CHARACTER VARYING
The SQL of ON UPDATE expression, if any.	
IS_VISIBLE	BOOLEAN

Whether column is visible (included into SELECT *).	
DEFAULT_ON_NULL	BOOLEAN
Whether value of DEFAULT expression is used when NULL value is inserted.	
SELECTIVITY	INTEGER
The selectivity of a column (0-100), used to choose the best index.	
REMARKS	CHARACTER VARYING
Optional remarks.	

COLUMN_PRIVILEGES

Contains information about privileges of columns. H2 doesn't have per-column privileges, so this view actually contains privileges of their tables.

GRANTOR	CHARACTER VARYING
NULL.	
GRANTEE	CHARACTER VARYING
The name of grantee.	
TABLE_CATALOG	CHARACTER VARYING
The catalog (database name).	
TABLE_SCHEMA	CHARACTER VARYING
The schema of the table.	
TABLE_NAME	CHARACTER VARYING
The name of the table.	
COLUMN_NAME	CHARACTER VARYING
The name of the column.	
PRIVILEGE_TYPE	CHARACTER VARYING
'SELECT', 'INSERT', 'UPDATE', or 'DELETE'.	
IS_GRANTABLE	CHARACTER VARYING
Whether grantee may grant rights to this object to others ('YES' or 'NO').	

CONSTANTS

Contains information about constants.

CONSTANT_CATALOG	CHARACTER VARYING
The catalog (database name).	
CONSTANT_SCHEMA	CHARACTER VARYING
The schema of the constant.	
CONSTANT_NAME	CHARACTER VARYING
The name of the constant.	
VALUE_DEFINITION	CHARACTER VARYING
The SQL of value.	
DATA_TYPE	CHARACTER VARYING
The SQL data type name.	
CHARACTER_MAXIMUM_LENGTH	BIGINT
The maximum length in characters for character string data types. For binary string data types contains the same value as CHARACTER_OCTET_LENGTH.	
CHARACTER_OCTET_LENGTH	BIGINT
The maximum length in bytes for binary string data types. For character string data types contains the same value as CHARACTER_MAXIMUM_LENGTH.	
CHARACTER_SET_CATALOG	CHARACTER VARYING
The catalog (database name) for character string data types.	
CHARACTER_SET_SCHEMA	CHARACTER VARYING
The name of public schema for character string data types.	
CHARACTER_SET_NAME	CHARACTER VARYING
The 'Unicode' for character string data types.	
COLLATION_CATALOG	CHARACTER VARYING
The catalog (database name) for character string data types.	
COLLATION_SCHEMA	CHARACTER VARYING

The name of public schema for character string data types.	
COLLATION_NAME	CHARACTER VARYING
The name of collation for character string data types.	
NUMERIC_PRECISION	INTEGER
The precision for numeric data types.	
NUMERIC_PRECISION_RADIX	INTEGER
The radix of precision (2 or 10) for numeric data types.	
NUMERIC_SCALE	INTEGER
The scale for numeric data types.	
DATETIME_PRECISION	INTEGER
The fractional seconds precision for datetime data types.	
INTERVAL_TYPE	CHARACTER VARYING
The data type of interval qualifier for interval data types.	
INTERVAL_PRECISION	INTEGER
The leading field precision for interval data types.	
MAXIMUM_CARDINALITY	INTEGER
The maximum cardinality for array data types.	
DTD_IDENTIFIER	CHARACTER VARYING
The data type identifier to read additional information from INFORMATION_SCHEMA.ELEMENT_TYPES for array data types, INFORMATION_SCHEMA.ENUM_VALUES for ENUM data type, and INFORMATION_SCHEMA.FIELDS for row value data types.	
DECLARED_DATA_TYPE	CHARACTER VARYING
The declared SQL data type name for numeric data types.	
DECLARED_NUMERIC_PRECISION	INTEGER
The declared precision, if any, for numeric data types.	
DECLARED_NUMERIC_SCALE	INTEGER

The declared scale, if any, for numeric data types.	
GEOMETRY_TYPE	CHARACTER VARYING
The geometry type constraint, if any, for geometry data types.	
GEOMETRY_SRID	INTEGER
The geometry SRID (Spatial Reference Identifier) constraint, if any, for geometry data types.	
REMARKS	CHARACTER VARYING
Optional remarks.	

CONSTRAINT_COLUMN_USAGE

Contains information about columns used in constraints.

TABLE_CATALOG	CHARACTER VARYING
The catalog (database name).	
TABLE_SCHEMA	CHARACTER VARYING
The schema of the table.	
TABLE_NAME	CHARACTER VARYING
The name of the table.	
COLUMN_NAME	CHARACTER VARYING
The name of the column.	
CONSTRAINT_CATALOG	CHARACTER VARYING
The catalog (database name).	
CONSTRAINT_SCHEMA	CHARACTER VARYING
The schema of the constraint.	
CONSTRAINT_NAME	CHARACTER VARYING
The name of the constraint.	

DOMAINS

Contains information about domains.

DOMAIN_CATALOG	CHARACTER VARYING
The catalog (database name).	
DOMAIN_SCHEMA	CHARACTER VARYING
The schema of domain.	
DOMAIN_NAME	CHARACTER VARYING
The name of domain.	
DATA_TYPE	CHARACTER VARYING
The SQL data type name.	
CHARACTER_MAXIMUM_LENGTH	BIGINT
The maximum length in characters for character string data types. For binary string data types contains the same value as CHARACTER_OCTET_LENGTH.	
CHARACTER_OCTET_LENGTH	BIGINT
The maximum length in bytes for binary string data types. For character string data types contains the same value as CHARACTER_MAXIMUM_LENGTH.	
CHARACTER_SET_CATALOG	CHARACTER VARYING
The catalog (database name) for character string data types.	
CHARACTER_SET_SCHEMA	CHARACTER VARYING
The name of public schema for character string data types.	
CHARACTER_SET_NAME	CHARACTER VARYING
The 'Unicode' for character string data types.	
COLLATION_CATALOG	CHARACTER VARYING
The catalog (database name) for character string data types.	
COLLATION_SCHEMA	CHARACTER VARYING
The name of public schema for character string data types.	
COLLATION_NAME	CHARACTER VARYING
The name of collation for character string data types.	
NUMERIC_PRECISION	INTEGER

The precision for numeric data types.	
NUMERIC_PRECISION_RADIX	INTEGER
The radix of precision (2 or 10) for numeric data types.	
NUMERIC_SCALE	INTEGER
The scale for numeric data types.	
DATETIME_PRECISION	INTEGER
The fractional seconds precision for datetime data types.	
INTERVAL_TYPE	CHARACTER VARYING
The data type of interval qualifier for interval data types.	
INTERVAL_PRECISION	INTEGER
The leading field precision for interval data types.	
DOMAIN_DEFAULT	CHARACTER VARYING
The SQL of DEFAULT expression, if any.	
MAXIMUM_CARDINALITY	INTEGER
The maximum cardinality for array data types.	
DTD_IDENTIFIER	CHARACTER VARYING
The data type identifier to read additional information from INFORMATION_SCHEMA.ELEMENT_TYPES for array data types, INFORMATION_SCHEMA.ENUM_VALUES for ENUM data type, and INFORMATION_SCHEMA.FIELDS for row value data types.	
DECLARED_DATA_TYPE	CHARACTER VARYING
The declared SQL data type name for numeric data types.	
DECLARED_NUMERIC_PRECISION	INTEGER
The declared precision, if any, for numeric data types.	
DECLARED_NUMERIC_SCALE	INTEGER
The declared scale, if any, for numeric data types.	
GEOMETRY_TYPE	CHARACTER VARYING

The geometry type constraint, if any, for geometry data types.	
GEOMETRY_SRID	INTEGER
The geometry SRID (Spatial Reference Identifier) constraint, if any, for geometry data types.	
DOMAIN_ON_UPDATE	CHARACTER VARYING
The SQL of ON UPDATE expression, if any.	
PARENT_DOMAIN_CATALOG	CHARACTER VARYING
The catalog (database name) for domains with parent domain.	
PARENT_DOMAIN_SCHEMA	CHARACTER VARYING
The schema of parent domain for domains with parent domain.	
PARENT_DOMAIN_NAME	CHARACTER VARYING
The name of parent domain for domains with parent domain.	
REMARKS	CHARACTER VARYING
Optional remarks.	

DOMAIN_CONSTRAINTS

Contains basic information about domain constraints. See also INFORMATION_SCHEMA.CHECK_CONSTRAINTS.

CONSTRAINT_CATALOG	CHARACTER VARYING
The catalog (database name).	
CONSTRAINT_SCHEMA	CHARACTER VARYING
The schema of the constraint.	
CONSTRAINT_NAME	CHARACTER VARYING
The name of the constraint.	
DOMAIN_CATALOG	CHARACTER VARYING
The catalog (database name).	
DOMAIN_SCHEMA	CHARACTER VARYING
The schema of domain.	

DOMAIN_NAME	CHARACTER VARYING
The name of domain.	
IS_DEFERRABLE	CHARACTER VARYING
'NO'.	
INITIALLY_DEFERRED	CHARACTER VARYING
'NO'.	
REMARKS	CHARACTER VARYING
Optional remarks.	

ELEMENT_TYPES

Contains information about types of array elements.

OBJECT_CATALOG	CHARACTER VARYING
The catalog (database name).	
OBJECT_SCHEMA	CHARACTER VARYING
The schema of the object.	
OBJECT_NAME	CHARACTER VARYING
The name of the object.	
OBJECT_TYPE	CHARACTER VARYING
The TYPE of the object ('CONSTANT', 'DOMAIN', 'TABLE', or 'ROUTINE').	
COLLECTION_TYPE_IDENTIFIER	CHARACTER VARYING
The DTD_IDENTIFIER value of the object.	
DATA_TYPE	CHARACTER VARYING
The SQL data type name.	
CHARACTER_MAXIMUM_LENGTH	BIGINT
The maximum length in characters for character string data types. For binary string data types contains the same value as CHARACTER_OCTET_LENGTH.	
CHARACTER_OCTET_LENGTH	BIGINT

The maximum length in bytes for binary string data types. For character string data types contains the same value as CHARACTER_MAXIMUM_LENGTH.	
CHARACTER_SET_CATALOG	CHARACTER VARYING
The catalog (database name) for character string data types.	
CHARACTER_SET_SCHEMA	CHARACTER VARYING
The name of public schema for character string data types.	
CHARACTER_SET_NAME	CHARACTER VARYING
The 'Unicode' for character string data types.	
COLLATION_CATALOG	CHARACTER VARYING
The catalog (database name) for character string data types.	
COLLATION_SCHEMA	CHARACTER VARYING
The name of public schema for character string data types.	
COLLATION_NAME	CHARACTER VARYING
The name of collation for character string data types.	
NUMERIC_PRECISION	INTEGER
The precision for numeric data types.	
NUMERIC_PRECISION_RADIX	INTEGER
The radix of precision (2 or 10) for numeric data types.	
NUMERIC_SCALE	INTEGER
The scale for numeric data types.	
DATETIME_PRECISION	INTEGER
The fractional seconds precision for datetime data types.	
INTERVAL_TYPE	CHARACTER VARYING
The data type of interval qualifier for interval data types.	
INTERVAL_PRECISION	INTEGER
The leading field precision for interval data types.	
MAXIMUM_CARDINALITY	INTEGER

The maximum cardinality for array data types.	
DTD_IDENTIFIER	CHARACTER VARYING
The data type identifier to read additional information from INFORMATION_SCHEMA.ELEMENT_TYPES for array data types, INFORMATION_SCHEMA.ENUM_VALUES for ENUM data type, and INFORMATION_SCHEMA.FIELDS for row value data types.	
DECLARED_DATA_TYPE	CHARACTER VARYING
The declared SQL data type name for numeric data types.	
DECLARED_NUMERIC_PRECISION	INTEGER
The declared precision, if any, for numeric data types.	
DECLARED_NUMERIC_SCALE	INTEGER
The declared scale, if any, for numeric data types.	
GEOMETRY_TYPE	CHARACTER VARYING
The geometry type constraint, if any, for geometry data types.	
GEOMETRY_SRID	INTEGER
The geometry SRID (Spatial Reference Identifier) constraint, if any, for geometry data types.	

ENUM_VALUES

Contains information about enum values.

OBJECT_CATALOG	CHARACTER VARYING
The catalog (database name).	
OBJECT_SCHEMA	CHARACTER VARYING
The schema of the object.	
OBJECT_NAME	CHARACTER VARYING
The name of the object.	
OBJECT_TYPE	CHARACTER VARYING
The TYPE of the object ('CONSTANT', 'DOMAIN', 'TABLE', or 'ROUTINE').	

ENUM_IDENTIFIER	CHARACTER VARYING
The DTD_IDENTIFIER value of the object.	
VALUE_NAME	CHARACTER VARYING
The name of enum value.	
VALUE_ORDINAL	CHARACTER VARYING
The ordinal of enum value.	

FIELDS

Contains information about fields of row values.

OBJECT_CATALOG	CHARACTER VARYING
The catalog (database name).	
OBJECT_SCHEMA	CHARACTER VARYING
The schema of the object.	
OBJECT_NAME	CHARACTER VARYING
The name of the object.	
OBJECT_TYPE	CHARACTER VARYING
The TYPE of the object ('CONSTANT', 'DOMAIN', 'TABLE', or 'ROUTINE').	
ROW_IDENTIFIER	CHARACTER VARYING
The DTD_IDENTIFIER value of the object.	
FIELD_NAME	CHARACTER VARYING
The name of the field of the row value.	
ORDINAL_POSITION	INTEGER
The ordinal position (1-based).	
DATA_TYPE	CHARACTER VARYING
The SQL data type name.	
CHARACTER_MAXIMUM_LENGTH	BIGINT
The maximum length in characters for character string data types. For binary	

string data types contains the same value as CHARACTER_OCTET_LENGTH.	
CHARACTER_OCTET_LENGTH	BIGINT
The maximum length in bytes for binary string data types. For character string data types contains the same value as CHARACTER_MAXIMUM_LENGTH.	
CHARACTER_SET_CATALOG	CHARACTER VARYING
The catalog (database name) for character string data types.	
CHARACTER_SET_SCHEMA	CHARACTER VARYING
The name of public schema for character string data types.	
CHARACTER_SET_NAME	CHARACTER VARYING
The 'Unicode' for character string data types.	
COLLATION_CATALOG	CHARACTER VARYING
The catalog (database name) for character string data types.	
COLLATION_SCHEMA	CHARACTER VARYING
The name of public schema for character string data types.	
COLLATION_NAME	CHARACTER VARYING
The name of collation for character string data types.	
NUMERIC_PRECISION	INTEGER
The precision for numeric data types.	
NUMERIC_PRECISION_RADIX	INTEGER
The radix of precision (2 or 10) for numeric data types.	
NUMERIC_SCALE	INTEGER
The scale for numeric data types.	
DATETIME_PRECISION	INTEGER
The fractional seconds precision for datetime data types.	
INTERVAL_TYPE	CHARACTER VARYING
The data type of interval qualifier for interval data types.	
INTERVAL_PRECISION	INTEGER

The leading field precision for interval data types.	
MAXIMUM_CARDINALITY	INTEGER
The maximum cardinality for array data types.	
DTD_IDENTIFIER	CHARACTER VARYING
The data type identifier to read additional information from INFORMATION_SCHEMA.ELEMENT_TYPES for array data types, INFORMATION_SCHEMA.ENUM_VALUES for ENUM data type, and INFORMATION_SCHEMA.FIELDS for row value data types.	
DECLARED_DATA_TYPE	CHARACTER VARYING
The declared SQL data type name for numeric data types.	
DECLARED_NUMERIC_PRECISION	INTEGER
The declared precision, if any, for numeric data types.	
DECLARED_NUMERIC_SCALE	INTEGER
The declared scale, if any, for numeric data types.	
GEOMETRY_TYPE	CHARACTER VARYING
The geometry type constraint, if any, for geometry data types.	
GEOMETRY_SRID	INTEGER
The geometry SRID (Spatial Reference Identifier) constraint, if any, for geometry data types.	

INDEXES

Contains information about indexes.

INDEX_CATALOG	CHARACTER VARYING
The catalog (database name).	
INDEX_SCHEMA	CHARACTER VARYING
The schema of the index.	
INDEX_NAME	CHARACTER VARYING
The name of the index.	

TABLE_CATALOG	CHARACTER VARYING
The catalog (database name).	
TABLE_SCHEMA	CHARACTER VARYING
The schema of the table.	
TABLE_NAME	CHARACTER VARYING
The name of the table.	
INDEX_TYPE_NAME	CHARACTER VARYING
The type of the index ('PRIMARY KEY', 'UNIQUE INDEX', 'SPATIAL INDEX', etc.)	
NULLS_DISTINCT	CHARACTER VARYING
'YES' for unique indexes with distinct null values, 'NO' for unique indexes with not distinct null values, 'ALL' for multi-column unique indexes where only rows with null values in all unique columns are distinct, NULL for other types of indexes.	
IS_GENERATED	BOOLEAN
Whether index is generated by a constraint and belongs to it.	
REMARKS	CHARACTER VARYING
Optional remarks.	
INDEX_CLASS	CHARACTER VARYING
The Java class name of index implementation.	

INDEX_COLUMNS

Contains information about columns used in indexes.

INDEX_CATALOG	CHARACTER VARYING
The catalog (database name).	
INDEX_SCHEMA	CHARACTER VARYING
The schema of the index.	
INDEX_NAME	CHARACTER VARYING
The name of the index.	

TABLE_CATALOG	CHARACTER VARYING
The catalog (database name).	
TABLE_SCHEMA	CHARACTER VARYING
The schema of the table.	
TABLE_NAME	CHARACTER VARYING
The name of the table.	
COLUMN_NAME	CHARACTER VARYING
The name of the column.	
ORDINAL_POSITION	INTEGER
The ordinal position (1-based).	
ORDERING_SPECIFICATION	CHARACTER VARYING
'ASC' or 'DESC'.	
NULL_ORDERING	CHARACTER VARYING
'FIRST', 'LAST', or NULL.	
IS_UNIQUE	BOOLEAN
Whether this column is a part of unique column list of a unique index (TRUE or FALSE).	

INFORMATION_SCHEMA_CATALOG_NAME

Contains a single row with the name of catalog (database name).

CATALOG_NAME	CHARACTER VARYING
The catalog (database name).	

IN_DOUBT

Contains information about prepared transactions.

TRANSACTION_NAME	CHARACTER VARYING
The name of prepared transaction.	
TRANSACTION_STATE	CHARACTER VARYING

The state of prepared transaction ('IN_DOUBT', 'COMMIT', or 'ROLLBACK').
--

KEY_COLUMN_USAGE

Contains information about columns used by primary key, unique, or referential constraint.

CONSTRAINT_CATALOG	CHARACTER VARYING
The catalog (database name).	
CONSTRAINT_SCHEMA	CHARACTER VARYING
The schema of the constraint.	
CONSTRAINT_NAME	CHARACTER VARYING
The name of the constraint.	
TABLE_CATALOG	CHARACTER VARYING
The catalog (database name).	
TABLE_SCHEMA	CHARACTER VARYING
The schema of the table.	
TABLE_NAME	CHARACTER VARYING
The name of the table.	
COLUMN_NAME	CHARACTER VARYING
The name of the column.	
ORDINAL_POSITION	INTEGER
The ordinal position (1-based).	
POSITION_IN_UNIQUE_CONSTRAINT	INTEGER
The ordinal position in the referenced unique constraint (1-based).	

LOCKS

Contains information about tables locked by sessions.

TABLE_SCHEMA	CHARACTER VARYING
The schema of the table.	

TABLE_NAME	CHARACTER VARYING
The name of the table.	
SESSION_ID	INTEGER
The identifier of the session.	
LOCK_TYPE	CHARACTER VARYING
'READ' or 'WRITE'.	

PARAMETERS

Contains information about parameters of routines.

SPECIFIC_CATALOG	CHARACTER VARYING
The catalog (database name).	
SPECIFIC_SCHEMA	CHARACTER VARYING
The schema of the overloaded version of routine.	
SPECIFIC_NAME	CHARACTER VARYING
The name of the overloaded version of routine.	
ORDINAL_POSITION	INTEGER
The ordinal position (1-based).	
PARAMETER_MODE	CHARACTER VARYING
'IN'.	
IS_RESULT	CHARACTER VARYING
'NO'.	
AS_LOCATOR	CHARACTER VARYING
'YES' for LOBs, 'NO' for others.	
PARAMETER_NAME	CHARACTER VARYING
The name of the parameter.	
DATA_TYPE	CHARACTER VARYING
The SQL data type name.	

CHARACTER_MAXIMUM_LENGTH	BIGINT
The maximum length in characters for character string data types. For binary string data types contains the same value as CHARACTER_OCTET_LENGTH.	
CHARACTER_OCTET_LENGTH	BIGINT
The maximum length in bytes for binary string data types. For character string data types contains the same value as CHARACTER_MAXIMUM_LENGTH.	
CHARACTER_SET_CATALOG	CHARACTER VARYING
The catalog (database name) for character string data types.	
CHARACTER_SET_SCHEMA	CHARACTER VARYING
The name of public schema for character string data types.	
CHARACTER_SET_NAME	CHARACTER VARYING
The 'Unicode' for character string data types.	
COLLATION_CATALOG	CHARACTER VARYING
The catalog (database name) for character string data types.	
COLLATION_SCHEMA	CHARACTER VARYING
The name of public schema for character string data types.	
COLLATION_NAME	CHARACTER VARYING
The name of collation for character string data types.	
NUMERIC_PRECISION	INTEGER
The precision for numeric data types.	
NUMERIC_PRECISION_RADIX	INTEGER
The radix of precision (2 or 10) for numeric data types.	
NUMERIC_SCALE	INTEGER
The scale for numeric data types.	
DATETIME_PRECISION	INTEGER
The fractional seconds precision for datetime data types.	
INTERVAL_TYPE	CHARACTER VARYING

The data type of interval qualifier for interval data types.	
INTERVAL_PRECISION	INTEGER
The leading field precision for interval data types.	
MAXIMUM_CARDINALITY	INTEGER
The maximum cardinality for array data types.	
DTD_IDENTIFIER	CHARACTER VARYING
The data type identifier to read additional information from INFORMATION_SCHEMA.ELEMENT_TYPES for array data types, INFORMATION_SCHEMA.ENUM_VALUES for ENUM data type, and INFORMATION_SCHEMA.FIELDS for row value data types.	
DECLARED_DATA_TYPE	CHARACTER VARYING
The declared SQL data type name for numeric data types.	
DECLARED_NUMERIC_PRECISION	INTEGER
The declared precision, if any, for numeric data types.	
DECLARED_NUMERIC_SCALE	INTEGER
The declared scale, if any, for numeric data types.	
PARAMETER_DEFAULT	CHARACTER VARYING
NULL.	
GEOMETRY_TYPE	CHARACTER VARYING
The geometry type constraint, if any, for geometry data types.	
GEOMETRY_SRID	INTEGER
The geometry SRID (Spatial Reference Identifier) constraint, if any, for geometry data types.	

QUERY_STATISTICS

Contains statistics of queries when query statistics gathering is enabled.

SQL_STATEMENT	CHARACTER VARYING
The SQL statement.	

EXECUTION_COUNT	INTEGER
The execution count.	
MIN_EXECUTION_TIME	DOUBLE PRECISION
The minimum execution time in milliseconds.	
MAX_EXECUTION_TIME	DOUBLE PRECISION
The maximum execution time in milliseconds.	
CUMULATIVE_EXECUTION_TIME	DOUBLE PRECISION
The total execution time in milliseconds.	
AVERAGE_EXECUTION_TIME	DOUBLE PRECISION
The average execution time in milliseconds.	
STD_DEV_EXECUTION_TIME	DOUBLE PRECISION
The standard deviation of execution time in milliseconds.	
MIN_ROW_COUNT	BIGINT
The minimum number of rows.	
MAX_ROW_COUNT	BIGINT
The maximum number of rows.	
CUMULATIVE_ROW_COUNT	BIGINT
The total number of rows.	
AVERAGE_ROW_COUNT	DOUBLE PRECISION
The average number of rows.	
STD_DEV_ROW_COUNT	DOUBLE PRECISION
The standard deviation of number of rows.	

REFERENTIAL CONSTRAINTS

Contains additional information about referential constraints.

CONSTRAINT_CATALOG	CHARACTER VARYING
The catalog (database name).	

CONSTRAINT_SCHEMA	CHARACTER VARYING
The schema of the constraint.	
CONSTRAINT_NAME	CHARACTER VARYING
The name of the constraint.	
UNIQUE_CONSTRAINT_CATALOG	CHARACTER VARYING
The catalog (database name).	
UNIQUE_CONSTRAINT_SCHEMA	CHARACTER VARYING
The schema of referenced unique constraint.	
UNIQUE_CONSTRAINT_NAME	CHARACTER VARYING
The name of referenced unique constraint.	
MATCH_OPTION	CHARACTER VARYING
'NONE'.	
UPDATE_RULE	CHARACTER VARYING
The rule for UPDATE in referenced table ('RESTRICT', 'CASCADE', 'SET DEFAULT', or 'SET NULL').	
DELETE_RULE	CHARACTER VARYING
The rule for DELETE in referenced table ('RESTRICT', 'CASCADE', 'SET DEFAULT', or 'SET NULL').	

RIGHTS

Contains information about granted rights and roles.

GRANTEE	CHARACTER VARYING
The name of grantee.	
GRANTEETYPE	CHARACTER VARYING
'USER' if grantee is a user, 'ROLE' if grantee is a role.	
GRANTEDROLE	CHARACTER VARYING
The name of the granted role for role grants.	
RIGHTS	CHARACTER VARYING

The set of rights ('SELECT', 'DELETE', 'INSERT', 'UPDATE', or 'ALTER ANY SCHEMA' separated with ', ') for table grants.	
TABLE_SCHEMA	CHARACTER VARYING
The schema of the table.	
TABLE_NAME	CHARACTER VARYING
The name of the table.	

ROLES

Contains information about roles.

ROLE_NAME	CHARACTER VARYING
The name of the role.	
REMARKS	CHARACTER VARYING
Optional remarks.	

ROUTINES

Contains information about user-defined routines, including aggregate functions.

SPECIFIC_CATALOG	CHARACTER VARYING
The catalog (database name).	
SPECIFIC_SCHEMA	CHARACTER VARYING
The schema of the overloaded version of routine.	
SPECIFIC_NAME	CHARACTER VARYING
The name of the overloaded version of routine.	
ROUTINE_CATALOG	CHARACTER VARYING
The catalog (database name).	
ROUTINE_SCHEMA	CHARACTER VARYING
The schema of the routine.	
ROUTINE_NAME	CHARACTER VARYING

The name of the routine.	
ROUTINE_TYPE	CHARACTER VARYING
'PROCEDURE', 'FUNCTION', or 'AGGREGATE'.	
DATA_TYPE	CHARACTER VARYING
The SQL data type name.	
CHARACTER_MAXIMUM_LENGTH	BIGINT
The maximum length in characters for character string data types. For binary string data types contains the same value as CHARACTER_OCTET_LENGTH.	
CHARACTER_OCTET_LENGTH	BIGINT
The maximum length in bytes for binary string data types. For character string data types contains the same value as CHARACTER_MAXIMUM_LENGTH.	
CHARACTER_SET_CATALOG	CHARACTER VARYING
The catalog (database name) for character string data types.	
CHARACTER_SET_SCHEMA	CHARACTER VARYING
The name of public schema for character string data types.	
CHARACTER_SET_NAME	CHARACTER VARYING
The 'Unicode' for character string data types.	
COLLATION_CATALOG	CHARACTER VARYING
The catalog (database name) for character string data types.	
COLLATION_SCHEMA	CHARACTER VARYING
The name of public schema for character string data types.	
COLLATION_NAME	CHARACTER VARYING
The name of collation for character string data types.	
NUMERIC_PRECISION	INTEGER
The precision for numeric data types.	
NUMERIC_PRECISION_RADIX	INTEGER
The radix of precision (2 or 10) for numeric data types.	

NUMERIC_SCALE	INTEGER
The scale for numeric data types.	
DATETIME_PRECISION	INTEGER
The fractional seconds precision for datetime data types.	
INTERVAL_TYPE	CHARACTER VARYING
The data type of interval qualifier for interval data types.	
INTERVAL_PRECISION	INTEGER
The leading field precision for interval data types.	
MAXIMUM_CARDINALITY	INTEGER
The maximum cardinality for array data types.	
DTD_IDENTIFIER	CHARACTER VARYING
The data type identifier to read additional information from INFORMATION_SCHEMA.ELEMENT_TYPES for array data types, INFORMATION_SCHEMA.ENUM_VALUES for ENUM data type, and INFORMATION_SCHEMA.FIELDS for row value data types.	
ROUTINE_BODY	CHARACTER VARYING
'EXTERNAL'.	
ROUTINE_DEFINITION	CHARACTER VARYING
Source code or NULL if not applicable or user doesn't have ADMIN privileges.	
EXTERNAL_NAME	CHARACTER VARYING
The name of the class or method.	
EXTERNAL_LANGUAGE	CHARACTER VARYING
'JAVA'.	
PARAMETER_STYLE	CHARACTER VARYING
'GENERAL'.	
IS_DETERMINISTIC	CHARACTER VARYING
Whether routine is deterministic ('YES' or 'NO').	

DECLARED_DATA_TYPE	CHARACTER VARYING
The declared SQL data type name for numeric data types.	
DECLARED_NUMERIC_PRECISION	INTEGER
The declared precision, if any, for numeric data types.	
DECLARED_NUMERIC_SCALE	INTEGER
The declared scale, if any, for numeric data types.	
GEOMETRY_TYPE	CHARACTER VARYING
The geometry type constraint, if any, for geometry data types.	
GEOMETRY_SRID	INTEGER
The geometry SRID (Spatial Reference Identifier) constraint, if any, for geometry data types.	
REMARKS	CHARACTER VARYING
Optional remarks.	

SCHEMATA

Contains information about schemas.

CATALOG_NAME	CHARACTER VARYING
The catalog (database name).	
SCHEMA_NAME	CHARACTER VARYING
The schema name.	
SCHEMA_OWNER	CHARACTER VARYING
The name of schema owner.	
DEFAULT_CHARACTER_SET_CATALOG	CHARACTER VARYING
The catalog (database name).	
DEFAULT_CHARACTER_SET_SCHEMA	CHARACTER VARYING
The name of public schema.	
DEFAULT_CHARACTER_SET_NAME	CHARACTER VARYING

'Unicode'.	
SQL_PATH	CHARACTER VARYING
NULL.	
DEFAULT_COLLATION_NAME	CHARACTER VARYING
The name of database collation.	
REMARKS	CHARACTER VARYING
Optional remarks.	

SEQUENCES

Contains information about sequences.

SEQUENCE_CATALOG	CHARACTER VARYING
The catalog (database name).	
SEQUENCE_SCHEMA	CHARACTER VARYING
The schema of the sequence.	
SEQUENCE_NAME	CHARACTER VARYING
The name of the sequence.	
DATA_TYPE	CHARACTER VARYING
The SQL data type name.	
NUMERIC_PRECISION	INTEGER
The precision for numeric data types.	
NUMERIC_PRECISION_RADIX	INTEGER
The radix of precision (2 or 10) for numeric data types.	
NUMERIC_SCALE	INTEGER
The scale for numeric data types.	
START_VALUE	BIGINT
The initial start value.	
MINIMUM_VALUE	BIGINT

The minimum value.	
MAXIMUM_VALUE	BIGINT
The maximum value.	
INCREMENT	BIGINT
The increment value.	
CYCLE_OPTION	CHARACTER VARYING
Whether values are cycled ('YES' or 'NO').	
DECLARED_DATA_TYPE	CHARACTER VARYING
The declared SQL data type name for numeric data types.	
DECLARED_NUMERIC_PRECISION	INTEGER
The declared precision, if any, for numeric data types.	
DECLARED_NUMERIC_SCALE	INTEGER
The declared scale, if any, for numeric data types.	
BASE_VALUE	BIGINT
The current base value.	
CACHE	BIGINT
The cache size.	
REMARKS	CHARACTER VARYING
Optional remarks.	

SESSIONS

Contains information about sessions. Only users with ADMIN privileges can see all sessions, other users can see only own session.

SESSION_ID	INTEGER
The identifier of the session.	
USER_NAME	CHARACTER VARYING
The name of the user.	

SERVER	CHARACTER VARYING
The name of the server used by remote connection.	
CLIENT_ADDR	CHARACTER VARYING
The client address and port used by remote connection.	
CLIENT_INFO	CHARACTER VARYING
Additional client information provided by remote connection.	
SESSION_START	TIMESTAMP(9) WITH TIME ZONE
When this session was started.	
ISOLATION_LEVEL	CHARACTER VARYING
The isolation level of the session ('READ UNCOMMITTED', 'READ COMMITTED', 'REPEATABLE READ', 'SNAPSHOT', or 'SERIALIZABLE').	
EXECUTING_STATEMENT	CHARACTER VARYING
The currently executing statement, if any.	
EXECUTING_STATEMENT_START	TIMESTAMP(9) WITH TIME ZONE
When the current command was started, if any.	
CONTAINS_UNCOMMITTED	BOOLEAN
Whether the session contains any uncommitted changes.	
SESSION_STATE	CHARACTER VARYING
The state of the session ('RUNNING', 'SLEEP', etc.)	
BLOCKER_ID	INTEGER
The identifier or blocking session, if any.	
SLEEP_SINCE	TIMESTAMP(9) WITH TIME ZONE
When the last command was finished if session is sleeping.	

SESSION_STATE

Contains the state of the current session.

STATE_KEY	CHARACTER VARYING
-----------	-------------------

The key.	
STATE_COMMAND	CHARACTER VARYING
The SQL command that can be used to restore the state.	

SETTINGS

Contains values of various settings.

SETTING_NAME	CHARACTER VARYING
The name of the setting.	
SETTING_VALUE	CHARACTER VARYING
The value of the setting.	

SYNONYMS

Contains information about table synonyms.

SYNONYM_CATALOG	CHARACTER VARYING
The catalog (database name).	
SYNONYM_SCHEMA	CHARACTER VARYING
The schema of the synonym.	
SYNONYM_NAME	CHARACTER VARYING
The name of the synonym.	
SYNONYM_FOR	CHARACTER VARYING
The name of the referenced table.	
SYNONYM_FOR_SCHEMA	CHARACTER VARYING
The name of the referenced schema.	
TYPE_NAME	CHARACTER VARYING
'SYNONYM'.	
STATUS	CHARACTER VARYING
'VALID'.	
REMARKS	CHARACTER VARYING

Optional remarks.

TABLES

Contains information about tables. See also INFORMATION_SCHEMA.COLUMNS.

TABLE_CATALOG	CHARACTER VARYING
The catalog (database name).	
TABLE_SCHEMA	CHARACTER VARYING
The schema of the table.	
TABLE_NAME	CHARACTER VARYING
The name of the table.	
TABLE_TYPE	CHARACTER VARYING
'BASE TABLE', 'VIEW', 'GLOBAL TEMPORARY', or 'LOCAL TEMPORARY'.	
IS_INSERTABLE INTO	CHARACTER VARYING
Whether the table is insertable ('YES' or 'NO').	
COMMIT_ACTION	CHARACTER VARYING
'DELETE', 'DROP', or 'PRESERVE' for temporary tables.	
STORAGE_TYPE	CHARACTER VARYING
'CACHE'D' for regular persisted tables, 'MEMORY' for in-memory tables or persisted tables with in-memory indexes, 'GLOBAL TEMPORARY' or 'LOCAL TEMPORARY' for temporary tables, 'EXTERNAL' for tables with external table engines, or 'TABLE LINK' for linked tables.	
REMARKS	CHARACTER VARYING
Optional remarks.	
LAST_MODIFICATION	BIGINT
The sequence number of the last modification, if applicable.	
TABLE_CLASS	CHARACTER VARYING
The Java class name of implementation.	
ROW_COUNT_ESTIMATE	BIGINT

The approximate number of rows if known or some default value if unknown. For regular tables contains the total number of rows including the uncommitted rows.

TABLE_CONSTRAINTS

Contains basic information about table constraints (check, primary key, unique, and referential).

CONSTRAINT_CATALOG	CHARACTER VARYING
The catalog (database name).	
CONSTRAINT_SCHEMA	CHARACTER VARYING
The schema of the constraint.	
CONSTRAINT_NAME	CHARACTER VARYING
The name of the constraint.	
CONSTRAINT_TYPE	CHARACTER VARYING
'CHECK', 'PRIMARY KEY', 'UNIQUE', or 'REFERENTIAL'.	
TABLE_CATALOG	CHARACTER VARYING
The catalog (database name).	
TABLE_SCHEMA	CHARACTER VARYING
The schema of the table.	
TABLE_NAME	CHARACTER VARYING
The name of the table.	
IS_DEFERRABLE	CHARACTER VARYING
'NO'.	
INITIALLY_DEFERRED	CHARACTER VARYING
'NO'.	
ENFORCED	CHARACTER VARYING
'YES' for non-referential constants. 'YES' for referential constants when checks for referential integrity are enabled for the both referenced and referencing	

tables and 'NO' when they are disabled.	
NULLS_DISTINCT	CHARACTER VARYING
'YES' for unique constraints with distinct null values, 'NO' for unique constraints with not distinct null values, 'ALL' for multi-column unique constraints where only rows with null values in all unique columns are distinct, NULL for other types of constraints.	
INDEX_CATALOG	CHARACTER VARYING
The catalog (database name).	
INDEX_SCHEMA	CHARACTER VARYING
The schema of the index.	
INDEX_NAME	CHARACTER VARYING
The name of the index.	
REMARKS	CHARACTER VARYING
Optional remarks.	

TABLE_PRIVILEGES

Contains information about privileges of tables. See INFORMATION_SCHEMA.CHECK_CONSTRAINTS, INFORMATION_SCHEMA.KEY_COLUMN_USAGE, and INFORMATION_SCHEMA.REFERENTIAL_CONSTRAINTS for additional information.

GRANTOR	CHARACTER VARYING
NULL.	
GRANTEE	CHARACTER VARYING
The name of grantee.	
TABLE_CATALOG	CHARACTER VARYING
The catalog (database name).	
TABLE_SCHEMA	CHARACTER VARYING
The schema of the table.	

TABLE_NAME	CHARACTER VARYING
The name of the table.	
PRIVILEGE_TYPE	CHARACTER VARYING
'SELECT', 'INSERT', 'UPDATE', or 'DELETE'.	
IS_GRANTABLE	CHARACTER VARYING
Whether grantee may grant rights to this object to others ('YES' or 'NO').	
WITH_HIERARCHY	CHARACTER VARYING
'NO'.	

TRIGGERS

Contains information about triggers.

TRIGGER_CATALOG	CHARACTER VARYING
The catalog (database name).	
TRIGGER_SCHEMA	CHARACTER VARYING
The schema of the trigger.	
TRIGGER_NAME	CHARACTER VARYING
The name of the trigger.	
EVENT_MANIPULATION	CHARACTER VARYING
'INSERT', 'UPDATE', 'DELETE', or 'SELECT'.	
EVENT_OBJECT_CATALOG	CHARACTER VARYING
The catalog (database name).	
EVENT_OBJECT_SCHEMA	CHARACTER VARYING
The schema of the table.	
EVENT_OBJECT_TABLE	CHARACTER VARYING
The name of the table.	
ACTION_ORIENTATION	CHARACTER VARYING
'ROW' or 'STATEMENT'.	

ACTION_TIMING	CHARACTER VARYING
'BEFORE', 'AFTER', or 'INSTEAD OF'.	
IS_ROLLBACK	BOOLEAN
Whether this trigger is executed on rollback.	
JAVA_CLASS	CHARACTER VARYING
The Java class name.	
QUEUE_SIZE	INTEGER
The size of the queue (is not actually used).	
NO_WAIT	BOOLEAN
Whether trigger is defined with NO WAIT clause (is not actually used).	
REMARKS	CHARACTER VARYING
Optional remarks.	

USERS

Contains information about users. Only users with ADMIN privileges can see all users, other users can see only themselves.

USER_NAME	CHARACTER VARYING
The name of the user.	
IS_ADMIN	BOOLEAN
Whether user has ADMIN privileges.	
REMARKS	CHARACTER VARYING
Optional remarks.	

VIEWS

Contains additional information about views. See INFORMATION_SCHEMA.TABLES for basic information.

TABLE_CATALOG	CHARACTER VARYING
The catalog (database name).	

TABLE_SCHEMA	CHARACTER VARYING
The schema of the table.	
TABLE_NAME	CHARACTER VARYING
The name of the table.	
VIEW_DEFINITION	CHARACTER VARYING
The query SQL, if applicable.	
CHECK_OPTION	CHARACTER VARYING
'NONE'.	
IS_UPDATABLE	CHARACTER VARYING
'NO'.	
INSERTABLE_INTO	CHARACTER VARYING
'NO'.	
IS_TRIGGER_UPDATABLE	CHARACTER VARYING
Whether the view has INSTEAD OF trigger for UPDATE ('YES' or 'NO').	
IS_TRIGGER_DELETABLE	CHARACTER VARYING
Whether the view has INSTEAD OF trigger for DELETE ('YES' or 'NO').	
IS_TRIGGER_INSERTABLE_INTO	CHARACTER VARYING
Whether the view has INSTEAD OF trigger for INSERT ('YES' or 'NO').	
STATUS	CHARACTER VARYING
'VALID' or 'INVALID'.	
REMARKS	CHARACTER VARYING
Optional remarks.	

Range Table

The range table is a dynamic system table that contains all values from a start to an end value. Non-zero step value may be also specified, default is 1. Start value, end value, and optional step value are converted to BIGINT data type. The table contains one column called X. If start value is greater than end value

and step is positive the result is empty. If start value is less than end value and step is negative the result is empty too. If start value is equal to end value the result contains only start value. Start value, start value plus step, start value plus step multiplied by two and so on are included in result. If step is positive the last value is less than or equal to the specified end value. If step is negative the last value is greater than or equal to the specified end value. The table is used as follows:

Examples:

```
SELECT X FROM SYSTEM_RANGE(1, 10);  
-- 1, 2, 3, 4, 5, 6, 7, 8, 9, 10  
SELECT X FROM SYSTEM_RANGE(1, 10, 2);  
-- 1, 3, 5, 7, 9  
SELECT X FROM SYSTEM_RANGE(1, 10, -1);  
-- No rows  
SELECT X FROM SYSTEM_RANGE(10, 2, -2);  
-- 10, 8, 6, 4, 2
```

Build

[Portability](#)

[Environment](#)

[Building the Software](#)

[Using Maven 2](#)

[Native Image](#)

[Using Eclipse](#)

[Translating](#)

[Submitting Source Code Changes](#)

[Reporting Problems or Requests](#)

[Automated Build](#)

[Generating Railroad Diagrams](#)

Portability

This database is written in Java and therefore works on many platforms.

Environment

To run this database, a Java Runtime Environment (JRE) version 11 or higher is required. It is also possible to compile a standalone executable with experimental [native image](#) build.

Building the Software

You need to install a JDK, for example the Oracle JDK version 11. Ensure that Java binary directory is included in the PATH environment variable, and that the environment variable JAVA_HOME points to your Java installation. On the command line, go to the directory h2 and execute the following command:

```
build -?
```

For Linux and OS X, use ./build.sh instead of build.

You will get a list of targets. If you want to build the jar file, execute (Windows):

```
build jar
```

To run the build tool in shell mode, use the command line option -:

```
./build.sh -
```

Using Apache Lucene

Apache Lucene 9.7.0 is used for testing.

Using Maven 2

Using a Central Repository

You can include the database in your Maven 2 project as a dependency.

Example:

```
<dependency>  
  <groupId>com.h2database</groupId>  
  <artifactId>h2</artifactId>  
  <version>2.5.250</version>  
</dependency>
```

New versions of this database are first uploaded to <http://hsqldb.sourceforge.net/m2-repo/> and then automatically synchronized with the main [Maven repository](#); however after a new release it may take a few hours before they are available there.

Maven Plugin to Start and Stop the TCP Server

A Maven plugin to start and stop the H2 TCP server is available from [Laird Nelson at GitHub](#). To start the H2 server, use:

```
mvn com.edugility.h2-maven-plugin:1.0-SNAPSHOT:spawn
```

To stop the H2 server, use:

```
mvn com.edugility.h2-maven-plugin:1.0-SNAPSHOT:stop
```

Using Snapshot Version

To build a h2-*-SNAPSHOT.jar file and upload it to the local Maven 2 repository, execute the following command:

```
build mavenInstallLocal
```

Afterwards, you can include the database in your Maven 2 project as a dependency:

```
<dependency>  
  <groupId>com.h2database</groupId>  
  <artifactId>h2</artifactId>  
  <version>1.0-SNAPSHOT</version>  
</dependency>
```

Native Image

There is an experimental support for compilation of native executables with native-image tool. To build an executable with H2 install GraalVM and use its updater to get the native-image tool:

```
gu install native-image
```

This tool can be used for compilation of native executables:

```
native-image --no-fallback -jar h2-VERSION.jar h2
```

Known limitations:

- If --no-fallback parameter was specified, system tray icon may not appear even if -Djava.awt.headless=false parameter of native-image tool was used, because native-image doesn't add all necessary configuration for working GUI.
- If --no-fallback parameter was specified, user-defined functions and triggers need an additional configuration.
- JAVA_OBJECT data type wasn't tested and may not work at all.
- Third-party loggers, ICU4J collators, and fulltext search weren't tested.

Using Eclipse

To create an Eclipse project for H2, use the following steps:

- Install Git and [Eclipse](#).
- Get the H2 source code from Github:
`git clone https://github.com/h2database/h2database`
- Download all dependencies:
`build.bat download(Windows)`
`./build.sh download(otherwise)`
- In Eclipse, create a new Java project from existing source code: File, New, Project, Java Project, Create project from existing source.
- Select the h2 folder, click Next and Finish.
- To resolve com.sun.javadoc import statements, you may need to manually add the file `<java.home>/../lib/tools.jar` to the build path.

Translating

The translation of this software is split into the following parts:

- H2 Console: `src/main/org/h2/server/web/res/_text_*.prop`
- Error messages: `src/main/org/h2/res/_messages_*.prop`

To translate the H2 Console, start it and select Preferences / Translate. After you are done, send the translated `*.prop` file to the Google Group. The web site is currently translated using Google.

Submitting Source Code Changes

If you'd like to contribute bug fixes or new features, please consider the following guidelines to simplify merging them:

- Only use Java 11 features (do not use Java 17/21/etc) (see [Environment](#)).
- Follow the coding style used in the project, and use Checkstyle (see above) to verify. For example, do not use tabs (use spaces instead). The checkstyle configuration is in `src/installer/checkstyle.xml`.

- A template of the Eclipse settings are in `src/installer/eclipse.settings/*`. If you want to use them, you need to copy them to the `.settings` directory. The formatting options (`eclipseCodeStyle`) are also included.
- Please provide test cases and integrate them into the test suite. For Java level tests, see `src/test/org/h2/test/TestAll.java`. For SQL level tests, see SQL files in `src/test/org/h2/test/scripts`.
- The test cases should cover at least 90% of the changed and new code; use a code coverage tool to verify that (see above). or use the build target coverage.
- Verify that you did not break other features: run the test cases by executing build test.
- Provide end user documentation if required (`src/docsrc/html/*`).
- Document grammar changes in `src/main/org/h2/res/help.csv`
- Provide a change log entry (`src/docsrc/html/changelog.html`).
- Verify the spelling using build spellcheck. If required add the new words to `src/tools/org/h2/build/doc/dictionary.txt`.
- Run `src/installer/buildRelease` to find and fix formatting errors.
- Verify the formatting using build docs and build javadoc.
- Submit changes using GitHub's "pull requests". You'll require a free [GitHub](#) account. If you are not familiar with pull requests, please read GitHub's [Using pull requests](#) page.

For legal reasons, patches need to be public in the form of an [issue report or attachment](#) or in the form of an email to the [group](#). Significant contributions need to include the following statement:

"I wrote the code, it's mine, and I'm contributing it to H2 for distribution multiple-licensed under the MPL 2.0, and the EPL 1.0 (<https://h2database.com/html/license.html>)."

Reporting Problems or Requests

Please consider the following checklist if you have a question, want to report a problem, or if you have a feature request:

- For bug reports, please provide a [short, self contained, correct \(compilable\), example](#) of the problem.
- Feature requests are always welcome, even if the feature is already on the [issue tracker](#) you can comment it. If you urgently need a feature, consider [providing a patch](#).
- Before posting problems, check the [FAQ](#) and do a [Google search](#).
- When got an unexpected exception, please try the [Error Analyzer tool](#). If this doesn't help, please report the problem, including the complete error message and stack trace, and the root cause stack trace(s).
- When sending source code, please use a public web clipboard such as [Pastebin](#) or [Mystic Paste](#) to avoid formatting problems. Please keep test cases as simple and short as possible, but so that the problem can still be reproduced. As a template, use: [HelloWorld.java](#). Method that simply call other methods should be avoided, as well as unnecessary exception handling. Please use the JDBC API and no external tools or libraries. The test should include all required initialization code, and should be started with the main method.
- For large attachments, use a public storage such as [Google Drive](#).
- Google Group versus issue tracking: Use the [Google Group](#) for questions or if you are not sure it's a bug. If you are sure it's a bug, you can create an [issue](#), but you don't need to (sending an email to the group is enough). Please note that only few people monitor the issue tracking system.
- For out-of-memory problems, please analyze the problem yourself first, for example using the command line option -XX:
+HeapDumpOnOutOfMemoryError (to create a heap dump file on out of memory) and a memory analysis tool such as the [Eclipse Memory Analyzer \(MAT\)](#).
- It may take a few days to get an answers. Please do not double post.

Automated Build

This build process is automated and runs regularly. The build process includes running the tests and code coverage, using the command line `./build.sh jar testCI`. The results are available on [CI workflow](#) page.

Generating Railroad Diagrams

The railroad diagrams of the [SQL grammar](#) are HTML, formatted as nested tables. The diagrams are generated as follows:

- The BNF parser (`org.h2.bnf.Bnf`) reads and parses the BNF from the file `help.csv`.
- The page parser (`org.h2.server.web.PageParser`) reads the template HTML file and fills in the diagrams.
- The rail images (one straight, four junctions, two turns) are generated using a simple Java application.

To generate railroad diagrams for other grammars, see the package `org.h2.jcr`. This package is used to generate the SQL-2 railroad diagrams for the JCR 2.0 specification.

History

[Change Log](#)

[History of this Database Engine](#)

[Why Java](#)

[Supporters](#)

Change Log

The up-to-date change log is available [here](#)

History of this Database Engine

The development of H2 was started in May 2004, but it was first published on December 14th 2005. The original author of H2, Thomas Mueller, is also the original developer of Hypersonic SQL. In 2001, he joined PointBase Inc. where he wrote PointBase Micro, a commercial Java SQL database. At that point, he had to discontinue Hypersonic SQL. The HSQLDB Group was formed to continued to work on the Hypersonic SQL codebase. The name H2 stands for Hypersonic 2, however H2 does not share code with Hypersonic SQL or HSQLDB. H2 is built from scratch.

Why Java

The main reasons to use a Java database are:

- Very simple to integrate in Java applications
- Support for many different platforms
- More secure than native applications (no buffer overflows)
- User defined functions (or triggers) run very fast
- Unicode support

Some think Java is too slow for low level operations, but this is no longer true. Garbage collection for example is now faster than manual memory management.

Developing Java code is faster than developing C or C++ code. When using Java, most time can be spent on improving the algorithms instead of porting the code to different platforms or doing memory management. Features such as Unicode and network libraries are already built-in. In Java, writing secure code is easier because buffer overflows can not occur. Features such as reflection can be used for randomized testing.

Java is future proof: a lot of companies support Java. Java is now open source.

To increase the portability and ease of use, this software depends on very few libraries. Features that are not available in open source Java implementations (such as Swing) are not used, or only used for optional features.

Supporters

Many thanks for those who reported bugs, gave valuable feedback, spread the word, and translated this project.

Also many thanks to the donors. To become a donor, use PayPal (at the very bottom of the main web page). Donators are:

- Martin Wildam, Austria
- [tagtraum industries incorporated, USA](#)
- [TimeWriter, Netherlands](#)
- [Cognitect, USA](#)
- [Code 42 Software, Inc., Minneapolis](#)
- [Code Lutin, France](#)
- [NetSuxxess GmbH, Germany](#)
- [Poker Copilot, Steve McLeod, Germany](#)
- [SkyCash, Poland](#)
- [Lumber-mill, Inc., Japan](#)
- [StockMarketEye, USA](#)
- [Eckenfelder GmbH & Co.KG, Germany](#)
- Jun Iyama, Japan
- Steven Branda, USA
- Anthony Goubard, Netherlands

- Richard Hickey, USA
- Alessio Jacopo D'Adamo, Italy
- Ashwin Jayaprakash, USA
- Donald Bleyl, USA
- Frank Berger, Germany
- Florent Ramiere, France
- Antonio Casqueiro, Portugal
- Oliver Computing LLC, USA
- Harpal Grover Consulting Inc., USA
- Elisabetta Berlini, Italy
- William Gilbert, USA
- Antonio Dieguez Rojas, Chile
- [Ontology Works, USA](#)
- Pete Haidinyak, USA
- William Osmond, USA
- Joachim Ansorg, Germany
- Oliver Soerensen, Germany
- Christos Vasilakis, Greece
- Fyodor Kupolov, Denmark
- Jakob Jenkov, Denmark
- Stéphane Chartrand, Switzerland
- Glenn Kidd, USA
- Gustav Trede, Sweden
- Joonas Pulakka, Finland
- Bjorn Darri Sigurdsson, Iceland
- Gray Watson, USA
- Erik Dick, Germany
- Pengxiang Shao, China
- Bilingual Marketing Group, USA
- Philippe Marschall, Switzerland
- Knut Staring, Norway
- Theis Borg, Denmark

- Mark De Mendonca Duske, USA
- Joel A. Garringer, USA
- Olivier Chafik, France
- Rene Schwietzke, Germany
- Jalpesh Patadia, USA
- Takanori Kawashima, Japan
- Terrence JC Huang, China
- [JiaDong Huang, Australia](#)
- Laurent van Roy, Belgium
- Qian Chen, China
- Clinton Hyde, USA
- Kritchai Phromros, Thailand
- Alan Thompson, USA
- Ladislav Jech, Czech Republic
- Dimitrijs Fedotovs, Latvia
- Richard Manley-Reeve, United Kingdom
- Daniel Cyr, ThirdHalf.com, LLC, USA
- Peter Jünger, Germany
- Dan Keegan, USA
- Rafel Israels, Germany
- Fabien Todescato, France
- Cristan Meijer, Netherlands
- Adam McMahon, USA
- Fábio Gomes Lisboa Gomes, Brasil
- Lyderic Landry, England
- Mederp, Morocco
- Joaquim Golay, Switzerland
- Clemens Quoss, Germany
- Kervin Pierre, USA
- Jake Bellotti, Australia
- Arun Chittanoor, USA

Frequently Asked Questions

I Have a Problem or Feature Request

Are there Known Bugs? When is the Next Release?

Is this Database Engine Open Source?

Is Commercial Support Available?

How to Create a New Database?

How to Connect to a Database?

Where are the Database Files Stored?

What is the Size Limit (Maximum Size) of a Database?

Is it Reliable?

Why is Opening my Database Slow?

My Query is Slow

H2 is Very Slow

Column Names are Incorrect?

Float is Double?

How to Translate this Project?

How to Contribute to this Project?

I Have a Problem or Feature Request

Please read the [support checklist](#).

Are there Known Bugs? When is the Next Release?

Usually, bugs get fixes as they are found. There is a release every few weeks.

Here is the list of known and confirmed issues:

- When opening a database file in a timezone that has different daylight saving rules: the time part of dates where the daylight saving doesn't match will differ. This is not a problem within regions that use the same rules (such as, within USA, or within Europe), even if the timezone itself is different. As a workaround, export the database to a SQL script using the old timezone, and create a new database in the new timezone.

- Old versions of Tomcat and Glassfish 3 set most static fields (final or non-final) to null when unloading a web application. This can cause a `NullPointerException`. In Tomcat ≥ 6.0 this behavior can be disabled by setting the system property `org.apache.catalina.loader.WebappClassLoader.ENABLE_CLEAR_REFERENCES=false`. A known workaround is to put the `h2*.jar` file in a shared lib directory (`common/lib`). Tomcat 8.5 and newer versions don't clear fields and don't have such property.
- Some problems have been found with right outer join. Internally, it is converted to left outer join, which does not always produce the same results as other databases when used in combination with other joins. This problem is fixed in H2 version 1.3.

For a complete list, see [Open Issues](#).

Is this Database Engine Open Source?

Yes. It is free to use and distribute, and the source code is included. See also under license.

Is Commercial Support Available?

No, currently commercial support is not available.

How to Create a New Database?

By default, a new database is automatically created if it does not yet exist when [embedded](#) URL is used. See [Creating New Databases](#).

How to Connect to a Database?

The database driver is `org.h2.Driver`, and the database URL starts with `jdbc:h2:.`. To connect to a database using JDBC, use the following code:

```
Connection conn = DriverManager.getConnection("jdbc:h2:~/test", "sa", "");
```


Where are the Database Files Stored?

When using database URLs like `jdbc:h2:~/test`, the database is stored in the user directory. For Windows, this is usually `C:\Documents and Settings\<userName>` or `C:\Users\<userName>`. If the base directory is not set (as in `jdbc:h2:./test`), the database files are stored in the directory where the application is started (the current working directory). When using the H2 Console application from the start menu, this is `<Installation Directory>/bin`. The base directory can be set in the database URL. A fixed or relative path can be used. When using the URL `jdbc:h2:file:./data/sample`, the database is stored in the directory `data` (relative to the current working directory). The directory is created automatically if it does not yet exist. It is also possible to use the fully qualified directory name (and for Windows, drive name). Example: `jdbc:h2:file:C:/data/test`

What is the Size Limit (Maximum Size) of a Database?

See [Limits and Limitations](#).

Is it Reliable?

That is not easy to say. It is still a quite new product. A lot of tests have been written, and the code coverage of these tests is higher than 80% for each package. Randomized stress tests are run regularly. But there are probably still bugs that have not yet been found (as with most software). Some features are known to be dangerous, they are only supported for situations where performance is more important than reliability. Those dangerous features are:

- Disabling the transaction log or `FileDescriptor.sync()` using `LOG=0` or `LOG=1`.
- Using the transaction isolation level `READ_UNCOMMITTED` (`LOCK_MODE 0`) while at the same time using multiple connections.
- Disabling database file protection using (setting `FILE_LOCK` to `NO` in the database URL).
- Disabling referential integrity using `SET REFERENTIAL_INTEGRITY FALSE`.

In addition to that, running out of memory should be avoided. In older versions, `OutOfMemory` errors while using the database could corrupt a databases.

This database is well tested using automated test cases. The tests run every night and run for more than one hour. But not all areas of this database are equally well tested. When using one of the following features for production, please ensure your use case is well tested (if possible with automated test cases). The areas that are not well tested are:

- Platforms other than Windows, Linux, Mac OS X, or runtime environments other than Oracle / OpenJDK 7, 8, 9.
- The features AUTO_SERVER and AUTO_RECONNECT.
- Cluster mode, 2-phase commit, savepoints.
- Fulltext search.
- Operations on LOBs over 2 GB.
- The optimizer may not always select the best plan.
- Using the ICU4J collator.

Areas considered experimental are:

- The PostgreSQL server
- Clustering (there are cases where transaction isolation can be broken due to timing issues, for example one session overtaking another session).
- Compatibility modes for other databases (only some features are implemented).
- The soft reference cache (CACHE_TYPE=SOFT_LRU). It might not improve performance, and out of memory issues have been reported.

Some users have reported that after a power failure, the database cannot be opened sometimes. In this case, use a backup of the database or the Recover tool. Please report such problems. The plan is that the database automatically recovers in all situations.

Why is Opening my Database Slow?

To find out what the problem is, use the H2 Console and click on "Test Connection" instead of "Login". After the "Login Successful" appears, click on it (it's a link). This will list the top stack traces. Then either analyze this yourself, or post those stack traces in the Google Group.

Other possible reasons are: the database is very big (many GB), or contains linked tables that are slow to open.

My Query is Slow

Slow SELECT (or DELETE, UPDATE, MERGE) statement can have multiple reasons. Follow this checklist:

- Run ANALYZE (see documentation for details).
- Run the query with EXPLAIN and check if indexes are used (see documentation for details).
- If required, create additional indexes and try again using ANALYZE and EXPLAIN.
- If it doesn't help please report the problem.

H2 is Very Slow

By default, H2 closes the database when the last connection is closed. If your application closes the only connection after each operation, the database is opened and closed a lot, which is quite slow. There are multiple ways to solve this problem, see [Database Performance Tuning](#).

Column Names are Incorrect?

For the query `SELECT ID AS X FROM TEST` the method `ResultSetMetaData.getColumnNames()` returns `ID`, I expect it to return `X`. What's wrong?

This is not a bug. According the JDBC specification, the method `ResultSetMetaData.getColumnNames()` should return the name of the column and not the alias name. If you need the alias name, use [ResultSetMetaData.getColumnLabel\(\)](#). Some other database don't work like this yet (they don't follow the JDBC specification). If you need compatibility with those databases, use the [Compatibility Mode](#).

This also applies to `DatabaseMetaData` calls that return a result set. The columns in the JDBC API are column labels, not column names.

Float is Double?

For a table defined as `CREATE TABLE TEST(X FLOAT)` the method `ResultSet.getObject()` returns a `java.lang.Double`, I expect it to return a `java.lang.Float`. What's wrong?

This is not a bug. According the JDBC specification, the JDBC data type `FLOAT` is equivalent to `DOUBLE`, and both are mapped to `java.lang.Double`. See also [Mapping SQL and Java Types - 8.3.10 FLOAT](#).

Use `REAL` or `FLOAT(24)` data type for `java.lang.Float` values.

How to Translate this Project?

For more information, see [Build/Translating](#).

How to Contribute to this Project?

There are various way to help develop an open source project like H2. The first step could be to [translate](#) the error messages and the GUI to your native language. Then, you could [provide patches](#). Please start with small patches. That could be adding a test case to improve the [code coverage](#) (the target code coverage for this project is 90%, higher is better). You will have to [develop, build and run the tests](#). Once you are familiar with the code, you could implement missing features from the [feature request list](#). I suggest to start with very small features that are easy to implement. Keep in mind to provide test cases as well.